
TotalDepth Reference

Release 0.4.0

Paul Ross

Oct 28, 2024

CONTENTS

1	TotalDepth.DeTif	3
2	TotalDepth.PlotLogs	5
3	TotalDepth.BIT API Reference	7
3.1	TotalDepth.BIT.ReadBIT	7
3.2	TotalDepth.BIT.ToLAS	13
4	TotalDepth.LAS API Reference	15
4.1	TotalDepth.LAS.ReadLASFiles	15
4.2	TotalDepth.LAS.core.LASConstants	15
4.3	TotalDepth.LAS.core.LASRead	15
5	TotalDepth.LIS API Reference	21
5.1	TotalDepth.LIS.core.EngVal (Engineering Values)	21
5.2	TotalDepth.LIS.core.File	24
5.3	TotalDepth.LIS.core.FileIndexer	27
5.4	TotalDepth.LIS.core.FrameSet	31
5.5	FrameSet API Reference	32
5.6	FrameSet Usage	38
5.7	TotalDepth.LIS.core.LogiRec (Logical Records)	38
5.8	TotalDepth.LIS.core.LogPass	50
5.9	TotalDepth.LIS.core.Mnem (Mnemonics)	57
5.10	TotalDepth.LIS.core.PhysRec (Physical Record Handler)	58
5.11	TotalDepth.LIS.core.RawStream (Raw Stream Handler)	62
5.12	TotalDepth.LIS.core.RepCode (Representation Codes)	63
5.13	TotalDepth.LIS.core.Rle (Run Length Encoding)	69
5.14	TotalDepth.LIS.core.TifMarker (TIF Marker Handling)	71
5.15	TotalDepth.LIS.core.Type01Plan (Binary Frame Data Random Access)	73
5.16	TotalDepth.LIS.core.Units (Unit Conversion)	75
5.17	TotalDepth.LIS.DumpFrameSet	78
5.18	TotalDepth.LIS.Index	78
5.19	TotalDepth.LIS.LisToHtml	79
5.20	TotalDepth.LIS.PlotLogPasses	80
5.21	TotalDepth.LIS.ProcLISPath	81
5.22	TotalDepth.LIS.RandomFrameSetRead	82
5.23	TotalDepth.LIS.ScanLogiData	82
5.24	TotalDepth.LIS.ScanLogiRec	82
5.25	TotalDepth.LIS.ScanPhysRec	82
5.26	TotalDepth.LIS.TableHistogram	83

5.27	TotalDepth.LIS.ToLAS	84
5.28	TotalDepth.LIS.lis_cmn_cmd_opts	84
6	TotalDepth.RP66V1 API Reference	85
6.1	TotalDepth.RP66V1.core.AbsentValue	85
6.2	TotalDepth.RP66V1.core.File	85
6.3	TotalDepth.RP66V1.core.Index	91
6.4	TotalDepth.RP66V1.core.LogPass	93
6.5	TotalDepth.RP66V1.core.LogicalFile	94
6.6	TotalDepth.RP66V1.core.RepCode	97
6.7	TotalDepth.RP66V1.core.Units	97
6.8	TotalDepth.RP66V1.core.XAxis	101
6.9	TotalDepth.RP66V1.core.stringify	102
6.10	TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor	102
6.11	TotalDepth.RP66V1.core.LogicalRecord.EFLR	104
6.12	TotalDepth.RP66V1.core.LogicalRecord.Encryption	108
6.13	TotalDepth.RP66V1.core.LogicalRecord.IFLR	108
6.14	TotalDepth.RP66V1.core.LogicalRecord.Semantics	109
6.15	TotalDepth.RP66V1.core.LogicalRecord.Types	109
6.16	TotalDepth.RP66V1.IndexPickle	110
6.17	TotalDepth.RP66V1.IndexXML	111
6.18	TotalDepth.RP66V1.LogRecIndex	113
6.19	TotalDepth.RP66V1.Scan	114
6.20	TotalDepth.RP66V1.ScanHTML	116
6.21	TotalDepth.RP66V1.SearchFF01	117
6.22	TotalDepth.RP66V1.ToLAS	117
6.23	TotalDepth.RP66V1.util API Reference	119
7	TotalDepth.DAT.DAT_parser	137
7.1	DAT Files	137
7.2	Performance	138
7.3	API	138
8	TotalDepth.common - Common Package Reference	139
8.1	TotalDepth.common.cmn_cmd_opts (Common Command Line Options)	139
8.2	TotalDepth.common.colorama	140
8.3	TotalDepth.common.data_table (Pretty Printing Tables)	140
8.4	TotalDepth.common.LogPass	141
8.5	TotalDepth.common.lookup_mnemonic	144
8.6	TotalDepth.common.process (Monitoring a Process's CPU and Memory)	151
8.7	TotalDepth.common.Rle (Run Length Encoding Data Compression)	153
8.8	TotalDepth.common.Slice (Data Slicing)	154
8.9	TotalDepth.common.statistics	156
8.10	TotalDepth.common.units	157
8.11	TotalDepth.common.xml (Importing XML Libraries)	159
8.12	TotalDepth.common.xxd	159
9	TotalDepth.util - Utility Package Reference	161
9.1	TotalDepth.util.CopyBinFiles	161
9.2	TotalDepth.util.DictTree (Tree-like Dictionary)	161
9.3	TotalDepth.util.DirWalk (Directory Walking)	165
9.4	TotalDepth.util.EBCDIC	166
9.5	TotalDepth.util.ExecTimer (Timing Code Execution)	168
9.6	TotalDepth.util.FileBuffer (Look-ahead File Buffer)	169
9.7	TotalDepth.util.FileStatus	170

9.8	TotalDepth.util.Histogram	171
9.9	TotalDepth.util.HtmlUtils (HTML Utilities)	172
9.10	TotalDepth.util.PatternSearch (Search for Patterns in Binary Files)	173
9.11	TotalDepth.util.RemoveDupeFiles (Remove Duplicate Files)	173
9.12	TotalDepth.util.XmlWrite (XML/XHTML/HTML Writer)	174
9.13	TotalDepth.util.archive (Archive Management)	176
9.14	TotalDepth.util.bin_file_type (Binary File Type Classification)	177
9.15	TotalDepth.util.gnuplot (Gnuplot Graphs Support)	178
9.16	Plot Package Reference	179
A	Glossary	223
B	Licence	233
B.1	OpenSource Licence	233
B.2	Other Licences	240
	Python Module Index	241
	Index	243

Contents:

TOTALDEPTH.DETIF

Strip TIF markers from a file or scan a file and reporting errors in TIF markers.

exception TotalDepth.DeTif.DeTifException

__weakref__

list of weak references to the object (if defined)

exception TotalDepth.DeTif.DeTifExceptionCompare

exception TotalDepth.DeTif.DeTifExceptionRead

class TotalDepth.DeTif.TifMarker(*tell, type, prev, next*)

tell: int

Alias for field number 0

type: int

Alias for field number 1

prev: int

Alias for field number 2

next: int

Alias for field number 3

__str__() → str

Return str(self).

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static __new__(_cls, tell: int, type: int, prev: int, next: int)

__repr__()

Return a nicely formatted representation string

TotalDepth.DeTif.**compare_next**(*this: TotalDepth.DeTif.TifMarker, nxt: TotalDepth.DeTif.TifMarker*) → None

Compares two adjacent TIF markers for consistency.

TotalDepth.DeTif.**has_tif_file**(*fobj: BinaryIO*) → bool

Returns True of the file in path apparently has TIF markers.

TotalDepth.DeTif.**has_tif**(*path: str*) → bool

Returns True of the file in path apparently has TIF markers.

`TotalDepth.DeTif.tif_scan_file_object(fobj: BinaryIO) → List[TotalDepth.DeTif.TifMarker]`

Scan a file object and return the list of TIF markers.

`TotalDepth.DeTif.tif_scan_path(path: str) → List[TotalDepth.DeTif.TifMarker]`

Scan a file at path and return the list of TIF markers.

`TotalDepth.DeTif.get_errors(tifs: List[TotalDepth.DeTif.TifMarker], file_size: int) → List[str]`

Return a list of TIF marker errors.

`TotalDepth.DeTif.strip_tif(file_in: BinaryIO, file_out: BinaryIO) → Tuple[int, int]`

Read file_in then strip TIF markers and write to file_out. The only error detected is negative reads. Returns a tuple of (tif_markers_stripped, bytes_written).

`TotalDepth.DeTif.strip_path(path_in: str, path_out: str) → Tuple[int, int]`

Read path_in then strip TIF markers and write to path_out.

`TotalDepth.DeTif.de_tif_file(path_in: str, path_out: str, nervous: bool, over_write: bool) → Tuple[int, int, int]`

De-TIFs a file path_in writing to path_out. Returns a tuple of (files_copied, tif_count, byte_count) where files_copied is 0 or 1, tif_count is the number of 12 byte TIF markers and byte_count is the number of bytes in path_in (if no file written) or path_out if the file is written.

TOTALDEPTH.PLOTLOGS

Created on Jan 24, 2012

@author: p2ross

class TotalDepth.PlotLogs.IndexTableValue(*scale, evFirst, evLast, evInterval, curves, numPoints, outPath*)

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static **__new__**(*_cls, scale, evFirst, evLast, evInterval, curves, numPoints, outPath*)

__repr__()

Return a nicely formatted representation string

curves

Alias for field number 4

evFirst

Alias for field number 1

evInterval

Alias for field number 3

evLast

Alias for field number 2

numPoints

Alias for field number 5

outPath

Alias for field number 6

scale

Alias for field number 0

class TotalDepth.PlotLogs.PlotLogInfo

Class that collates information about the results of plotting log passes. This can, for example, write out an index.html page with links to SVG pages.

__init__()

__str__()

Return str(self).

addPlotResult(*theInPath, theOutPath, theLpIdx, theFilmID, theScale, theEvFirst, theEvLast, theCurveS, ptsPlotted*)

Adds a successful plot. theInPath - The file path to the input file. theOutPath - The file path to the output file. theLpIdx - Integer index of the LogPass in the input file. theFilmID - The FILM ID as a Mnem.

theScale - Plot scale as an number. theEvFirst - The first X axis as an EngVal. theEvLast - The last X axis as an EngVal. theCurveS - A list of Mnem of the curves plotted. ptsPlotted - Number of points plotted.

writeHTML(*theFilePath, theDesc*)

Write the index.html table.

retRelPath(*d, f*)

Given directory d and file path of f this returns relative path to f from d.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.PlotLogs.**PlotLogPasses**(*fpIn, fpOut, opts*)

Takes an input path, output path and generates SVG file(s) from LIS.

__init__(*fpIn, fpOut, opts*)

Constructor.

fpIn and fpOut are file or directory paths. fpOut will be created if necessary.

recursive is a flag to control directory recursion.

keepGoing is a flag passed to the LIS File.FileRead object.

lgFormatS is a list of strings the correspond to the **LgFormat UniqueId XML attribute**. If absent the LIS file FILM/PRES etc. tables are used.

apiHeader is a flag to control whether a API header is extracted from CONS tables is to be plotted on the top of the log.

property usesInternalRecords

True if we are going to use the internal records of a file to describe the plot. False if we are going to use external records, such as LgFormat XML files to describe the plot.

__weakref__

list of weak references to the object (if defined)

TotalDepth.PlotLogs.**plotLogPassesMP**(*dIn, dOut, opts*)

Multiprocessing code to plot log passes. Returns a PlotLogInfo object.

TOTALDEPTH.BIT API REFERENCE

Contents:

3.1 TotalDepth.BIT.ReadBIT

This processes Dresser Atlas BIT files. Dresser Atlas BIT files are TIF encoded and consist of a set of Log Passes:

- First block, this gives a description of the file.
- Subsequent blocks are frame data.

TIF markers are used to separate Log Passes and delineate the file. A TIF marker type 1 ends the Log Pass and a pair of type 1 markers ends the readable file.

Here is an example of TIF markers in a file:

```
$ tddetif -n data/DresserAtlasBIT/special/29_10-_3Z/DWL_FILE/29_10-_3Z_dwl_DWL_WIRE_
↳1644659.bit -v
Cmd: DWL_FILE/29_10-_3Z_dwl_DWL_WIRE_1644659.bit -v
Detected 187 TIF Markers in data/DresserAtlasBIT/special/29_10-_3Z/DWL_FILE/29_10-_3Z_
↳dwl_DWL_WIRE_1644659.bit
[    0] TifMarker: 0x00000000 Type: 0x00000000 Prev: 0x00000000 Next: 0x00000120
↳Length: 0x00000120 Payload: 0x00000114
[    1] TifMarker: 0x00000120 Type: 0x00000000 Prev: 0x00000000 Next: 0x000003ac
↳Length: 0x0000028c Payload: 0x00000280
[    2] TifMarker: 0x000003ac Type: 0x00000000 Prev: 0x00000120 Next: 0x00000638
↳Length: 0x0000028c Payload: 0x00000280
...
[   91] TifMarker: 0x0000e658 Type: 0x00000000 Prev: 0x0000e3cc Next: 0x0000e8e4
↳Length: 0x0000028c Payload: 0x00000280
[   92] TifMarker: 0x0000e8e4 Type: 0x00000000 Prev: 0x0000e658 Next: 0x0000eb70
↳Length: 0x0000028c Payload: 0x00000280
[   93] TifMarker: 0x0000eb70 Type: 0x00000001 Prev: 0x0000e8e4 Next: 0x0000eb7c
↳Length: 0x0000000c Payload: 0x00000000
[   94] TifMarker: 0x0000eb7c Type: 0x00000000 Prev: 0x0000eb70 Next: 0x0000ec9c
↳Length: 0x00000120 Payload: 0x00000114
[   95] TifMarker: 0x0000ec9c Type: 0x00000000 Prev: 0x0000eb7c Next: 0x0000ef28
↳Length: 0x0000028c Payload: 0x00000280
...
[  184] TifMarker: 0x0001cf48 Type: 0x00000000 Prev: 0x0001ccbc Next: 0x0001d1d4
↳Length: 0x0000028c Payload: 0x00000280
[  185] TifMarker: 0x0001d1d4 Type: 0x00000001 Prev: 0x0001cf48 Next: 0x0001d1e0
↳Length: 0x0000000c Payload: 0x00000000
```

(continues on next page)

(continued from previous page)

```
[ 186] TifMarker: 0x0001d1e0 Type: 0x00000001 Prev: 0x0001d1d4 Next: 0x0001d1ec
↪Length: 0x0000000c Payload: 0x00000000
Execution time = 0.020 (S)
Bye, bye!
```

Example first block without TIF markers, length 276 (0x114) bytes:

```
0000000c: 0002 0000 5348 454c 4c20 4558 5052 4f20 ....SHELL EXPRO
0000001c: 552e 4b2e 2020 2020 2020 3234 204f 4354 U.K.      24 OCT
0000002c: 2038 3420 2020 2020 204d 414e 5346 4945 84      MANSFIE
0000003c: 4c44 2f44 4f44 4453 2020 2020 2020 2020 LD/DODDS
0000004c: 2020 2020 2020 2020 2020 2020 000a 0018 ....
0000005c: 0054 2020 3220 3920 2f20 3120 3020 2d20 .T  2 9 / 1 0 -
0000006c: 3320 2020 2020 2020 2020 2020 2020 2020 3
0000007c: 2020 2020 2020 2020 2020 2020 2020 2020
0000008c: 2020 2020 2020 2020 2020 2020 2020 2020
0000009c: 2020 2020 2020 2020 2020 2020 0012 000b ....
000000ac: 0006 2020 000a 0000 434f 4e44 534e 2020 .. ....CONDSN
000000bc: 5350 2020 4752 2020 4341 4c20 5445 4e20 SP  GR  CAL TEN
000000cc: 5350 4420 4143 5120 4143 2020 5254 2020 SPD ACQ AC  RT
000000dc: 2020 2020 2020 2020 2020 2020 2020 2020
000000ec: 2020 2020 2020 2020 2020 2020 2020 2020
000000fc: 2020 2020 2020 2020 443a 6600 4438 fe00      D:f.D8..
0000010c: 4040 0000 0000 0000 4210 0000 4d4e 3233 @@.....B...MN23
0000011c: 394a 2031
```

Decomposed:

- 4 bytes unknown.
- 160 bytes ASCII description, there is some structure here but it is as yet unknown.
- C, the count of channels as big endian two byte format '>H' or '>h'.
- A two byte null
- Channel names, 4 bytes each, this is always 80 bytes long.
- Five 4 byte floats start, stop, step, 0, ???.
- Unknown tail of eight bytes.

Total = 4 + 160 + 2 + 2 + 80 + 5 * 4 + 8 = 276 (0x114) bytes.

Of note is that while the TIF markers are little endian many values within the file are big endian.

3.1.1 Decoding the Description

This is 160 bytes long. Example:

```
$ xxd -s +16 data/DresserAtlasBIT/special/29_10-_3Z/DWL_FILE/29_10-_3Z_dwl_DWL_WIRE_
↪1644659.bit | head -n 20
00000010: 5348 454c 4c20 4558 5052 4f20 552e 4b2e SHELL EXPRO U.K.
00000020: 2020 2020 2020 3234 204f 4354 2038 3420      24 OCT 84
00000030: 2020 2020 204d 414e 5346 4945 4c44 2f44      MANSFIELD/D
00000040: 4f44 4453 2020 2020 2020 2020 2020 2020 ODDS
```

(continues on next page)

(continued from previous page)

```

00000050: 2020 2020 2020 2020 000a 0018 0054 2020      ....T
00000060: 3220 3920 2f20 3120 3020 2d20 3320 2020  2 9 / 1 0 - 3
00000070: 2020 2020 2020 2020 2020 2020 2020 2020
00000080: 2020 2020 2020 2020 2020 2020 2020 2020
00000090: 2020 2020 2020 2020 2020 2020 2020 2020
000000a0: 2020 2020 2020 2020 0012 000b 0006 2020      .....
000000b0: 000a 0000 434f 4e44 534e 2020 5350 2020  ....CONDSN  SP

```

Looks like we have $4 * 16 + 8 = 72$ bytes of ASCII to 0x58 as a description.

Either:

Then 24 bytes of binary data to 0x70 ???:

```
000a 0018 0054 2020 3220 3920 2f20 3120 3020 2d20 3320 2020
```

$3 * 16 + 8 = 56$ ASCII spaces. hmm divided by 4 is 19. hmm maybe together $24 + 56 = 80$ and $80 / 4$ is 20. Channel units?

Then 8 bytes of stuff: 0012 000b 0006 2020

Total is: $72 + 24 + 56 + 8 == 160$

Or, alternatively: 72 ($4 * 16 + 8$) bytes of ASCII. Then five bytes: 000a 0018 00 Then 75 bytes of ASCII: 54 2020 3220 3920 2f20 3120 3020 2d20 3320 2020 ... Then 8 bytes of stuff: 0012 000b 0006 2020 Total is: $72 + 5 + 75 + 8 == 160$

3.1.2 Decoding the frames

Each subsequent block is a subdivided into the number of channels and read into that channel. For example a block of data 0x280 (640) bytes long with 10 channels is decomposed into 10 sub-blocks of 64 bytes each. Each sub-block contains 16 floats.

There is a directory that has both BIT and LIS files in it.

LIS: 29_10-_3Z/DWL_FILE/29_10-_3Z_dwl_DWL_WIRE_1988494.lis BIT: 29_10-_3Z_dwl_DWL_WIRE_1644659.bit and 29_10-_3Z_dwl_DWL_WIRE_1644660.bit

Log Pass 0 has X axis: 14950.000 (FT) to 14582.250 (FT) Interval -367.750 (FT) Total number of frames 1472 Overall frame spacing -0.250 (FT)

Corresponding BIT file 29_10-_3Z_dwl_DWL_WIRE_1644659.bit has:

```
LogPassRange(depth_from=14950.000891089492, depth_to=14590.000869631818, spacing=0.
→2500000149011621, unknown_a=0.0,
unknown_b=16.000000953674373)
```

Frames from spacing: 1441

A striking feature of the LIS Log Pass 0 file is that the SP is fixed throughout at -249.709. This value appears nowhere else in the LIS Log Pass. The BIT file 29_10-_3Z_dwl_DWL_WIRE_1644659.bit corresponds with the following:

- The binary data is assumed to start at $0x128 + 4$
- Each channel is sequential but read in blocks of 16 floats (64 bytes).
- After 16 floats are read for each channel (every 160 floats, or 640 bytes) then 12 bytes are read and discarded.

- Although the BIT file states 1441 frames from spacing the LIS file has read 1472 (0x5c0) frames (modulo 16) with the remaining values as 0.0001 for all channels.

The 12 bytes read after every 640 bytes look like this: 4 nulls, two values unknown, two nulls, two values unknown, two nulls. These are TIF markers.

The 0.0001 figure is actually 9.999999615829415e-05 or 0x3d 0x68 0xdb 0x8b

exception TotalDepth.BIT.ReadBIT.ExceptionTotalDepthBIT

Simple specialisation of an exception class for TotalDepth.BIT.

exception TotalDepth.BIT.ReadBIT.ExceptionTotalDepthBIT_TIF

When TIF markers go wrong.

exception TotalDepth.BIT.ReadBIT.ExceptionTotalDepthBITFirstBlock

Constructor from first block of data.

exception TotalDepth.BIT.ReadBIT.ExceptionTotalDepthBITDataBlocks

Constructor from first block of data.

TotalDepth.BIT.ReadBIT.bytes_to_float(*b: bytes*) → float

Returns a float from four bytes.

https://en.wikipedia.org/wiki/IBM_hexadecimal_floating-point

Example: -118.625 -> b'Âv '.

NOTE: This is the same as RP66V1 ISINGL (5) Representation Code.

TotalDepth.BIT.ReadBIT.float_to_bytes(*f: float*) → bytes

Returns four bytes from a float.

https://en.wikipedia.org/wiki/IBM_hexadecimal_floating-point

Example: b'Âv ' -> -118.625

NOTE: This is the same as RP66V1 ISINGL (5) Representation Code.

TotalDepth.BIT.ReadBIT.read_float(*file: BinaryIO*) → float

Returns a float from the current read position.

TotalDepth.BIT.ReadBIT.gen_floats(*b: bytes*) → Sequence[float]

Yields a sequence of floats from the bytes.

TotalDepth.BIT.ReadBIT.is_bit_file(*fobj: BinaryIO*) → bool

Returns True the file looks like a Western Atlas BIT file, False otherwise.

TotalDepth.BIT.ReadBIT.is_bit_file_from_path(*file_path: str*) → bool

Returns True the file looks like a Western Atlas BIT file, False otherwise.

class TotalDepth.BIT.ReadBIT.TifMarker(*tell: int, type: int, prev: int, next: int*)

Contains a TIF marker with its file position.

tell: int

Alias for field number 0

type: int

Alias for field number 1

prev: int

Alias for field number 2

next: int

Alias for field number 3


```

__getnewargs__()
    Return self as a plain tuple. Used by copy and pickle.

static __new__(_cls, tell: int, type: int, prev: int, next: int)

__repr__()
    Return a nicely formatted representation string

class TotalDepth.BIT.ReadBIT.TifType(value)
    Type of TIF marker. Type 0 is normal data. Type 1 is end of Log Pass. Type 2 is end of file.

class TotalDepth.BIT.ReadBIT.TifMarkedBytes(tell: int, tif_type: TotalDepth.BIT.ReadBIT.TifType,
                                              payload: bytes)

    This is yielded by yield_tif_blocks().

    tell: int
        Alias for field number 0

    tif_type: TotalDepth.BIT.ReadBIT.TifType
        Alias for field number 1

    payload: bytes
        Alias for field number 2

__getnewargs__()
    Return self as a plain tuple. Used by copy and pickle.

static __new__(_cls, tell: int, tif_type: TotalDepth.BIT.ReadBIT.TifType, payload: bytes)

__repr__()
    Return a nicely formatted representation string

TotalDepth.BIT.ReadBIT.yield_tif_blocks(file: BinaryIO) →
    Sequence[TotalDepth.BIT.ReadBIT.TifMarkedBytes]

    Generate the payload from blocks of the file.

class TotalDepth.BIT.ReadBIT.LogPassRange(depth_from: float, depth_to: float, spacing: float, unknown_a:
                                          float, unknown_b: float)

    POD container for the five floats that describe the range of the BIT Log Pass.

    depth_from: float
        Alias for field number 0

    depth_to: float
        Alias for field number 1

    spacing: float
        Alias for field number 2

    unknown_a: float
        Alias for field number 3

    unknown_b: float
        Alias for field number 4

__getnewargs__()
    Return self as a plain tuple. Used by copy and pickle.

static __new__(_cls, depth_from: float, depth_to: float, spacing: float, unknown_a: float, unknown_b:
              float)

__repr__()
    Return a nicely formatted representation string

```

`TotalDepth.BIT.ReadBIT.read_bytes_from_offset(b: bytes, count: int, offset: int) → Tuple[bytes, int]`
 Slices a bytes object and increments the offset. Usage:

```
result, offset = read_bytes_from_offset(b, count, offset)
```

class `TotalDepth.BIT.ReadBIT.BITFrameArray`(*ident: str, tif_block: TotalDepth.BIT.ReadBIT.TifMarkedBytes*)

Represents a Log Pass from a BIT file. This has a number of fields, some are BIT specific but `self.frame_array` is a [TotalDepth.common.LogPass.FrameArray](#).

__init__(*ident: str, tif_block: TotalDepth.BIT.ReadBIT.TifMarkedBytes*)

Example initial block, length 0x114, 276 bytes:

```
00000000c: 0002 0000 5348 454c 4c20 4558 5052 4f20 ...SHELL EXPRO
00000001c: 552e 4b2e 2020 2020 2020 3234 204f 4354 U.K.      24 OCT
00000002c: 2038 3420 2020 2020 204d 414e 5346 4945 84      MANSFIE
00000003c: 4c44 2f44 4f44 4453 2020 2020 2020 2020 LD/DODDS
00000004c: 2020 2020 2020 2020 2020 2020 000a 0018 ....
00000005c: 0054 2020 3220 3920 2f20 3120 3020 2d20 .T 2 9 / 1 0 -
00000006c: 3320 2020 2020 2020 2020 2020 2020 2020 3
00000007c: 2020 2020 2020 2020 2020 2020 2020 2020
00000008c: 2020 2020 2020 2020 2020 2020 2020 2020
00000009c: 2020 2020 2020 2020 2020 2020 0012 000b ....
0000000ac: 0006 2020 000a 0000 434f 4e44 534e 2020 .. ....CONDSN
0000000bc: 5350 2020 4752 2020 4341 4c20 5445 4e20 SP GR CAL TEN
0000000cc: 5350 4420 4143 5120 4143 2020 5254 2020 SPD ACQ AC RT
0000000dc: 2020 2020 2020 2020 2020 2020 2020 2020
0000000ec: 2020 2020 2020 2020 2020 2020 2020 2020
0000000fc: 2020 2020 2020 2020 443a 6600 4438 fe00 D:f.D8..
00000010c: 4040 0000 0000 0000 4210 0000 4d4e 3233 @@.....B...MN23
00000011c: 394a 2031
```

long_str() → str

Returns a multi-line string describing self.

add_block(*block: bytes*) → None

Adds a data block of frame data to my temporary data structure(s).

complete() → None

Converts the existing frame data to a `LogPass.FrameArray`. This adds a computed X-axis and removes temporary data structures.

__weakref__

list of weak references to the object (if defined)

`TotalDepth.BIT.ReadBIT.create_bit_frame_array_from_file`(*file: BinaryIO*) →

List[[TotalDepth.BIT.ReadBIT.BITFrameArray](#)]

Given a file this returns a list of `BITFrameArray` objects.

`TotalDepth.BIT.ReadBIT.create_bit_frame_array_from_path`(*file_path: str*) →

List[[TotalDepth.BIT.ReadBIT.BITFrameArray](#)]

Given a file path this returns a list of `BITFrameArray` objects or raises.

class `TotalDepth.BIT.ReadBIT.FileSizeTime`(*name, size, time*)

name: str

Alias for field number 0

size: int
Alias for field number 1

time: float
Alias for field number 2

__str__()
Return str(self).

__getnewargs__()
Return self as a plain tuple. Used by copy and pickle.

static __new__(_cls, name: str, size: int, time: float)

__repr__()
Return a nicely formatted representation string

TotalDepth.BIT.ReadBIT.**print_summarise_frame_array**(frame_array: TotalDepth.BIT.ReadBIT.BITFrameArray) → None

Summarise all the channels in the Frame Array.

TotalDepth.BIT.ReadBIT.**print_process_file**(file_path: str, verbose: int, summary: bool) → TotalDepth.BIT.ReadBIT.FileSizeTime

Process a BIT file and print out a summary.

TotalDepth.BIT.ReadBIT.**print_process_directory**(directory: str, recursive: bool, verbose: int, summary: bool) → Dict[str, TotalDepth.BIT.ReadBIT.FileSizeTime]

Processes a complete directory for BIT files.

TotalDepth.BIT.ReadBIT.**main**() → int
Main entry point.

3.2 TotalDepth.BIT.ToLAS

This reads Dresser Atlas BIT files and writes LAS files.

TotalDepth.BIT.ToLAS.**bit_frame_array_to_las_file**(bit_frame_array: TotalDepth.BIT.ReadBIT.BITFrameArray, input_file: str, logical_file_number: int, frame_slice: Optional[TotalDepth.common.Slice.Slice], channel_name_sub_set: Set[str], field_width: int, float_format: str, las_file: TextIO) → None

Writes a single Frame Array to an open LAS file.

TotalDepth.BIT.ToLAS.**single_bit_path_to_las_path**(bit_path: str, _array_reduction: str, path_out: str, frame_slice: TotalDepth.common.Slice.Slice, channels: Set[str], field_width: int, float_format: str) →

TotalDepth.LAS.core.WriteLAS.LASWriteResult

Given a path to BIT file this reads the file and writes out a LAS file for each frame array. The LAS file names are numbered sequentially.

TotalDepth.BIT.ToLAS.**main**() → int
Main entry point.

TOTALDEPTH.LAS API REFERENCE

Contents:

4.1 TotalDepth.LAS.ReadLASFiles

Reads LAS files and provides statistics on their content.

This also measures the performance of the LAS parser.

Created on Jan 12, 2012

@author: paulross

`TotalDepth.LAS.ReadLASFiles.main()`

Main entry point.

4.2 TotalDepth.LAS.core.LASConstants

4.3 TotalDepth.LAS.core.LASRead

Reads LAS files.

Created on Jan 11, 2012

4.3.1 Research

Finding occurrences of STEP:

```
$ grep -rIh 'STEP\.' LAS_1.2_2.0/ | sed -e 's/^[[[:space:]]*/' | tr -s ' ' | sort -b |  uniq
```

From that file occurrences of STEP of zero in around 24,000 files:

```
$ grep -rI 'STEP' LAS_1.2_2.0/ | sed -e 's/^[[[:space:]]*/' | tr -s ' ' | grep 'STEP.F 0.00' | wc -l $ grep -rI  
'STEP' LAS_1.2_2.0/ | sed -e 's/^[[[:space:]]*/' | tr -s ' ' | grep 'STEP.M 0.00' | wc -l $ grep -rI 'STEP'  
LAS_1.2_2.0/ | sed -e 's/^[[[:space:]]*/' | tr -s ' ' | grep 'STEP.S 0.00' | wc -l
```

F: 629 M: 114 S: 29 Total 772 or about 3.2%

exception `TotalDepth.LAS.core.LASRead.ExceptionLASRead`

Specialisation of exception for LASRead.

exception TotalDepth.LAS.core.LASRead.**ExceptionLASReadSection**

Specialisation of exception for LASRead when handling sections.

exception TotalDepth.LAS.core.LASRead.**ExceptionLASReadSectionArray**

Specialisation of exception for LASRead when handling array section.

exception TotalDepth.LAS.core.LASRead.**ExceptionLASReadData**

Specialisation of exception for LASRead when reading data.

exception TotalDepth.LAS.core.LASRead.**ExceptionLASKeyError**

Equivalent to KeyError when looking stuff up.

TotalDepth.LAS.core.LASRead.**LAS_FILE_EXT** = **' .las'**

LAS file extension

TotalDepth.LAS.core.LASRead.**LAS_FILE_EXT_LOWER** = **' .las'**

LAS lower case file extension

TotalDepth.LAS.core.LASRead.**has_las_extension**(*fp*)

Returns True if the file extension is a LAS one.

TotalDepth.LAS.core.LASRead.**RE_COMMENT** = **re.compile('^\\s*#(.*?)\$')**

Regex to match a comment Section 5.1 of “LAS Version 2.0: A Digital Standard for Logs, Update February 2017”

TotalDepth.LAS.core.LASRead.**DEBUG_LINE_BY_LINE** = **False**

logger.debug call here can add about 50% of the processing time

TotalDepth.LAS.core.LASRead.**generate_lines**(*text_file*)

Given an file-like object this generates non-blank, non-comment lines. It’s a co-routine so can accept a line to put back.

TotalDepth.LAS.core.LASRead.**SECT_TYPES** = **'VWCPOA'**

All section identifiers

TotalDepth.LAS.core.LASRead.**SECT_TYPES_WITH_DATA_LINES** = **'VWCP'**

Section with data lines

TotalDepth.LAS.core.LASRead.**SECT_TYPES_WITH_INDEX** = **'VWCPA'**

Section with index value in column 0

TotalDepth.LAS.core.LASRead.**RE_SECT_HEAD** = **re.compile('^~([VWCPOA])(.+)*\$')**

Regex to match a section head

TotalDepth.LAS.core.LASRead.**SECT_DESCRIPTION_MAP** = {'A': 'ASCII Log Data', 'C': 'Curve Information Section', 'O': 'Other Information Section', 'P': 'Parameter Information Section', 'V': 'Version Information Section', 'W': 'Well Information Section'}

Map of section identifiers to description

TotalDepth.LAS.core.LASRead.**RE_LINE_FIELD_0** = **re.compile('^\\s*([^ .:]+)\\s*\$')**

The ‘MENM’ field, no spaces, dots or colons. ONE group Perhaps Allow spaces in mnemonic. For example: “SWU -CPX.V/V 00 000 00 00: 75 Unlimited Formation Water Saturation (Complex Litho Model)” This is specifically excluded by LAS 2.0 (Page 4, 2017-01) but it happens. Also: “Spaces are permitted in front of the mnemonic and between the end of the mnemonic and the dot.”

TotalDepth.LAS.core.LASRead.**RE_LINE_FIELD_1** = **re.compile('^([^ :]+)(.+)*\$')**

The data field which is the middle of the line between the first ‘.’ and last ‘:’ This is composed of optional units and value. Units must follow the dot immediately and contain no colons or spaces Note: Group 2 may have leading and trailing spaces TWO groups

TotalDepth.LAS.core.LASRead.**string_to_value**(*value: Any*) → Any

Convert a value to an integer, float, bool or string. If a string it is stripped.

```

class TotalDepth.LAS.core.LASRead.SectLine(mnem: str, unit: str, valu: Any, desc: str)
    Captures the four fields from a single line: mnem unit valu desc

    mnem: str
        Alias for field number 0

    unit: str
        Alias for field number 1

    valu: Any
        Alias for field number 2

    desc: str
        Alias for field number 3

    __getnewargs__()
        Return self as a plain tuple. Used by copy and pickle.

    static __new__(_cls, mnem: str, unit: str, valu: Any, desc: str)

    __repr__()
        Return a nicely formatted representation string

class TotalDepth.LAS.core.LASRead.LASSection(section_type: str, raise_on_error: bool = True)
    Contains data on a section.

    SECTION_MNEMONIC_ORDER_AND_VALUES = {'V': (('VERS', (1.2, 2.0)), ('WRAP', (True, False)))}
        Contains the allowable values of certain mnemonics in a section. The appropriate section must have these
        mnemonics in the given order and the values of those mnemonics must be one of the given values.

    __init__(section_type: str, raise_on_error: bool = True)

    __str__()
        Return str(self).

    __len__()
        Number of members.

    __contains__(mnemonic: str)
        Membership test.

    add_member_line(line_number, line)
        Given a line this decomposes it to its _members. line_number is the position of the line l in the file starting
        at 1. Empty lines are ignored.

    create_index()
        Creates an index of {key : ordinal, ...}. key is the first object in each member. This will be a MNEM for
        most sections but a depth as a float for an array section.

    finalise()
        Finalisation of section, this updates the internal representation.

    __getitem__(key) → TotalDepth.LAS.core.LASRead.SectLine
        Returns an entry, key can be int or str.

    mnemonics()
        Returns a list of mnemonics.

    units()
        Returns a list of mnemonic units.

    keys()
        Returns the keys in the internal map.

```

find(*m*)

Returns the member ordinal for mnemonic *m* or -1 if not found. This can be used for finding the array column for a particular curve.

__weakref__

list of weak references to the object (if defined)

```
class TotalDepth.LAS.core.LASRead.LASSectionArray(section_type: str, wrap: bool, curve_section:  
                                                TotalDepth.LAS.core.LASRead.LASSection, null:  
                                                Union[int, float] = - 999.25, raise_on_error: bool  
                                                = True)
```

Contains data on an array section.

```
__init__(section_type: str, wrap: bool, curve_section: TotalDepth.LAS.core.LASRead.LASSection, null:  
        Union[int, float] = - 999.25, raise_on_error: bool = True)
```

add_member_line(*line_number: int, line: str*) → None

Process a line in an array section.

__getitem__(*key*) → *TotalDepth.common.LogPass.FrameChannel*

Returns an entry, key can be int or str.

__len__()

Number of members.

create_index()

Creates an index of {key : ordinal, ...}. key is the first object in each member. This will be a MNEM for most sections but a depth as a float for an array section.

finalise()

Finalisation.

frame_size()

Returns the number of data points in a frame.

```
class TotalDepth.LAS.core.LASRead.LASBase(identity: str)
```

Base class for LAS reading and writing. This provides common functionality to child classes.

UNITS_LAS_TO_LIS = {'F': 'FEET', 'mts': 'M' }

Mapping of commonly seen LAS units to proper LIS units Both key and value must be strings

__init__(*identity: str*)**__len__**()

Number of sections.

__getitem__(*key*)

Returns a section, key can be int or str.

generate_sections() → Sequence[*TotalDepth.LAS.core.LASRead.LASSection*]

Yields up each section.

property null_value: float

The NULL value, defaults to -999.25.

property x_axis_units

The X axis units.

property x_axis_start: TotalDepth.LIS.core.EngVal.EngVal

The Xaxis start value as an EngVal.

property x_axis_stop: TotalDepth.LIS.core.EngVal.EngVal

The Xaxis end value as an EngVal.

property x_axis_step: `TotalDepth.LIS.core.EngVal.EngVal`
 The Xaxis step value as an EngVal.

is_log_down() $\rightarrow$ bool
 Returns True if X axis is increasing i.e. for time or down log.

get_wsd_mnemonic(*mnemonic*) $\rightarrow$ Tuple[Optional[str], Optional[str]]
 Returns a tuple of (value, units) for a Mnemonic that may appear in either a Well section or a Parameter section. Units may be None if empty. Returns (None, None) if nothing found.

get_all_wsd_mnemonics() $\rightarrow$ Set[str]
 Returns a set of mnemonics from the Well section and the Parameter section.

has_output_mnemonic(*mnemonic: str*)
 Returns True if theMnem, a Mnem.Mnem() object is an output in the Curve section. It will use the alternate names table LGFORMAT_LAS from LASConstants to interpret curves that are not exact matches.

curve_mnemonics(*ordered=False*)
 Returns list of curve names actually declared in the Curve section. List will be unordered if ordered is False.

curve_units_as_str(*m*)
 Given a curve as a Mnem.Mnem() this returns the units as a string. May raise KeyError.

__weakref__
 list of weak references to the object (if defined)

class `TotalDepth.LAS.core.LASRead.LASRead(file_path: Union[str, TextIO], file_identity: str = "", raise_on_error: bool = True)`
 Reads a LAS file.

__init__ (*file_path: Union[str, TextIO], file_identity: str = "", raise_on_error: bool = True*)
 Reads a LAS file from theFp that is either a string (file path) or a file like object. If raise_on_error is True then some errors will terminate processing, if False then some errors will be ignored.

number_of_frames() $\rightarrow$ int
 Returns the number of frames of data in an 'A' record if I have one.

number_of_data_points()
 Returns the number of frame data points in an 'A' record if I have one.

TOTALDEPTH.LIS API REFERENCE

Contents:

5.1 TotalDepth.LIS.core.EngVal (Engineering Values)

This module handles engineering values i.e. a real numeric value associated with a unit-of-measure.

5.1.1 Operator Overloading

EngVals can be operated on by a real number or an other EngVal.

Addition and subtraction

The operators $+$, $-$, $+=$, $-=$ work on mixed real numbers and EngVals as expected. If two EngVal objects then unit conversion is performed before the operation. If the two EngVals have units in different categories an ExceptionUnit will be raised.

Division

Operators $/$, $/=$ are implemented.

Division of an EngVal by a real number preserves the EngVal units.

Division of an EngVal by an EngVal results in the following:

- It preserves the EngVal units if the denominator units are DIMENSIONLESS.
- **The result will have DIMENSIONLESS EngVal units if the numerator and denominator have units that are freely convertible i.e. in the same unit category.**
- An ExceptionUnit will be raised.

Multiplication

Operators `*`, `*=` are implemented for reals and EngVals. If an EngVal the units must be DIMENSIONLESS.

5.1.2 Notes

rval operators are implemented and this can result in type promotion. For example the result of `4.0 * EngVal(16, b'INCH')` is an `EngVal(64.0, b'INCH')`.

Created on 24 Nov 2010

5.1.3 EngVal Reference

exception TotalDepth.LIS.core.EngVal.ExceptionEngVal

class TotalDepth.LIS.core.EngVal.EngVal(*theVal, theUom=Mnem(b'\x00\x00\x00\x00')*)

Represents an engineering value that consists of a numeric value and a unit of measure (usually a string).

__init__(*theVal, theUom=Mnem(b'\x00\x00\x00\x00')*)

dimensionless()

Returns True if the measure is dimensionless.

__str__()

String representation.

pStr()

Returns a 'pretty' ASCII string.

strFormat(*theFormat, incPrefix=True*)

Returns as a string with the value to the specified format (must be capable of handling floating point values).

__add__(*other*)

Overload self+other, returned result has the sum of self and other. other can be an EngVal or a Real number. The units chosen are self's.

__radd__(*other*)

Right value addition, see **__add__**().

__sub__(*other*)

Overload self-other, returned result has the sum of self and other. The units chosen are self's.

__rsub__(*other*)

Right value subtraction, see **__sub__**().

__mul__(*other*)

Overload self*other. other must be a real number or a dimensionless EngVal.

__rmul__(*other*)

Right value multiplication, see **__mul__**().

__truediv__(*other*)

Overload self/other. other must be a real number or an EngVal. If the units of other are dimensionless then treat as a real number. If the units are in the same category as me convert them and return a dimensionless EngVal.

__rtruediv__(*other*)

Right value true division, see **__truediv__**().

__iadd__(*other*)

Addition in place, *other* must be a real number or an EngVal where it is converted to my units.

__isub__(*other*)

Subtraction in place, *other* must be a real number or an EngVal where it is converted to my units.

__imul__(*other*)

Overload self *= *other*. *other* must be a real number or a dimensionless EngVal.

__itruediv__(*other*)

Overload self /= *other*. *other* must be a real number or a dimensionless EngVal.

__lt__(*other*)

True if self < *other* False otherwise. If *other* is an EngVal unit conversion is performed which may raise an ExceptionUnit. If *other* is a real number then self units are ignored.

__le__(*other*)

True if self <= *other* False otherwise. If *other* is an EngVal unit conversion is performed which may raise an ExceptionUnit. If *other* is a real number then self units are ignored.

__eq__(*other*)

True if self == *other* False otherwise. If *other* is an EngVal unit conversion is performed which may raise an ExceptionUnit. If *other* is a real number then self units are ignored.

__ne__(*other*)

True if self != *other* False otherwise. If *other* is an EngVal unit conversion is performed which may raise an ExceptionUnit. If *other* is a real number then self units are ignored.

__gt__(*other*)

True if self > *other* False otherwise. If *other* is an EngVal unit conversion is performed which may raise an ExceptionUnit. If *other* is a real number then self units are ignored.

__ge__(*other*)

True if self >= *other* False otherwise. If *other* is an EngVal unit conversion is performed which may raise an ExceptionUnit. If *other* is a real number then self units are ignored.

convert(*theUnits*)

Convert my value to the supplied units in-place. May raise an ExceptionUnits.

getInUnits(*theUnits*)

Returns my value to the supplied units. May raise an ExceptionUnits.

newEngValInUnits(*theUnits*)

Returns a new EngVal converting me to the supplied units. May raise an ExceptionUnits.

newEngValInOpticalUnits()

Returns a new EngVal converting me to the 'optical' units if possible. For example a value in b'.1IN" will be converted to b'FEET'.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.LIS.core.EngVal.**EngValRc**(*theVal*, *theUom*, *theRc=None*)

An engineering value with a integer Representation Code.

__init__(*theVal*, *theUom*, *theRc=None*)

__str__()

String representation.

encode()

Encode my value to my RepCode returning a bytes object. May raise an ExceptionEngVal.

5.2 TotalDepth.LIS.core.File

Reads or writes LIS data to a physical file.

Created on 14 Nov 2010

exception TotalDepth.LIS.core.File.**ExceptionFile**

Specialisation of exception for Physical files.

exception TotalDepth.LIS.core.File.**ExceptionFileRead**

Specialisation of exception for reading Physical files.

exception TotalDepth.LIS.core.File.**ExceptionFileWrite**

Specialisation of exception for writing Physical files.

class TotalDepth.LIS.core.File.**FileBase**(*theFile, theFileId, mode, keepGoing*)

LIS file handler. This handles Physical Records (and TIF records).

__init__(*theFile, theFileId, mode, keepGoing*)

__weakref__

list of weak references to the object (if defined)

class TotalDepth.LIS.core.File.**FileRead**(*theFile, theFileId=None, keepGoing=False, pad_modulo: int = 0, pad_non_null: bool = False*)

LIS file reader, this offers the caller a number of incremental read operations. This handles Physical Records (and TIF records).

theFile - A file like object or string, if the latter it assumed to be a path.

theFileId - File identifier, this could be a path for example. If None the RawStream will try and cope with it.

keepGoing - If True we do our best to keep going.

__init__(*theFile, theFileId=None, keepGoing=False, pad_modulo: int = 0, pad_non_null: bool = False*)

Constructor with: *theFile* - A file like object or string, if the latter it assumed to be a path. *theFileId* - File identifier, this could be a path for example. If None the RawStream will try and cope with it. *keepGoing* - If True we do our best to keep going.

readLrBytes(*theLen=- 1*)

Reads *theLen* LogicalData bytes and returns it or None if nothing left to read for this logical record. If *theLen* is -1 all the remaining Logical data is read. This positions the file at the *_next_* Logical Record or EOF.

skipLrBytes(*theLen=- 1*)

Skips logical data and returns a count of skipped bytes. If *theLen* is -1 all the remaining Logical data is read.

seekCurrentLrStart()

Setting the file position directly to the beginning of a PRH or TIF marker (if present) for the current Logical Record.

skipToNextLr()

Skips the rest of the current Logical Data and positions the file at the start of the next Logical Record.

tellLr()

Returns the absolute file position of the start current Logical record. This value can be safely used in **seekLr**.

tell()

Returns the absolute position of the file.

ldIndex()

Returns the index position in the current logical data.

seekLr(*offset*)

External setting of file position directly to the beginning of a PRH or TIF marker (if present). The caller is fully responsible for getting this right!

hasLd()

Returns True if there is logical data to be read, False otherwise. NOTE: This will return False on file initialisation and only return True once the Physical Record Header (i.e. one or more logical bytes) has been read.

rewind()

Sets the file position to the beginning of file.

property isEOF

True if at EOF.

unpack(*theStruct*)

Unpack some logical bytes using the supplied format. format ~ a struct.Struct object. Returns a tuple of the number of objects specified by the format or None.

```
class TotalDepth.LIS.core.File.FileWrite(theFile, theFileId=None, keepGoing=False, hasTif=False,  
                                         thePrLen=65535,  
                                         thePrt=<TotalDepth.LIS.core.PhysRec.PhysRecTail object>)
```

LIS file writer. This handles Physical Records (and TIF records).

theFile - A file like object or string, if the latter it assumed to be a path.

theFileId - File identifier, this could be a path for example. If None the RawStream will try and cope with it.

keepGoing - If True we do our best to keep going.

hasTif - Insert TIF markers or not.

thePrLen - Max Physical Record length, defaults to the maximum possible length.

thePrt - Physical Records Trailer settings (defaults to PhysRec.PhysRecTail()).

```
__init__(theFile, theFileId=None, keepGoing=False, hasTif=False, thePrLen=65535,  
         thePrt=<TotalDepth.LIS.core.PhysRec.PhysRecTail object>)
```

Constructor with: *theFile* - A file like object or string, if the latter it assumed to be a path. *theFileId* - File identifier, this could be a path for example. If None the RawStream will try and cope with it. *keepGoing* - If True we do our best to keep going. *hasTif* - Insert TIF markers or not. *thePrLen* - Max Physical Record length, defaults to the maximum possible length. *thePrt* - Physical Records Trailer settings (defaults to PhysRec.PhysRecTail()).

write(*theLr*)

Writes the Logical Record to the file. Returns the tell() of the start of the LR.

close()

Closes the file.

```
class TotalDepth.LIS.core.File.PhysicalRecordSettings(pad_modulo: int, pad_non_null: bool)
```

Container for the settings to read Physical Records.

pad_modulo: int

Alias for field number 0

pad_non_null: bool

Alias for field number 1

__str__() → str

Return str(self).

__repr__() → str
Return repr(self).

__getnewargs__()
Return self as a plain tuple. Used by copy and pickle.

static __new__(cls, pad_modulo: int, pad_non_null: bool)

TotalDepth.LIS.core.File.best_physical_record_pad_settings(file_path_or_object: Union[str, _io.BytesIO], pr_limit=0) → Union[None, *TotalDepth.LIS.core.File.PhysicalRecordSettings*]

This attempts to find the best settings to read Physical Records. It returns a PhysicalRecordSettings on success or None on failure. pr_limit limits the number of Physical Records to test if you want higher performance otherwise all Physical Records are read.

Typically 38Mb file with 46276 Physical Records processed in 0.380 (s) so 10ms/Mb or 100Mb/s. This is proportionate so if limited to 100 records this would be around 0.001 (s)

TotalDepth.LIS.core.File.file_read_with_best_physical_record_pad_settings(file_path_or_object: Union[str, _io.BytesIO], file_id=None, pr_limit=0) → Union[None, *TotalDepth.LIS.core.File.FileRead*]

Explores a LIS file with the 'best' Physical Record settings and returns a FileRead object or None.

TotalDepth.LIS.core.File.scan_file_no_output(file_path_or_object: Union[str, _io.BytesIO], keep_going: bool, pad_modulo: int, pad_non_null: bool, pr_limit=0)

Reads a file as if it was a LIS file and returns the number of Physical Records.

Typically 38Mb file processed in 0.381 seconds so 10ms/Mb or 100Mb/s.

TotalDepth.LIS.core.File.scan_file_with_different_padding(file_path_or_object: Union[str, _io.BytesIO], keep_going: bool, pr_limit=0) → Dict[*TotalDepth.LIS.core.File.PhysicalRecordSettings*, int]

Tries all different padding options and returns a dict with the number of Physical Records parsed.

TotalDepth.LIS.core.File.ret_padding_options_with_max_records(pad_opts_to_prs: Dict[*TotalDepth.LIS.core.File.PhysicalRecordSettings*, int]) → List[*TotalDepth.LIS.core.File.PhysicalRecordSettings*]

Returns the list of padding options that maximises the number of Physical Records parsed from the structure provided by scan_file_with_different_padding().

5.3 TotalDepth.LIS.core.FileIndexer

#Contents: Indexes LIS files.

Created on 10 Feb 2011

@author: p2ross

exception TotalDepth.LIS.core.FileIndexer.ExceptionFileIndex

Specialisation of exception for the LIS file indexer.

class TotalDepth.LIS.core.FileIndexer.IndexObjBase(*tell, lrType, theF*)

Base class for indexed objects.

tell - The file position of the Logical Record as an integer.

lrType - The Logical Record type as an integer.

theF - The LIS File object. The file ID is recorded for later error checking.

__init__(*tell, lrType, theF*)

__str__()

Return str(self).

tocStr()

Returns a 'pretty' string suitable for a table of contents.

property tell

The file offset of the Logical Record.

property lrType

The Logical Record type, in integer.

property logicalRecord

The underlying LogiRec object or None.

property isDelimiter

True if this represents a delimiter record e.g. File Head/Tail.

canAdd(*iflrType*)

Returns True if this can accumulate another IFLR.

iflrType()

Returns the IFLR type that this EFLR can describe.

setLogicalRecord(*theFile*)

Sets the logicalRecord property to an LogiRec object from theFile.

jsonObject()

Return an Python object that can be JSON encoded.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.LIS.core.FileIndexer.IndexNone(*tell, lrType, theF*)

NULL class just takes the LR information and skips to next LR.

__init__(*tell, lrType, theF*)

setLogicalRecord(*theFile*)

Sets the logicalRecord property to an LogiRec object from theFile.

class TotalDepth.LIS.core.FileIndexer.IndexUnknownInternalFormat(*tell, lrType, theF*)

Binary verbatim class for things like encrypted records, images, raw table dumps and so on.

__init__(*tell, lrType, theF*)

setLogicalRecord(*theFile*)

Sets the logicalRecord property to an LogiRec.LrTable() object.

class TotalDepth.LIS.core.FileIndexer.**IndexLrFull**(*tell, lrType, theF, theClass*)

Takes a full LR and assigns it to self._lr.

theClass is a cls that is use to instantiate a Logical Record object at the current file position.

__init__(*tell, lrType, theF, theClass*)

setLogicalRecord(*theFile*)

Sets the logicalRecord property to an LogiRec object from theFile.

class TotalDepth.LIS.core.FileIndexer.**IndexFileHeadTail**(*tell, lrType, theF, theClass*)

Indexes a File header or trailer. The full Logical record is retained.

__init__(*tell, lrType, theF, theClass*)

tocStr()

Returns a 'pretty' string suitable for a table of contents.

class TotalDepth.LIS.core.FileIndexer.**IndexFileHead**(*tell, lrType, theF*)

Indexes a File header. The full Logical record is retained.

__init__(*tell, lrType, theF*)

class TotalDepth.LIS.core.FileIndexer.**IndexFileTail**(*tell, lrType, theF*)

Indexes a File trailer. The full Logical record is retained.

__init__(*tell, lrType, theF*)

class TotalDepth.LIS.core.FileIndexer.**IndexTapeHeadTail**(*tell, lrType, theF, theClass*)

Indexes a Tape header or trailer. The full Logical record is retained.

__init__(*tell, lrType, theF, theClass*)

tocStr()

Returns a 'pretty' string suitable for a table of contents.

class TotalDepth.LIS.core.FileIndexer.**IndexTapeHead**(*tell, lrType, theF*)

Indexes a Tape header. The full Logical record is retained.

__init__(*tell, lrType, theF*)

class TotalDepth.LIS.core.FileIndexer.**IndexTapeTail**(*tell, lrType, theF*)

Indexes a Tape trailer. The full Logical record is retained.

__init__(*tell, lrType, theF*)

class TotalDepth.LIS.core.FileIndexer.**IndexReelHeadTail**(*tell, lrType, theF, theClass*)

Indexes a Reel header or trailer. The full Logical record is retained.

__init__(*tell, lrType, theF, theClass*)

tocStr()

Returns a 'pretty' string suitable for a table of contents.

class TotalDepth.LIS.core.FileIndexer.**IndexReelHead**(*tell, lrType, theF*)

Indexes a Reel header. The full Logical record is retained.

__init__(*tell, lrType, theF*)

class TotalDepth.LIS.core.FileIndexer.**IndexReelTail**(*tell, lrType, theF*)

Indexes a Reel trailer. The full Logical record is retained.

```

    __init__(tell, lrType, theF)
class TotalDepth.LIS.core.FileIndexer.IndexTable(tell, lrType, theF)
    Table type logical records. Here we capture the first component block so that we know the name of the table.
    __init__(tell, lrType, theF)
    property name
        The name of the table from the first component block.
    __str__()
        Return str(self).
    tocStr()
        Returns a 'pretty' string suitable for a table of contents.
    setLogicalRecord(theFile)
        Sets the logicalRecord property to an LogiRec.LrTable() object.
    jsonObject()
        Return an Python object that can be JSON encoded.
class TotalDepth.LIS.core.FileIndexer.IndexLogPass(tell, lrType, theF, xAxisIndex=0)
    The index of a Log Pass. This contains a LogPass object.

    xAxisIndex is the channel index that is regarded as the X axis. This is the indirect axis if present or defaults to
    channel 0.
    __init__(tell, lrType, theF, xAxisIndex=0)
    property logPass
        The LogPass object.
    __str__()
        Return str(self).
    tocStr()
        Returns a 'pretty' string suitable for a table of contents.
    canAdd(iflrType)
        Returns the IFLR type that this EFLR can describe.
    iflrType()
        Returns the IFLR type that this EFLR can describe.
    add(tell, lrType, theF)
        Add an IFLR.
    jsonObject()
        Return an Python object that can be JSON encoded.
class TotalDepth.LIS.core.FileIndexer.PlotRecordSet
    A POD class that can contain a set of references to the essential (plus optional) logical records for plotting.
    __init__()
    __str__()
        Return str(self).
    canPlotFromInternalRecords()
        True if I have a valid value that could be yielded, i.e. the minimum information from the file that allows a
        plot. In practice this means a FILM, PRES record and a LogPass.

```

canPlotFromExternalRecords()

True if I have a valid value that could be yielded, i.e. the minimum information from the file that allows a plot using some external definition of what has to be plotted. In practice this means a LogPass.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.LIS.core.FileIndexer.**FileIndex**(*theF, xAxisIndex=0*)

Create an index for the LIS file, theF is a LIS File object.

xAxisIndex is the channel index that is regarded as the X axis (default 0). This is currently ignored in the absence of a reasonable use case.

__init__(*theF, xAxisIndex=0*)**longDesc**() → str

Returns a string that is the long description of this object.

property lrTypeS: Sequence[int]

Returns a list of Logical Record types (integers) that is in the index.

numLogPasses() → int

Returns the number of IndexLogPass objects in the index.

genLogPasses() → Sequence[TotalDepth.LIS.core.FileIndexer.IndexLogPass]

Yields all IndexLogPass objects in the index.

genPlotRecords(*fromInternalRecords=True*) → Sequence[TotalDepth.LIS.core.FileIndexer.PlotRecordSet]

This provides the minimal information for creating a Plot. It yields a PlotRecordSet that has (tell_FILM, tell_PRES, tell_AREA, tell_PIP, IndexLogPass) objects that are not separated by delimiter records. tell_FILM is the file offset of the FILM record. tell_PRES is the file offset of the PRES record. tell_AREA is None or the file offset of the AREA record. tell_PIP is None or the file offset of the PIP record. IndexLogPass is a IndexLogPass object.

genAll() → Sequence[TotalDepth.LIS.core.FileIndexer.IndexObjBase]

Generates each index object (child class of IndexObjBase) for example a IndexLogPass.

jsonObject() → Dict[str, Any]

Return an Python object that can be JSON encoded.

__weakref__

list of weak references to the object (if defined)

5.3.1 Usage

5.3.2 Examples

Using a LIS Indexer

Source:

```
myFilePath = "Spam/Eggs.LIS"
# Open a LIS file
myFi = File.FileRead(myFilePath, theFileId=myFilePath, keepGoing=True)
# Index the LIS file
myIdx = FileIndexer.FileIndex(myFi)
# Ask the index for all the LogPass objects, these do not have the frameSet populated yet
```

(continues on next page)

(continued from previous page)

```
for aLr in myIdx.genAll():
    print(aLr)
```

Typical result:

```
tell: 0x00000000 type=128 <TotalDepth.LIS.core.FileIndexer.IndexFileHead object at_
↳0x10197ff90>
tell: 0x00000050 type= 34 name=b'TOOL' <TotalDepth.LIS.core.FileIndexer.IndexTable_
↳object at 0x10197ffd0>
tell: 0x000001ea type= 34 name=b'INPU' <TotalDepth.LIS.core.FileIndexer.IndexTable_
↳object at 0x10197fd10>
tell: 0x000002342 type= 34 name=b'OUTP' <TotalDepth.LIS.core.FileIndexer.IndexTable_
↳object at 0x101b0b1d0>
tell: 0x000003622 type= 34 name=b'CONS' <TotalDepth.LIS.core.FileIndexer.IndexTable_
↳object at 0x101b0bed0>
tell: 0x00000456a type= 34 name=b'CONS' <TotalDepth.LIS.core.FileIndexer.IndexTable_
↳object at 0x101b0b590>
tell: 0x0000064aa type= 34 name=b'PRES' <TotalDepth.LIS.core.FileIndexer.IndexTable_
↳object at 0x101b0b050>
tell: 0x000006efe type= 34 name=b'FILM' <TotalDepth.LIS.core.FileIndexer.IndexTable_
↳object at 0x101b0bfd0>
tell: 0x000006fc6 type= 34 name=b'AREA' <TotalDepth.LIS.core.FileIndexer.IndexTable_
↳object at 0x101b0bb50>
<TotalDepth.LIS.core.LogPass.LogPass object at 0x101b06490>
<TotalDepth.LIS.core.LogPass.LogPass object at 0x101b12410>
tell: 0x0017cbee type=129 <TotalDepth.LIS.core.FileIndexer.IndexFileTail object at_
↳0x101b173d0>
```

5.3.3 Testing

The unit tests are in test/TestFileIndexer.Py

5.4 TotalDepth.LIS.core.FrameSet

This describes the LIS FrameSet that contains the binary frame data. It is effectively a wrapper around a 2-D numpy array with specific APIs to interface that with a channel specific shape. Importantly FrameSets can be partial in that they need not hold all the data for every frame and every channel. Instead they can hold data for the frames and channels specified by the caller.

The FrameSet module is located in src/TotalDepth/LIS/core and can be imported thus:

```
from TotalDepth.LIS.core import FrameSet
```

5.4.1 FrameSet Internals

The internal representation of the FrameSet uses only Python and numpy structures and types, it is thus ignorant of the LIS file format or data structures apart from that described below.

5.4.2 LIS Dependencies

In the current version of the code there are a number of dependencies on knowledge of the LIS file format (or at least TotalDepth's representation of that format):

- ChArTe objects are constructed from LIS Datum Specification Block objects.
- XAxisDecl consists of a straight copy of various Entry Block values.
- XAxisDecl relies on the LIS value of the up/down flag.
- FrameSet is constructed with a DFSR and extracts the absent value from the DFSR as well as constructing ChArTe and XAxisDecl objects.
- **FrameSet has quite a lot of dependencies of Representation codes these are mainly used in two ways.**
 - Interpreting Dipmeter sub-channels.
 - `setFrameBytes()` uses the RepCode module directly to convert bytes to values for a channel.

These dependencies restrict the use of FrameSet to processing LIS data, if they were removed then FrameSet could be used for other file formats but there is no obvious use case for that as, apart from LIS, TotalDepth supports LAS (trivially simple frame sets) and will support RP66 at some point. The latter may trigger a refactoring of the FrameSet module.

5.5 FrameSet API Reference

The FrameSet module provides a means of representing LIS frame data.

Created on 10 Jan 2011

exception TotalDepth.LIS.core.FrameSet.ExceptionFrameSet

Specialisation of exception for FrameSet.

exception TotalDepth.LIS.core.FrameSet.ExceptionFrameSetEmpty

Raised when an illegal operation is performed on a FrameSet.

exception TotalDepth.LIS.core.FrameSet.ExceptionFrameSetNULLSpacing

Raised when FrameSet depends on a frame spacing that can not be determined.

exception TotalDepth.LIS.core.FrameSet.ExceptionFrameSetMixedChannels

Raised when generating values for multiple channels where the channels are not of the same shape i.e. number of samples, butsts.

TotalDepth.LIS.core.FrameSet.chkIdx(*i*, *l*, *msg*)

Global function for checking indexes and raising an IndexError. *msg* is expected to have two format fields that take *i* and *l* respectively.

TotalDepth.LIS.core.FrameSet.sliceDefaults(*theSl*)

Returns a new slice with start=None as 0 and step=None as 1.

class TotalDepth.LIS.core.FrameSet.DataSeqBase

Base class for a sequence of objects.

`__init__()`

__weakref__

list of weak references to the object (if defined)

class TotalDepth.LIS.core.FrameSet.**SuChArTe**(*samples, bursts*)

Sub-channel Array Template.

property numValues

The total number of values in this sub-channel.

index(*theS, theB*)

Returns the index of a particular sample or channel with bounds checking.

WARNING: This not correct for Dipmeter sub-channels.

__str__()

String representation.

class TotalDepth.LIS.core.FrameSet.**ChArTe**(*theDsb*)

Channel Array Template. Constructed with a DatumSpecBlock object.

Implementation note: `_index()` and `_dipmeterIndex()` have no bounds checking but are significantly faster for those routines that access then with pre-checked limits.

__init__(*theDsb*)

Constructor with a DatumSpecBlock object.

__str__()

String representation.

property numSubChannels

Number of sub-channels.

property lisSize

Number of bytes per frame in the LIS representation.

subChOffsRange(*sc*)

Returns a range object that is the sub-channel offset in the frame relative to the start of the channel.

subChOffsSlice(*sc, chOfs=0*)

Returns a slice object that is the sub-channel offset in the frame relative to the start of the channel.

index(*theSc, theSa, theBu*)

Returns the index of a particular sub-channel, sample and burst. Order is (fastest changing first): burst, sample. For those with sub-channels: sub-channel, sample This has bounds checking.

dipmeterIndex(*theSc, theSa, theBu*)

Returns dipmeter data index. Bounds checking is performed.

class TotalDepth.LIS.core.FrameSet.**XAxisDecl**(*upDown, frameSpacing, frameSpacingUnits, recordingMode, depthUnits, depthRepCode*)

property isLogUp

True if “up” log (x decreasing).

property isLogDown

True if “down” log (x decreasing).

property isIndirectX

True if has indirect X axis.

class TotalDepth.LIS.core.FrameSet.**FrameSet**(*theDfsr, theFrameSlice, theChS=None, xAxisIndex=0*)

Contains the representation of a list of Frames and thus a representation of and 'matrix' (non literal) of: (frame, channel, sub-channel, sample, burst). Commonly shortened to: (fr, ch, sc, sa, bu) Effectively this is a wrapper around a numpy 2-D array that we treat specially to get our 5-D array (LIS channels are not homogeneous).

Constructed with a DFSR, a slice of frame indexes and an optional list of external channel indexes (defaults to all channels). Duplicates in the theChS will be removed and it will be sorted.

xAxisIndex is the external channel index of the X axis (ignored if indirect X).

NUMPY_DATA_TYPE = 'float64'

Data type used in the underlying numpy array.

__init__(*theDfsr, theFrameSlice, theChS=None, xAxisIndex=0*)

Constructed with a DFSR, a slice of frame indexes and an optional list of external channel indexes (defaults to all channels). Duplicates in the theChS will be removed and it will be sorted. xAxisIndex is the external channel index of the X axis (ignored if indirect X).

__str__()

String representation.

longStr()

Returns a long string that describes me.

dumpFrames(*theS=<colorama.ansitowin32.StreamWrapper object>*)

Dump the frames to the stream.

property nbytes

Returns the number of bytes in the underlying array implementation.

property lisSize

The number of LIS bytes that make up this FrameSet.

property numFrames

The number of frames currently in this FrameSet.

property valuesPerFrame

The number of values in each frame currently in this FrameSet.

property numValues

The total number of values in the array.

property frames

Gives access to the raw numpy array.

frame(*fr*)

Returns a specific frame.

xAxisValue(*fr*)

Returns the X axis value for the frame when indirect X is used or None.

xAxisStep(*numFr*)

The distance stepped by numFr.

intFrameNum(*theExtFrameNum*)

Given an external frame number this returns the internal frame number. Will raise an IndexError if the internal frame number is out of range or the external frame number is not in the caller specified slice object.

extFrameNum(*theIntFrameNum*)

Given an internal frame number this returns the external frame number. This does `_not_` test that the internal frame number exists.

property numChannels

The number of `_internal_` channels.

numSubChannels(*theChExt*)

Number of sub-channels for an external channel.

numSamples(*theChExt, theSc*)

Number of samples for the external channel, sub-channel.

numBursts(*theChExt, theSc*)

Number of bursts for the external channel, sub-channel.

property isIndirectX

True if there is an indirect X axis, False otherwise.

property xAxisDecl

The XAxisDecl object created from the DFSR.

setFrameBytes(*by, fr, chFrom, chTo*)

Given bytes, convert from Rep Codes to 'float64' and populates the appropriate frame and channel(s). This has random write access so the caller needs to be sensitive to the frame and channel location. *fr* is the internal frame position to write the data to. *chFrom, chTo* are inclusive and represent external channel indices.

WARNING: It is rare to use this API directly, instead a FrameSet object is usually a member of a LogPass object and that uses this API as a LogPass is capable of finding and reading bytes objects in a file (a LogPass uses RLE and Type01Plan objects to do this efficiently).

setIndirectX(*fr, val*)

Sets an indirect X axis value directly, for example with an EXTRAPOLATE event.

internalChIdx(*chIdxExt*)

Return the internal channel index from the external one.

valueIdxStartExtCh(*ch*)

The index in the frame of the first value for external channel.

valueIdxInFrame(*ch, sc, sa, bu*)

The index in the frame of the value for (external channel, sub-channel, sample, burst). TODO: Provide an API that returns a numpy view of a ch/sc on the frameset.

value(*fr, ch, sc, sa, bu*)

Returns a single value from an: internal *fr*, external *ch, sc, sa, bu*. This is very good at random access but can be quite slow for iteration compared to the generators.

frameView(*chIdxExt, sc*)

Returns a numpy array that is a view of the current frame set for an external channel and sub channel.

frame_channel_sub_channel_values(*frame_index, channel_index, sub_channel_index*)

Returns a numpy array that is a view of the values for the frame, external channel and sub channel.

genExtChIndexes()

Generates an ordered list of external channel indexes for this (possibly) partial frameset.

genChScValues(*ch, sc=0, chIsExternal=True*)

Generates values for the external channel and sub channel. *sc* is ignored unless the channel has > 1 sub-channels. If *chIsExternal* is True then *ch* is the external channel index otherwise it is the internal index. The value order is sample/burst.

genChScPoints(*ch, sc=0, chIsExt=True*)

Generates (*xAxis, values*) as numbers for the external channel and sub-channel. *sc* is ignored unless the channel has > 1 sub-channels. If *chIsExt* is True then *ch* is the external channel index otherwise it is the

internal index.

genMultipleChScPoints(*theChScS, chIsExt=True*)

Generates (xAxis, (values, ...)) as numbers for the list of (channel, sub-channel) indexes. An Exception-FrameSetMixedChannels will be raised if all ch/sc channels are not of the same form i.e. number of samples and bursts. If chIsExt is True then ch is the external channel index otherwise it is the internal index.

genAll()

Yields 6 item tuples (fr ext, ch ext, sc, sa, bu value).

accumulate(*theAccs*)

Calls .add() on every accumulator (with a unary function) for every internal channel and returns an numpy array of (numSubCh, len(theAccs)) doubles. Each accumulator is expected to have __init__(), add() and value() that returns a double implemented. Will raise a ExceptionFrameSetEmpty if there are no values to analyse. Will return None if theAccs is zero length.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.LIS.core.FrameSet.AccMin

Accumulates the minimum value.

__init__()

add(*v*)

Add a new value.

value()

Return the result.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.LIS.core.FrameSet.AccMax

Accumulates the maximum value.

__init__()

add(*v*)

Add a new value.

value()

Return the result.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.LIS.core.FrameSet.AccMean

Accumulates the mean value.

__init__()

add(*v*)

Add a new value.

value()

Return the result.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.LIS.core.FrameSet.AccStDev

Accumulates the standard deviation.

```

    __init__()
    add(v)
        Add a new value.
    value()
        Return the result.
    __weakref__
        list of weak references to the object (if defined)
class TotalDepth.LIS.core.FrameSet.AccCount
    Accumulates the number of values.
    __init__()
    add(v)
        Add a new value.
    value()
        Return the result.
    __weakref__
        list of weak references to the object (if defined)
class TotalDepth.LIS.core.FrameSet.AccDelta
    Base class for accumulating a count of first order differences.
    __init__()
    add(v)
        Add a new value.
    value()
        Return the result.
    __weakref__
        list of weak references to the object (if defined)
class TotalDepth.LIS.core.FrameSet.AccInc
    Counting how many values are an increase from the previous value.
    add(v)
        Add a new value.
class TotalDepth.LIS.core.FrameSet.AccEq
    Counting how many values are equal to the previous value.
    add(v)
        Add a new value.
class TotalDepth.LIS.core.FrameSet.AccDec
    Counting how many values are less than the previous value.
    add(v)
        Add a new value.
class TotalDepth.LIS.core.FrameSet.AccBias
    Measures increment, equal, decrement and computes bias which is: (inc - dec) / total.
    __init__()
    add(v)
        Add a new value.

```

value()

Return the result.

class TotalDepth.LIS.core.FrameSet.AccDrift

Measures drift i.e. the movement between the first and the last value.

__init__()

add(v)

Add a new value.

value()

Return the result.

class TotalDepth.LIS.core.FrameSet.AccActivity

Measures curve activity.

__init__()

add(v)

Add a new value.

value()

Return the result.

5.6 FrameSet Usage

TODO:

5.7 TotalDepth.LIS.core.LogiRec (Logical Records)

Handles LIS Logical Records.

exception TotalDepth.LIS.core.LogiRec.ExceptionLr

Specialisation of exception for Logical Records.

exception TotalDepth.LIS.core.LogiRec.ExceptionLrNotImplemented

Logical Records that have no implementation here.

exception TotalDepth.LIS.core.LogiRec.ExceptionCbWrite

Raised when creating a component block with Python native types that has illogical or conflicting data.

TotalDepth.LIS.core.LogiRec.LR_TYPE_NORMAL_DATA = 0

0x00 Normal data record containing log data

TotalDepth.LIS.core.LogiRec.LR_TYPE_ALTERNATE_DATA = 1

0x01 Alternate data.

TotalDepth.LIS.core.LogiRec.LR_TYPE_JOB_ID = 32

0x20 Job identification

TotalDepth.LIS.core.LogiRec.LR_TYPE_WELL_DATA = 34

0x22 Well site data

TotalDepth.LIS.core.LogiRec.LR_TYPE_TOOL_INFO = 39

0x27 Tool string info

TotalDepth.LIS.core.LogiRec.LR_TYPE_ENCRYPTED_TABLE = 42

0x2a Encrypted table dump

TotalDepth.LIS.core.LogiRec.LR_TYPE_TABLE_DUMP = 47
0x2f Table dump

TotalDepth.LIS.core.LogiRec.LR_TYPE_DATA_FORMAT = 64
0x40 Data format specification record

TotalDepth.LIS.core.LogiRec.LR_TYPE_DATA_DESCRIPTOR = 65
0x41 Data descriptor (not defined in the LIS79 Description Reference Manual)

TotalDepth.LIS.core.LogiRec.LR_TYPE_TU10_BOOT = 95
0x5f TU10 software boot

TotalDepth.LIS.core.LogiRec.LR_TYPE_BOOTSTRAP_LOADER = 96
0x60 Bootstrap loader

TotalDepth.LIS.core.LogiRec.LR_TYPE_CP_KERNEL = 97
0x61 CP-kernel loader boot

TotalDepth.LIS.core.LogiRec.LR_TYPE_PROGRAM_FILE_HEAD = 100
0x64 Program file header

TotalDepth.LIS.core.LogiRec.LR_TYPE_PROGRAM_OVER_HEAD = 101
0x65 Program overlay header

TotalDepth.LIS.core.LogiRec.LR_TYPE_PROGRAM_OVER_LOAD = 102
0x66 Program overlay load

TotalDepth.LIS.core.LogiRec.LR_TYPE_FILE_HEAD = 128
0x80 File header

TotalDepth.LIS.core.LogiRec.LR_TYPE_FILE_TAIL = 129
0x81 File trailer

TotalDepth.LIS.core.LogiRec.LR_TYPE_TAPE_HEAD = 130
0x82 Tape header

TotalDepth.LIS.core.LogiRec.LR_TYPE_TAPE_TAIL = 131
0x83 Tape trailer

TotalDepth.LIS.core.LogiRec.LR_TYPE_REEL_HEAD = 132
0x84 Reel header

TotalDepth.LIS.core.LogiRec.LR_TYPE_REEL_TAIL = 133
0x85 Reel trailer

TotalDepth.LIS.core.LogiRec.LR_TYPE_EOF = 137
0x89 Logical EOF (end of file)

TotalDepth.LIS.core.LogiRec.LR_TYPE_BOT = 138
0x8a Logical BOT (beginning of tape)

TotalDepth.LIS.core.LogiRec.LR_TYPE_EOT = 139
0x8b Logical EOT (end of tape)

TotalDepth.LIS.core.LogiRec.LR_TYPE_EOM = 141
0x8d Logical EOM (end of medium)

TotalDepth.LIS.core.LogiRec.LR_TYPE_OPERATOR_INPUT = 224
0xe0 Operator command inputs

TotalDepth.LIS.core.LogiRec.LR_TYPE_OPERATOR_RESPONSE = 225
0xe1 Operator response inputs

`TotalDepth.LIS.core.LogiRec.LR_TYPE_SYSTEM_OUTPUT = 227`
0xe3 System outputs to operator

`TotalDepth.LIS.core.LogiRec.LR_TYPE_FLIC_COMMENT = 232`
0xe8 FLIC comment

`TotalDepth.LIS.core.LogiRec.LR_TYPE_BLANK_RECORD = 234`
0xea Blank record/CSU comment

`TotalDepth.LIS.core.LogiRec.LR_TYPE_PICTURE = 85`
0x55 Picture

`TotalDepth.LIS.core.LogiRec.LR_TYPE_IMAGE = 86`
0x56 Image

`TotalDepth.LIS.core.LogiRec.LR_TYPE_ALL = (0, 1, 32, 34, 39, 42, 47, 64, 65, 95, 96, 97, 100, 101, 102, 128, 129, 130, 131, 132, 133, 137, 138, 139, 141, 224, 225, 227, 232, 234, 85, 86)`
All possible Logical Records Types

`TotalDepth.LIS.core.LogiRec.LR_TYPE_LOG_DATA = (0, 1)`
Logical Records Types for Log data

`TotalDepth.LIS.core.LogiRec.LR_TYPE_TABLE_DATA = (32, 34, 39)`
Logical Records Types for Table data

`TotalDepth.LIS.core.LogiRec.LR_TYPE_DELIMITER_START = (132, 130, 128)`
Logical Records Types for delimiter start records.

`TotalDepth.LIS.core.LogiRec.LR_TYPE_DELIMITER_END = (133, 131, 129)`
Logical Records Types for delimiter end records.

`TotalDepth.LIS.core.LogiRec.LR_TYPE_DELIMITER = (132, 130, 128, 133, 131, 129)`
Logical Records Types for all delimiter records. Delimiter records 'bookend' dynamic and static data that makes up a LIS file.

`TotalDepth.LIS.core.LogiRec.LR_TYPE_MARKER = (137, 138, 139, 141)`
Logical Records Types for all marker records. Marker records terminate original physical media such as a tape.

`TotalDepth.LIS.core.LogiRec.LR_TYPE_DELIMITER_MARKER = (132, 130, 128, 133, 131, 129, 137, 138, 139, 141)`
Logical Records Types for all delimiter and marker records.

`TotalDepth.LIS.core.LogiRec.LR_FIXED_FORMAT = {95: 594, 101: 52, 128: 58, 129: 58, 130: 128, 131: 128, 132: 128, 133: 128, 137: 0, 138: 0, 139: 0, 141: 0}`
Table of fixed format Logical Record types and their fixed length in bytes.

`TotalDepth.LIS.core.LogiRec.isDelimiter(theType)`
Returns True if the Logical Record Type is a Delimiter record.

`TotalDepth.LIS.core.LogiRec.LR_TYPE_UNKNOWN_INTERNAL_FORMAT = (42, 47, 95, 96, 97, 100, 101, 102, 224, 225, 227, 232, 234, 85, 86)`
Logical Records Types with no known format so just treat these as unformatted binary data

TotalDepth.LIS.core.LogiRec.LR_DESCRIPTION_MAP = {0: 'Normal data record containing log data', 1: 'Alternate data.', 32: 'Job identification', 34: 'Well site data', 39: 'Tool string info', 42: 'Encrypted table dump', 47: 'Table dump', 64: 'Data format specification record', 65: 'Data descriptor (not defined in the LIS79 Description Reference Manual)', 85: 'Picture', 86: 'Image', 95: 'TU10 software boot', 96: 'Bootstrap loader', 97: 'CP-kernel loader boot', 100: 'Program file header', 101: 'Program overlay header', 102: 'Program overlay load', 128: 'File header', 129: 'File trailer', 130: 'Tape header', 131: 'Tape trailer', 132: 'Reel header', 133: 'Reel trailer', 137: 'Logical EOF (end of file)', 138: 'Logical BOT (beginning of tape)', 139: 'Logical EOT (end of tape)', 141: 'Logical EOM (end of medium)', 224: 'Operator command inputs', 225: 'Operator response inputs', 227: 'System outputs to operator', 232: 'FLIC comment', 234: 'Blank record/CSU comment'}

Map of {Logical Records Type : description, ...}

TotalDepth.LIS.core.LogiRec.LR_DESCRIPTION_UNKNOWN = 'Unknown Logical Record type.'

Description string for unknown Logical Records Type

TotalDepth.LIS.core.LogiRec.STRUCT_LR_HEAD = <_struct.Struct object>

Logical Record header (type and attributes)

TotalDepth.LIS.core.LogiRec.STRUCT_LR_FILE_HEAD_TAIL = <_struct.Struct object>

Logical Record field interpretation via the struct module

TotalDepth.LIS.core.LogiRec.STRUCT_LR_REEL_TAPE_HEAD_TAIL = <_struct.Struct object>

Logical Record reel/tape head/tail via the struct module

TotalDepth.LIS.core.LogiRec.STRUCT_COMPONENT_BLOCK_PREAMBLE = <_struct.Struct object>

Component Block preamble as a struct.Struct()

TotalDepth.LIS.core.LogiRec.STRUCT_ENTRY_BLOCK_PREAMBLE = <_struct.Struct object>

Entry Block preamble as a struct.Struct()

TotalDepth.LIS.core.LogiRec.STRUCT_DSB = <_struct.Struct object>

Datum Specification Block structure. NOTE: Due to the funny way API codes are done we read then as a single 32 bit unsigned integer and then do a decimal masking operation to extract the four sub-fields

class TotalDepth.LIS.core.LogiRec.LrBase(*theType, theAttr*)

Base class for Logical Records. Constructed with and integer type and integer attributes.

__init__(*theType, theAttr*)

Base class constructor, theType and theAttr are bytes.

init(*theLen*)

Returns a string of spaces of the supplied length.

__str__()

String representation.

property desc

Description of the LR type.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.LIS.core.LogiRec.LrMarker(*theType, theAttr*)

A marker record such as EOF BOT EOT EOM.

__init__(*theType, theAttr*)

Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.LrEOF(*attr=0*)

A EOF marker record.

__init__(*attr=0*)

Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.**LrEOFRead**(*theFile*)

A EOF marker record read from a file.

__init__(*theFile*)

Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.**LrBOT**(*attr=0*)

A BOT marker record.

__init__(*attr=0*)

Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.**LrBOTRead**(*theFile*)

A BOT marker record read from a file.

__init__(*theFile*)

Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.**LrEOT**(*attr=0*)

A EOT marker record.

__init__(*attr=0*)

Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.**LrEOTRead**(*theFile*)

A EOT marker record read from a file.

__init__(*theFile*)

Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.**LrEOM**(*attr=0*)

A EOM marker record.

__init__(*attr=0*)

Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.**LrEOMRead**(*theFile*)

A EOM marker record read from a file.

__init__(*theFile*)

Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.**LrWithDateField**(*theType, theAttr*)

ABC for classes that have the YY/MM/DD date field.

__init__(*theType, theAttr*)

Base class constructor, theType and theAttr are bytes.

property ymd

Returns the YY/MM/DD date field to a year, month, day tuple or None.

class TotalDepth.LIS.core.LogiRec.**LrFileHeadTail**(*theType, theAttr*)

Parent class of FileHead, FileTail that have identical structure.

__init__(*theType, theAttr*)

Base class constructor, theType and theAttr are bytes.

read(*theFile*)

Read from a LIS physical file.


```

property contFileName
    Continuation file name.

class TotalDepth.LIS.core.LogiRec.LrFileHead(theType, theAttr)
    Specific class of File Head.

    __init__(theType, theAttr)
        Base class constructor, theType and theAttr are bytes.

property prevFileName
    Previous file name.

class TotalDepth.LIS.core.LogiRec.LrFileHeadRead(theFile)
    Specific class of File head read from a file.

    __init__(theFile)
        Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.LrFileTail(theType, theAttr)
    Specific class of File Tail.

    __init__(theType, theAttr)
        Base class constructor, theType and theAttr are bytes.

property nextFileName
    Next file name.

class TotalDepth.LIS.core.LogiRec.LrFileTailRead(theFile)
    Specific class of File tail read from a file.

    __init__(theFile)
        Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.LrReelTapeHeadTail(theType, theAttr)
    Parent class of Reel/Tape Head/Tail that have identical structure.

    __init__(theType, theAttr)
        Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.LrTapeHeadTail(theType, theAttr)
    Tape head or tail Logical Record.

    __init__(theType, theAttr)
        Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.LrTapeHead(theType, theAttr)
    Tape head Logical Record.

    __init__(theType, theAttr)
        Base class constructor, theType and theAttr are bytes.

property prevTapeName
    Previous tape name.

class TotalDepth.LIS.core.LogiRec.LrTapeHeadRead(theFile)
    Specific class of Tape head read from a file.

    __init__(theFile)
        Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.LrTapeTail(theType, theAttr)
    Tape tail Logical Record.

```

__init__(*theType, theAttr*)
Base class constructor, theType and theAttr are bytes.

property nextTapeName
Next tape name.

class TotalDepth.LIS.core.LogiRec.**LrTapeTailRead**(*theFile*)
Specific class of Tape tail read from a file.

__init__(*theFile*)
Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.**LrReelHeadTail**(*theType, theAttr*)
Reel head or tail Logical Record.

__init__(*theType, theAttr*)
Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.**LrReelHead**(*theType, theAttr*)
Reel head Logical Record.

__init__(*theType, theAttr*)
Base class constructor, theType and theAttr are bytes.

property prevReelName
Previous reel name.

class TotalDepth.LIS.core.LogiRec.**LrReelHeadRead**(*theFile*)
Specific class of Reel head read from a file.

__init__(*theFile*)
Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.**LrReelTail**(*theType, theAttr*)
Reel tail Logical Record.

__init__(*theType, theAttr*)
Base class constructor, theType and theAttr are bytes.

property nextReelName
Next reel name.

class TotalDepth.LIS.core.LogiRec.**LrReelTailRead**(*theFile*)
Specific class of Reel tail read from a file.

__init__(*theFile*)
Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.**LrMisc**(*theType, theAttr*)
Miscellaneous Logical Record.

__init__(*theType, theAttr*)
Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.**LrMiscRead**(*theFile*: [TotalDepth.LIS.core.File.FileRead](#))
Miscellaneous Logical Record read from a LIS file.

__init__(*theFile*: [TotalDepth.LIS.core.File.FileRead](#))
Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.**LrUnknown**(*theType, theAttr*)
Logical Record that does not fall into any other category.

```

    __init__(theType, theAttr)
        Base class constructor, theType and theAttr are bytes.
class TotalDepth.LIS.core.LogiRec.LrUnkownRead(theFile: TotalDepth.LIS.core.File.FileRead)
    Logical Record that does not fall into any other category.
    __init__(theFile: TotalDepth.LIS.core.File.FileRead)
        Base class constructor, theType and theAttr are bytes.
TotalDepth.LIS.core.LogiRec.COMPONENT_BLOCK_TABLE = 73
    Type of first Component Blocks in the table
TotalDepth.LIS.core.LogiRec.COMPONENT_BLOCK_DATUM_BLOCK_START = 0
    Type of first Component Blocks in the Datum Block i.e. row
TotalDepth.LIS.core.LogiRec.COMPONENT_BLOCK_DATUM_BLOCK_ENTRY = 69
    Component Block type that describes an entry in a Datum Block i.e. a cell
exception TotalDepth.LIS.core.LogiRec.ExceptionLrTable
    Specialisation of exception for Table Logical Records.
exception TotalDepth.LIS.core.LogiRec.ExceptionLrTableInit
    Table __init__() issues.
exception TotalDepth.LIS.core.LogiRec.ExceptionLrTableInternaStructuresCorrupt
    Raised when there are inconsistencies with the IR of the table.
exception TotalDepth.LIS.core.LogiRec.ExceptionLrTableCompose
    Table creation (not from file) issues.
exception TotalDepth.LIS.core.LogiRec.ExceptionLrTableRow
    TableRow issues.
exception TotalDepth.LIS.core.LogiRec.ExceptionLrTableRowInit
    TableRow __init__() issues.
exception TotalDepth.LIS.core.LogiRec.ExceptionCbEngValInit
    CbEngVal.__init__() issues such as unknown rep code.
class TotalDepth.LIS.core.LogiRec.CbEngVal
    Contains the data from a Component Block and has an EngVal
    CB_TYPES = (73, 0, 69)
        Allowable Component Block types
    __init__()
    __str__()
        Return str(self).
    setValue(v)
        Sets the value to v.
    lisBytes()
        Return a bytes() array from the internal representation.
    property value
        The value of the Component Block or None.
    property status
        Returns True if the value is b'ALLO', False otherwise.
    __weakref__
        list of weak references to the object (if defined)

```

class TotalDepth.LIS.core.LogiRec.CbEngValRead(*the_file*: TotalDepth.LIS.core.File.FileRead)

Contains the data from a Component Block and has an EngVal read from a file.

__init__(*the_file*: TotalDepth.LIS.core.File.FileRead)

Initialise. This will raise a TypeError if theFile.unpack returns None i.e. when not enough data to create a Component Block.

class TotalDepth.LIS.core.LogiRec.CbEngValWrite(*t, v, m, **kwargs*)

A Component Block and has an EngVal created directly.

__init__(*t, v, m, **kwargs*)

Initialise component block with type, value, mnemonic and optional key words.

class TotalDepth.LIS.core.LogiRec.TableRow(*theCb*)

Represents a row of a table and consists of CbEngVal objects.

__init__(*theCb*)

genCells()

yields each CbEngVal.

__len__()

The number of cells in the row.

property value

The name of the row i.e. the value of block 0.

addCb(*theCb*)

Adds a component block onto the end of the row. Returns the mnemonic field from the component block.

__getitem__(*key*)

If key is an integer or slice this returns a CbEngVal by index(es). If key is a bytes() object then this returns a CbEngVal by label. May raise a KeyError or IndexError.

__contains__(*key*)

Returns True if this row has a column named key.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.LIS.core.LogiRec.LrTable(*theType, theAttr*)

Table-like Logical Record.

__init__(*theType, theAttr*)

Base class constructor. theType, theAttr are byte objects i.e. integers 0 to 255

genRows()

yields each TableRow. TODO: parameter to sort or reverse.

genRowNames(*sort=0*)

yields each TableRow name. If sort > 0 then the results are sorted. If sort < 0 the results are reverse sorted

retRowByMnem(*m*: TotalDepth.LIS.core.Mnem.Mnem) → TotalDepth.LIS.core.LogiRec.TableRow

Returns the TableRow from a Mnem.Mnem object. i.e. the table row that has a b'MNEM' column whose value matches m. May raise a KeyError. Note: an IndexError would mean that self.__mnemRowIndex is corrupt.

property isSingleParam

True if this table is a list of single parameters (i.e. type 0 blocks).

property value

The name of the table or None i.e. the value of the first block.

__getitem__(*key*)
If key is an integer or slice this returns block by index(es). If key is a bytes() object then this returns row by label. May raise a KeyError or IndexError.

__contains__(*key*)
Returns True if this row has a row named key.

__len__()
Number of rows in the table.

rowLabels()
Returns dictionary view of row values (unordered).

rowMnems()
Returns dictionary view of row Mnem.Mnem objects (unordered).

colLabels()
Returns ordered super-set of column values. Not all rows have these.

startNewRow(*theCbEv*)
Starts a new row from the component block. Returns theCbEv.mnem value.

addDatumBlock(*theCbEv*)
Adds a component block to the last row. Returns theCbEv value.

genRowValuesInColOrder(*theRow*)
Yields table cells in a particular row in column order.

genLisBytes()
Yields chunks of binary LIS data (actually each component block).

class TotalDepth.LIS.core.LogiRec.**LrTableRead**(*theFile*)
A table-like Logical Record read from a LIS file.

__init__(*theFile*)
Base class constructor. theType, theAttr are byte objects i.e. integers 0 to 255

class TotalDepth.LIS.core.LogiRec.**LrTableWrite**(*theType, theName, theMnemS, theTable*)
Creates a table from internal Python data structures.

theType is the table type e.g. b'FILM'

theMnemS is the list of column names, theTable members must fit this size.

If an element of the table is a tuple or a list it is assumed to be (value, units).

__init__(*theType, theName, theMnemS, theTable*)
Construct a table from internal data that must be bytes/float/int. theType is the table type e.g. b'FILM'
theMnemS is the list of column names, theTable members must fit this size. If an element of the table is a tuple or a list it is assumed to be (value, units).

exception TotalDepth.LIS.core.LogiRec.**ExceptionEntryBlock**
Specialisation of exception for Entry Blocks.

exception TotalDepth.LIS.core.LogiRec.**ExceptionEntryBlockSetInit**
Exception for EntryBlockSet.__init__().

class TotalDepth.LIS.core.LogiRec.**EntryBlock**(*type, size, repCode, value*)

lisBytes()
Returns the LIS bytes for the Entry Block.

class TotalDepth.LIS.core.LogiRec.EntryBlockRead(*theFile*)

An entry block read from a LIS file.

static __new__(*self, theFile*)

class TotalDepth.LIS.core.LogiRec.EntryBlockSet

Represents the set of Entry Blocks in a DFSR.

ATTR_MAP = {'absentValue': 12, 'dataType': 1, 'depthRepCode': 15, 'depthUnits': 14, 'dsbType': 2, 'frameSpacing': 8, 'frameSpacingUnits': 9, 'optLogScale': 5, 'recordingMode': 13, 'upDown': 4}

Map of supported attributes i.e. those that are 'interesting'

EB_DOC = {0: 'Terminator, size is chosen to make total size even.', 1: 'Data Record Type. The type of the IFLR (0 | 1) that this describes.', 2: 'Datum Spec Block Type. How to interpret the DSBs. Only 0 is defined.', 3: 'Data Frame Size. Not required.', 4: 'Up/Down. 1=up, 255=down, 0=neither.', 5: 'Optical Log Scale. 1=Feet, 255=Meters, 0=Time.', 6: 'Data Reference Point. The distance of the data reference point above the tool reference point. Essentially add this to depth to find the depth axis of un-memorised data such as tension.', 7: 'Units for Data Reference Point.', 8: 'Frame Spacing.', 9: 'Units for Frame Spacing.', 10: 'Undefined.', 11: 'Maximum Frames per Record.', 12: 'Absent value.', 13: 'Depth Recording Mode. 1=Indirect X, 0=Direct X', 14: 'Units of Depth when depth Recording Mode=1', 15: 'Representation Code for depth when Depth Recording Mode=1', 16: 'Datum Spec Block sub-Type. How to interpret the DSBs (0 | 1)'}
Documentation about each Entry Block

BLOCKS_TO_SKIP = (10,)

List of block numbers that are not written out, also _setLisSizeEven() and lisSize() ignore these.

__init__()

__str__()

Return str(self).

__getattr__(*name*)

Returns the Entry Block corresponding to the name.

__getitem__(*key*)

This returns an Entry block by integer index.

property logUp

True if the logging direction is up (X decreasing). Note: not logUp and not logDown is possible to be True e.g. time log.

property logDown

True if the logging direction is down (X increasing). Note: not logUp and not logDown is possible to be True e.g. time log.

property xInc

True if the logging X increases (down or time log).

property opticalLogScale

Returns the Units corresponding to Entry Block 5: 'Optical Log Scale' This will be a LENG or TIME unit or empty if undefined.

lisSize()

Returns the total size of the Entry Block set.

setEntryBlock(*theEb*)

Sets an Entry Block.

readFromFile(*theFile*)
 Reads from a File object. NOTE: *theFile*.hasLd() must be True so the Logical Record Header must have been read already.

lisBytes()
 Returns the Entry Block set as an array of bytes.

lisByteList()
 Returns a list of bytes() objects, one for each entry block.

__weakref__
 list of weak references to the object (if defined)

exception TotalDepth.LIS.core.LogiRec.**ExceptionDatumSpecBlock**
 Specialisation of exception for Datum Specification Blocks.

class TotalDepth.LIS.core.LogiRec.**DatumSpecBlock**
 This represents as Datum Specification Block.

__init__()

property isNull
 True if this block is compromised in any way and should be ignored when composing a DFSR. The critical test is whether the data from this channel will be in the frame.

samples(*theSc*)
 Returns the number of samples in a sub-channel.

bursts(*theSc*)
 Returns the (samples, burst) for a sub-channel (bursts are invariant over sub-channels).

values()
 Returns the total number of discrete values per frame for a single channel.

subChMnem(*theSc*)
 Returns the curve Mnemonic for a particular sub-channel or None if unknown.

__weakref__
 list of weak references to the object (if defined)

class TotalDepth.LIS.core.LogiRec.**DatumSpecBlockRead(*theF*)**
 This represents as Datum Specification Block read from a file.

__init__(*theF*)

class TotalDepth.LIS.core.LogiRec.**LrDFSR(*theType, theAttr*)**
 Data Format Specification Record.

__init__(*theType, theAttr*)
 Base class constructor, *theType* and *theAttr* are bytes.

class TotalDepth.LIS.core.LogiRec.**LrDFSRRRead(*theFile*)**
 Data Format Specification Record read from a file.

__init__(*theFile*)
 Base class constructor, *theType* and *theAttr* are bytes.

class TotalDepth.LIS.core.LogiRec.**LrNormalAlternateData(*theType, theAttr*)**
 Class for Normal and Alternate data i.e. curve data.

__init__(*theType, theAttr*)
 Base class constructor, *theType* and *theAttr* are bytes.

class TotalDepth.LIS.core.LogiRec.LrNormalAlternateDataRead(*theFile*)

Class for Normal and Alternate data i.e. curve data.

__init__(*theFile*)

Base class constructor, theType and theAttr are bytes.

class TotalDepth.LIS.core.LogiRec.LrFactory

Provides a despatch mechanism for generating Logical Records. This can be sub-classed to create different sets of Logical Records. For example the Indexer creates minimal logical records.

__weakref__

list of weak references to the object (if defined)

__init__()

class TotalDepth.LIS.core.LogiRec.LrFactoryRead

A factory for generating complete Logical Records from a file.

__init__()

retLrFromFile(*theFile*)

Given a LIS file this reads one Logical Record, and returns the appropriate Logical Record object or None.

5.8 TotalDepth.LIS.core.LogPass

This describes the LIS LogPass class that encapsulates a Log Pass. A log pass is the binary log data plus the record (DFSR) that describes the format of the binary data. Internally the binary data is converted to a FrameSet that wraps a numpy array.

The LogPass module is located in src/TotalDepth/LIS/core and can be imported thus:

```
from TotalDepth.LIS.core import LogPass
```

5.8.1 LogPass API Reference

The LogPass module contains a single class LogPass that is fundamental to the way that TotalDepth handles LIS binary data.

A Log Pass is defined as a single continuous recording of log data. “Main Log”, “Repeat Section” are separate examples of Log Pass(es).

exception TotalDepth.LIS.core.LogPass.ExceptionLogPass

Specialisation of exception for LogPass.

exception TotalDepth.LIS.core.LogPass.ExceptionLogPassCtor

Specialisation of exception for LogPass __init__().

exception TotalDepth.LIS.core.LogPass.ExceptionLogPassNoType01Data

Raised on access when there is no frame data loaded by addType01Data().

exception TotalDepth.LIS.core.LogPass.ExceptionLogPassNoFrameSet

Raised when FrameSet access is required but there has been no call to setFrameSet().

exception TotalDepth.LIS.core.LogPass.ExceptionLogPassKeyError

Raised when an internal KeyError is raised in a FrameSet access.

TotalDepth.LIS.core.LogPass.EVENT_SEEK_LR = 'seekLr'

Event to seek to the start of a new Logical Record

TotalDepth.LIS.core.LogPass.EVENT_READ = 'read'

Event to read bytes from frame

TotalDepth.LIS.core.LogPass.EVENT_SKIP = 'skip'

Event to skip bytes in frame

TotalDepth.LIS.core.LogPass.EVENT_EXTRAPOLATE = 'extrapolate'

Event to extrapolate X axis

class TotalDepth.LIS.core.LogPass.LogPass(*theDfsr, theFileId, xAxisIndex=0*)

Contains the information about a log pass which is defined as a LIS Data Format Specification Record (DFSR) plus any number of Type 0/1 Logical Records.

A LogPass must be created with a LIS DFSR and a LIS file ID. It can be then informed with addType01Data() which records the file positions of Type 0/1 records. When required the actual frame data can be populated with setFrameSet().

theDfsr - The DFSR.

theFileId - The ID of the file, this will be checked against any File object passed to me.

xAxisIndex - The index of the DSB block that describes the X axis, if indirect X this is ignored.

__init__(*theDfsr, theFileId, xAxisIndex=0*)

Constructed with an EFLR i.e. a DFSR

theDfsr - The DFSR.

theFileId - The ID of the file, this will be checked against any File object passed to me.

xAxisIndex - The index of the DSB block that describes the X axis, if indirect X this is ignored.

property dfsr

The DFSR used for construction.

property type01Plan

The Frame plan, a Type01Plan.FrameSetPlan() object.

property rle

The Logical Record Run Length Encoding as a Rle.RLEType01() object.

property frameSet

The Frame Set as a FrameSet.FrameSet() object or None if not initialised.

property iflrType

Returns the IFLR type that this LogPass describes.

property xAxisIndex

The channel index that corresponds to the X axis.

property isIndirectX

True if indirect X axis, False if explicit X axis channel.

property numBytes

The number of bytes in the underlying frame set (i.e. LIS) representation for the curve data. Returns None if the frame set is not initialised.

property nullValue

The NULL or absent value as specified in the DFSR.

longStr()

Returns a long (multiline) descriptive string.

__str__()

Returns a long (multiline) descriptive string.

frameSetLongStr()

Returns a long (multiline) descriptive string of the Frame Set or N/A if not initialised.

curveUnitsAsStr(*chMnem*)

Given a curve as a Mnem.Mnem() this returns the units as a string.

curveUnits(*chMnem*)

Given a curve as a Mnem.Mnem() this returns the units as a bytes object.

property xAxisFirstVal

The numerical value of the X axis of the first frame.

property xAxisLastVal

The numerical value of the X axis of the last frame.

property xAxisFirstEngVal

The EngVal (value, units) of the X axis of the first frame.

property xAxisLastEngVal

The EngVal (value, units) of the X axis of the first frame.

property xAxisSpacing

The numerical value of the X axis frame spacing. This is the extreme range of X axis values divided by the number of frames - 1. This is +ve if the Xaxis increases, -ve if it decreases.

property xAxisUnits

The units of the X axis.

property totalFrames

The total number of frames.

property xAxisFirstValOptical

The numerical value of the X axis of the first frame in 'optical' units.

property xAxisLastValOptical

The numerical value of the X axis of the last frame in 'optical' units.

property xAxisSpacingOptical

The numerical value of the X axis frame spacing. This is the extreme range of X axis values divided by the number of frames - 1.

property xAxisUnitsOptical

Returns the actual units to 'optical' i.e. user friendly units. For example if the Xaxis was in b'.1IN' the 'optical' units would be b'FEET'.

frameFromX(*theEv*)

Returns the estimated frame number from the X axis, and EngValue.

hasOutpMnem(*theMnem*)

Returns True if theMnem is in this LogPass (i.e. is in the DFSR).

outpMnemS()

Returns all of the OUTP Mnems in this LogPass (i.e. is in the DFSR).

retExtChIndexList(*theMnemS*)

Returns a sorted, unique list of external channel indexes for a list of mnemonics. May raise a Exception-LogPassKeyError.

genFrameSetHeadings()

This generates a name and units for each value in a frame in the current frame set. It is useful for heading up a frame dump.

genFrameSetScNameUnit(*toAscii=True*)

This generates a name and units for sub-channel in a frame in the current frame set. It is useful for heading up a accumulate() dump.

genFrameSetChIndexScNameUnit(*toAscii=True*)

This generates an index, name and units for sub-channel in a frame in the current frame set. Like genFrameSetScNameUnit() but with the channel index as well.

addType01Data(*tellLr, lrType, lrLen, xAxisVal*)

Add an Type 0/1 logical record entry.

tellLr - the Logical Record start position in the file.

lrType - the type of the IFLR. Will raise an ExceptionLogPass if this does not match the DFSR.

lrLen - the length of the Logical record, not including the LRH.

xAxisVal - the value of the X Axis of the first frame of the Logical Record.

setFrameSetChX(*theFi, theChS, Xstart, Xstop, frStep=1*)

Loads a FramesSet using 'external' values from a File object. theChS is a list of channel mnemonics or None for all channels. Xstart, Xstop are EngVal of the start stop. frStep is not number of frames to step over, default means all frames.

setFrameSet(*theFile, theFrSl=None, theChList=None*)

Populates the frames set.

theFile - The File object. Will raise an ExceptionLogPass is the file ID does not match that in the constructor.

theFrSl - A slice object that describes when LogPass frames are to be used (default all). Will raise an ExceptionLogPass is there are no frames to load i.e. addType01Data() has not been called.

theChList - A list of external channel indexes (i.e. DSB block indexes) to populate the frame set with (default all).

genOutpPoints(*theMnem*)

Wrapper around the frameset generator, in fact this returns exactly that generator.

jsonObject()

Return an Python object that can be JSON encoded.

__weakref__

list of weak references to the object (if defined)

gen_mnemonic_units() → Sequence[Tuple[bytes, bytes]]

Yields a sequence of (mnemonic, units) for each of the recorded Datum Specification Blocks. This does not include sub-channels.

5.8.2 LogPass Usage

Typically a LogPass will be created directly or via a LIS FileIndexer. The latter technique is recommended as it is simpler.

Direct usage

LogPass objects are used via a three step process and this reflects the sequential process of reading a LIS file:

1. Construction with a DFSR object (once).
2. Updating with the location of the binary data records that contain the frame data (once per IFLR).
3. Populating the FrameSet with real values from the file (many times).

Construction with a DFSR

A LogPass object needs to be constructed with an instance of a LogiRec.LrDFSRRead. The LogPass will take a reference to the DFSR so the caller need not. The caller can always access the DFSR with the `.dfsr` property.

Assuming myF is a LIS File object positioned at the start of the DFSR:

```
myLp = LogPass.LogPass(LogiRec.LrDFSRRead(myF), 'FileID')
```

At this stage no FrameSet is created so the resource usage is minimal.

Add Binary Data Records

The method `addType01Data(tellLr, lrType, lrLen, xAxisVal)` adds the position of an IFLR that contain frame data. This method takes these arguments:

Argument	Description
<code>tellLr</code>	The Logical Record start position (as a <code>size_t</code>) in the file.
<code>lrType</code>	The type of the IFLR [0 1]. Will raise an <code>ExceptionLogPass</code> if this does not match the DFSR.
<code>lrLen</code>	The length of the Logical Record in bytes, not including the LRH.
<code>xAxisVal</code>	The value of the X Axis as a number of the first frame of the Logical Record.

This call should be made for each relevant IFLR as they are encountered in sequence.

At this stage no FrameSet is created and the arguments are encoded into an RLE object so the resource usage is minimal.

5.8.3 Populating the FrameSet

Once the preceding stages have been done the LogPass can be populated any number of times from the LIS file.

The method `setFrameSet(theFile, theFrSl=None, theChList=None)` constructs a new frame set with the appropriate values.

Argument	Description
<code>the-File</code>	The File object. Will raise an <code>ExceptionLogPass</code> if the file ID does not match that used in the constructor.
<code>the-FrSl</code>	A slice object that describes when LogPass frames are to be used (default all). Will raise an <code>ExceptionLogPass</code> if there are no frames to load i.e. <code>addType01Data()</code> has not been called.
<code>theCh-List</code>	A list of external channel indexes (i.e. DSB block indexes) to populate the frame set with (default all).

Examples

Setting a frame set for all frames and all channels:

```
myLogPass.setFrameSet(myFile)
# The above line is equivalent to:
myLogPass.setFrameSet(myFile, theFrSl=None, theChList=None)
```

Setting a frame set for frames [0:16:4] i.e. frame indexes (0,4,8,12) and channels [0, 4, 7]

```
myLogPass.setFrameSet(myFile, theFrSl=slice(0,16,4), theChList=[0, 4, 7])
```

Any previous FrameSet will be freed and a new FrameSet of the appropriate dimension is created so the resource usage can be significant.

5.8.4 Using a LIS Indexer

A *TotalDepth.LIS.core.FileIndexer* [TotalDepth.LIS.core.FileIndexer.FileIndex] object will perform the necessary construction of a LogPass and the population with Logical Record positions with `addType01Data()` leaving the user just to call `setFrameSet()`. Thus a FileIndex object imposes low resource usage until the user wishes to populate the frame set.

An indexer will index a LIS file that has multiple Log Passes (e.g. repeat section, main log etc.) so the indexer provides an iteration method for Log Passes:

```
myFilePath = "Spam/Eggs.LIS"
# Open a LIS file, keepGoing for luck!
myFi = File.FileRead(myFilePath, theFileId=myFilePath, keepGoing=True)
# Index the LIS file
myIdx = FileIndexer.FileIndex(myFi)
# Ask the index for all the LogPass objects, these do not have the frameSet populated yet
for alp in myIdx.genLogPasses():
    # Load the FrameSet, all channels [None], all frames [None]
    alp.logPass.setFrameSet(myFi, None, None)
    # The FrameSet is fully populated here...
    # Do something with it...
```

5.8.5 Testing

The unit tests are in `test/TestLogPass.py`. This should take under a second to execute.

Running the tests under coverage:

```
$ coverage run test/TestLogPass.py
TestClass.py script version "0.8.0", dated 10 Jan 2011
Author: Paul Ross
Copyright (c) Paul Ross

testSetUpTearDown (__main__.TestLogPass_LowLevel)
TestLogPass_LowLevel: Tests setUp() and tearDown(). ... ok
test_00 (__main__.TestLogPass_LowLevel)
TestLogPass_LowLevel.test_00(): Construction. ... ok
test_01 (__main__.TestLogPass_LowLevel)
```

(continues on next page)

(continued from previous page)

```

TestLogPass_LowLevel.test_01(): _sliceFromList(). ... ok
test_02 (__main__.TestLogPass_LowLevel)
TestLogPass_LowLevel.test_02(): exercise various properties. ... ok
test_10 (__main__.TestLogPass_LowLevel)
TestLogPass_LowLevel.test_10(): nullValue(). ... ok

8<----- snip ----->8

test_00 (__main__.TestLogPass_UpIndirect)
TestLogPass_UpIndirect.test_00(): 3 LR, 5 fr, 4 ch. setFrameSet() All. ... ok
test_01 (__main__.TestLogPass_UpIndirect)
TestLogPass_UpIndirect.test_01(): 3 LR, 5 fr, 4 ch. setFrameSet() theFrSl=slice(0,16,2).
↳ ... ok
test_02 (__main__.TestLogPass_UpIndirect)
TestLogPass_UpIndirect.test_02(): 3 LR, 5 fr, 4 ch. setFrameSet() theFrSl=slice(2,4,2). .
↳ ... ok
test_03 (__main__.TestLogPass_UpIndirect)
TestLogPass_UpIndirect.test_03(): 3 LR, 5 fr, 4 ch. setFrameSet() theFrSl=slice(1,5,2). .
↳ ... ok
test_10 (__main__.TestLogPass_UpIndirect)
TestLogPass_UpIndirect.test_10(): genFrameSetHeadings() ... ok

-----
Ran 54 tests in 0.507s

OK
CPU time =    0.510 (S)
Bye, bye!

$ coverage report -m
Name      Stmts   Miss  Cover   Missing
-----
↳ -----

8<----- snip ----->8

LogPass    273     27   90%   142, 167, 182, 187, 203, 216-218, 232, 258, 273-275, 319,
↳ 324, 412, 416, 457-459, 513, 527, 541, 557, 572, 646-647

8<----- snip ----->8

-----
↳ -----
TOTAL      4586   1783   61%

```

5.9 TotalDepth.LIS.core.Mnem (Mnemonics)

Represents a MNEM (Mnemonic) as bytes. Created on May 26, 2011

@author: paulross

`TotalDepth.LIS.core.Mnem.ORDS_WS = (32, 9, 10, 13, 11, 12)`

A tuple of the ordinal values of whitespace characters

`TotalDepth.LIS.core.Mnem.ORDS_REPLACE = (0, 32, 9, 10, 13, 11, 12)`

A tuple of the ordinal values of characters that can be replaced with PAD_CHAR

`TotalDepth.LIS.core.Mnem.LEN_MNEM = 4`

Standard LIS mnemonic length

class `TotalDepth.LIS.core.Mnem.Mnem(m, len_mnem=4)`

Represents a four byte mnemonic where trailing nulls and spaces are not considered significant. This preserves original length but replaces trailing and whitespace characters with the PAD_CHAR.

m must be a bytes object or an 'ascii' str that can be converted to a bytes object.

If len_mnem is positive then m is truncated or padded as necessary to achieve that length.

If len_mnem is zero then all of m is considered significant, no padding is performed.

If len_mnem is less than zero then all characters of m are considered significant and padding up to -1*len_mnem characters of m is performed if m smaller than that.

__init__(m, len_mnem=4)

Constructor that prunes trailing nulls and spaces.

property `m`

The raw bytes of the mnemonic.

__str__()

String representation.

pStr(strip=False)

Returns a 'pretty' ascii string. If strip then trailing padding is removed.

__repr__()

repr() representation.

__hash__()

Hashing, this makes bytes() and Mnem() objects interchangeable.

__eq__(other)

True if self == other False otherwise. If other is not a Mnem it is coerced into one before the comparison is made..

__ne__(other)

True if self != other False otherwise. If other is not a Mnem it is coerced into one before the comparison is made..

__lt__(other)

True if self < other False otherwise. If other is not a Mnem it is coerced into one before the comparison is made..

__iter__()

Byte by byte iteration.

__weakref__

list of weak references to the object (if defined)

5.10 TotalDepth.LIS.core.PhysRec (Physical Record Handler)

5.10.1 Physical Record Attributes

Bit positions here are different from the LIS standard Section 2.3.1.1 and Figure 2.11 on page 2-17.

There they describe the PRL as bits 0-15 and thee PR attributes as bits 16-31.

Our attributes bits are from 15-0 so our attribute bits are 31 - the standard's.

Bit	Description
15	Unused, reserved
14	Physical record type (only 0 is defined)
13-12	00 - No checksum, 01 - 16bit checksum, 10, 11 - Undefined
11	Unused, reserved
10	If 1 File number is present in trailer
09	If 1 Record number is present in trailer
08	Unused
07	Unused, reserved
06	If 1 then a previous parity error has occurred
05	If 1 then a previous checksum error has occurred
04	Unused
03	Unused, reserved
02	Unused
01	If 1 there is a predecessor Physical Record
00	If 1 there is a successor Physical Record

exception TotalDepth.LIS.core.PhysRec.ExceptionPhysRec

Specialisation of exception for Physical Records.

exception TotalDepth.LIS.core.PhysRec.ExceptionPhysRecEOF

Physical Record unexpected EOF.

exception TotalDepth.LIS.core.PhysRec.ExceptionPhysRecUndefinedChecksum

Physical Record encountered undefined checksum bit.

exception TotalDepth.LIS.core.PhysRec.ExceptionPhysRecUnknownType

Encountered unknown type 1 Physical Record.

exception TotalDepth.LIS.core.PhysRec.ExceptionPhysRecWrite

Physical Record writing.

exception TotalDepth.LIS.core.PhysRec.ExceptionPhysRecTail

Physical Record Trailer exception.

TotalDepth.LIS.core.PhysRec.PR_PRH_LEN_FORMAT = <_struct.Struct object>

PR Header 4 bytes long, two big-endian 16 bit numbers The struct.Struct() format for the Physical Record Header

TotalDepth.LIS.core.PhysRec.PR_PRH_ATTR_FORMAT = <_struct.Struct object>

The struct.Struct() format for the Physical Record Header attributes

TotalDepth.LIS.core.PhysRec.PR_PRH_LENGTH = 4

The length of the Physical Record Header

TotalDepth.LIS.core.PhysRec.PR_ATTRIBUTE_BITS = 16

Number of bits in the 2 byte attributes

`TotalDepth.LIS.core.PhysRec.PR_SUCCESOR_ATTRIBUTE_BIT = 0`
 Successor bit position

`TotalDepth.LIS.core.PhysRec.PR_PREDECESSOR_ATTRIBUTE_BIT = 1`
 Predessor bit position

`TotalDepth.LIS.core.PhysRec.PR_OLD_CHECK_ERROR_BIT = 5`
 Checksum error bit position

`TotalDepth.LIS.core.PhysRec.PR_OLD_PARITY_ERROR_BIT = 6`
 Parity error bit position

`TotalDepth.LIS.core.PhysRec.PR_RECORD_NUMBER_BIT = 9`
 Bit position to indicate there is a record number in the trailer

`TotalDepth.LIS.core.PhysRec.PR_FILE_NUMBER_BIT = 10`
 Bit position to indicate there is a file number in the trailer

`TotalDepth.LIS.core.PhysRec.PR_CHECKSUM_BIT = 12`
 Bit position to indicate there is a 16bit checksum in the trailer

`TotalDepth.LIS.core.PhysRec.PR_CHECKSUM_UNDEFINED_BIT = 13`
 Bit position to indicate checksum is undefined

`TotalDepth.LIS.core.PhysRec.PR_TYPE_BIT = 14`
 Bit position to indicate Physical Record Type

`TotalDepth.LIS.core.PhysRec.PR_ATTRIBUTE_UNUSED_ONLY_MASK = 276`
 Unused only bits - i.e. Unused but not Unused, reserved

`TotalDepth.LIS.core.PhysRec.PR_ATTRIBUTE_UNUSED_RESERVED_MASK = 34952`
 Unused, reserved bit mask

`TotalDepth.LIS.core.PhysRec.PR_ATTRIBUTE_UNUSED_MASK = 0`
 Unused and Unused, reserved bits, 0x899C

`TotalDepth.LIS.core.PhysRec.PR_PRT_REC_NUM_FORMAT = <_struct.Struct object>`
 The struct.Struct() format for the Physical Record Trailer record number, unsigned short.

`TotalDepth.LIS.core.PhysRec.PR_PRT_REC_NUM_LEN = 2`
 The length of the Physical Record Trailer for the record number

`TotalDepth.LIS.core.PhysRec.PR_PRT_REC_NUM_MIN = 0`
 The minimum record number, two byte unsigned.

`TotalDepth.LIS.core.PhysRec.PR_PRT_REC_NUM_MAX = 65535`
 The maximum record number, two byte unsigned.

`TotalDepth.LIS.core.PhysRec.PR_PRT_FILE_NUM_FORMAT = <_struct.Struct object>`
 The struct.Struct() format for the Physical Record Trailer file number, unsigned short.

`TotalDepth.LIS.core.PhysRec.PR_PRT_FILE_NUM_LEN = 2`
 The length of the Physical Record Trailer for the file number

`TotalDepth.LIS.core.PhysRec.PR_PRT_FILE_NUM_MIN = 0`
 The minimum file number, two byte unsigned.

`TotalDepth.LIS.core.PhysRec.PR_PRT_FILE_NUM_MAX = 65535`
 The maximum file number, two byte unsigned.

`TotalDepth.LIS.core.PhysRec.PR_PRT_CHECKSUM_FORMAT = <_struct.Struct object>`
 The struct.Struct() format for the Physical Record Trailer checksum, unsigned short.

TotalDepth.LIS.core.PhysRec.PR_PRT_CHECKSUM_LEN = 2

The length of the Physical Record Trailer for the checksum

TotalDepth.LIS.core.PhysRec.PR_MAX_LENGTH = 65535

Maximum possible Physical Record length represented by an unsigned 16 bit int

class TotalDepth.LIS.core.PhysRec.PhysRecBase(*theFileId: str, keepGoing: bool*)

Base class for physical record read and write. TODO: Checksum reading, writing and testing.

__init__(*theFileId: str, keepGoing: bool*)

Constructor, initialise data common to child classes.

theFileId - The ID of the file being read, usually the path.

keepGoing - If True continue parsing files that are not standard compliant.

close()

Close the underlying stream, further operations will raise a ValueError.

strHeader(*inc_attributes_short: bool*)

Returns the header string to go at the top of a list of **__str__**(). If *inc_attributes_short* then the header is suitable for using the string created by **attribute_str_short**().

__str__()

Return **str**(self).

attribute_str() → str

Human readable attributes.

attribute_str_short() → str

Human readable attributes.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.LIS.core.PhysRec.PhysRecRead(*theFile, theFileId: str = "", keepGoing: bool = False, pad_modulo: int = 0, pad_non_null: bool = False*)

Specialisation of PhysRecBase for reading streams.

__init__(*theFile, theFileId: str = "", keepGoing: bool = False, pad_modulo: int = 0, pad_non_null: bool = False*)

Constructor with a file path or file-like object. TODO: checksum.

pad_modulo - If non zero this is used to consume pad bytes. Usually this is 4 so that **tell**() will be increased to % *pad_modulo*.

pad_is_null - If True only consume pad bytes if they are "".

__str__()

Return **str**(self).

readLrBytes(*theSize=- 1, theLd=None*)

Reads theSize logical data bytes and returns it as a **bytes**() object. If theSize is -1 all logical data for this logical record is returned. If theLd is not None it is extended and returned, otherwise a new **bytes**() object is created and returned. Returns None on end of logical record.

skiplrBytes(*theSize=- 1*)

Skips logical data and returns a count of skipped bytes. If theSize is -1 all logical data for this logical record is skipped positioning the stream at end of this logical record (Note: not the beginning of the next logical record). Returns 0 on end of logical record.

skipToNextLr()

Skips all remaining logical data, the PR trailer, and the next PR header. This positions stream at the start of the next logical record. May raise an `ExceptionPhysRecEOF` if there is no further Logical or Physical record. Returns the number of Logical Data bytes skipped.

tellLr()

Returns the absolute file position of the start current Logical record. This value can be safely used in `seekLr`.

tell()

Returns the absolute position of the file.

seekLr(*offset*)

External setting of file position directly to the beginning of a PRH or TIF marker (if present). The caller is fully responsible for getting this right!

seekCurrentLrStart()

Setting the file position directly to the beginning of a PRH or TIF marker (if present) for the current Logical Record.

hasLd()

Returns True if there is logical data to be read, False otherwise. NOTE: This will return False on file initialisation and only return True once the Physical Record Header has been read.

ldRemaingInPr()

Returns the number of bytes remaining in this particular Physical Record. NOTE: The can be 0 and `hasLd()` be True if at the end of a Physical Record that has a successor record.

genLd()

A generator that produces a tuple of (logical data, `isLrStart`) where: logical data - A `bytes()` object for the logical data in the current PR. `isLrStart` - A boolean that is True of that LogicalData is the start of a logical record. NOTE: This rewinds the current state of this instance.

genPr()

A generator that iterates through all the Physical Records from the current position. This seeks through the Logical Data so should be as fast as possible.

```
class TotalDepth.LIS.core.PhysRec.PhysRecTail (hasRecNum=False, fileNum=None,  
                                              hasChecksum=False)
```

Represents Physical Record Tail fields.

```
__init__ (hasRecNum=False, fileNum=None, hasChecksum=False)
```

These three fields are Rep Code type 79 (16 bit signed integer).

```
__str__ ()
```

Return `str(self)`.

```
static normalise_integer (value: int, value_min: int, value_max) → int
```

Given a value this normalises it to be within the range `value_min <= value <= value_max`.

```
property prhAttr
```

Returns the PRH attributes, to be or'd with any other attributes.

```
hasTail ()
```

Returns True if any PRT field is present, False otherwise.

```
computeChecksum (theB)
```

Computes the checksum of the byte stream.

```
__weakref__
```

list of weak references to the object (if defined)

```
class TotalDepth.LIS.core.PhysRec.PhysRecWrite(theFile, theFileId=None, keepGoing=False,  
                                              hasTif=False, thePrLen=65535,  
                                              thePrt=<TotalDepth.LIS.core.PhysRec.PhysRecTail  
                                              object>)
```

Specialisation of PhysRecBase for writing to files.

```
__init__(theFile, theFileId=None, keepGoing=False, hasTif=False, thePrLen=65535,  
         thePrt=<TotalDepth.LIS.core.PhysRec.PhysRecTail object>)
```

Constructor with: *theFile* - A file like object or string, if the latter it assumed to be a path. *theFileId* - File identifier, this could be a path for example. If None the RawStream will try and cope with it. *keepGoing* - If True we do our best to keep going. *hasTif* - Insert TIF markers or not. *thePrLen* - Max Physical Record length, defaults to the maximum possible length. *thePrt* - Physical Records Trailer settings (defaults to PhysRec.PhysRecTail()).

```
close()
```

Close the Physical Record Handler and the underlying stream.

```
writeLr(theLr)
```

Splits a Logical Record into into Physical Records and writes them to the stream. These Physical Records have trailer records if required. Returns the tell() of the start of the LR.

5.11 TotalDepth.LIS.core.RawStream (Raw Stream Handler)

The RawStream handler provides low-level stream I/O functionality.

```
exception TotalDepth.LIS.core.RawStream.ExceptionRawStream
```

Specialisation of exception for RawStream.

```
exception TotalDepth.LIS.core.RawStream.ExceptionRawStreamEOF
```

RawStream premature EOF.

```
class TotalDepth.LIS.core.RawStream.RawStream(f, mode='rb', fileId=None)
```

Class that creates a I/O stream from a file path or file-like object and provides various low level functionality on it such as unpacking.

f - A file like object or string, if the latter it assumed to be a path.

mode - The file mode, defaults to binary read.

fileId - If *f* is a string is this is present then this is used as the file name. If *f* is not a string then *f.name* is used with *fileId* as a fallback.

```
__init__(f, mode='rb', fileId=None)
```

Construct with: *f* - A file like object or string, if the latter it assumed to be a path. *mode* - The file mode, defaults to binary read. *fileId* - If *f* is a string is this is present then this is used as the file name. If *f* is not a string then *f.name* is used with *fileId* as a fallback.

```
__enter__()
```

Context Manager support.

```
__exit__(exc_type, exc_value, traceback)
```

Context manager finalisation, this closes the underlying stream.

```
property stream
```

Exposes the underlying stream.

```
tell()
```

Return the file's current position, like stdio's ftell.

seek(*offset*, *whence=0*)

Set the file's current position, like stdio's fseek. The whence argument is optional and defaults to os.SEEK_SET or 0 (absolute file positioning); other values are os.SEEK_CUR or 1 (seek relative to the current position) and os.SEEK_END or 2 (seek relative to the file's end). There is no return value. Not all file objects are seekable.

read(*theLen*)

Reads and returns theLen bytes.

write(*theB*)

Writes theB bytes.

__weakref__

list of weak references to the object (if defined)

close()

Closes the underlying stream.

readAndUnpack(*theStruct*)

Reads from the stream and unpacks binary data according to the struct module format. This returns a tuple.

theStruct - A formatted instance of struct.Struct().

packAndWrite(*theStruct*, **args*)

Packs binary data from args and writes it to the stream.

theStruct - A formatted instance of struct.Struct().

args - The data to write.

5.12 TotalDepth.LIS.core.RepCode (Representation Codes)

This describes how TotalDepth.LIS handles representation codes, it covers several modules.

5.12.1 Design

There is a top level module RepCode that imports all sub-modules and provides some fundamental definitions. Sub-modules pRepCode and cRepCode provide alternative implementations in Python or Cython.

5.12.2 RepCode Module

Description

This provides the main interface to Representation Code processing as well as some fundamental definitions.

This module is the top level module that imports other sub-modules implemented in Python, Cython or C/C++. The Cython implementations take precedence as this module imports the sub-modules thus:

```
from TotalDepth.LIS.core.pRepCode import *
from TotalDepth.LIS.core.cRepCode import *
```

Usage

It is designed to use thus:

```
from TotalDepth.LIS.core import RepCode
```

5.12.3 Reference

Module for translating raw LIS to an appropriate internal type for LIS Representation Codes ('RepCodes').

This aggregates pRepCode and cRepCode with the latter overwriting the former.

Created on 11 Nov 2010

@author: p2ross

exception TotalDepth.LIS.core.RepCode.**ExceptionRepCodeRead**

Exception for unknown Representation codes in look up tables.

exception TotalDepth.LIS.core.RepCode.**ExceptionRepCodeWrite**

Exception for unknown Representation codes in look up tables.

exception TotalDepth.LIS.core.RepCode.**ExceptionRepCodeNoLength**

Exception for indeterminate length when using Rep Code 65.

TotalDepth.LIS.core.RepCode.**read49**(*theFile*)

Returns a Representation Code 49 value from a File object.

TotalDepth.LIS.core.RepCode.**readBytes49**(*arg*)

Returns a Representation Code 49 value from a bytes object.

TotalDepth.LIS.core.RepCode.**read50**(*theFile*)

Returns a Representation Code 50 value from a File object.

TotalDepth.LIS.core.RepCode.**readBytes50**(*arg*)

Returns a Representation Code 50 value from a bytes object.

TotalDepth.LIS.core.RepCode.**read56**(*theFile*)

Returns a Representation Code 56 value from a File object.

TotalDepth.LIS.core.RepCode.**readBytes56**(*arg*)

Returns a Representation Code 56 value from a bytes object.

TotalDepth.LIS.core.RepCode.**read66**(*theFile*)

Returns a Representation Code 66 value from a File object.

TotalDepth.LIS.core.RepCode.**readBytes66**(*arg*)

Returns a Representation Code 66 value from a bytes object.

TotalDepth.LIS.core.RepCode.**writeBytes66**(*v*)

Converts a value to a Rep Code 66 and returns the bytes.

TotalDepth.LIS.core.RepCode.**read68**(*theFile*)

Returns a Representation Code 68 value from a File object.

TotalDepth.LIS.core.RepCode.**readBytes68**(*arg*)

Returns a Representation Code 68 value from a bytes object.

TotalDepth.LIS.core.RepCode.**writeBytes68**(*v*)

Converts a value to a Rep Code 68 and returns the bytes.

`TotalDepth.LIS.core.RepCode.read70(theFile)`

Returns a Representation Code 70 value from a File object.

`TotalDepth.LIS.core.RepCode.readBytes70(arg)`

Returns a Representation Code 70 value from a bytes object.

`TotalDepth.LIS.core.RepCode.read73(theFile)`

Returns a Representation Code 73 value from a File object.

`TotalDepth.LIS.core.RepCode.readBytes73(arg)`

Returns a Representation Code 73 value from a bytes object.

`TotalDepth.LIS.core.RepCode.writeBytes73(v)`

Converts a value to a Rep Code 73 and returns the bytes.

`TotalDepth.LIS.core.RepCode.read77(theFile)`

Returns a Representation Code 77 value from a File object.

`TotalDepth.LIS.core.RepCode.readBytes77(arg)`

Returns a Representation Code 77 value from a bytes object.

`TotalDepth.LIS.core.RepCode.read79(theFile)`

Returns a Representation Code 79 value from a File object.

`TotalDepth.LIS.core.RepCode.readBytes79(arg)`

Returns a Representation Code 79 value from a bytes object.

`TotalDepth.LIS.core.RepCode.readBytes130(by)`

Reads Dipmeter RepCode 130 and returns a list of (integer) values.

`TotalDepth.LIS.core.RepCode.readBytes234(by)`

Reads Dipmeter RepCode 234 and returns a list of (integer) values.

`TotalDepth.LIS.core.RepCode.readRepCode(theRc, theFile, theLen=None)`

Reads a Representation Code from the file and returns a value. If theRc is 65 (string) then theLen must be supplied, up to that length of bytes will be returned.

`TotalDepth.LIS.core.RepCode.readBytes(theRc, theB, theLen=None)`

Reads a Representation Code from the a bytes() object. If theRc is 65 (string) then theLen must be supplied, up to that length of bytes will be returned.

`TotalDepth.LIS.core.RepCode.writeBytes(v, r)`

Takes a value v and a Representation Code r and converts this to a bytes() object.

5.12.4 pRepCode Module

This contains Python implementations.

Usage

This module is not designed to imported directly, use RepCode instead. This module can be imported only for test purposes thus:

```
from TotalDepth.LIS.core import pRepCode
```

Reference

Python module for translating raw LIS to an appropriate internal type for LIS Representation Codes ('RepCodes').

Created on 11 Nov 2010

@author: p2ross

exception TotalDepth.LIS.core.pRepCode.**ExceptionRepCode**

Specialisation of exception for Representation codes.

exception TotalDepth.LIS.core.pRepCode.**ExceptionRepCodeUnknown**

Specialisation of exception for unknown Representation codes.

TotalDepth.LIS.core.pRepCode.**isInt**(*r*)

Returns True if the Rep Code is represented by an integer.

TotalDepth.LIS.core.pRepCode.**lisSize**(*r*)

Returns the size in bytes for a single instance of a representation code. Zero means variable length. May raise ExceptionRepCodeUnknown.

TotalDepth.LIS.core.pRepCode.**wordLength**(*r*)

Returns the word length in bytes used by a representation code. NOTE: This is subtly different from lisSize as it take into account dipmeter sub-channels Zero means variable length. May raise ExceptionRepCodeUnknown.

TotalDepth.LIS.core.pRepCode.**maxValue**(*r*)

Returns the maximum value for various representation codes. May raise ExceptionRepCodeUnknown.

TotalDepth.LIS.core.pRepCode.**minValue**(*r*)

Returns the minimum value for various representation codes. May raise ExceptionRepCodeUnknown.

TotalDepth.LIS.core.pRepCode.**minMaxValue**(*r*)

Returns a pair; (minimum value, maximum value) for various representation codes. May raise ExceptionRepCodeUnknown.

TotalDepth.LIS.core.pRepCode.**from49**(*theWord*)

Returns a double from Rep code 49 0x31, 16bit floating point representation. Value +153 is 0100 1100 1000 1000 or 0x4C88. Value -153 is 1011 0011 1000 1000 or 0xB388.

TotalDepth.LIS.core.pRepCode.**to49**(*theVal*)

Converts a double to Rep code 49 0x31, 16bit floating point representation. Value +153 is 0100 1100 1000 1000 or 0x4C88. Value -153 is 1011 0011 1000 1000 or 0xB388.

TotalDepth.LIS.core.pRepCode.**read49**(*theFile*)

Returns a Representation Code 49 value from a File object.

TotalDepth.LIS.core.pRepCode.**from50**(*theWord*)

Returns a double from Rep code 0x32, 32bit floating point representation. Value +153 is 0x00084C80 Value -153 is 0x0008B380

TotalDepth.LIS.core.pRepCode.**to50**(*theVal*)

Converts a double to Rep code 0x32, 32bit floating point representation.

TotalDepth.LIS.core.pRepCode.**read50**(*theFile*)

Returns a Representation Code 50 value from a File object.

TotalDepth.LIS.core.pRepCode.**from56**(*theWord*)

Returns an integer from Rep code 0x38, signed char representation.

TotalDepth.LIS.core.pRepCode.**to56**(*theVal*)

Converts a double to Rep code 0x38, signed char representation.

`TotalDepth.LIS.core.pRepCode.read56(theFile)`
Returns a Representation Code 56 value from a File object.

`TotalDepth.LIS.core.pRepCode.from66(theWord)`
Returns a Rep code 0x42, unsigned byte from theWord, expected to be an integer.

`TotalDepth.LIS.core.pRepCode.to66(theVal)`
Converts theVal to representation code 66 integer from theWord.

`TotalDepth.LIS.core.pRepCode.read66(theFile)`
Returns a Representation Code 66 value from a File object.

`TotalDepth.LIS.core.pRepCode.from68(theWord)`
Returns a double from a Rep code 68 word (a 32 bit integer).

`TotalDepth.LIS.core.pRepCode.to68(v)`
Returns Representation code 68 as a 32 bit integer from a double.

`TotalDepth.LIS.core.pRepCode.read68(theFile)`
Returns a Representation Code 68 value from a File object.

`TotalDepth.LIS.core.pRepCode.from70(theWord)`
Returns a double from a Rep code 0x46, 32bit fixed point.

`TotalDepth.LIS.core.pRepCode.to70(v)`
Returns Rep code 0x46, 32bit fixed point from an int or double.

`TotalDepth.LIS.core.pRepCode.read70(theFile)`
Returns a Representation Code 70 value from a File object.

`TotalDepth.LIS.core.pRepCode.from73(theWord)`
Returns an integer from a Rep code 0x49, 32bit signed integer.

`TotalDepth.LIS.core.pRepCode.to73(v)`
Returns Rep code 0x49, 32bit signed integer from an int or double.

`TotalDepth.LIS.core.pRepCode.read73(theFile)`
Returns a Representation Code 73 value from a File object.

`TotalDepth.LIS.core.pRepCode.from77(theWord)`
Returns a Rep code 0x4D, 8bit mask from theWord, expected to be an integer.

`TotalDepth.LIS.core.pRepCode.to77(theVal)`
Converts theVal to Rep code 0x4D, 8bit mask from theVal.

`TotalDepth.LIS.core.pRepCode.read77(theFile)`
Returns a Representation Code 77 value from a File object.

`TotalDepth.LIS.core.pRepCode.from79(theWord)`
Returns an integer from Rep code 0x4F, 16bit signed integer.

`TotalDepth.LIS.core.pRepCode.to79(theVal)`
Converts a double to Rep code 0x4F, 16bit signed integer.

`TotalDepth.LIS.core.pRepCode.read79(theFile)`
Returns a Representation Code 79 value from a File object.

5.12.5 cRepCode Module

This contains Cython implementations. By the import mechanism used by RepCode these implementations take precedence over the implementations in pRepCode.

Usage

This module is not designed to be imported directly, use RepCode instead. This module can be imported only for test purposes thus:

```
from TotalDepth.LIS.core import cRepCode
```

Reference

Cython module for translating raw LIS to an appropriate internal type for LIS Representation Codes ('RepCodes').

Internal types are double, int or string.

`TotalDepth.LIS.core.cRepCode.from50()`

Returns a double from Rep code 0x32, 32bit floating point representation. Value +153 is 0x00084C80 Value -153 is 0x0008B380

`TotalDepth.LIS.core.cRepCode.from56()`

Returns an integer from a Rep code 56 word (a 8 bit signed integer).

`TotalDepth.LIS.core.cRepCode.from66()`

Returns an integer from a Rep code 66 word (a 8 bit unsigned integer).

`TotalDepth.LIS.core.cRepCode.from68()`

Returns a double from a Rep code 68 word (a 32 bit integer).

`TotalDepth.LIS.core.cRepCode.from73()`

Returns an integer from a Rep code 73 word (a 32 bit integer).

`TotalDepth.LIS.core.cRepCode.from77()`

Returns an integer from a Rep code 77 word (a 8 bit unsigned integer).

`TotalDepth.LIS.core.cRepCode.from79()`

Returns an integer from a Rep code 79 word (a 16 bit integer).

`TotalDepth.LIS.core.cRepCode.to68()`

Returns Representation code 68 as a 32 bit integer from a double.

5.12.6 Testing

The unit tests are in `test/TestRepCode.py` and `test/TestRepCode68.py`.

5.13 TotalDepth.LIS.core.Rle (Run Length Encoding)

The RLE module provides Run Length Encoding suitable for recording the file positions of a set of LIS Logical Records that represent frame data.

Created on 5 Jan 2011

5.13.1 class RLEItem()

A generic item in a Run Length Encoding list.

5.13.2 class RLE()

A generic Run Length Encoding list.

5.13.3 class RLEItemType01()

A specialised item in a Run Length Encoding list for type 0/1 LIS Logical Records.

5.13.4 class RLEType01()

A specialised Run Length Encoding list for type 0/1 LIS Logical Records.

5.13.5 API Reference

class TotalDepth.LIS.core.Rle.**RLEItemType01**(*tellLrPos*, *numFrameS*, *xAxisValue*)

Specialisation of an RLEItem for type 0 and type 1 LIS Logical Records. This is a RLEItem for the Logical Record but within we have a RLE() object for the X axis values.

tellLrPos - the position in the LIS file of the start of the Logical Record.

numFrameS - integer number of frames in this Logical Record.

xAxisValue - The value of the X axis of the first frame in the Logical Record.

__init__(*tellLrPos*, *numFrameS*, *xAxisValue*)

__str__()

String representation.

property numFrames

Total number of frames.

add(*tellLrPos*, *numFrameS*, *xAxisValue*)

Returns True if v has been absorbed in this entry. False means a new entry is required. A new entry is required if the *tellLrPos* is not regular or *numFrameS* is different than before.

values()

Generates ordered tuples of (value, number of frames, xaxis value).

value(*i*)

Returns the i'th tuple of (i, (value, number of frames, xaxis value)).

totalFrames()

Returns the total number of frames in this RLE item.

tellLrForFrame(*fNum*)

Returns the Logical Record position that contains the integer frame number.

xAxisFirst()

Returns the first X-axis value loaded.

xAxisLast()

Returns the first X-axis value loaded.

class TotalDepth.LIS.core.Rle.**RLEType01**(*theXUnits, *args*)

Class that represents Run Length Encoding for type 0/1 logical records.

theXUnits - the X axis units.

__init__(*theXUnits, *args*)

Constructor, optionally takes a unary function to convert all values with.

__str__()

String representation.

property **xAxisUnits**

X axis units.

property **hasXaxisData**

True if there is X axis data.

add(*tellLrPos, numFrameS, xAxisValue*)

Adds a value to this RLE object.

tellLrForFrame(*fNum*)

Returns the (*lr_seek*, *frame_offset*) i.e. the Logical Record position that contains the integer frame number and the number of excess frames.

totalFrames()

Returns the total number of frames in this RLE object.

xAxisFirst()

Returns the first X-axis value loaded or None if nothing loaded.

xAxisLast()

Returns the last X-axis value at the start of the last Logical Record loaded or None if nothing loaded.

xAxisLastFrame()

Returns the last X-axis value of the last frame loaded or None if nothing loaded.

frameSpacing()

Returns the frame spacing from the first/last entries, or None if nothing loaded. Returned value is -ve for decreasing X (up logs), +ve for increasing X (down and time logs).

5.14 TotalDepth.LIS.core.TifMarker (TIF Marker Handling)

5.14.1 TIF Markers

These are 3x32bit little-endian integers at the beginning of each Physical Record.

Word[0]

This is the TIF set type, 0 for a normal TIF set. 1 for an EOF set.

Word[1]

This is the physical file location of the start of the previous set.

Word[2]

This is the physical file location of the start of the next set.

A dump looks like this:

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00000000:	00	00	00	00	00	00	00	00	4A	00	00	00	00	3E	00	00
00000010:	80	00	32	30	30	30	39	39	2E	44	41	54	20	20	20	20
00000020:	20	20	20	20	20	20	20	20	20	20	20	39	39	2F	30	
00000030:	34	2F	32	39	20	20	31	30	32	34	20	20	20	20	20	20
00000040:	20	20	20	20	20	20	2E	20	20	20	00	00	00	00	00	00
00000050:	00	00	56	04	00	00	04	00	00	01	40	00	01	01	42	00
00000060:	02	01	42	00	03	04	49	00	00	02	34	04	01	42	01	05
...																
00000450:	20	20	44	45	47	20	00	00	00	00	4A	00	00	00	62	08
00000460:	00	00	04	00	00	03	00	00	00	00	00	00	00	04	20	20
00000470:	00	01	44	20	20	20	20	20	31	30	41	20	20	20	20	20
...																
00000860:	46	4E	00	00	00	00	56	04	00	00	6E	0C	00	00	04	00
00000870:	00	03	4F	52	20	20	20	20	20	20	20	20	20	20	20	20
...																
...																
...																
0016E770:	18	00	BA	83	18	00	00	00	00	00	30	E5	16	00	C0	E7
0016E780:	16	00	00	3E	00	00	81	00	32	30	30	30	39	39	2E	44
...																
0016E7B0:	20	20	20	20	20	20	20	20	20	20	20	20	2E	20	20	20
0016E7C0:	01	00	00	00	76	E7	16	00	CC	E7	16	00	01	00	00	00
0016E7D0:	C0	E7	16	00	D8	E7	16	00								

Or:

tell()	TIF type	TIF back	TIF next
00000000:	00 00 00 00	00 00 00 00	00 00 00 4A
0000004A:	00 00 00 00	00 00 00 00	00 00 04 56
00000456:	00 00 00 00	00 00 00 4A	00 00 08 62

(continues on next page)

(continued from previous page)

```

000000862:  00 00 00 00      00 00 04 56      00 00 0C 6E
...
0016E776:  00 00 00 00      00 16 E5 30      00 16 E7 C0
0016E7C0:  00 00 00 01      00 16 E7 76      00 16 E7 CC
0016E7CC:  00 00 00 01      00 16 E7 C0      00 16 E7 D8
0016E7D8: EOF

```

exception TotalDepth.LIS.core.TifMarker.**ExceptionTifMarker**

Specialisation of exception for Physical Records.

TotalDepth.LIS.core.TifMarker.**TIF_WORD_BYTES** = 4

Number of bytes in a TIF word

TotalDepth.LIS.core.TifMarker.**TIF_NUM_WORDS** = 3

Number of words in a TIF marker

TotalDepth.LIS.core.TifMarker.**TIF_TOTAL_BYTES** = 12

Number of bytes in a TIF marker

TotalDepth.LIS.core.TifMarker.**TIF_WORD_FORMAT** = <_struct.Struct object>

struct.Struct() format for a TIF word

TotalDepth.LIS.core.TifMarker.**TIF_WORD_FORMAT_WRONG_SEX** = <_struct.Struct object>

struct.Struct() format for a TIF word written wrongly as little-endian

TotalDepth.LIS.core.TifMarker.**TIF_WORD_ALL_FORMAT_WRONG_SEX** = <_struct.Struct object>

struct.Struct() format for a TIF marker written wrongly as little-endian

TotalDepth.LIS.core.TifMarker.**TIF_FIRST_WORD_LIMIT** = 65547

The maximum possible size of the first 'next' word. If larger than this then the words are written wrongly as little-endian and need to be reversed This is calculated as the maximum PR length + TIF bytes.

class TotalDepth.LIS.core.TifMarker.**TifMarkerBase**(raiseOnError=True)

Base class for TIF markers.

__init__(raiseOnError=True)

Constructor, initialises internals.

strHeader()

Header string for an ASCII dump.

__str__()

String representation.

markers()

Current values of markers as a tuple of three integers.

property eof

True if I have encountered a EOF marker.

reset()

Resets the TIF markers to all zero, this means hasPrevious is False.

reportError(theMsg)

Reports the error. I constructed with raiseOnError as True this will raise a ExceptionTifMarker otherwise it will write the error to the log.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.LIS.core.TifMarker.TifMarkerRead(*theStream*, *allowPrPadding=False*)

Class for reading TIF markers. This will automatically determine if TIF markers are present and automatically correct ill-formed little-endian TIF markers.

theStream - the file stream.

allowPrPadding - If True this will consume spurious padding bytes after the Physical Record tail i.e. the TIF markers determine the Physical Record structure rather than the Physical Record Headers.

__init__(*theStream*, *allowPrPadding=False*)

Constructor, initialises internals. *allowPrPadding* - If true this allows padding bytes after the PRT.

property *hasPrevious*

True if a Physical Record has been read, cleared on reset().

reset()

Calling reset() means that the caller is probably randomly accessing the file so we can not error check the previous marker in the same way that we can if we are reading the file linearly.

read(*theStream*)

Read TIF markers from a RawStream object. Returns the stream tell() or None of the start of the TIF marker. This is not necessarily the same as the stream tell() seen by the caller as we might consume PR padding.

class TotalDepth.LIS.core.TifMarker.TifMarkerWrite

Class for writing TIF markers.

__init__()

Constructor, initialises internals.

write(*theStream*, *theLen*)

Write TIF markers to a RawStream object. *theLen* must be the length of the Physical Record including the PRH and PRT.

close(*theStream*)

Write TIF EOF markers.

5.15 TotalDepth.LIS.core.Type01Plan (Binary Frame Data Random Access)

Given a DFSR the FrameSetPlan gives offsets to any part of the Logical Record for any frame and channel.

exception TotalDepth.LIS.core.Type01Plan.ExceptionFrameSetPlan

Specialisation of exception for FrameSetPlan.

exception TotalDepth.LIS.core.Type01Plan.ExceptionFrameSetPlanNegLen

Specialisation of exception for negative/too small length arguments to FrameSetPlan.

exception TotalDepth.LIS.core.Type01Plan.ExceptionFrameSetPlanOverrun

Exception channel number greater than available.

TotalDepth.LIS.core.Type01Plan.EVENT_READ = 'read'

Read event

TotalDepth.LIS.core.Type01Plan.EVENT_SKIP = 'skip'

Skip event

TotalDepth.LIS.core.Type01Plan.EVENT_EXTRAPOLATE = 'extrapolate'

Extrapolate event

class TotalDepth.LIS.core.Type01Plan.**FrameSetPlan**(*dfsr*)

Given a DFSR the FrameSetPlan gives offsets to any part of the frame set within a Logical Record.

NOTE: All offsets, lengths etc. are relative to the end of the LRH i.e. add LR_HEADER_LENGTH to get absolute Logical Data position.

__init__(*dfsr*)

__str__()

Return str(self).

property indirectSize

The size, in bytes of the indirect X axis value. 0 for explicit X axis.

property frameSize

Frame size in bytes.

property numChannels

Number of channels.

channelSize(*i*)

The size of the given channel.

numFrames(*recLen*)

Returns the number of frames that will fit into the record length. May raise a ExceptionFrameSetPlan. May raise ExceptionFrameSetPlanNegLen if arguments are negative.

NOTE: record length should not include size of LRH.

chOffset(*frame, ch*)

Returns the offset into the LR (after LRH) to the start of a particular channel and frame.

Will raise ExceptionFrameSetPlanNegLen if arguments are negative or an IndexError if ch out of range.

skipToEndOfFrame(*ch*)

Returns the skip distance into the LR to the end of frame after a particular channel.

genOffsets(*theChIndexS*)

Yields an indefinite set of (frame, channel, offset) values for the set of channels. Will raise ExceptionFrameSetPlanNegLen if any channel negative. Will raise KeyError if any channel index is out of range.

__weakref__

list of weak references to the object (if defined)

genEvents(*theFSlice, theChIndexS*)

For a single Logical Record type 0/1 this yields an set of events.

theFSlice is a frame slice object (default start=0, step=1), step 0 is interpreted as step 1.

theChIndexS is as list of indexes of channels that need to be read.

The events are 5 member tuples: (event_type, size, frame, channel_start, channel_stop)

event_type is 'skip' or 'read' or 'extrapolate':

skip: ('skip', size, frame_number, channel_start, channel_stop)

read: ('read', size, frame_number, channel_start, channel_stop)

extrapolate: ('extrapolate', frames, frame_number, None, None)

NOTE: No 'seek', 0, ... events are generated.

size is the number of bytes to read or skip or, for 'extrapolate' the number of frames to extrapolate the implied X axis.

frame_number is the frame number in the predicted Logical Record.

In the case of 'skip' or 'extrapolate' its is the frame number in the Logical Record being moved to. The frame number can be None if there is no prediction, for example when reading an indirect X at the beginning of an LR.

channel_start, channel_stop are the channel numbers (inclusive) in the DFSR DSB block None being the indirect X axis.

Thus: ('read', size, frame_number, None, 3) Means read size bytes and interpret them as channels: None, 0, 1, 2, 3. 'extrapolate' events mean project the Xaxis forward by the specified number of frames (this will not be bestrided with two seek events but preceded or followed by a single seek event (if any). 'read' and 'seek' events are synchronous as are 'read' and 'extrapolate'. However 'seek' and 'extrapolate' are asynchronous i.e. 'seek'-'extrapolate' is equivalent to 'extrapolate'-'seek'.

Will raise ExceptionFrameSetPlanNegLen if any channel negative or fStop < fStart. Will raise IndexError if any channel index is out of range.

Be aware: This code is tricky.

5.16 TotalDepth.LIS.core.Units (Unit Conversion)

Provides unit conversion for LIS79.

The __RAW_UNIT_MAP is the master map from which all other data is derived. Its format is as follows:

key - The unit category as four bytes representing uppercase ASCII characters. value - A tuple of three fields:

- [0] - Descriptive string of the unit category.
- [1] - The base unit name as four bytes representing uppercase ASCII characters.
- [2] - A tuple the contents of which is a four or five item tuple:
 - **If four members:**
 - * [2][0] - The unit name as four bytes representing uppercase ASCII characters.
 - * [2][1] - The multiplier as a float.
 - * [2][2] - Descriptive string of the units.
 - * **[2][3] - The unit name that this is an alternate for as four bytes** representing uppercase ASCII characters, or four spaces.
 - **If five members:**
 - * [2][0] - The unit name as four bytes representing uppercase ASCII characters.
 - * [2][1] - The multiplier as a float.
 - * [2][2] - The offset as a float.
 - * [2][3] - Descriptive string of the units.
 - * **[2][4] - The unit name that this is an alternate for as four bytes** representing uppercase ASCII characters or four spaces.

The unit name should also be unique.

TODO: Clean up units by making reciprocal e.g. 1/6.0 rather than 0.166666...

TODO: Check each unit for errors.

exception TotalDepth.LIS.core.Units.**ExceptionUnits**

Specialisation of exception for Unit conversion.

exception TotalDepth.LIS.core.Units.**ExceptionUnitsUnknownUnit**

When a unit does not exist.

exception TotalDepth.LIS.core.Units.**ExceptionUnitsUnknownCategory**

When a unit category does not exist.

exception TotalDepth.LIS.core.Units.**ExceptionUnitsNoUnitInCategory**

When a unit does not exist in a category.

exception TotalDepth.LIS.core.Units.**ExceptionUnitsMismatchedCategory**

When a two units do not exist in the same category.

class TotalDepth.LIS.core.Units.**UnitConvertCategory**(*theCat, theDesc, theBaseUnitName, theUnitS*)

Internal module data structure that represents a category of units such as linear length.

theCat is the unit category.

theDesc is the description of that category.

theBaseUnitName is the name of the base units for the category. For example for linear length this is b'M '.

theUnitS is a list of unit names.

__init__(*theCat, theDesc, theBaseUnitName, theUnitS*)

units()

Returns a list of unit names for this category.

unitConvertor(*u*)

Returns a UnitConvert object corresponding to the name u. Will raise a ExceptionUnitsNoUnitInCategory if not found.

convert(*v, u_1, u_2*)

Returns a value converted from one units to another. e.g. convert(1.2, "FEET", "INCH")

__weakref__

list of weak references to the object (if defined)

class TotalDepth.LIS.core.Units.**UnitConvert**(*tup*)

Internal data structure for this module that represents a particular unit of measure. Takes a 4 or 5 member tuple from __RAW_UNIT_MAP.

__init__(*tup*)

convert(*val, other*)

Convert a value from me to the other where other is a UnitConvert object.

__weakref__

list of weak references to the object (if defined)

TotalDepth.LIS.core.Units.**unitCategories**()

Returns a list of the unit categories.

TotalDepth.LIS.core.Units.**hasUnitCategory**(*c*)

Returns True if I have that unit category e.g. b"TIME".

TotalDepth.LIS.core.Units.**hasUnit**(*u*)

Returns True if I have that unit e.g. b"FEET".

TotalDepth.LIS.core.Units.**category**(*unit*)

Returns the category of the unit. May raise a ExceptionUnitsUnknownUnit.

`TotalDepth.LIS.core.Units.categoryDescription(theCat)`

Returns the description of a unit category.

`TotalDepth.LIS.core.Units.units(theCat=None)`

Returns an unordered list of unit names. If category is None all unit names are returned, otherwise the unit names for a particular category are returned. This may raise a `ExceptionUnitsUnknownCategory` if the category does not exist.

`TotalDepth.LIS.core.Units.retUnitConvertCategory(c)`

Returns a `UnitConvertCategory` object for the category. May raise a `ExceptionUnits` or descendent.

`TotalDepth.LIS.core.Units.retUnitConvert(u)`

Returns a `UnitConvert` object for the unit. May raise a `ExceptionUnits` or descendent.

`TotalDepth.LIS.core.Units.unitDescription(u)`

Returns the description of the unit. e.g. Given “.1IN” returns “Tenth-inches”. May raise a `ExceptionUnits` or descendent.

`TotalDepth.LIS.core.Units.realUnitName(u)`

Returns the real unit name or None if *u* is the ‘real’ unit e.g. the ‘real’ unit name for `b’FT ’` is `b’FEET’`. May raise a `ExceptionUnits` or descendent.

`TotalDepth.LIS.core.Units.convert(v, u_1, u_2)`

Returns a value converted from one units to another. e.g. `convert(1.2, b’FEET’, b’INCH’)`.

Will raise an `ExceptionUnitsUnknownUnit` if either unit is unknown.

Will raise an `ExceptionUnitsMissmatchedCategory` if both units do not belong to the same unit category.

`TotalDepth.LIS.core.Units.opticalUnits(u)`

If possible returns the ‘optical’ units i.e. user friendly units. For example the ‘optical’ units of `b’.1IN’` are `b’FEET’`. Failure returns the argument.

5.16.1 Examples

Converting bytes objects:

```
from TotalDepth.LIS.core import Units

v = Units.convert(1.0, b"M ", b"FEET")
# v is now 3.281
```

5.16.2 Testing

The unit tests are in `test/TestUnits.py`.

5.17 TotalDepth.LIS.DumpFrameSet

Reads a LIS file and writes out separated values of each frame.

Created on 25 Mar 2011

`TotalDepth.LIS.DumpFrameSet.dump_frame_sets(fp, keep_going, output_frames: bool, output_summary: bool, channels, separator: str = ' ')`

Dump the frame values to stdout. channels is a set of Mnems, if non-empty then only these channels, if present, are written out.

5.18 TotalDepth.LIS.Index

Indexes LIS files and reports performance.

Created on 24 Feb 2011

@author: p2ross

Indexing errors on LIS files:

[34] TotalDepth.LIS.core.TifMarker.ExceptionTifMarker: TIF read() expected 0x50, got tell: 0x4A, Shortfall: 0x6 Fixed.

[24] TotalDepth.LIS.core.LogiRec.ExceptionEntryBlock: EntryBlockSet.setEntryBlock(): type 10 excluded from EntryBlockSet Fixed.

[2] TotalDepth.LIS.core.Type01Plan.ExceptionFrameSetPlan: Can not fit integer number of frames length 120 into LR length 824, modulo 104 [indirect size 0]. Two file have different problems:

5.18.1 13576.S1

W:openLISsrcTotalDepth.LIS>python Index.py -rk -140pLogicTestDataLIS]76.S1 ... TotalDepth.LIS.core.Type01Plan.ExceptionFrameSetPlan: Can not fit integer number of frames length 120 into LR length 824, modulo 104 [indirect size 0].

Looks like the last PR is truncated: ... TIF True >: 0x 0 0x 19006 0x 197b6 PR: 0x 193de 972 0x9600 962 0x006c 0x0001 0xa2e1 0x00 0x00 [962] TIF True >: 0x 0 0x 193de 0x 19b06 PR: 0x 197b6 836 0x9600 826 0x006d 0x0001 0x0304 0x00 0x00 [826] Missing 962-826 bytes 136 bytes.

5.18.2 13610.S1

W:openLISsrcTotalDepth.LIS>python Index.py -rk -140pLogicTestDataLIS^10.S1 ... TotalDepth.LIS.core.Type01Plan.ExceptionFrameSetPlan: Can not fit integer number of frames length 7176 into LR length 13354, modulo 6178 [indirect size 0].

This looks like a bad PR header at 0x3a986 that has set a successor bit: W:openLISsrcTotalDepth.LIS>python ScanPhysRec.pypLogicTestDataLIS^10.S1 Cmd: ScanPhysRec.pypLogicTestDataLIS^10.S1 TIF ? : Type Back Next PR: tell() Length Attr LD_len RecNum FilNum ChkSum LR Attr [Total LD] TIF True >: 0x 0 0x 0 0x 4a PR: 0x 0 62 0x8000 58 ———— 0x80 0x00 [58] TIF True >: 0x 0 0x 0 0x 3ac PR: 0x 4a 854 0x8000 850 ———— 0x40 0x00 [850] ... TIF True >: 0x 0 0x 390f4 0x 395ae PR: 0x 394ec 182 0x8001 178 ———— 0x00 0x00 TIF True >: 0x 0 0x 394ec 0x 399a6 PR: 0x 395ae 1004 0x8003 1000 ———— + ———— TIF True >: 0x 0 0x 395ae 0x 39d9e PR: 0x 399a6 1004 0x8003 1000 ———— + ———— TIF True >: 0x 0 0x 399a6 0x 3a196 PR: 0x 39d9e 1004 0x8003 1000 ———— + ———— TIF True >: 0x 0 0x 39d9e 0x 3a58e PR: 0x 3a196 1004 0x8003 1000 ———— + ———— TIF True >: 0x 0 0x 3a196 0x 3a986 PR: 0x 3a58e 1004 0x8003 1000 ————

_____ + _____ TIF True >: 0x 0 0x 3a58e 0x 3ad7e PR: 0x 3a986 1004 0x8003 1000 _____ + _____
 2011-03-09 19:50:13,710 WARNING Physical record at 0x3AD7E is successor but has no predecessor bit set.

TIF True >: 0x 0 0x 3a986 0x 3ae40 PR: 0x 3ad7e 182 0x8001 178 _____ + _____ TIF True >: 0x 0 0x 3ad7e 0x 3b238 PR: 0x 3ae40 1004 0x8003 1000 _____ + _____ TIF True >: 0x 0 0x 3ae40 0x 3b630 PR: 0x 3b238 1004 0x8003 1000 _____ + _____ TIF True >: 0x 0 0x 3b238 0x 3ba28 PR: 0x 3b630 1004 0x8003 1000 _____ + _____ TIF True >: 0x 0 0x 3b630 0x 3be20 PR: 0x 3ba28 1004 0x8003 1000 _____ + _____ TIF True >: 0x 0 0x 3ba28 0x 3c218 PR: 0x 3be20 1004 0x8003 1000 _____ + _____ TIF True >: 0x 0 0x 3be20 0x 3c610 PR: 0x 3c218 1004 0x8003 1000 _____ + _____ TIF True >: 0x 0 0x 3c218 0x 3ca08 PR: 0x 3c610 1004 0x8002 1000 _____ + _____ [13356]

[1] TotalDepth.LIS.core.pRepCode.ExceptionRepCodeUnknown: Unknown representation code: 0 Fixed by being a bit more cautious about dealing with DSB blocks that are 'null'.

TotalDepth.LIS.Index.**index_dir_single_process**(*d, r, t, v, k*) → Dict[str,
 TotalDepth.LIS.Index.IndexTimer]
 Recursively process a directory using a single process.

TotalDepth.LIS.Index.**generate_file_paths**(*d, r*)
 Generates file paths, recursive if necessary.

5.19 TotalDepth.LIS.LisToHtml

Created on Jun 14, 2011

@author: paulross

```
class TotalDepth.LIS.LisToHtml.FileInfo(pathIn, pathOut, lisSize, numLr, cpuTime, exception)

    pathIn: str
        Alias for field number 0
    pathOut: str
        Alias for field number 1
    lisSize: int
        Alias for field number 2
    numLr: int
        Alias for field number 3
    cpuTime: float
        Alias for field number 4
    exception: bool
        Alias for field number 5

    __str__()
        Return str(self).

    __getnewargs__()
        Return self as a plain tuple. Used by copy and pickle.

    static __new__(_cls, pathIn: str, pathOut: str, lisSize: int, numLr: int, cpuTime: float, exception: bool)

    __repr__()
        Return a nicely formatted representation string

class TotalDepth.LIS.LisToHtml.LisToHtml(fpIn, fpOut, recursive, keepGoing, accCh=True)
    Takes an input path, output path and generates HTML file(s) form LIS.
```

__init__(*fpIn, fpOut, recursive, keepGoing, accCh=True*)

Write an HTML page about a LIS file. If accChan is True a summary table of the data is written.

TotalDepth.LIS.LisToHtml.**processFile**(*fpIn, fpOut, keepGoing*) →

TotalDepth.LIS.LisToHtml.IndexSummary

Used by the multiprocessing code.

5.20 TotalDepth.LIS.PlotLogPasses

Created on May 23, 2011

@author: p2ross

class TotalDepth.LIS.PlotLogPasses.**IndexTableValue**(*scale, evFirst, evLast, curves, numPoints, outPath*)

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static **__new__**(*_cls, scale, evFirst, evLast, curves, numPoints, outPath*)

__repr__()

Return a nicely formatted representation string

curves

Alias for field number 3

evFirst

Alias for field number 1

evLast

Alias for field number 2

numPoints

Alias for field number 4

outPath

Alias for field number 5

scale

Alias for field number 0

class TotalDepth.LIS.PlotLogPasses.**PlotLogInfo**

Class that collates information about the results of plotting log passes. This can, for example, write out an index.html page with links to SVG pages.

__init__()

__str__()

Return str(self).

addPlotResult(*theInPath, theOutPath, theLpIdx, theFilmID, theScale, theEvFirst, theEvLast, theCurveS, ptsPlotted*)

Adds a successful plot. theInPath - The file path to the input file. theOutPath - The file path to the output file. theLpIdx - Integer index of the LogPass in the input file. theFilmID - The FILM ID as a Mnem. theScale - Plot scale as an number. theEvFirst - The first X axis as an EngVal. theEvLast - The last X axis as an EngVal. theCurveS - A list of Mnem of the curves plotted. ptsPlotted - Number of points plotted.

writeHTML(*theFilePath, theDesc*)

Write the index.html table.

retRelPath(*d, f*)

Given directory *d* and file path of *f* this returns relative path to *f* from *d*.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.LIS.PlotLogPasses.**PlotLogPasses**(*fpIn, fpOut, recursive=False, keepGoing=True, lgFormatS=None, apiHeader=False*)

Takes an input path, output path and generates SVG file(s) from LIS.

__init__(*fpIn, fpOut, recursive=False, keepGoing=True, lgFormatS=None, apiHeader=False*)

Constructor.

fpIn and *fpOut* are file or directory paths. *fpOut* will be created if necessary.

recursive is a flag to control directory recursion.

keepGoing is a flag passed to the LIS File.FileRead object.

lgFormatS is a list of strings the correspond to the **LgFormat UniqueId XML attribute**. If absent the LIS file FILM/PRES etc. tables are used.

apiHeader is a flag to control whether a API header is extracted from CONS tables is to be plotted on the top of the log.

__weakref__

list of weak references to the object (if defined)

TotalDepth.LIS.PlotLogPasses.**plotLogPassesMP**(*dIn, dOut, fnMatch, recursive, keepGoing, lgFormatS, apiHeader, jobs*)

Multiprocessing code to plot log passes. Returns a PlotLogInfo object.

5.21 TotalDepth.LIS.ProcLISPath

Created on Jun 14, 2011

@author: paulross

exception TotalDepth.LIS.ProcLISPath.**ExceptionProcLisPath**

class TotalDepth.LIS.ProcLISPath.**ProcLISPathBase**(*fpIn, fpOut, recursive, keepGoing*)

Takes an input path, output path and processes LIS files.

__init__(*fpIn, fpOut, recursive, keepGoing*)

__weakref__

list of weak references to the object (if defined)

TotalDepth.LIS.ProcLISPath.**procLISPath**(*dIn, dOut, fnMatch, recursive, keepGoing, jobs, fileFn, resultObj=None*)

Multiprocessing code to process LIS files. *dIn*, *dOut* are directories.

fnMatch is a glob string.

recursive is a boolean to control recursion.

keepGoing is passed to fileFn

jobs is number of jobs; -1 single process, 0 number of available CPUs

fileFn is the operational function that will take a tuple of: (*fIn*, *fOut*, *keepGoing*) and return a result that can be added to the *resultObj* or None. This should not raise.

resultObj is accumulation of the results of fileFn or None, this it returned.

TotalDepth.LIS.ProcLISPath.**procLISPathMP**(dIn, dOut, fnMatch, recursive, keepGoing, jobs, fileFn, resultObj=None)

Multiprocessing code to process LIS files.

dIn, dOut are directories.

fnMatch is a glob string.

recursive is a boolean to control recursion.

keepGoing is passed to fileFn

fileFn is the operational function that will take a tuple of: (fIn, fOut, keepGoing) and return a result that can be added to the resultObj or None. This should not raise.

resultObj is accumulation of the results of fileFn or None, this it returned.

5.22 TotalDepth.LIS.RandomFrameSetRead

Created on 25 Feb 2011

@author: p2ross

TotalDepth.LIS.RandomFrameSetRead.**regexStdout**(fp)

Opens a file on path fp and regexes it for data.

5.23 TotalDepth.LIS.ScanLogiData

Created on 10 Nov 2010

@author: p2ross

TotalDepth.LIS.ScanLogiData.**ret_int_dump_list**(arg_string: str) → List[int]

Splits a string and returns a list of integers from hex/dec.

5.24 TotalDepth.LIS.ScanLogiRec

Created on 10 Nov 2010

@author: p2ross

5.25 TotalDepth.LIS.ScanPhysRec

Created on 10 Nov 2010

@author: p2ross

class TotalDepth.LIS.ScanPhysRec.**PhysRecScanResult**(path, pr_count, error)

path: str

Alias for field number 0


```

pr_count: int
    Alias for field number 1

error: bool
    Alias for field number 2

__str__()
    Return str(self).

__getnewargs__()
    Return self as a plain tuple. Used by copy and pickle.

static __new__(_cls, path: str, pr_count: int, error: bool)

__repr__()
    Return a nicely formatted representation string

class TotalDepth.LIS.ScanPhysRec.PhysRecScanResultWithPad(result, pad_modulo, pad_non_null)

    result: TotalDepth.LIS.ScanPhysRec.PhysRecScanResult
        Alias for field number 0

    pad_modulo: int
        Alias for field number 1

    pad_non_null: bool
        Alias for field number 2

    __str__()
        Return str(self).

    __getnewargs__()
        Return self as a plain tuple. Used by copy and pickle.

    static __new__(_cls, result: TotalDepth.LIS.ScanPhysRec.PhysRecScanResult, pad_modulo: int,
        pad_non_null: bool)

    __repr__()
        Return a nicely formatted representation string

```

5.26 TotalDepth.LIS.TableHistogram

Provides a count of elements in LIS tables.

Created on May 24, 2011

@author: paulross

```

class TotalDepth.LIS.TableHistogram.TableMatcher(lrType: int, nameTable: bytes, nameRow: bytes,
        nameCol: bytes)

    Tests if a table entry matches.

    lrType: int
        Alias for field number 0

    nameTable: bytes
        Alias for field number 1

    nameRow: bytes
        Alias for field number 2

```

nameCol: `bytes`
Alias for field number 3

__getnewargs__()
Return self as a plain tuple. Used by copy and pickle.

static __new__(_cls, lrType: int, nameTable: bytes, nameRow: bytes, nameCol: bytes)

__repr__()
Return a nicely formatted representation string

5.27 TotalDepth.LIS.ToLAS

Reads a LIS file and writes out LAS files.

Created on 25 Mar 2011

class TotalDepth.LIS.ToLAS.LisLogicalFile

Contains a representation of a LIS Logical File which is a series of indexes followed by a Log Pass

__init__()

add_index(index_entry: TotalDepth.LIS.core.FileIndexer.IndexObjBase) → None

Adds an index or Log Pass.

is_end() → bool

Can be added to.

__weakref__

list of weak references to the object (if defined)

TotalDepth.LIS.ToLAS.**stringify**(value: Any, float_format: str) → str

Convert an object to a string respecting the requested floating point format.

TotalDepth.LIS.ToLAS.**write_las_file**(path_in: str, array_reduction: str, path_out: str, frame_slice: Union[TotalDepth.common.Slice.Slice, TotalDepth.common.Slice.Sample], channels: Set[str], field_width: int, float_format: str, lis_file: TotalDepth.LIS.core.File.FileRead, logical_file_index: int, lis_logical_file: TotalDepth.LIS.ToLAS.LisLogicalFile) → int

Writes a single LAS file corresponding to a LIS logical file (set of CONS tables and a LogPass).

5.28 TotalDepth.LIS.lis_cmn_cmd_opts

Common command line options for LIS tools.

TOTALDEPTH.RP66V1 API REFERENCE

Contents:

6.1 TotalDepth.RP66V1.core.AbsentValue

Absent Value - a value that represents a missing value.

Unlike LIS79 which specifies an absent value in the DFSR, RP66V1 has no means of explicitly specifying an absent value. Observation of real-world files shows the -999.25 is used for floats and -999 for integer representation codes.

`TotalDepth.RP66V1.core.AbsentValue.absent_value_from_rep_code(rep_code: int) → Optional[Union[float, int]]`

Returns the absent value depending on the Representation Code.

6.2 TotalDepth.RP66V1.core.File

Handles low level RP66V1 operations.

TODO: Replace this with the C/C++ implementation.

exception `TotalDepth.RP66V1.core.pFile.ExceptionFile`

exception `TotalDepth.RP66V1.core.pFile.ExceptionEOF`
Premature EOF.

exception `TotalDepth.RP66V1.core.pFile.ExceptionTIF`

exception `TotalDepth.RP66V1.core.pFile.ExceptionTIFEOF`

exception `TotalDepth.RP66V1.core.pFile.ExceptionVisibleRecord`

exception `TotalDepth.RP66V1.core.pFile.ExceptionVisibleRecordEOF`

exception `TotalDepth.RP66V1.core.pFile.ExceptionLogicalRecordSegmentHeaderAttributes`

exception `TotalDepth.RP66V1.core.pFile.ExceptionLogicalRecordSegmentHeader`

exception `TotalDepth.RP66V1.core.pFile.ExceptionLogicalRecordSegmentHeaderEOF`

exception `TotalDepth.RP66V1.core.pFile.ExceptionLogicalRecordSegmentHeaderSequence`

exception `TotalDepth.RP66V1.core.pFile.ExceptionFileRead`

exception `TotalDepth.RP66V1.core.pFile.ExceptionFileReadEOF`

exception `TotalDepth.RP66V1.core.pFile.ExceptionFileReadPositionsInconsistent`

exception TotalDepth.RP66V1.core.pFile.ExceptionStorageUnitLabel

Exception specialisation for this module.

TotalDepth.RP66V1.core.pFile.TIF_FILE_PREFIX =

`b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00'`

The first one must be StorageUnitLabel.SIZE + TIF_SIZE = 92 or 0x5c little endian 0000 0000 0000 0000 5c00 0000 This is here just to raise an exception for TIF marked RP66V1 files.

class TotalDepth.RP66V1.core.pFile.StorageUnitLabel(*by: bytes*)

The Storage Unit Label that must be at the beginning of a file. See [RP66V1 Section 2.3.2 Storage Unit Label (SUL)].

Unique Storage Unit Labels seen in practice:

```
$ grep -rIh StorageUnitLabel <XML indexes> | sort | uniq -c
  16 <StorageUnitLabel dlis_version="V1.00" maximum_record_length="16384"
↳sequence_number="1"
    storage_set_identififier="CUSTOMER
↳
    storage_unit_structure="RECORD"/>
   2 <StorageUnitLabel dlis_version="V1.00" maximum_record_length="16384"
↳sequence_number="1"
    storage_set_identififier="PRODUCER
↳
    storage_unit_structure="RECORD"/>
   1 <StorageUnitLabel dlis_version="V1.00" maximum_record_length="8192" sequence_
↳number="1"
    storage_set_identififier="
↳
    storage_unit_structure="RECORD"/>
  34 <StorageUnitLabel dlis_version="V1.00" maximum_record_length="8192" sequence_
↳number="1"
    storage_set_identififier="DLIS ATLAS 1
↳
    storage_unit_structure="RECORD"/>
 537 <StorageUnitLabel dlis_version="V1.00" maximum_record_length="8192" sequence_
↳number="1"
    storage_set_identififier="Default Storage Set
↳
    storage_unit_structure="RECORD"/>
```

__init__(*by: bytes*)

as_bytes() → bytes

Returns the bytes that encode this Storage Unit Label.

__str__() → str

Return str(self).

__weakref__

list of weak references to the object (if defined)

TotalDepth.RP66V1.core.pFile.create_storage_unit_label(*storage_unit_sequence_number: int,*
dlis_version: bytes,
maximum_record_length: int,
storage_set_identififier: bytes) → *TotalDepth.RP66V1.core.pFile.StorageUnitLabel*

Create a StorageUnitLabel from the given values.

class TotalDepth.RP66V1.core.pFile.**VisibleRecord**(*fobj: Optional[BinaryIO]*)

RP66V1 visible records. See [RP66V1 Section 2.3.6] (sic) - Place marker for error in the standard in this case

__init__(*fobj: Optional[BinaryIO]*)

as_bytes() → bytes

The Visible Record represented in raw bytes.

read(*fobj: BinaryIO*) → None

Read a new Visible Record and check it. This may throw a `ExceptionVisibleRecord`.

read_next(*fobj: BinaryIO*) → None

Move to next Visible Record and read it. This may throw a `ExceptionVisibleRecord`.

__format__(*format_spec*) → str

Default object formatter.

__eq__(*other*)

Return self==value.

__str__() → str

Return str(self).

__weakref__

list of weak references to the object (if defined)

class TotalDepth.RP66V1.core.pFile.**LogicalRecordSegmentHeader**(*fobj: BinaryIO*)

RP66V1 Logical Record Segment Header. See [RP66V1 2.2.2.1]

__init__(*fobj: BinaryIO*)

Constructor. position: The file position of the start of the LRSH.

length: The *Logical Record Segment Length* is a two-byte, unsigned integer (Representation Code UNORM) that specifies the length, in bytes, of the Logical Record Segment. The Logical Record Segment Length is required to be even. The even length ensures that 2-byte checksums can be computed, when present, and permits some operating systems to handle DLIS data more efficiently without degrading performance with other systems. There is no limitation on a Logical Record length. Logical Record Segments must contain at least sixteen (16) bytes. This requirement facilitates mapping the Logical Format to those Physical Formats that require a minimum physical record length.

attributes: The *Logical Record Segment Attributes* consist of a one-byte bit string that specifies the Attributes of the Logical Record Segment. Its structure is defined in Figure 2-3. Since its structure is defined explicitly in Figure 2-3, no Representation Code is assigned to it.

record_type: The *Logical Record Type* is a one-byte, unsigned integer (Representation Code USHORT) that specifies the Type of the Logical Record. Its value indicates the general semantic content of the Logical Record. The same value must be used in all Segments of a Logical Record. Logical Record Types are specified in Appendix A.

IFLRs: Numeric codes 0-127 are reserved for Public IFLRs. Codes 128-255 are reserved for Private IFLRs. 0 is Frame Data, 1 is unformatted data.

EFLRs: Numeric codes 0-127 are reserved for Public EFLRs. Codes 128-255 are reserved for Private EFLRs. 0 is FILE-HEADER, 1 is ORIGIN and so on.

read(*fobj: BinaryIO*) → None

Read a new Logical Record Segment Header. This may throw a `ExceptionVisibleRecord` or `ExceptionLogicalRecordSegmentHeaderEOF`.

as_bytes() → bytes

The LRSH represented in raw bytes.

__str__() → str
Return str(self).

property next_position: int
File position of the start of the next Logical Record Segment Header.

property logical_data_position: int
File position of the start of the Logical Data.

property logical_data_length
Returns the length of the logical data, including padding but excluding the tail.

__weakref__
list of weak references to the object (if defined)

class TotalDepth.RP66V1.core.pFile.**LogicalRecordPosition**(*vr: TotalDepth.RP66V1.core.pFile.VisibleRecord,*
lrsh: TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHeader)

Class that contains the file position of the Logical Record Segment Header and the immediately prior Visible Record.

__init__(*vr: TotalDepth.RP66V1.core.pFile.VisibleRecord, lrsh: TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHeader*)

__str__()
Return str(self).

__eq__(*other*)
Return self==value.

__weakref__
list of weak references to the object (if defined)

class TotalDepth.RP66V1.core.pFile.**LogicalDataDescription**(*attributes: TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHeaderAttributes,*
lr_type: int, ld_length: int)

At this level this describes the raw Logical Data that can be converted into a Logical Record.

attributes: TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHeaderAttributes
Alias for field number 0

lr_type: int
Alias for field number 1

ld_length: int
Alias for field number 2

__str__() → str
Return str(self).

__getnewargs__()
Return self as a plain tuple. Used by copy and pickle.

static **__new__**(*cls, attributes: TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHeaderAttributes,*
lr_type: int, ld_length: int)

__repr__()
Return a nicely formatted representation string

```
class TotalDepth.RP66V1.core.pFile.LRPosDesc(position:
                                             TotalDepth.RP66V1.core.pFile.LogicalRecordPosition,
                                             description:
                                             TotalDepth.RP66V1.core.pFile.LogicalDataDescription)
```

This contains the position and description of a Logical Record suitable for an indexer.

It contains:

- **LogicalRecordPosition:** This is the absolute file position of the Visible Record and LRSH. This will be of interest to indexers that mean to use `get_file_logical_data()` as this is a required argument.
- **LogicalDataDescription:** This provides some basic information about the Logical Data such as attributes Logical Record type and the Logical Data length. This will be of interest to indexers to offer up to their callers.

position: *TotalDepth.RP66V1.core.pFile.LogicalRecordPosition*

Alias for field number 0

description: *TotalDepth.RP66V1.core.pFile.LogicalDataDescription*

Alias for field number 1

__str__() → str
Return str(self).

__getnewargs__()
Return self as a plain tuple. Used by copy and pickle.

static __new__(*_cls, position:* *TotalDepth.RP66V1.core.pFile.LogicalRecordPosition, description:* *TotalDepth.RP66V1.core.pFile.LogicalDataDescription*)

__repr__()
Return a nicely formatted representation string

```
class TotalDepth.RP66V1.core.pFile.LogicalData(by: bytes)
```

Class that holds data bytes and can successively read them maintaining an index of what has been read.

__init__(*by: bytes*)

peek() → int
Return the next bytes without incrementing the index. May raise an IndexError if there is no data left.

read() → int
Return the next byte and increment the index. May raise an IndexError if there is no data left.

seek(*length: int*) → None
Increments the index. There is no error checking.

view_remaining(*length: int*) → bytes
Read only method to return a slice of length from the current index. Usage `ld.view_remaining(ld.remain)` to see all the remaining data.

chunk(*length: int*) → bytes
Return the next length bytes and increment the index. May raise an IndexError if there is not enough data.

property remain: int
The number of bytes remaining.

property sha1: `_hashlib.openssl_sha1`
Lazy SHA1 evaluation of the complete binary data.

rewind() → None
Reset the index to 0.

__bool__()

True if there is some data remaining.

__len__()

Total length of the binary data.

__getitem__(*index*)

Return a byte and the given index.

__str__() → str

String representation.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.RP66V1.core.pFile.**FileLogicalData**(*vr*: TotalDepth.RP66V1.core.pFile.VisibleRecord,
lrsh: TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHeader)

Class that contains information about a Logical Record within a physical file. This is lazily evaluated with only the VisibleRecord and LogicalRecordSegmentHeader provided to the constructor. Eager evaluation is done with one or more add()'s followed by a seal().

__init__(*vr*: TotalDepth.RP66V1.core.pFile.VisibleRecord, *lrsh*:
TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHeader)

add_bytes(*by*: bytes) → None

Add some raw data that is part of aa Logical Record.

seal() → None

All of the Logical Record has been read into this class so seal it to prevent any more data being added. This also creates a LogicalData object that encapsulates the logical data.

is_sealed() → bool

Returns True if this is sealed so no more bytes can be added.

__len__() → int

Number of bytes of data whether sealed or unsealed.

__str__() → str

Return str(self).

__weakref__

list of weak references to the object (if defined)

class TotalDepth.RP66V1.core.pFile.**FileRead**(*path_or_file*: Union[str, BinaryIO])
RP66V1 file reader.

__init__(*path_or_file*: Union[str, BinaryIO])

iter_visible_records() → Sequence[TotalDepth.RP66V1.core.pFile.VisibleRecord]

Iterate across the file yielding the Visible Records as VisibleRecord objects. The iteration can be further divided by calling iter_LRSHs_for_VR()

iter_LRSHs_for_visible_record(*vr_given*: TotalDepth.RP66V1.core.pFile.VisibleRecord) → Sequence[TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHeader]

Iterate across the Visible Record yielding the Logical Record Segments as LogicalRecordSegmentHeader objects. This leaves the file positioned at the next Visible Record or EOF.

iter_LRSHs_for_visible_record_and_logical_data_fragment (*vr_given: TotalDepth.RP66V1.core.pFile.VisibleRecord*)
 → Sequence[Tuple[TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHeader, bytes]]

Iterate across the Visible Record yielding the Logical Record Segments and the Logical Data fragment as (LogicalRecordSegmentHeader, bytes) objects. This leaves the file positioned at the next Visible Record or EOF. TODO: Drop this from pFile/cFile as it is only used in one rare case in Scan.py?

__weakref__

list of weak references to the object (if defined)

iter_logical_records() → Sequence[TotalDepth.RP66V1.core.pFile.FileLogicalData]

Iterate across the file from the beginning yielding FileLogicalData objects.

iter_logical_record_positions() → Sequence[TotalDepth.RP66V1.core.pFile.LRPosDesc]

Iterate across the file from the beginning yielding a LRPosDesc which contains:

- **LogicalRecordPosition:** This is the absolute file position of the Visible Record and LRSH. This will be of interest to indexers that mean to use `get_file_logical_data()` as this is a required argument.
- **LogicalDataDescription:** This provides some basic information about the Logical Data such as attributes Logical Record type and the Logical Data length. This will be of interest to indexers to offer up to their callers.

get_file_logical_data (*position: TotalDepth.RP66V1.core.pFile.LogicalRecordPosition, offset: int = 0, length: int = -1*) → TotalDepth.RP66V1.core.pFile.FileLogicalData

Returns a FileLogicalData object from the Logic Record position (Visible Record Position and Logical Record Segment Header position). This allows random access to the file to an index that has the Logical Record Positions. This will leave the file at EOF or at the beginning of the next Visible Record or LRSH.

Param position A LogicalRecordPosition that specifies the visible record and LRSH position of the first LRSH for the Logical Record data.

Param offset An integer offset into the Logical Record data, default 0.

Param length An integer length the required Logical Record data, default of -1 is all.

validate_positions() → None

Iterate through the Visible Records and Logical Record Segment Headers and raise a ExceptionFileReadPositionsInconsistent on the first inconsistent position.

6.3 TotalDepth.RP66V1.core.Index

RP66V1 file indexer at the level of Visible Records and the first Logical Record Segment Header.

This allows random access of Logical Record data.

In the taxonomy of indexes this is a 'mid level' index as it indexes:

- Below: Internally it discovers and records Visible Records and Logical Record Segment Headers (where `is_first()` is True).
- Above: Externally it provides an API to a sequence of Logical Records and the data that makes up those Logical Records.

TODO: Replace this with the C/C++ implementation.

exception TotalDepth.RP66V1.core.pIndex.ExceptionIndex

Base class for exceptions in this module.

class TotalDepth.RP66V1.core.pIndex.LogicalRecordIndex(*path_or_file: Union[str, _io.BytesIO]*)

This maintains an index of visible record and Logical Record Segment Header (LRSH) positions where the LRSH is the first in the Logical Record.

The index is a list of File.LRPosDesc objects that contain:

- **.position** A LogicalRecordPosition which has the absolute file position of the Visible Record and LRSH. This will be of interest to indexers that mean to use `get_file_logical_data()` as this is a required argument.
- **.description** A LogicalDataDescription which provides some basic information about the Logical Data such as the LRSH attributes, Logical Record type and the Logical Data length. This will be of interest to indexers to offer up to their callers.

__init__(*path_or_file: Union[str, _io.BytesIO]*)

property sul: TotalDepth.RP66V1.core.pFile.StorageUnitLabel

The file's Storage Unit Label.

property visible_record_positions: List[int]

A list of Visible Record positions. This is used by the XML index for example.

get_file_logical_data(*index: int, offset: int = 0, length: int = - 1*) →

TotalDepth.RP66V1.core.pFile.FileLogicalData

Returns a FileLogicalData object from the Logical Record position in the index (Visible Record Position and Logical Record Segment Header position).

This allows random access to any Logical Record in the file. The caller can construct a more sophisticated index such as a sequence of Logical Files which contain Logical Records that can be EFLRs or IFLRs and interpreted accordingly.

If offset or length are use then the result will be the partial data from that offset and length.

Param index The index of the Logical Record.

Param offset An integer offset into the Logical Record data, default 0.

Param length An integer length the Logical Record data, default of -1 is all.

get_file_logical_data_at_position(*position: TotalDepth.RP66V1.core.pFile.LogicalRecordPosition, offset: int = 0, length: int = - 1*) →

TotalDepth.RP66V1.core.pFile.FileLogicalData

Returns a FileLogicalData object from the Logical Record position.

This allows random access to any Logical Record in the file. The caller can construct a more sophisticated index such as a sequence of Logical Files which contain Logical Records that can be EFLRs or IFLRs and interpreted accordingly.

If offset or length are use then the result will be the partial data from that offset and length.

Param position The Logical Record position in the file.

Param offset An integer offset into the Logical Record data, default 0.

Param length An integer length the Logical Record data, default of -1 is all.

validate()

Perform validation checks.

__weakref__

list of weak references to the object (if defined)

6.4 TotalDepth.RP66V1.core.LogPass

This provides a representation of the structure of recorded data.

exception TotalDepth.RP66V1.core.LogPass.ExceptionLogPass

exception TotalDepth.RP66V1.core.LogPass.ExceptionFrameChannel

exception TotalDepth.RP66V1.core.LogPass.ExceptionFrameArray

exception TotalDepth.RP66V1.core.LogPass.ExceptionFrameArrayInit

exception TotalDepth.RP66V1.core.LogPass.ExceptionLogPassInit

exception TotalDepth.RP66V1.core.LogPass.ExceptionLogPassProcessIFLR

class TotalDepth.RP66V1.core.LogPass.RP66V1FrameChannel(*ident: Hashable, long_name: bytes, units: bytes, dimensions: List[int], np_dtype: numpy.dtype, rep_code: int*)

This represents a single channel in a frame. It is file format independent and can be used depending on the source of the information: LIS/LAS/RP66V1 file, XML index, Postgres database etc.

__init__(*ident: Hashable, long_name: bytes, units: bytes, dimensions: List[int], np_dtype: numpy.dtype, rep_code: int*)

Constructor.

Parameters

- **ident** – Some hashable identity.
- **long_name** – A description of the channel
- **units** – Units of Measure.
- **dimensions** – A list of dimensions of each value. [1] is a single value per frame. [4, 1024] is a 4 * 1024 matrix such as sonic waveform of 1024 samples with 4 waveforms per frame.
- **np_dtype** – The numpy dtype to use.
- **rep_code** – Integer representation code that this channel is encoded in.

property ident: Hashable

Overload base class.

property len_input_bytes: int

The number of RP66V1 bytes to read for one frame of this channel.

read(*ld: TotalDepth.RP66V1.core.pFile.LogicalData, frame_number: int*) → None

Reads the Logical Data into the numpy frame at the specified frame number.

This is currently RP66V1 specific. In future designs this can be sub-classed by format (LAS, LIS, RP66V1 etc.)

seek(*ld: TotalDepth.RP66V1.core.pFile.LogicalData*) → None

Increments the logical data without reading any values into the array.

class TotalDepth.RP66V1.core.LogPass.RP66V1FrameArray(*ident: Hashable, description: Union[str, bytes]*)

In the olden days we would record this on a single chunk of continuous film.

read(*ld: TotalDepth.RP66V1.core.pFile.LogicalData, frame_number: int*) → None

Reads the Logical Data into the numpy frame.

read_partial(*ld*: TotalDepth.RP66V1.core.pFile.LogicalData, *frame_number*: int, *channels*: Set[Hashable]) → None

Reads the Logical Data into the numpy frame for the nominated channels.

property x_axis_len_input_bytes: int

The number of RP66V1 bytes to read for one frame of the X axis. This can be useful for partial reads of an IFLR from file if only the X axis is interesting, for example for indexing.

Will raise an ExceptionFrameChannel if the X axis is not represented by a fixed length Representation Code.

init_x_axis_array(*number_of_frames*: int) → None

Initialises an empty Numpy array for the first channel suitable to fill with <frames> number of frame data.

read_x_axis(*ld*: TotalDepth.RP66V1.core.pFile.LogicalData, *frame_number*: int) → None

Reads the first channel of the Logical Data into the numpy frame.

TotalDepth.RP66V1.core.LogPass.**frame_channel_from_RP66V1**(*channel_object*: TotalDepth.RP66V1.core.LogicalRecord.EFLR.Object) → TotalDepth.RP66V1.core.LogPass.RP66V1FrameChannel

Create a file format agnostic FrameChannel from an EFLR.Object (row) in a CHANNEL EFLR.

TotalDepth.RP66V1.core.LogPass.**frame_array_from_RP66V1**(*frame_object*: TotalDepth.RP66V1.core.LogicalRecord.EFLR.Object, *channel_eflr*: TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormatted) → TotalDepth.RP66V1.core.LogPass.RP66V1FrameArray

Create a file format agnostic FrameArray from an EFLR.Object (row) in a FRAME EFLR and a CHANNEL EFLR.

TotalDepth.RP66V1.core.LogPass.**log_pass_from_RP66V1**(*frame*: TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormattedL, *channels*: TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormattedL) → TotalDepth.common.LogPass.LogPass

Create a file format agnostic FrameArray from a FRAME type EFLR and a CHANNEL type EFLR.

6.5 TotalDepth.RP66V1.core.LogicalFile

Represents a RP66V1 file as a ‘logical’ level.

exception TotalDepth.RP66V1.core.LogicalFile.ExceptionLogicalFile

exception TotalDepth.RP66V1.core.LogicalFile.ExceptionLogicalFileCtor

exception TotalDepth.RP66V1.core.LogicalFile.ExceptionLogicalFileAdd

exception TotalDepth.RP66V1.core.LogicalFile.ExceptionLogicalFileMissingData

exception TotalDepth.RP66V1.core.LogicalFile.ExceptionLogicalIndex

exception TotalDepth.RP66V1.core.LogicalFile.ExceptionLogicalIndexCtor

```

class TotalDepth.RP66V1.core.LogicalFile.PositionEFLR(lrsh_position: To-
                                                    talDepth.RP66V1.core.pFile.LogicalRecordPosition,
                                                    eflr: To-
                                                    talDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormattedLogicalRecord)
    POD class that represents the Logical Record Segment Header position in the file of the Explicitly Formatted
    Logical Record and the EFLR itself.

    lrsh_position: TotalDepth.RP66V1.core.pFile.LogicalRecordPosition
        Alias for field number 0

    eflr: TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormattedLogicalRecord
        Alias for field number 1

    __getnewargs__()
        Return self as a plain tuple. Used by copy and pickle.

    static __new__(_cls, lrsh_position: TotalDepth.RP66V1.core.pFile.LogicalRecordPosition, eflr:
        TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormattedLogicalRecord)

    __repr__()
        Return a nicely formatted representation string

```

```

class TotalDepth.RP66V1.core.LogicalFile.LogicalFile(logical_record_index: To-
                                                    talDepth.RP66V1.core.pIndex.LogicalRecordIndex,
                                                    file_logical_data: To-
                                                    talDepth.RP66V1.core.pFile.FileLogicalData,
                                                    fhlr: To-
                                                    talDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormattedLogicalRecord)

```

This represents a RP66V1 Logical File.

From the standard [RP66V1 Definitions]:

Logical File A sequence of two or more contiguous Logical Records in a Storage Set that begins with a File Header Logical Record and contains no other File Header Logical Records. A Logical File must have at least one OLR (Origin) Logical Record immediately following the File Header Logical Record. A Logical File supports user-level organization of data.

For the File Header Logical Record see [RP66V1 Section 5.1 File Header Logical Record (FHLR)] For the Origin Logical Record see [RP66V1 Section 5.2 Origin Logical Record (OLR)]

This is actually two/multi stage construction with the FHLR first. The OLR is extracted from the first add().

TODO: Check the FHLR references the ORIGIN record. [RP66V1 Section 5.1 File Header Logical Record (FHLR)]: “The Origin Subfield of the Name of the File-Header Object must reference the Defining Origin (see §5.2.1).” Is this done in practice?

TODO: Handle multiple ORIGIN records. This is actually allowed in the standard: “A Logical File must have at least one OLR (Origin) Logical Record immediately following the File Header Logical Record”.

TODO: Handle multiple CHANNEL records in a Logical File.

```

DUPE_EFLR_CHANNEL_LOGGER(msg, *args, **kwargs)

```

Log ‘msg % args’ with severity ‘WARNING’.

To pass exception information, use the keyword argument exc_info with a true value, e.g.

```
logger.warning(“Houston, we have a %s”, “bit of a problem”, exc_info=1)
```

```

__init__(logical_record_index: TotalDepth.RP66V1.core.pIndex.LogicalRecordIndex, file_logical_data:
        TotalDepth.RP66V1.core.pFile.FileLogicalData, fhlr:
        TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormattedLogicalRecord)

```

property file_header_logical_record:

TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormattedLogicalRecord

Returns the FILE-HEADER EFLR see [RP66V1 Section 5.1 File Header Logical Record (FHLR)].

property origin_logical_record:

TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormattedLogicalRecord

Returns the ORIGIN EFLR see [RP66V1 Section 5.2 Origin Logical Record (OLR)].

property defining_origin: TotalDepth.RP66V1.core.LogicalRecord.EFLR.Object

Returns the Defining Origin of the Logical File. This is the first row of the ORIGIN Logical Record. From [RP66V1 Section 5.2.1 Origin Objects]: “The first Object in the first ORIGIN Set is the Defining Origin for the Logical File in which it is contained, and the corresponding Logical File is called the Origin’s Parent File. It is intended that no two Logical Files will ever have Defining Origins with all Attribute Values identical.”

property has_log_pass: bool

Return True if a log pass has been created from a CHANNEL and a FRAME record and some relevant IFLRs.

static is_next(*eflr*: [TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormattedLogicalRecord](#))
→ bool

Returns True if the given EFLR belongs to the next Logical Record.

add_eflr(*file_logical_data*: [TotalDepth.RP66V1.core.pFile.FileLogicalData](#), *eflr*:
[TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormattedLogicalRecord](#)) → None

Adds an EFLR in sequence from the file.

Will raise a `ExceptionLogicalFileAdd` if the EFLR is a FILE-HEADER as that signals the next Logical File.

add_iflr(*file_logical_data*: [TotalDepth.RP66V1.core.pFile.FileLogicalData](#), *iflr*:
[TotalDepth.RP66V1.core.LogicalRecord.IFLR.IndirectlyFormattedLogicalRecord](#)) → None

Adds a IFLR entry to the index. The IFLR just contains the object name and frame number. This extracts the X axis from the first value in the IFLR free data and appends this to the `iflr_position_map`.

num_frames(*frame_array*: [TotalDepth.RP66V1.core.LogPass.RP66V1FrameArray](#)) → int

Return the number of frames in the FrameArray

populate_frame_array(*frame_array*: [TotalDepth.RP66V1.core.LogPass.RP66V1FrameArray](#), *frame_slice*:
Optional[Union[TotalDepth.common.Slice.Slice,
TotalDepth.common.Slice.Sample]] = None, *channels*: *Optional[Set[Hashable]] =*
None) → int

Populates a FrameArray with channel values.

`frame_array` must be a member of the LogPass in this LogicalFile.

`frame_slice` Allows partial population in the X axis.

`channels` Allows partial population of specific channels.

The FrameArray will be populated and this returns the number of frames populated.

__weakref__

list of weak references to the object (if defined)

class [TotalDepth.RP66V1.core.LogicalFile.LogicalIndex](#)(*path_or_file*: *Union[str, BinaryIO]*)

This takes a RP66V1 file and indexes it into a sequence of Logical Files.

__init__(*path_or_file*: *Union[str, BinaryIO]*)

__len__() → int

Returns the number of Logical Files.

__getitem__(*item*) → *TotalDepth.RP66V1.core.LogicalFile.LogicalFile*

Returns the Logical Files at position *item*.

property storage_unit_label: *TotalDepth.RP66V1.core.pFile.StorageUnitLabel*

The Storage Unit Label. This comes from the LogicalRecordIndex.

property visible_record_positions: *List[int]*

A list of Visible Record positions. This is used by the XML index for example.

__enter__()

Context manager support.

__exit__(*exc_type*, *exc_val*, *exc_tb*)

Context manager support.

__weakref__

list of weak references to the object (if defined)

6.6 TotalDepth.RP66V1.core.RepCode

6.7 TotalDepth.RP66V1.core.Units

Handles RP66V1 units.

There are two approaches:

6.7.1 Standard Based Parser

Formal but slow and does not appreciate that there are a load of non-standard implementations out there.

References: [RP66V1 Appendix b, B.27 Code UNITS: Units Expression] [“Energetics Unit Symbol Grammar Specification” Section 2.2 Unit Symbol Construction Grammar]

```
UnitSymbol ::= [ Multiplier ' ' ] FactorExpression .

FactorExpression ::= OneOrMore |
    ( '1' | OneOrMore | Division ) '/' (
Division ::= '(' OneOrMore '/' Divisor ')'

OneOrMore ::= Factor | Factors .

Divisor ::= Factor | '(' Factors ')' .

Factors ::= Factor '.' Factor { '.' Factor

Factor ::= UnitComponent [ Exponent ] .

UnitComponent ::= PrefixedAtom | Atom | SpecialAtom [ Qualifier ] .

PrefixedAtom ::= ( SIPrefix | BinaryPrefix ) Atom .

Atom ::= Letter { Letter } [ Qualifier ] .
```

(continues on next page)

(continued from previous page)

```

SpecialAtom ::= '%' | 'inH2O' | 'cmH2O' .

Qualifier ::= '[' [ AT ] QualPart { COMMA QualPart } ']' .

AT ::= '@' .

COMMA ::= ',' .

QualPart ::= LetterOrDigit { LetterOrDigit

LetterOrDigit ::= Letter | Digit .

Letter ::= E | LTTR .

LTTR ::= 'A' | 'B' | 'C' | 'D' | 'L' | 'M' | 'N' | 'O' | 'W' | 'X' | 'Y' | 'Z' | 'a' | 'b' |
↳ 'c' | 'd' | 'l' | 'm' | 'n'
    | 'o' | 'w' | 'x' | 'y' | 'z' . 'P' | 'e' | 'p' | } .
    } .
    'F' | 'G' | 'H' | 'Q' | 'R' | 'S'
    'f' | 'g' | 'h'
    'q' | 'r' | 's'
    | 'I' | 'J' | 'K' |
    | 'T' | 'U' | 'V' |
    | 'i' | 'j' | 'k' |
    | 't' | 'u' | 'v' |

Exponent ::= GtOneDigit | '(' ( NonZeroInt '.' FractionalPart | '0' '.' FractionalPart )
↳ ')' .

Multiplier ::= '1' E PowerOfTen [ '/' GtOneInt ] | '1' '/' GtOneInt |
    Number [ E PowerOfTen ] [ '/' GtOneInt ] .

E ::= 'E' .

PowerOfTen ::= [ '-' ] GtOneInt .

Number ::= GtOneInt |
    NonZeroInt '.' FractionalPart | '0' '.' FractionalPart .

GtOneInt ::= GtOneDigit | NonZeroDigit Digit { Digit } .

FractionalPart ::= { Digit } NonZeroDigit .

NonZeroInt ::= NonZeroDigit { Digit } .

Digit ::= '0' | NonZeroDigit .

GtOneDigit ::= '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9' .

NonZeroDigit ::= '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9' .

```

(continues on next page)

(continued from previous page)

```

SIPrefix ::= 'y' | 'z' | 'a' | 'f' | 'p' | 'n' | 'u' | 'm' | 'c' | 'd' |
'da' | 'h' | 'k' | 'M' | 'G' | 'T' | 'P' | 'E' | 'Z' | 'Y' .

BinaryPrefix ::= 'Ki' | 'Mi' | 'Gi' | 'Ti' | 'Pi' | 'Ei' | 'Zi' | 'Yi' .

```

Something like this:

```

import re
import typing

# Token = collections.namedtuple('Token', ['type', 'value', 'line', 'column'])
class Token(typing.NamedTuple):
    type: str
    value: str
    line: int
    column: int

def tokenise_units(code: str) -> typing.Sequence[Token]:
    # See also [RP66V1 Appendix B, B.27 Code UNITS: Units Expression]
    token_specification = [
        ('BinaryPrefix', r'Ki|Mi|Gi|Ti|Pi|Ei|Zi|Yi'),
        ('SiPrefix', r'y|z|a|f|p|n|u|m|c|d|da|h|k|M|G|T|P|E|Z|Y'),
        ('NonZeroDigit', r'1|2|3|4|5|6|7|8|9'),
        ('Digit', r'0|1|2|3|4|5|6|7|8|9'),
        ('GtOneDigit', r'2|3|4|5|6|7|8|9'),
        ('E', r'E'),
        ('Letter', r'[A-Za-z]'),
        ('COMMA', r','),
        ('AT', r'\@'),
        ('SpecialAtom', r'%|inH2O|cmH2O'),
        ('PARENLEFT', r'\('),
        ('PARENRIGHT', r'\)'),
        ('MULTIPLY', r'\*'),
        ('DIVIDE', r'/'),
        ('BLANK', r' '),
        ('DOT', r'.'),
        ('HYPHEN', r'-'),
        ('NEWLINE', r'\n'),
        ('ID', r'[A-Za-z]+'),
        ('MISSMATCH', r'.'),
    ]
    tok_regex = '|'.join('(?P<%s>%s)' % pair for pair in token_specification)
    line_num = 1
    line_start = 0
    for mo in re.finditer(tok_regex, code):
        kind = mo.lastgroup
        value = mo.group()
        column = mo.start() - line_start
        if kind == 'NUMBER':
            value = float(value) if '.' in value else int(value)

```

(continues on next page)

(continued from previous page)

```

    # elif kind == 'ID' and value in keywords:
    #     kind = value
    # elif kind == 'NEWLINE':
    #     line_start = mo.end()
    #     line_num += 1
    #     continue
    # elif kind == 'SKIP':
    #     continue
    elif kind == 'MISMATCH':
        raise RuntimeError(f'{value!r} unexpected on line {line_num}')
    yield Token(kind, value, line_num, column)

for token in tokenise_units('0.1m/s2'):
for token in tokenise_units('627264E5/15499969 m2'):
    print(token)

```

This is likely to be slow, some pre-processing may help.

6.7.2 Lookup

This uses online or offline data structures. The primary source is Schlumberger's Oilfield Services Data Dictionary (OSDD): <https://www.apps.slb.com/cmd/units.aspx> It is quick and largely respects existing implementations.

Other data providers (by PRODUCER-CODE) may have alternate mappings that can be put into PRODUCER_CODE_MAPPING_OF_UNIT_CODE.

See `src/TotalDepth/TP66V1/util/XMLReadUnits.py` for some analysis.

exception `TotalDepth.RP66V1.core.Units.ExceptionRP66V1Units`

Base class exception for this module.

`TotalDepth.RP66V1.core.Units.PRODUCER_CODE_MAPPING_OF_UNIT_CODE: Dict[int, Dict[bytes, bytes]] = {280: {b'gapi': b'GAPI', b'ltrs': b'dm3', b'sec': b'SEC'}}`

This permits different producer codes to map into Schlumberger's Oilfield Services Data Dictionary (OSDD).

This is just an example, see `src/TotalDepth/TP66V1/util/XMLReadUnits.py` for some analysis of test files.

`TotalDepth.RP66V1.core.Units.convert(value: float, unit_from: bytes, unit_to: bytes, producer_code: int = 0) → float`

Converts a value from one unit to another. This uses `TotalDepth.common.units` with an additional producer code mapping.

Examples:

Code	Name	Standard Form	Dimension	Scale	Offset
DEGC	'degree celsius'	degC	Temperature	1	-273.15
DEGF	'degree fahrenheit'	degF	Temperature	0.5555555555555556	-459.67

So conversion from, say DEGC to DEGF is:

```

((value - DEGC.offset) * DEGC.scale) / DEGF.scale + DEGF.offset

((0.0 - -273.15) * 1.0) / 0.5555555555555556 + -459.67 == 32.0

```

`TotalDepth.RP66V1.core.Units.convert_function(unit_from: TotalDepth.common.units.Unit, unit_to: TotalDepth.common.units.Unit, producer_code: int = 0) → Callable`

Return a partial function to convert from one RP66V1 units to another.

6.8 TotalDepth.RP66V1.core.XAxis

Provides analysis and navigation along the X axis of RP66V1 logs. RP66V1 specific code based on TotalDepth/common/XAxis.py

class `TotalDepth.RP66V1.core.XAxis.IFLRReference(logical_record_position: TotalDepth.RP66V1.core.pFile.LogicalRecordPosition, frame_number: int, x_axis: Union[int, float])`

POD class that represents the position of the IFLR in the file.

logical_record_position: `TotalDepth.RP66V1.core.pFile.LogicalRecordPosition`

Alias for field number 0

frame_number: `int`

Alias for field number 1

x_axis: `Union[int, float]`

Alias for field number 2

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static **__new__**(*_cls*, *logical_record_position*: TotalDepth.RP66V1.core.pFile.LogicalRecordPosition, *frame_number*: int, *x_axis*: Union[int, float])

__repr__()

Return a nicely formatted representation string

class `TotalDepth.RP66V1.core.XAxis.XAxis(ident: bytes, long_name: bytes, units: bytes)`

This represents an X axis of a log pass for a particular object in that log pass. It has an ident, long name and units. It accumulates, for every IFLR in the set, the VR position LRSH position, frame number and X axis value.

__init__(*ident*: bytes, *long_name*: bytes, *units*: bytes)

append(*position*: TotalDepth.RP66V1.core.pFile.LogicalRecordPosition, *frame_number*: int, *x_axis*: Union[int, float]) → None

Add a IFLRReference to the XAxis.

__getitem__(*item*) → TotalDepth.RP66V1.core.XAxis.IFLRReference

Return the IFLRReference for the index.

__len__() → int

Return the number of IFLRs.

property summary: `TotalDepth.common.XAxis.XAxisSummary`

Lazily compute the summary.

__weakref__

list of weak references to the object (if defined)

6.9 TotalDepth.RP66V1.core.stringify

TotalDepth.RP66V1.core.stringify.**stringify_object_by_type**(*obj: Any*) → str
Convert objects to strings for HTML or text presentation.

6.10 TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor

Implements the Component Descriptor [RP66V1 Section 3.2.2.1 Component Descriptor]

References:

RP66V1: <http://w3.energistics.org/rp66/v1/rp66v1.html>

Specifically section 3:

http://w3.energistics.org/rp66/v1/rp66v1_sec3.html

exception

TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.**ExceptionComponentDescriptor**
General Exception class for Component Descriptor errors.

exception

TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.**ExceptionComponentDescriptorInit**
Exception class for Component Descriptor `__init__` errors.

exception TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.

ExceptionComponentDescriptorAccessError

Exception class for Component Descriptor data access errors.

class TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.**RoleType**(*role: str, type: str*)
Contains the role and type such as ('ABSATR', 'Absent Attribute')

role: str

Alias for field number 0

type: str

Alias for field number 1

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static __new__(_cls, role: str, type: str)

__repr__()

Return a nicely formatted representation string

class TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.**CharacteristicRepCodeGlobalDefault**(*characteristic: str, rep_code: Optional[str], global_default: Any*)

Contains the characteristic, rep_code and global_default such as ('Name', 'IDENT', b'')

characteristic: str

Alias for field number 0

rep_code: Optional[str]

Alias for field number 1

global_default: Any

Alias for field number 2

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static __new__(_cls, characteristic: str, rep_code: Optional[str], global_default: Any)

__repr__()

Return a nicely formatted representation string

class TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.**ComponentDescriptor**(*descriptor: int*)

Component Descriptor that extracts the necessary attributes from an integer.

See: [RP66V1 Section 3.2.2.1 Component Descriptor]

__init__(*descriptor: int*)

__eq__(*other*)

Return self==value.

__str__()

Return str(self).

property role: str

The Component Descriptor role.

property type: str

The Component Descriptor type.

property is_attribute_group: bool

Is an attribute, absent attribute or invariant attribute.

property is_set_group: bool

Is a set, redundant set or replacement set.

property is_absent_attribute: bool

Is an absent attribute.

property is_attribute: bool

Is an attribute.

property is_invariant_attribute: bool

Is an invariant attribute.

property is_object: bool

Is an object.

property is_redundant_set: bool

Is a redundant set.

property is_replacement_set: bool

Is a replacement set.

property is_set: bool

Is a set.

property has_set_T: int

Has a type, must be in the set group otherwise an ExceptionComponentDescriptorAccessError will be raised.

property has_set_N: int

Has a name, must be in the set group otherwise an `ExceptionComponentDescriptorAccessError` will be raised.

property has_object_N: int

Has a name, must be an object otherwise an `ExceptionComponentDescriptorAccessError` will be raised.

property has_attribute_L: int

Has a label, must be in the attribute group otherwise an `ExceptionComponentDescriptorAccessError` will be raised.

property has_attribute_C: int

Has a count, must be in the attribute group otherwise an `ExceptionComponentDescriptorAccessError` will be raised.

property has_attribute_R: int

Has a Representation Code, must be in the attribute group otherwise an `ExceptionComponentDescriptorAccessError` will be raised.

property has_attribute_U: int

Has units, must be in the attribute group otherwise an `ExceptionComponentDescriptorAccessError` will be raised.

property has_attribute_V: int

Has a value, must be in the attribute group otherwise an `ExceptionComponentDescriptorAccessError` will be raised.

__weakref__

list of weak references to the object (if defined)

6.11 TotalDepth.RP66V1.core.LogicalRecord.EFLR

Implements the Explicitly Formatted Logical Record (EFLR) [RP66V1 Section 3 Logical Record Syntax]

References: RP66V1: <http://w3.energistics.org/rp66/v1/rp66v1.html>

Specifically section 3: http://w3.energistics.org/rp66/v1/rp66v1_sec3.html

exception TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExceptionEFLR

General Exception class for EFLR errors.

exception TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExceptionEFLRSet

Exception class for EFLR Set errors.

exception TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExceptionEFLRSetDuplicateObjectNames

Exception class for EFLR Set with duplicate object names.

exception TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExceptionEFLRAttribute

Exception class for EFLR Attribute errors.

exception TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExceptionEFLRTemplate

Exception class for EFLR Template errors.

exception TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExceptionEFLRTemplateDuplicateLabel

Exception class for EFLR Template with duplicate object names.

exception TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExceptionEFLRObject

Exception class for EFLR Object errors.

exception TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExceptionEFLRObjectDuplicateLabel

Exception class for EFLR Object with duplicate labels.

class TotalDepth.RP66V1.core.LogicalRecord.EFLR.**Set**(*ld*: TotalDepth.RP66V1.core.pFile.LogicalData)
 Class that represents a component set. See [RP66V1 3.2.2.1 Component Descriptor]

__init__(*ld*: TotalDepth.RP66V1.core.pFile.LogicalData)

__str__() → str
 String representation.

__eq__(*other*) → bool
 Equality operator.

__weakref__
 list of weak references to the object (if defined)

class TotalDepth.RP66V1.core.LogicalRecord.EFLR.**AttributeBase**(*component_descriptor*: TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor)
 Class that represents a component attribute. See [RP66V1 3.2.2.1 Component Descriptor]

__init__(*component_descriptor*: TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.ComponentDescriptor)

__eq__(*other*)
 Equality operator.

__str__() → str
 String representation.

stringify_value(*stringify_function*: Callable) → str
 Return the value as a string.

__weakref__
 list of weak references to the object (if defined)

class TotalDepth.RP66V1.core.LogicalRecord.EFLR.**TemplateAttribute**(*component_descriptor*: TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor, *ld*: TotalDepth.RP66V1.core.pFile.LogicalData)
 Class that represents a component template. See [RP66V1 3.2.2.1 Component Descriptor]

__init__(*component_descriptor*: TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.ComponentDescriptor, *ld*: TotalDepth.RP66V1.core.pFile.LogicalData)

class TotalDepth.RP66V1.core.LogicalRecord.EFLR.**Attribute**(*component_descriptor*: TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor, *ld*: TotalDepth.RP66V1.core.pFile.LogicalData, *template_attribute*: TotalDepth.RP66V1.core.LogicalRecord.EFLR.TemplateAttribute)
 Class that represents a component attribute. See [RP66V1 3.2.2.1 Component Descriptor]

__init__(*component_descriptor*: TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.ComponentDescriptor, *ld*: TotalDepth.RP66V1.core.pFile.LogicalData, *template_attribute*: TotalDepth.RP66V1.core.LogicalRecord.EFLR.TemplateAttribute)

class TotalDepth.RP66V1.core.LogicalRecord.EFLR.**Template**
 Class that represents a component template. See [RP66V1 3.2.2.1 Component Descriptor]

__init__()

read(*ld*: TotalDepth.RP66V1.core.pFile.LogicalData)

Populate the template with the Logical Data.

__len__() → int

Return the number of columns described by this Template.

__getitem__(*item*) → TotalDepth.RP66V1.core.LogicalRecord.EFLR.TemplateAttribute

Get a TemplateAttribute by name or integer index.

__eq__(*other*) → bool

Equality operator.

__str__() → str

String representation.

header_as_strings(*stringify_function*: Callable) → List[str]

Return the TemplateAttributes as strings.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.RP66V1.core.LogicalRecord.EFLR.Object(*ld*:

TotalDepth.RP66V1.core.pFile.LogicalData,

template: To-

talDepth.RP66V1.core.LogicalRecord.EFLR.Template)

Class that represents a component object. See [RP66V1 3.2.2.1 Component Descriptor]. Essentially this is one row in the table as a list of Attributes.

__init__(*ld*: TotalDepth.RP66V1.core.pFile.LogicalData, *template*:

TotalDepth.RP66V1.core.LogicalRecord.EFLR.Template)

__len__() → int

Return the number of attributes (columns) for this row.

__getitem__(*item*) → Optional[TotalDepth.RP66V1.core.LogicalRecord.EFLR.AttributeBase]

Get an Attribute (column) by name or integer index.

__eq__(*other*) → bool

Equality operator.

__str__() → str

String representation.

values_as_strings(*stringify_function*: Callable) → List[str]

Return the Attribute values as strings.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.RP66V1.core.LogicalRecord.EFLR.PublicEFLRType(*code*: int, *type*: bytes, *description*:

str, *allowable_set_types*:

Set[bytes])

From [RP66V1 Appendix A: Logical Record Types] Figure A-2. Numeric Codes for Public EFLR Types.

code: int

Alias for field number 0

type: bytes

Alias for field number 1

description: str

Alias for field number 2

allowable_set_types: [Set\[bytes\]](#)

Alias for field number 3

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static **__new__**(*_cls, code: int, type: bytes, description: str, allowable_set_types: Set[bytes]*)

__repr__()

Return a nicely formatted representation string

```
TotalDepth.RP66V1.core.LogicalRecord.EFLR.PUBLIC_EFLR_TYPES: Dict[int,
TotalDepth.RP66V1.core.LogicalRecord.EFLR.PublicEFLRType] = {0: PublicEFLRType(code=0,
type=b'FHLR', description='File header', allowable_set_types={b'FILE-HEADER'}), 1:
PublicEFLRType(code=1, type=b'OLR', description='Origin',
allowable_set_types={b'WELL-REFERENCE', b'ORIGIN'}), 2: PublicEFLRType(code=2,
type=b'AXIS', description='Coordinate Axis', allowable_set_types={b'AXIS'}), 3:
PublicEFLRType(code=3, type=b'CHANNL', description='Channel-related information',
allowable_set_types={b'CHANNEL'}), 4: PublicEFLRType(code=4, type=b'FRAME',
description='Frame Data', allowable_set_types={b'PATH', b'FRAME'}), 5:
PublicEFLRType(code=5, type=b'STATIC', description='Static Data',
allowable_set_types={b'EQUIPMENT', b'CALIBRATION-COEFFICIENT',
b'CALIBRATION-MEASUREMENT', b'CALIBRATION', b'PROCESS', b'TOOL', b'ZONE', b'GROUP',
b'SPICE', b'PARAMETER', b'COMPUTATION'}), 6: PublicEFLRType(code=6, type=b'SCRIPT',
description='Textual Data', allowable_set_types={b'MESSAGE', b'COMMENT'}), 7:
PublicEFLRType(code=7, type=b'UPDATE', description='Update Data',
allowable_set_types={b'UPDATE'}), 8: PublicEFLRType(code=8, type=b'UDI',
description='Unformatted Data Identifier', allowable_set_types={b'NO-FORMAT'}), 9:
PublicEFLRType(code=9, type=b'LNAME', description='Long Name',
allowable_set_types={b'LONG-NAME'}), 10: PublicEFLRType(code=10, type=b'SPEC',
description='Specification', allowable_set_types={b'ATTRIBUTE', b'EFLR', b'UNIT-SYMBOL',
b'IFLR', b'CODE', b'OBJECT-TYPE', b'REPRESENTATION-CODE', b'SPECIFICATION'}), 11:
PublicEFLRType(code=11, type=b'DICT', description='Dictionary',
allowable_set_types={b'OPTION', b'BASE-DICTIONARY', b'IDENTIFIER', b'LEXICON'})}
```

From [RP66V1 Appendix A: Logical Record Types] Figure A-2. Numeric Codes for Public EFLR Types

class TotalDepth.RP66V1.core.LogicalRecord.EFLR.**ExplicitlyFormattedLogicalRecord**(*lr_type: int, ld: TotalDepth.RP66V1.core.pFile.*

Represents a RP66V1 Explicitly Formatted Logical Record (EFLR). Effectively this is a table containing a list of rows, each row is represented by an Object.

DUPE_OBJECT_STRATEGY = 3

The strategy for dealing with duplicate objects.

DUPE_OBJECT_LOGGER(*msg, *args, **kwargs*)

What level to log duplicate object operations.

__init__(*lr_type: int, ld: TotalDepth.RP66V1.core.pFile.LogicalData*)

__len__() → int

Returns the number of rows in the table.

__getitem__(*item*) → *TotalDepth.RP66V1.core.LogicalRecord.EFLR.Object*

Get an Object (row) by name or integer index.

__str__() → str

Short string representation.

__eq__(*other*) → bool
Equality operator.

__weakref__
list of weak references to the object (if defined)

str_long() → str
Returns a long string representing the table.

table_as_strings(*stringify_function: Callable, sort: bool*) → List[List[str]]
Returns a list of strings representing the table.

is_key_value() → bool
True if this is a key/value table.

key_values(*stringify_function: Callable, sort: bool*) → List[List[str]]
Returns a list of stringified key values. Will raise ExceptionEFLR if not a key/value table.

property shape: Tuple[int, int]
Shape as (rows, columns)

TotalDepth.RP66V1.core.LogicalRecord.EFLR.**reduced_object_map**(*eflr: TotalDepth.RP66V1.core.LogicalRecord.EFLR.Explicitly*)
→ Dict[bytes, int]

This returns a reduced lookup map that refers to the latest object by count. Key is the object IDENT, value is the ordinal into self.

6.12 TotalDepth.RP66V1.core.LogicalRecord.Encryption

Handles encryption in Logical Record Segments [RP66V1 Section 2.2.2.1], which is to say we do very little.

References:

RP66V1: <http://w3.energistics.org/rp66/v1/rp66v1.html>

TODO: Ensure this is consistent across the storage unit?

6.13 TotalDepth.RP66V1.core.LogicalRecord.IFLR

Indirectly Formatted Logical Records

exception TotalDepth.RP66V1.core.LogicalRecord.IFLR.**ExceptionIFLR**

class TotalDepth.RP66V1.core.LogicalRecord.IFLR.**IndirectlyFormattedLogicalRecord**(*lr_type: int, ld: TotalDepth.RP66V1.core.pFile.*

Indirectly Formatted Logical Record has an OBNAME as its identity, a UVARI as the frame number then free data. This just reads the OBNAME and UVARI but not the free data.

Reference: [RP66V1 Section 3.3 Indirectly Formatted Logical Record]

Reference: [RP66V1 Section 5.6.1 Frames] “The Frame Number is an integer (Representation Code UVARI) specifying the numerical order of the Frame in the Frame Type, counting sequentially from one.”

__init__(*lr_type: int, ld: TotalDepth.RP66V1.core.pFile.LogicalData*)

__str__()
Return str(self).

__weakref__

list of weak references to the object (if defined)

6.14 TotalDepth.RP66V1.core.LogicalRecord.Semantics

The DLIS Semantics of Logical Records.

References:

[RP66V1 Chapter 4: Semantic Terminology and Rules] [RP66V1 Chapter 5: Semantics: Static and Frame Data]

Links:

http://w3.energistics.org/rp66/v1/rp66v1_sec4.html http://w3.energistics.org/rp66/v1/rp66v1_sec5.html

TODO: Use this for validating EFLRs.

class TotalDepth.RP66V1.core.LogicalRecord.Semantics.Restrictions(**kwargs)

Imposes semantic restrictions.

__init__(**kwargs)**__weakref__**

list of weak references to the object (if defined)

```
TotalDepth.RP66V1.core.LogicalRecord.Semantics.FREQUENTLY_USED_ATTRIBUTES = {b'AXIS':
<TotalDepth.RP66V1.core.LogicalRecord.Semantics.Restrictions object>, b'DESCRIPTION':
<TotalDepth.RP66V1.core.LogicalRecord.Semantics.Restrictions object>, b'DIMENSION':
<TotalDepth.RP66V1.core.LogicalRecord.Semantics.Restrictions object>, b'LONG-NAME':
<TotalDepth.RP66V1.core.LogicalRecord.Semantics.Restrictions object>}
```

[RP66V1 Section 4.4]

```
TotalDepth.RP66V1.core.LogicalRecord.Semantics.SEMANTICS: Dict[bytes, Dict[bytes,
TotalDepth.RP66V1.core.LogicalRecord.Semantics.Restrictions]] = {b'FRAME': {b'CHANNELS':
<TotalDepth.RP66V1.core.LogicalRecord.Semantics.Restrictions object>, b'DESCRIPTION':
<TotalDepth.RP66V1.core.LogicalRecord.Semantics.Restrictions object>, b'DIRECTION':
<TotalDepth.RP66V1.core.LogicalRecord.Semantics.Restrictions object>, b'ENCRYPTED':
<TotalDepth.RP66V1.core.LogicalRecord.Semantics.Restrictions object>, b'INDEX-MAX':
<TotalDepth.RP66V1.core.LogicalRecord.Semantics.Restrictions object>, b'INDEX-MIN':
<TotalDepth.RP66V1.core.LogicalRecord.Semantics.Restrictions object>, b'INDEX-TYPE':
<TotalDepth.RP66V1.core.LogicalRecord.Semantics.Restrictions object>, b'SPACING':
<TotalDepth.RP66V1.core.LogicalRecord.Semantics.Restrictions object>}}
```

[RP66V1 Section 5]

6.15 TotalDepth.RP66V1.core.LogicalRecord.Types

Implements the Logical Record Syntax.

References: [RP66V1 Section 3 Logical Record Syntax]

In particular: [RP66V1 Appendix A logical Record Types]

TotalDepth.RP66V1.core.LogicalRecord.Types.

TypeDescriptionDataDescriptorObjectReferenceType

alias of *TotalDepth.RP66V1.core.LogicalRecord.Types.TypeDescriptionAllowableSetTypes*

```
class TotalDepth.RP66V1.core.LogicalRecord.Types.TypeDescriptionAllowableSetTypes(type, de-
                                                                 description,
                                                                 allow-
                                                                 able_set_types)

    __getnewargs__()
        Return self as a plain tuple. Used by copy and pickle.

    static __new__(_cls, type, description, allowable_set_types)

    __repr__()
        Return a nicely formatted representation string

    allowable_set_types
        Alias for field number 2

    description
        Alias for field number 1

    type
        Alias for field number 0
```

6.16 TotalDepth.RP66V1.IndexPickle

Read RP66V1 files and saves the index as a pickle file.

```
class TotalDepth.RP66V1.IndexPickle.IndexResult(path_in, size_input, size_index, time_index,
                                                                 time_write, time_read, exception, ignored)

    path_in: str
        Alias for field number 0

    size_input: int
        Alias for field number 1

    size_index: int
        Alias for field number 2

    time_index: float
        Alias for field number 3

    time_write: float
        Alias for field number 4

    time_read: float
        Alias for field number 5

    exception: bool
        Alias for field number 6

    ignored: bool
        Alias for field number 7

    __getnewargs__()
        Return self as a plain tuple. Used by copy and pickle.

    static __new__(_cls, path_in: str, size_input: int, size_index: int, time_index: float, time_write: float,
                                                                 time_read: float, exception: bool, ignored: bool)
```

`__repr__()`

Return a nicely formatted representation string

`TotalDepth.RP66V1.IndexPickle.unpickle(path: str) → TotalDepth.RP66V1.core.LogicalFile.LogicalIndex`
Un-pickles a Logical Index from the given path.

`TotalDepth.RP66V1.IndexPickle.index_dir_multiprocessing(dir_in: str, dir_out: str, jobs: int, recurse: bool, read_back: bool) → Dict[str, TotalDepth.RP66V1.IndexPickle.IndexResult]`

Multiprocessing code to plot log passes. Returns a dict of {path_in : IndexResult, ... }

6.17 TotalDepth.RP66V1.IndexXML

exception `TotalDepth.RP66V1.IndexXML.ExceptionRP66V1IndexXMLRead`

exception `TotalDepth.RP66V1.IndexXML.ExceptionIndexXML`

exception `TotalDepth.RP66V1.IndexXML.ExceptionIndexXMLRead`

exception `TotalDepth.RP66V1.IndexXML.ExceptionLogPassXML`

`TotalDepth.RP66V1.IndexXML.xml_write_value(xml_stream: TotalDepth.util.XmlWrite.XmlStream, value: Any) → None`

Write a value to the XML stream with specific type as an attribute. This writes either a <Value> or an <ObjectName> element.

`TotalDepth.RP66V1.IndexXML.frame_channel_to_XML(channel: TotalDepth.RP66V1.core.LogPass.RP66V1FrameChannel, xml_stream: TotalDepth.util.XmlWrite.XmlStream) → None`

Writes a XML Channel node suitable for RP66V1.

Example:

```
<Channel C="0" I="DEPTH" O="35" count="1" dimensions="1" long_name="Depth Channel"
↪rep_code="7" units="m"/>
```

`TotalDepth.RP66V1.IndexXML.frame_array_to_XML(frame_array: TotalDepth.RP66V1.core.LogPass.RP66V1FrameArray, iflr_data: Sequence[TotalDepth.RP66V1.core.XAxis.IFLRReference], xml_stream: TotalDepth.util.XmlWrite.XmlStream) → None`

Writes a XML FrameArray node suitable for RP66V1.

Example:

```
<FrameArray C="0" I="0B" O="11" description="">
  <Channels channel_count="9">
    <Channel C="0" I="DEPT" O="11" count="1" dimensions="1" long_name="MWD Tool
↪Measurement Depth" rep_code="2" units="0.1 in"/>
    <Channel C="0" I="INC" O="11" count="1" dimensions="1" long_name="Inclination"
↪rep_code="2" units="deg"/>
    <Channel C="0" I="AZI" O="11" count="1" dimensions="1" long_name="Azimuth" rep_
↪code="2" units="deg"/>
    ...
```

(continues on next page)

(continued from previous page)

```

</Channels>
<IFLR count="83">
  <FrameNumbers count="83" rle_len="1">
    <RLE datum="1" repeat="82" stride="1"/>
  </FrameNumbers>
  <LRSH count="83" rle_len="2">
    <RLE datum="0x14ac" repeat="61" stride="0x30"/>
    <RLE datum="0x2050" repeat="20" stride="0x30"/>
  </LRSH>
  <Xaxis count="83" rle_len="42">
    <RLE datum="0.0" repeat="1" stride="75197.0"/>
    <RLE datum="154724.0" repeat="1" stride="79882.0"/>
  </Xaxis>
</IFLR>
</FrameArray>

```

`TotalDepth.RP66V1.IndexXML.log_pass_to_XML`(*log_pass*: `<module 'TotalDepth.common.LogPass' from
'/Users/engun/Documents/workspace/TotalDepth/src/TotalDepth/common/LogPass.py'>`,
iflr_data_map: `Dict[Hashable,
Sequence[TotalDepth.RP66V1.core.XAxis.IFLRReference]]`,
xml_stream: `TotalDepth.util.XmlWrite.XmlStream`) → `None`

Writes a XML LogPass node suitable for RP66V1. Example:

```

<LogPass count="4">
  <FrameArray C="0" I="600T" O="44" description="">
    ...
  <FrameArray>
    ...
</LogPass>

```

`TotalDepth.RP66V1.IndexXML.write_logical_file_sequence_to_xml`(*logical_index*: `TotalDepth.RP66V1.core.LogicalFile.LogicalIndex`,
output_stream: `TextIO`, *private*:
bool) → `None`

Takes a LogicalIndex and writes the index to an XML stream.

class `TotalDepth.RP66V1.IndexXML.IndexResult`(*path_input*, *size_input*, *size_index*, *time*, *exception*,
ignored)

path_input: `str`
Alias for field number 0

size_input: `int`
Alias for field number 1

size_index: `int`
Alias for field number 2

time: `float`
Alias for field number 3

exception: `bool`
Alias for field number 4

ignored: `bool`

Alias for field number 5

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static __new__(*_cls, path_input: str, size_input: int, size_index: int, time: float, exception: bool, ignored: bool*)

__repr__()

Return a nicely formatted representation string

TotalDepth.RP66V1.IndexXML.index_dir_multiprocessing(*dir_in: str, dir_out: str, private: bool, jobs: int*) → Dict[str, *TotalDepth.RP66V1.IndexXML.IndexResult*]

Multiprocessing code to index in XML. Returns a dict of {path_in : IndexResult, ... }

6.18 TotalDepth.RP66V1.LogRecIndex

Exercises the LogicalRecordIndex on real files.

TotalDepth.RP66V1.LogRecIndex.unpickle(*path: str*) → *TotalDepth.RP66V1.core.pIndex.LogicalRecordIndex*

Unpickle a file and return an Index.LogicalRecordIndex.

class TotalDepth.RP66V1.LogRecIndex.IndexResult(*path_in: str, size_input: int, size_index: int, index_time: float, time_write: float, time_read_back: float, exception: bool, ignored: bool*)

Result of indexing a RP66V1 file.

path_in: str

Alias for field number 0

size_input: int

Alias for field number 1

size_index: int

Alias for field number 2

index_time: float

Alias for field number 3

time_write: float

Alias for field number 4

time_read_back: float

Alias for field number 5

exception: bool

Alias for field number 6

ignored: bool

Alias for field number 7

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static __new__(*_cls, path_in: str, size_input: int, size_index: int, index_time: float, time_write: float, time_read_back: float, exception: bool, ignored: bool*)

__repr__()

Return a nicely formatted representation string

`TotalDepth.RP66V1.LogRecIndex.index_dir_multiprocessing`(*dir_in: str, dir_out: str, jobs: int, recurse: bool, read_back: bool, validate: bool*) → Dict[str, *TotalDepth.RP66V1.LogRecIndex.IndexResult*]

Multiprocessing code to plot log passes. Returns a dict of {path_in : IndexResult, ... }

`TotalDepth.RP66V1.LogRecIndex.index_a_single_file`(*path_in: str, path_out: str, read_back: bool, validate: bool*) → *TotalDepth.RP66V1.LogRecIndex.IndexResult*

Read a single file and return an IndexResult.

`TotalDepth.RP66V1.LogRecIndex.index_dir_or_file`(*path_in: str, path_out: str, recurse: bool, read_back: bool, validate: bool*) → Dict[str, *TotalDepth.RP66V1.LogRecIndex.IndexResult*]

Index a directory or file and return the results.

`TotalDepth.RP66V1.LogRecIndex.plot_gnuplot`(*data: Dict[str, TotalDepth.RP66V1.LogRecIndex.IndexResult], gnuplot_dir: str*) → None

Plot the results as a Gnuplot graph.

`TotalDepth.RP66V1.LogRecIndex.main()` → int
Main entry point.

6.19 TotalDepth.RP66V1.Scan

Scans a RP66V1 file and prints out the summary.

This produces text output at various levels of encapsulation:

–VR ~ Visible Records only. –LRSH ~ Logical Record segments. –LD ~ Logical data i.e. all Logical Record segments concatenated for each Logical Record. –EFLR ~ Explicitly Formatted Logical Records. –IFLR ~ Implicitly Formatted Logical Records. –LR ~ All data, including frame data from all Logical Records.

`TotalDepth.RP66V1.Scan.scan_RP66V1_file_visible_records`(*fobj: BinaryIO, fout: TextIO, **kwargs*) → None

Scans the file reporting Visible Records, optionally Logical Record Segments as well.

class `TotalDepth.RP66V1.Scan.LRSHSummary`(*vr_position, lrsh_position, lrsh_attributes, lrsh_record_type*)

vr_position: int
Alias for field number 0

lrsh_position: int
Alias for field number 1

lrsh_attributes: TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHeaderAttributes
Alias for field number 2

lrsh_record_type: int
Alias for field number 3

__getnewargs__()
Return self as a plain tuple. Used by copy and pickle.


```

static __new__(_cls, vr_position: int, lrsh_position: int, lrsh_attributes:
                TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHeaderAttributes, lrsh_record_type:
                int)

__repr__()
    Return a nicely formatted representation string

TotalDepth.RP66V1.Scan.scan_RP66V1_LRSH_consistency(fobj: BinaryIO, fout: TextIO, **kwargs) →
                                                    None

    Look at the consistency of the sequence of LRSH.

TotalDepth.RP66V1.Scan.scan_RP66V1_file_logical_data(fobj: BinaryIO, fout: TextIO, **kwargs) →
                                                    None

    Scans the file reporting the raw Logical Data.

TotalDepth.RP66V1.Scan.scan_RP66V1_file_EFLR_IFLR(fobj: BinaryIO, fout: TextIO, **kwargs) → None

    Scans the file reporting the individual EFLR and IFLR.

TotalDepth.RP66V1.Scan.scan_RP66V1_file_data_content(fobj: BinaryIO, fout: TextIO, *, rp66v1_path:
                                                    str, frame_slice:
                                                    TotalDepth.common.Slice.Slice, eflr_as_table:
                                                    bool) → None

    Scans all of every EFLR and IFLR in the file using a ScanFile object.

TotalDepth.RP66V1.Scan.dump_RP66V1_test_data(fobj: BinaryIO, fout: TextIO, **kwargs) → None

    Scans the file reporting Visible Records, optionally Logical Record Segments as well.

class TotalDepth.RP66V1.Scan.IndexResult(size_input, size_output, time, exception, ignored)

    __getnewargs__()
        Return self as a plain tuple. Used by copy and pickle.

    static __new__(_cls, size_input, size_output, time, exception, ignored)

    __repr__()
        Return a nicely formatted representation string

    exception
        Alias for field number 3

    ignored
        Alias for field number 4

    size_input
        Alias for field number 0

    size_output
        Alias for field number 1

    time
        Alias for field number 2

```

6.20 TotalDepth.RP66V1.ScanHTML

Scans a RP66V1 file and writes out the summary in HTML.

`TotalDepth.RP66V1.ScanHTML.html_write_table_of_contents`(*logical_file_sequence*: `TotalDepth.RP66V1.core.LogicalFile.LogicalIndex`,
xhtml_stream:
`TotalDepth.util.XmlWrite.XhtmlStream`)
→ `None`

Write out the table of contents.

`TotalDepth.RP66V1.ScanHTML.html_write_body`(*logical_file_sequence*:
`TotalDepth.RP66V1.core.LogicalFile.LogicalIndex`,
frame_slice: `TotalDepth.common.Slice.Slice`, *xhtml_stream*:
`TotalDepth.util.XmlWrite.XhtmlStream`, *sort_eflr*: `bool`) →
`TotalDepth.common.ToHTML.HTMLBodySummary`

Write out the <body> of the document.

`TotalDepth.RP66V1.ScanHTML.html_scan_RP66V1_file_data_content`(*path_in*: `str`, *fout*: `TextIO`,
label_process: `bool`, *frame_slice*:
`TotalDepth.common.Slice.Slice`,
sort_eflr: `bool`) →
`TotalDepth.common.ToHTML.HTMLBodySummary`

Scans all of every EFLR and IFLR in the file and writes to HTML. Similar to `TotalDepth.RP66V1.core.Scan.scan_RP66V1_file_data_content` Returns the text to use as a link.

`TotalDepth.RP66V1.ScanHTML.scan_a_single_file`(*path_in*: `str`, *path_out*: `str`, *label_process*: `bool`,
frame_slice: `Union[TotalDepth.common.Slice.Slice,`
`TotalDepth.common.Slice.Sample]`, *sort_eflr*: `bool`) →
`TotalDepth.common.ToHTML.HTMLResult`

Scan a single file and write out an HTML summary.

class `TotalDepth.RP66V1.ScanHTML.FileNameLinkHref`(*file_name*, *link_text*, *href*)

file_name: `str`
Alias for field number 0

link_text: `str`
Alias for field number 1

href: `str`
Alias for field number 2

__getnewargs__()
Return self as a plain tuple. Used by copy and pickle.

static **__new__**(*_cls*, *file_name*: `str`, *link_text*: `str`, *href*: `str`)

__repr__()
Return a nicely formatted representation string

`TotalDepth.RP66V1.ScanHTML.scan_dir_multiprocessing`(*dir_in*, *dir_out*, *jobs*, *frame_slice*:
`Union[TotalDepth.common.Slice.Slice,`
`TotalDepth.common.Slice.Sample]`, *sort_eflr*:
`bool`) → `Dict[str,`
`TotalDepth.common.ToHTML.HTMLResult]`

Multiprocessing code to plot log passes. Returns a dict of {*path_in* : `HTMLResult`, ... }

`TotalDepth.RP66V1.ScanHTML.scan_dir_or_file(path_in: str, path_out: str, recursive: bool, label_process: bool, frame_slice: Union[TotalDepth.common.Slice.Slice, TotalDepth.common.Slice.Sample], sort_eflr: bool) → Dict[str, TotalDepth.common.ToHTML.HTMLResult]`
 Scans a directory or file putting the results in path_out. Returns a dict of {path_in : HTMLResult, ... }

6.21 TotalDepth.RP66V1.SearchFF01

`TotalDepth.RP66V1.SearchFF01.main() → int`
 Searches for likely visible record headers which have 0xff01 as bytes two and three.

6.22 TotalDepth.RP66V1.ToLAS

Converts RP66V1 files to LAS files.

References:

- General: <http://www.cwls.org/las/>
- LAS Version 2: http://www.cwls.org/wp-content/uploads/2017/02/Las2_Update_Feb2017.pdf

Example reference to the LAS documentation in this source code:

[LAS2.0 Las2_Update_Feb2017.pdf Section 5.3 ~V (Version Information)]

`TotalDepth.RP66V1.ToLAS.write_las_header(input_file: str, logical_file: TotalDepth.RP66V1.core.LogicalFile.LogicalFile, logical_file_number: int, frame_array_ident: str, output_stream: TextIO) → None`

Writes the LAS opening such as:

```
~Version Information Section
VERS.          2.0                : CWLS Log ASCII Standard - VERSION 2.
↪0
WRAP.          NO                 : One Line per depth step
PROD.          TotalDepth         : LAS Producer
PROG.          TotalDepth.RP66V1.ToLAS 0.1.1 : LAS Program name and version
CREA.          2012-11-14 10:50    : LAS Creation date [YYYY-MMM-DD
↪hh:mm]
DLIS_CREA.     2012-11-10 22:06    : DLIS Creation date and time [YYYY-
↪MMM-DD hhmm]
SOURCE.        SOME-FILE-NAME.dlis : DLIS File Name
FILE-ID.       SOME-FILE-ID       : File Identification from the FILE-
↪HEADER Logical Record
LOGICAL-FILE.  3                  : Logical File number in the DLIS file
FRAME-ARRAY.   60B                : Identity of the Frame Array in the
↪Logical File
```

Reference: [LAS2.0 Las2_Update_Feb2017.pdf Section 5.3 ~V (Version Information)]

`TotalDepth.RP66V1.ToLAS.DLIS_TO_WELL_INFORMATION_LAS_EFLR_MAPPING: Dict[bytes, Dict[bytes, str]] = {b'PARAMETER': {b'APIN': 'API ', b'COUN': 'CNTY', b'LATT': 'LATT', b'LOC ': 'LOC ', b'LONG': 'LONG', b'NATI': 'CTRY', b'STAT': 'STAT', b'UWI ': 'UWI '}}`

Mapping of DLIS EFLR Type and Object Name to LAS WELL INFORMATION section and Mnemonic. The Object

LONG-NAME is used as the LAS description. We prefer to take data from the ORIGIN EFLR as it is more clearly specified [RP66V1 5.2 Origin Logical Record (OLR)] whereas the PARAMETER EFLR tables are fairly free form. See also [RP66V1 Section 5.8.2 Parameter Objects]

TotalDepth.RP66V1.ToLAS.WELL_INFORMATION_FROM_ORIGIN: Dict[bytes, str] = {b'COMPANY': 'COMP', b'CREATION-TIME': 'DATE', b'FIELD-NAME': 'FLD ', b'PRODUCER-NAME': 'SRVC', b'WELL-NAME': 'WELL'}

[RP66V1 Section 5.2 Origin Logical Record (OLR)]

TotalDepth.RP66V1.ToLAS.extract_well_information_from_origin(logical_file: TotalDepth.RP66V1.core.LogicalFile.LogicalFile) → Dict[str, TotalDepth.LAS.core.WriteLAS.UnitValueDescription]

Extracts partial well information from the ORIGIN record. Example:

```
Objects [1]:
OBNAME: 0: 11 C: 0 I: b'DLIS_DEFINING_ORIGIN'
CD: 001 00001 L: b'FILE-ID' C: 1 R: ASCII U: b'' V: [b'auto_las_survey']
CD: 001 00001 L: b'FILE-SET-NAME' C: 1 R: IDENT U: b'' V: [b'']
CD: 001 00001 L: b'FILE-SET-NUMBER' C: 1 R: UVARI U: b'' V: [1]
CD: 001 00001 L: b'FILE-NUMBER' C: 1 R: UVARI U: b'' V: [1]
CD: 001 00001 L: b'FILE-TYPE' C: 1 R: IDENT U: b'' V: [b'DEPH-LOG']
CD: 001 00001 L: b'PRODUCT' C: 1 R: ASCII U: b'' V: [b'RX6']
CD: 001 00001 L: b'VERSION' C: 1 R: ASCII U: b'' V: [b'v0.0']
CD: 000 00000 L: b'PROGRAMS' C: 1 R: ASCII U: b'' V: None
CD: 001 00001 L: b'CREATION-TIME' C: 1 R: DTIME U: b'' V: [<class 'TotalDepth.
RP66V1.core.RepCode.DateTime'> 2015-08-16 04:57:12.000 STD]
CD: 001 00001 L: b'ORDER-NUMBER' C: 1 R: ASCII U: b'' V: [b'0000']
CD: 000 00000 L: b'DESCENT-NUMBER' C: 1 R: ULONG U: b'' V: None
CD: 001 00001 L: b'RUN-NUMBER' C: 1 R: ULONG U: b'' V: [1]
CD: 000 00000 L: b'WELL-ID' C: 1 R: IDENT U: b'' V: None
CD: 001 00001 L: b'WELL-NAME' C: 1 R: ASCII U: b'' V: [b'PRASLIN 1
']
CD: 001 00001 L: b'FIELD-NAME' C: 1 R: ASCII U: b'' V: [b'PRASLIN PROSPECT
']
CD: 001 00001 L: b'PRODUCER-CODE' C: 1 R: UNORM U: b'' V: [440]
CD: 001 00001 L: b'PRODUCER-NAME' C: 1 R: ASCII U: b'' V: [b'PathFinder']
CD: 001 00001 L: b'COMPANY' C: 1 R: ASCII U: b'' V: [b'BURU ENERGY
']
CD: 001 00001 L: b'NAME-SPACE-NAME' C: 1 R: IDENT U: b'' V: [b'PF']
CD: 000 00000 L: b'NAME-SPACE-VERSION' C: 1 R: UVARI U: b'' V: None
```

TotalDepth.RP66V1.ToLAS.write_well_information_to_las(logical_file: TotalDepth.RP66V1.core.LogicalFile.LogicalFile, frame_array: Optional[TotalDepth.RP66V1.core.LogPass.RP66V1FrameArray], frame_slice: Union[TotalDepth.common.Slice.Slice, TotalDepth.common.Slice.Sample], ostream: TextIO) → None

Writes the well information section.

Reference: [LAS2.0 Las2_Update_Feb2017.pdf Section 5.4 ~W (Well Information)]

`TotalDepth.RP66V1.ToLAS.write_parameter_section_to_las(logical_file: TotalDepth.RP66V1.core.LogicalFile.LogicalFile, ostream: TextIO) → None`

Write the PARAMETER tables to LAS.

`TotalDepth.RP66V1.ToLAS.las_file_name(path_out: str, logical_file_index: int, frame_array_ident: bytes) → str`

Returns the output file name.

`TotalDepth.RP66V1.ToLAS.write_logical_index_to_las(logical_index: TotalDepth.RP66V1.core.LogicalFile.LogicalIndex, array_reduction: str, path_out: str, frame_slice: Union[TotalDepth.common.Slice.Slice, TotalDepth.common.Slice.Sample], channels: Set[str], field_width: int, float_format: str) → List[str]`

Take a Logical Index for a Logical File within a RP66V1 file and write out a set of LAS 2.0 files.

`TotalDepth.RP66V1.ToLAS.single_rp66v1_file_to_las(path_in: str, array_reduction: str, path_out: str, frame_slice: TotalDepth.common.Slice.Slice, channels: Set[str], field_width: int, float_format: str) → TotalDepth.LAS.core.WriteLAS.LASWriteResult`

Convert a single RP66V1 file to a set of LAS files.

`TotalDepth.RP66V1.ToLAS.main() → int`
Main entry point.

6.23 TotalDepth.RP66V1.util API Reference

Contents:

6.23.1 TotalDepth.RP66V1.util.IndexXMLRead

Example of reading the index XML files and mining them for data.

`class TotalDepth.RP66V1.util.IndexXMLRead.IndexResult(size_input, size_index, time, exception, ignored, analysis)`

`__getnewargs__()`

Return self as a plain tuple. Used by copy and pickle.

`static __new__(_cls, size_input, size_index, time, exception, ignored, analysis)`

`__repr__()`

Return a nicely formatted representation string

analysis

Alias for field number 5

exception

Alias for field number 3

ignored

Alias for field number 4

size_index

Alias for field number 1

size_input

Alias for field number 0

time

Alias for field number 2

`TotalDepth.RP66V1.util.IndexXMLRead.read_a_single_index(xml_path_in: str, eflr_set_type: List[str])`→ *To-**talDepth.RP66V1.util.IndexXMLRead.IndexResult*

Reads a single XML index and analyses it.

6.23.2 TotalDepth.RP66V1.util.XMLReadUnits

Reads XML files generated from RP66V1 files and explores them for the units that they use.

Example output of 1885 files:

Producer code names:

-1 TERRASCIENCES, Inc.	:	1
0	:	6
15 INTEQ	:	2
126 CROCKER DATA PROCESSING	:	3
126 WEATHERFORD	:	13
150 ATLAS	:	256
280 Halliburton	:	273
440 PathFinder	:	2
440 Schlumberger	:	2,733

Producer code channel units:

-1	:	4
-1 %	:	8
-1 GAPI	:	2
-1 UNITS	:	1
-1 brne	:	1
-1 deg	:	4
-1 degC	:	2
-1 frac	:	15
-1 g/cc	:	2
-1 in	:	1
-1 m	:	14
-1 m/hr	:	1
-1 ohmm	:	7
-1 ppm	:	7
-1 us/f	:	4
-1 v/v	:	2
0	:	22
0 F3	:	2
0 MMHO	:	54
0 MS10	:	3
0 b/e	:	1
0 deg	:	16
0 degC	:	4

(continues on next page)

(continued from previous page)

0 flg	:	4
0 g	:	18
0 g/cm3	:	4
0 gAPI	:	5
0 gauss	:	3
0 in	:	47
0 lb	:	1
0 lbf	:	6
0 m	:	6
0 m/min	:	3
0 m/s	:	4
0 mV	:	1
0 ms	:	10
0 ohm.m	:	5
0 unitless	:	22
0 us	:	16
0 us/ft	:	5
0 v/v	:	8
15 deg	:	8
15 m	:	2
15 ohm.m	:	3
126	:	25
126 CCPS	:	4
126 DC/S	:	1
126 G	:	3
126 GAUSS	:	3
126 MMHO	:	106
126 MPM	:	2
126 b/e	:	2
126 b/elec	:	3
126 cps	:	1
126 deg	:	20
126 degC	:	14
126 ft/s	:	6
126 g	:	20
126 g/cc	:	10
126 g/cm3	:	44
126 gAPI	:	28
126 gapi	:	3
126 gauss	:	5
126 in	:	100
126 inch	:	5
126 lb	:	1
126 lbf	:	11
126 m	:	13
126 m/h	:	2
126 m/min	:	6
126 m3	:	4
126 mA	:	1
126 mD	:	1
126 mV	:	26
126 microseconds	:	10

(continues on next page)

(continued from previous page)

126 milliVolts	:	6
126 min	:	2
126 mm	:	3
126 mmho/m	:	30
126 ms	:	9
126 ohm.m	:	54
126 psi	:	1
126 pu	:	63
126 t	:	2
126 unitless	:	2
126 us	:	19
126 us/ft	:	11
126 v/v	:	38
150	:	8,033
150 %	:	96
150 1/psi	:	70
150 1/s	:	51
150 1E-2 T/m	:	18
150 1E-5 Oe	:	130
150 K	:	5
150 L	:	423
150 V	:	322
150 b/e	:	9
150 bbl	:	2
150 cP	:	35
150 cm	:	98
150 cm3	:	624
150 cm3/s	:	196
150 cu	:	3
150 dAPI	:	70
150 dB	:	6
150 dB/ft	:	261
150 deg	:	715
150 degC	:	1,083
150 degC/min	:	124
150 degF	:	556
150 degF/min	:	34
150 ft	:	32
150 ft/s	:	70
150 ft3	:	18
150 ft3/bbl	:	70
150 g/cm3	:	67
150 gAPI	:	424
150 h	:	893
150 in	:	1,637
150 kHz	:	107
150 lbf	:	1,036
150 m	:	1,267
150 m/min	:	300
150 m3	:	90
150 mA	:	103
150 mD	:	28

(continues on next page)

(continued from previous page)

150 mG	:	131
150 mS/m	:	79
150 mV	:	835
150 min	:	328
150 mm	:	67
150 ms	:	538
150 ohm.m	:	346
150 pF	:	35
150 ppm	:	127
150 psi	:	2,252
150 psi/min	:	347
150 pu	:	800
150 s	:	1,147
150 uV	:	175
150 us	:	342
150 us/ft	:	535
280	:	1,646
280 %	:	66
280 0.00	:	2
280 0.001/ohm	:	7
280 0.01	:	1
280 0.01 L	:	66
280 0.1	:	1
280 0.1 L/S	:	96
280 0.1 in	:	2
280 08.3	:	4
280 1.0/	:	8
280 1.0/S	:	24
280 1/PS	:	32
280 100	:	18
280 100 pu	:	12
280 C/C	:	4
280 DECP	:	9
280 G	:	14
280 G/CC	:	9
280 GOR	:	48
280 IN_2	:	1
280 Kv/K	:	35
280 MILS	:	2
280 NESW	:	1
280 NONE	:	2,003
280 QLTY	:	2
280 RPM	:	57
280 S	:	406
280 SEC	:	3
280 V	:	49
280 api	:	133
280 b/e	:	28
280 cP	:	14
280 cc	:	45
280 ccps	:	90
280 cnts	:	1

(continues on next page)

(continued from previous page)

280 cps	:	32
280 dB	:	1
280 decp	:	616
280 deg	:	212
280 degC	:	272
280 degF	:	123
280 dist	:	29
280 fph	:	4
280 fpm	:	11
280 ft	:	11
280 ft3	:	20
280 g	:	28
280 g/c3	:	22
280 g/cc	:	152
280 gAPI	:	175
280 gapi	:	1
280 gm/c	:	29
280 in	:	288
280 kHz	:	10
280 kg/m3	:	8
280 klb	:	5
280 lb	:	3
280 lbm	:	77
280 lbs	:	140
280 ltrs	:	11
280 m	:	316
280 m/mi	:	4
280 m/min	:	44
280 m3	:	60
280 mA	:	29
280 mD	:	23
280 mG	:	4
280 mPsec	:	2
280 mS	:	49
280 mSEC	:	2
280 mV	:	51
280 md/cp	:	96
280 min	:	66
280 mm	:	148
280 mmho	:	3
280 mmho/m	:	48
280 mmo/	:	1
280 mmo/m	:	76
280 mpm	:	38
280 ms	:	167
280 nT	:	22
280 no.	:	29
280 ohm.	:	7
280 ohm.m	:	58
280 ohmm	:	390
280 pF	:	36
280 ppm	:	75

(continues on next page)

(continued from previous page)

280	pres	:	29
280	psi	:	597
280	psia	:	965
280	psig	:	31
280	pu	:	127
280	sec	:	8
280	serv	:	29
280	time	:	29
280	uS	:	8
280	uS/f	:	6
280	uS/ft	:	3
280	ucts	:	7
280	us	:	78
280	us/m	:	5
280	uspf	:	101
280	v/v	:	5
280	vol	:	29
280	x8.3ms	:	6
280	z1x1	:	174
280	z1x2	:	174
280	z1y1	:	174
280	z1y2	:	174
280	z2x1	:	29
280	z2x2	:	29
280	z2y1	:	29
280	z2y2	:	29
280	zoom	:	203
440		:	327,687
440	%	:	16,349
440	----	:	587
440	0.1 deg/m	:	1
440	0.1 in	:	29,587
440	0.5 ms	:	1
440	1/30 deg/m	:	6
440	1/min	:	100
440	1/s	:	11,907
440	10 ms	:	9
440	1000 ft.lbf	:	65
440	1000 kPa.s/m	:	23
440	1000 kgf	:	53
440	1000 lbf	:	100
440	96.487 C/g	:	4
440	A	:	7,696
440	A/m	:	1,948
440	BAD_UNIT-?	:	14
440	CPS	:	1
440	GPa	:	27
440	Hz	:	15,966
440	L/min	:	5
440	MPa	:	1
440	Mrayl	:	703
440	N	:	3,073

(continues on next page)

(continued from previous page)

440	N.m	:	4
440	Oe	:	67
440	Pa	:	20
440	Pa.s	:	6
440	S	:	4
440	V	:	79,974
440	W	:	26
440	b/cm3	:	2
440	b/e	:	2
440	bar	:	8
440	c/min	:	5,748
440	c/s	:	43
440	cP	:	561
440	cm3	:	13,512
440	cm3/s	:	6,155
440	cu	:	214
440	dB	:	4,245
440	dB.mW	:	48
440	dB/m	:	1,461
440	deg	:	8,578
440	deg/100ft	:	2
440	deg/30m	:	88
440	deg/ft	:	6
440	deg/m	:	334
440	degC	:	25,640
440	degF	:	1,753
440	ft	:	719
440	ft/h	:	4,875
440	ft/min	:	721
440	ft/s2	:	368
440	ft2	:	6
440	ft3	:	28
440	ft3/bbl	:	1,014
440	ft3/ft3	:	136
440	g	:	44
440	g/cm3	:	8,827
440	gAPI	:	14,847
440	gal/min	:	61
440	gn	:	60
440	h	:	1,730
440	in	:	18,022
440	in2	:	87
440	kN.m	:	1
440	kPa	:	42,487
440	kPa/h	:	244
440	kg/m3	:	37
440	kgf/kgf	:	320
440	km.daN	:	2
440	lbf	:	4,957
440	lbm/gal	:	94
440	m	:	13,969
440	m/h	:	595

(continues on next page)

(continued from previous page)

440 m/min	:	1,792
440 m/s	:	313
440 m/s2	:	5,142
440 m2	:	885
440 m3	:	728
440 m3/m	:	126
440 m3/m3	:	20,861
440 mA	:	11,418
440 mD	:	1,024
440 mD.ft	:	6
440 mD.m	:	216
440 mPa.s	:	222
440 mS/m	:	3,407
440 mSv/h	:	5
440 mT	:	775
440 mV	:	5,218
440 mgn	:	11
440 min	:	5,312
440 min/ft	:	21
440 min/m	:	102
440 mm	:	1
440 mm2	:	7
440 ms	:	33,095
440 nT	:	7
440 nW	:	6
440 nm	:	135
440 ohm	:	223
440 ohm.m	:	19,449
440 ppk	:	42
440 ppm	:	790
440 psi	:	165,969
440 pu	:	1,522
440 rad	:	18
440 s	:	8,699
440 uA	:	864
440 uV	:	2,420
440 unitless	:	6
440 us	:	9,722
440 us/ft	:	8,995
440 us/m	:	28

Producer code attribute units:

-1	:	1,458
-1 GAPI	:	3
-1 dCpm	:	1
-1 deg	:	3
-1 degC	:	4
-1 frac	:	3
-1 g/cc	:	3
-1 in	:	1
-1 lb/g	:	1
-1 m	:	33
-1 ohmm	:	3

(continues on next page)

(continued from previous page)

-1 ppm	:	2
-1 us/f	:	4
0	:	4,665
0 DEG	:	2
0 Hrs	:	1
0 M	:	17
0 Metres	:	8
0 SEC/QT	:	1
0 deg C	:	8
0 degC	:	20
0 g/c3	:	1
0 g/cm3	:	4
0 in	:	8
0 inches	:	6
0 kg/m	:	4
0 lb/USg	:	1
0 lb/ft	:	4
0 m	:	70
0 mL/30min	:	4
0 metres	:	19
0 ml/30Min	:	2
0 ohm-m	:	6
0 ohm.m	:	16
0 sec/qt	:	5
0 us	:	22
0 us/ft	:	4
15	:	800
15 Hrs	:	2
15 M	:	3
15 m	:	17
126	:	13,775
126 CP	:	14
126 Celsius	:	18
126 DEG C	:	2
126 FT	:	3
126 IN	:	13
126 INCHES	:	2
126 M	:	11
126 METRES	:	4
126 OHM-M	:	4
126 degC	:	97
126 g/cc	:	14
126 g/cm3	:	3
126 grams/cc	:	3
126 in	:	126
126 lbf/ft	:	51
126 m	:	328
126 metres	:	35
126 ml/30Min	:	15
126 mm	:	3
126 ohm.m	:	43
126 ohmm	:	6

(continues on next page)

(continued from previous page)

126 pounds/ft	:	5
126 sec/qt	:	6
126 us	:	8
150	:	15,502,799
150 %	:	921
150 Hz	:	188
150 cP	:	98
150 cm	:	383
150 cm3	:	131
150 cm3/s	:	7
150 dB/ft	:	300
150 deg	:	1,119
150 degC	:	538
150 degC/min	:	7
150 degF	:	50
150 ft	:	44
150 g/cm3	:	147
150 gAPI	:	181
150 in	:	1,255
150 kHz	:	792
150 lbf	:	186
150 lbm	:	905
150 lbm/ft	:	172
150 lbm/gal	:	140
150 m	:	10,387
150 m/min	:	23
150 mD/cP	:	14
150 mS/m	:	30
150 mV	:	87
150 mm	:	301
150 ms	:	214
150 ohm.m	:	357
150 ppm	:	14
150 psi	:	203
150 psi/min	:	87
150 pu	:	7
150 s	:	19,681
150 us	:	6,689
150 us/ft	:	258
280	:	260,292
280 \$/da	:	32
280 %	:	133
280 0.1 in	:	4
280 Hz	:	46
280 N	:	19
280 cP	:	18
280 cps	:	12
280 cptm	:	77
280 deg	:	1,093
280 degC	:	1,484
280 degF	:	402
280 f-p	:	6

(continues on next page)

(continued from previous page)

280 fph	:	12
280 ft	:	190
280 g	:	211
280 g/cc	:	221
280 gpm	:	6
280 hr	:	79
280 in	:	714
280 kHz	:	145
280 kPaa	:	19
280 kg/m3	:	22
280 kgm3	:	18
280 klb	:	6
280 lbpf	:	159
280 lbs	:	930
280 m	:	5,789
280 m/hr	:	25
280 mV	:	12
280 mm	:	981
280 mm/m	:	15
280 mpm	:	753
280 mptm	:	12
280 ms	:	289
280 nT	:	209
280 ohmm	:	504
280 pH	:	92
280 pa	:	2
280 ppg	:	120
280 ppm	:	273
280 psi	:	119
280 psia	:	867
280 psig	:	10
280 rpm	:	6
280 s	:	342
280 s/qt	:	78
280 sec	:	144
280 sg	:	6
280 spf	:	8
280 spl	:	2
280 spqt	:	12
280 ucts	:	220
280 us	:	385
280 us/m	:	31
280 uspf	:	215
440	:	35,769,005
440 %	:	34,429
440 0.01 degF/ft	:	26
440 0.1 in	:	7,110
440 0.5 ms	:	11,097
440 1/s	:	147,530
440 1000 1/s	:	143
440 1000 ft.lbf	:	1
440 1000 kPa.s/m	:	15

(continues on next page)

(continued from previous page)

440	1000 kgf	:	1
440	1000 lbf	:	1
440	1000 lbm	:	834
440	1E-4 cm ² /s	:	83
440	1E-5 Oe	:	3
440	1E-6 1/Pa	:	214
440	1E-6 1/psi	:	13
440	96.487 C/g	:	2
440	A	:	2,614
440	A/m	:	581
440	Hz	:	7,413
440	K	:	180
440	Mrayl	:	453
440	N	:	11,282
440	Oe	:	192
440	V	:	316,781
440	bbl	:	999
440	c/min	:	4,829
440	cP	:	611,762
440	cm	:	1,441
440	cm ³	:	1,771,845
440	cm ³ /h	:	5
440	cm ³ /min	:	1,316
440	cm ³ /s	:	124
440	cu	:	342
440	d	:	6
440	dAPI	:	763
440	dB	:	2,300
440	dB/m	:	283
440	deg	:	167,842
440	degC	:	209,611
440	degC/km	:	428
440	degC/m	:	2,166
440	degF	:	8,410
440	degF/ft	:	15
440	ft	:	2,043,783
440	ft/h	:	3,556
440	ft/s ²	:	9
440	ft ³	:	841
440	ft ³ /bbl	:	13
440	ft ³ /ft ³	:	13
440	g	:	24
440	g/cm ³	:	34,157
440	gAPI	:	74,124
440	gn	:	48
440	h	:	1,493
440	in	:	913,413
440	kHz	:	34
440	kN.m	:	1
440	kPa	:	860,887
440	kPa/h	:	118
440	keV	:	4,726

(continues on next page)

(continued from previous page)

440 kg	:	69,794
440 kg/m	:	511
440 kg/m3	:	85
440 kgf/kgf	:	212
440 kohm	:	88
440 lbf	:	9,681
440 lbf/lbf	:	23
440 lbm	:	635
440 lbm/ft	:	2,997
440 lbm/gal	:	1,608
440 lbm/min	:	32
440 m	:	398,845
440 m/h	:	58
440 m/min	:	19
440 m/s	:	14
440 m/s2	:	5,330
440 m2/g	:	64
440 m3	:	275,811
440 m3/m3	:	1,132
440 mA	:	4,238
440 mD	:	1,223,325
440 mD.ft	:	1
440 mD/cP	:	1,633,855
440 mPa.s	:	1,442
440 mS/m	:	1,884
440 mT	:	5,997
440 mV	:	8,523
440 mV/m	:	111
440 mbar	:	224
440 mgn	:	52
440 min	:	164,455
440 mm	:	544
440 mm2	:	96
440 mol/kg	:	96
440 ms	:	3,007
440 nT	:	974
440 nW	:	4
440 nm	:	459
440 ohm	:	1,017
440 ohm.m	:	14,967
440 ppk	:	45
440 ppm	:	85,551
440 psi	:	4,190,937
440 psi/h	:	4
440 pu	:	1,668
440 rad/ft3	:	13
440 rad/m3	:	52
440 s	:	1,764,018
440 s/m3	:	5
440 uA	:	3,719
440 uV	:	7,845
440 us	:	12,236

(continues on next page)

(continued from previous page)

```
440 us/ft      :      7,500
440 us/m       :      20
```

```
2020-08-31 11:28:31,102 - units.py      - 152 - 1801 - (MainThread) - INFO      -
↳ Loading all unit standard forms into the cache
```

Channels:

Producer code: -1

Found code: ['', '%', 'GAPI', 'degC', 'g/cc', 'in', 'm', 'ohmm', 'ppm']

Missing code: ['UNITS', 'brne', 'deg', 'frac', 'm/hr', 'us/f', 'v/v']

Found standard form: ['deg', 'degC', 'in', 'm', 'ppm']

Missing standard form: ['', '%', 'GAPI', 'UNITS', 'brne', 'frac', 'g/cc', 'm/hr', 'ohmm',
↳ 'us/f', 'v/v']

Producer code: 0

Found code: ['', 'F3', 'MMHO', 'b/e', 'degC', 'g', 'g/cm3', 'gAPI', 'gauss', 'in', 'lbf',
↳ 'm', 'm/min', 'm/s', 'mV', 'ms', 'ohm.m', 'us', 'us/ft']

Missing code: ['MS10', 'deg', 'flg', 'lb', 'unitless', 'v/v']

Found standard form: ['deg', 'degC', 'g', 'g/cm3', 'gAPI', 'in', 'lbf', 'm', 'm/min',
↳ 'm/s', 'ms', 'ohm.m', 'us', 'us/ft']Missing standard form: ['', 'F3', 'MMHO', 'MS10', 'b/e', 'flg', 'gauss', 'lb', 'mV',
↳ 'unitless', 'v/v']

Producer code: 15

Found code: ['m', 'ohm.m']

Missing code: ['deg']

Found standard form: ['deg', 'm', 'ohm.m']

Missing standard form: []

Producer code: 126

Found code: ['', 'G', 'MMHO', 'b/e', 'cps', 'degC', 'ft/s', 'g', 'g/cc', 'g/cm3', 'gAPI',
↳ 'gauss', 'in', 'lbf', 'm', 'm/h', 'm/min', 'm3', 'mA', 'mD', 'mV', 'min', 'mm',
↳ 'mmho/m', 'ms', 'ohm.m', 'psi', 'pu', 't', 'us', 'us/ft']Missing code: ['CCPS', 'DC/S', 'GAUSS', 'MPM', 'b/elec', 'deg', 'gapi', 'inch', 'lb',
↳ 'microseconds', 'milliVolts', 'unitless', 'v/v']Found standard form: ['deg', 'degC', 'ft/s', 'g', 'g/cm3', 'gAPI', 'in', 'lbf', 'm',
↳ 'm/h', 'm/min', 'm3', 'mD', 'min', 'mm', 'ms', 'ohm.m', 'psi', 'pu', 't', 'us', 'us/ft',
↳ '']Missing standard form: ['', 'CCPS', 'DC/S', 'G', 'GAUSS', 'MMHO', 'MPM', 'b/e', 'b/elec',
↳ 'cps', 'g/cc', 'gapi', 'gauss', 'inch', 'lb', 'mA', 'mV', 'microseconds', 'milliVolts',
↳ 'mmho/m', 'unitless', 'v/v']

Producer code: 150

Found code: ['', '%', '1/psi', '1/s', '1E-5 Oe', 'K', 'L', 'V', 'b/e', 'bbl', 'cP', 'cm',
↳ 'cm3', 'cm3/s', 'cu', 'dAPI', 'dB', 'dB/ft', 'degC', 'degC/min', 'degF', 'degF/min',
↳ 'ft', 'ft/s', 'ft3', 'ft3/bbl', 'g/cm3', 'gAPI', 'h', 'in', 'kHz', 'lbf', 'm', 'm/min',
↳ 'm3', 'mA', 'mD', 'mS/m', 'mV', 'min', 'mm', 'ms', 'ohm.m', 'pF', 'ppm', 'psi',
↳ 'psi/min', 'pu', 's', 'uV', 'us', 'us/ft']

Missing code: ['1E-2 T/m', 'deg', 'mG']

Found standard form: ['1/psi', '1/s', '1E-5 Oe', 'K', 'L', 'V', 'bbl', 'cP', 'cm', 'cm3',
↳ 'cm3/s', 'cu', 'dAPI', 'dB', 'dB/ft', 'deg', 'degC', 'degC/min', 'degF', 'degF/min',
↳ 'ft', 'ft/s', 'ft3', 'ft3/bbl', 'g/cm3', 'gAPI', 'h', 'in', 'kHz', 'lbf', 'm', 'm/min',
↳ 'm3', 'mA', 'mD', 'mS/m', 'min', 'mm', 'ms', 'ohm.m', 'ppm', 'psi', 'psi/min', 'pu', 's',
↳ 'uV', 'us', 'us/ft']

(continued from previous page)

Missing standard form: ['', '%', '1E-2 T/m', 'b/e', 'mA', 'mG', 'mV', 'pF']

Producer code: 280

Found code: ['', '%', '0.1 in', 'C/C', 'DECP', 'G', 'G/CC', 'NONE', 'RPM', 'S', 'SEC',
 → 'V', 'b/e', 'cP', 'cc', 'cps', 'dB', 'degC', 'degF', 'ft', 'ft3', 'g', 'g/cc', 'gAPI',
 → 'in', 'kHz', 'kg/m3', 'lbm', 'm', 'm/mi', 'm/min', 'm3', 'mA', 'mD', 'mS', 'mV', 'min',
 → 'mm', 'mmho', 'mmho/m', 'ms', 'nT', 'ohm.m', 'ohmm', 'pF', 'ppm', 'psi', 'psig', 'pu',
 → 'uS', 'us', 'us/m']

Missing code: ['0.00', '0.001/ohm', '0.01', '0.01 L', '0.1', '0.1 L/S', '08.3', '1.0/',
 → '1.0/S', '1/PS', '100', '100 pu', 'GOR', 'IN_2', 'Kv/K', 'MILS', 'NESW', 'QLTY', 'api',
 → 'ccps', 'cnts', 'decP', 'deg', 'dist', 'fph', 'fpm', 'g/c3', 'gapi', 'gm/c', 'klbf',
 → 'lb', 'lbs', 'ltrs', 'mG', 'mPsec', 'mSEC', 'md/cp', 'mmo/', 'mmo/m', 'mpm', 'no.',
 → 'ohm.', 'pres', 'psia', 'sec', 'serv', 'time', 'uS/f', 'uS/ft', 'ucts', 'uspf', 'v/v',
 → 'vol', 'x8.3ms', 'z1x1', 'z1x2', 'z1y1', 'z1y2', 'z2x1', 'z2x2', 'z2y1', 'z2y2', 'zoom'
 → '']

Found standard form: ['0.01', '0.1 in', 'S', 'V', 'cP', 'dB', 'deg', 'degC', 'degF',
 → 'ft', 'ft3', 'g', 'gAPI', 'in', 'kHz', 'kg/m3', 'lbm', 'm', 'm/min', 'm3', 'mD', 'mS',
 → 'min', 'mm', 'ms', 'nT', 'ohm.m', 'ppm', 'psi', 'pu', 'uS', 'us', 'us/m']

Missing standard form: ['', '%', '0.00', '0.001/ohm', '0.01 L', '0.1', '0.1 L/S', '08.3',
 → '1.0/', '1.0/S', '1/PS', '100', '100 pu', 'C/C', 'DECP', 'G', 'G/CC', 'GOR', 'IN_2',
 → 'Kv/K', 'MILS', 'NESW', 'NONE', 'QLTY', 'RPM', 'SEC', 'api', 'b/e', 'cc', 'ccps', 'cnts'
 → ', 'cps', 'decP', 'dist', 'fph', 'fpm', 'g/c3', 'g/cc', 'gapi', 'gm/c', 'klbf', 'lb',
 → 'lbs', 'ltrs', 'm/mi', 'mA', 'mG', 'mPsec', 'mSEC', 'mV', 'md/cp', 'mmho', 'mmho/m',
 → 'mmo/', 'mmo/m', 'mpm', 'no.', 'ohm.', 'ohmm', 'pF', 'pres', 'psia', 'psig', 'sec',
 → 'serv', 'time', 'uS/f', 'uS/ft', 'ucts', 'uspf', 'v/v', 'vol', 'x8.3ms', 'z1x1', 'z1x2'
 → ', 'z1y1', 'z1y2', 'z2x1', 'z2x2', 'z2y1', 'z2y2', 'zoom']

Producer code: 440

Found code: ['', '%', '----', '0.1 deg/m', '0.1 in', '0.5 ms', '1/min', '1/s', '10 ms',
 → '1000 ft.lbf', '1000 kPa.s/m', '1000 kgf', '1000 lbf', 'A', 'A/m', 'CPS', 'GPa', 'Hz',
 → 'L/min', 'MPa', 'Mrayl', 'N', 'N.m', 'Oe', 'Pa', 'Pa.s', 'S', 'V', 'W', 'b/cm3', 'b/e'
 → ', 'bar', 'c/min', 'c/s', 'cP', 'cm3', 'cm3/s', 'cu', 'dB', 'dB.mW', 'dB/m', 'deg/100ft'
 → ', 'deg/30m', 'deg/ft', 'deg/m', 'degC', 'degF', 'ft', 'ft/h', 'ft/min', 'ft/s2', 'ft2'
 → ', 'ft3', 'ft3/bbl', 'ft3/ft3', 'g', 'g/cm3', 'gAPI', 'gal/min', 'gn', 'h', 'in', 'in2'
 → ', 'kN.m', 'kPa', 'kPa/h', 'kg/m3', 'kgf/kgf', 'km.daN', 'lbf', 'lbm/gal', 'm', 'm/h',
 → 'm/min', 'm/s', 'm/s2', 'm2', 'm3', 'm3/m', 'm3/m3', 'mA', 'mD', 'mD.ft', 'mD.m', 'mPa.
 → s', 'mS/m', 'mSv/h', 'mT', 'mV', 'mgn', 'min', 'min/ft', 'min/m', 'mm', 'mm2', 'ms',
 → 'nT', 'nW', 'nm', 'ohm', 'ohm.m', 'ppk', 'ppm', 'psi', 'pu', 'rad', 's', 'uA', 'uV',
 → 'us', 'us/ft', 'us/m']

Missing code: ['1/30 deg/m', '96.487 C/g', 'BAD_UNIT-?', 'deg', 'unitless']

Found standard form: ['0.1 deg/m', '0.1 in', '0.5 ms', '1/30 deg/m', '1/min', '1/s',
 → '10 ms', '1000 ft.lbf', '1000 kgf', '1000 lbf', 'A', 'A/m', 'GPa', 'Hz', 'L/min', 'MPa'
 → ', 'N', 'N.m', 'Oe', 'Pa', 'Pa.s', 'S', 'V', 'b/cm3', 'bar', 'c/min', 'c/s', 'cP', 'cm3'
 → ', 'cm3/s', 'cu', 'dB', 'dB.mW', 'dB/m', 'deg', 'deg/ft', 'deg/m', 'degC', 'degF', 'ft'
 → ', 'ft/h', 'ft/min', 'ft/s2', 'ft2', 'ft3', 'ft3/bbl', 'ft3/ft3', 'g', 'g/cm3', 'gAPI',
 → 'gal/min', 'h', 'in', 'in2', 'kN.m', 'kPa', 'kPa/h', 'kg/m3', 'kgf/kgf', 'km.daN',
 → 'lbf', 'lbm/gal', 'm', 'm/h', 'm/min', 'm/s', 'm/s2', 'm2', 'm3', 'm3/m', 'm3/m3', 'mD'
 → ', 'mD.ft', 'mD.m', 'mPa.s', 'mS/m', 'mSv/h', 'mT', 'min', 'min/ft', 'min/m', 'mm',
 → 'mm2', 'ms', 'nT', 'nW', 'nm', 'ohm', 'ohm.m', 'ppk', 'ppm', 'psi', 'pu', 'rad', 's',
 → 'uA', 'uV', 'us', 'us/ft', 'us/m']

Missing standard form: ['', '%', '----', '1000 kPa.s/m', '96.487 C/g', 'BAD_UNIT-?', 'CPS'
 → ', 'Mrayl', 'W', 'b/e', 'deg/100ft', 'deg/30m', 'gn', 'mA', 'mV', 'mgn', 'unitless']

(continues on next page)

(continued from previous page)

Attributes:

Producer code: -1

Found code: ['', 'GAPI', 'degC', 'g/cc', 'in', 'm', 'ohmm', 'ppm']

Missing code: ['dCpm', 'deg', 'frac', 'lb/g', 'us/f']

Found standard form: ['deg', 'degC', 'in', 'm', 'ppm']

Missing standard form: ['', 'GAPI', 'dCpm', 'frac', 'g/cc', 'lb/g', 'ohmm', 'us/f']

Producer code: 0

Found code: ['', 'DEG', 'M', 'deg C', 'degC', 'g/cm3', 'in', 'kg/m', 'lb/ft', 'm', 'mL/30min', 'ohm.m', 'us', 'us/ft']

Missing code: ['Hrs', 'Metres', 'SEC/QT', 'g/c3', 'inches', 'lb/USg', 'metres', 'mL/30Min', 'ohm-m', 'sec/qt']

Found standard form: ['degC', 'g/cm3', 'in', 'kg/m', 'm', 'ohm.m', 'us', 'us/ft']

Missing standard form: ['', 'DEG', 'Hrs', 'M', 'Metres', 'SEC/QT', 'deg C', 'g/c3', 'inches', 'lb/USg', 'lb/ft', 'mL/30min', 'metres', 'mL/30Min', 'ohm-m', 'sec/qt']

Producer code: 15

Found code: ['', 'M', 'm']

Missing code: ['Hrs']

Found standard form: ['m']

Missing standard form: ['', 'Hrs', 'M']

Producer code: 126

Found code: ['', 'CP', 'FT', 'IN', 'INCHES', 'M', 'METRES', 'degC', 'g/cc', 'g/cm3', 'in', 'm', 'mm', 'ohm.m', 'ohmm', 'us']

Missing code: ['Celsius', 'DEG C', 'OHM-M', 'grams/cc', 'lbf/ft', 'metres', 'mL/30Min', 'pounds/ft', 'sec/qt']

Found standard form: ['degC', 'g/cm3', 'in', 'm', 'mm', 'ohm.m', 'us']

Missing standard form: ['', 'CP', 'Celsius', 'DEG C', 'FT', 'IN', 'INCHES', 'M', 'METRES', 'OHM-M', 'g/cc', 'grams/cc', 'lbf/ft', 'metres', 'mL/30Min', 'ohmm', 'pounds/ft', 'sec/qt']

Producer code: 150

Found code: ['', '%', 'Hz', 'cP', 'cm', 'cm3', 'cm3/s', 'dB/ft', 'degC', 'degC/min', 'degF', 'ft', 'g/cm3', 'gAPI', 'in', 'kHz', 'lbf', 'lbm', 'lbm/ft', 'lbm/gal', 'm', 'm/min', 'mD/cP', 'mS/m', 'mV', 'mm', 'ms', 'ohm.m', 'ppm', 'psi', 'psi/min', 'pu', 's', 'us', 'us/ft']

Missing code: ['deg']

Found standard form: ['Hz', 'cP', 'cm', 'cm3', 'cm3/s', 'dB/ft', 'deg', 'degC', 'degC/min', 'degF', 'ft', 'g/cm3', 'gAPI', 'in', 'kHz', 'lbf', 'lbm', 'lbm/ft', 'lbm/gal', 'm', 'm/min', 'mD/cP', 'mS/m', 'mm', 'ms', 'ohm.m', 'ppm', 'psi', 'psi/min', 'pu', 's', 'us', 'us/ft']

Missing standard form: ['', '%', 'mV']

Producer code: 280

Found code: ['', '%', '0.1 in', 'Hz', 'N', 'cP', 'cps', 'degC', 'degF', 'ft', 'g', 'g/cc', 'gpm', 'in', 'kHz', 'kPaa', 'kg/m3', 'm', 'mV', 'mm', 'ms', 'nT', 'ohmm', 'ppm', 'psi', 'psig', 'rpm', 's', 'us', 'us/m']

Missing code: ['\$/da', 'cptm', 'deg', 'f-p', 'fph', 'hr', 'kgm3', 'klb', 'lbpf', 'lbs', 'm/hr', 'mmo/m', 'mpm', 'mptm', 'pH', 'pa', 'ppg', 'psia', 's/qt', 'sec', 'sg', 'spf', 'spl', 'spqt', 'ucts', 'uspf']

(continues on next page)

(continued from previous page)

```

Found standard form: ['0.1 in', 'Hz', 'N', 'cP', 'deg', 'degC', 'degF', 'ft', 'g', 'in
→', 'kHz', 'kg/m3', 'm', 'mm', 'ms', 'nT', 'ppm', 'psi', 's', 'us', 'us/m']
Missing standard form: ['', '$/da', '%', 'cps', 'cptm', 'f-p', 'fph', 'g/cc', 'gpm', 'hr
→', 'kPaa', 'kgm3', 'klb', 'lbp', 'lbs', 'm/hr', 'mV', 'mmo/m', 'mpm', 'mptm', 'ohmm',
→'pH', 'pa', 'ppg', 'psia', 'psig', 'rpm', 's/qt', 'sec', 'sg', 'spf', 'spl', 'spqt',
→'ucts', 'uspf']

Producer code: 440
Found code: ['', '%', '0.01 degF/ft', '0.1 in', '0.5 ms', '1/s', '1000 1/s', '1000 ft.
→lbf', '1000 kPa.s/m', '1000 kgf', '1000 lbf', '1000 lbm', '1E-4 cm2/s', '1E-5 Oe', '1E-
→6 1/Pa', '1E-6 1/psi', 'A', 'A/m', 'Hz', 'K', 'Mrayl', 'N', 'Oe', 'V', 'bbl', 'c/min',
→'cP', 'cm', 'cm3', 'cm3/h', 'cm3/min', 'cm3/s', 'cu', 'd', 'dAPI', 'dB', 'dB/m', 'degC
→', 'degC/km', 'degC/m', 'degF', 'degF/ft', 'ft', 'ft/h', 'ft/s2', 'ft3', 'ft3/bbl',
→'ft3/ft3', 'g', 'g/cm3', 'gAPI', 'gn', 'h', 'in', 'kHz', 'kN.m', 'kPa', 'kPa/h', 'keV',
→'kg', 'kg/m', 'kg/m3', 'kgf/kgf', 'kohm', 'lbf', 'lbf/lbf', 'lbm', 'lbm/ft', 'lbm/gal
→', 'lbm/min', 'm', 'm/h', 'm/min', 'm/s', 'm/s2', 'm2/g', 'm3', 'm3/m3', 'mA', 'mD',
→'mD.ft', 'mD/cP', 'mPa.s', 'mS/m', 'mT', 'mV', 'mV/m', 'mbar', 'mgn', 'min', 'mm', 'mm2
→', 'mol/kg', 'ms', 'nT', 'nW', 'nm', 'ohm', 'ohm.m', 'ppk', 'ppm', 'psi', 'psi/h', 'pu
→', 'rad/ft3', 'rad/m3', 's', 's/m3', 'uA', 'uV', 'us', 'us/ft', 'us/m']
Missing code: ['96.487 C/g', 'deg']
Found standard form: ['0.01 degF/ft', '0.1 in', '0.5 ms', '1/s', '1000 1/s', '1000 ft.
→lbf', '1000 kgf', '1000 lbf', '1000 lbm', '1E-4 cm2/s', '1E-5 Oe', '1E-6 1/Pa', '1E-6
→1/psi', 'A', 'A/m', 'Hz', 'K', 'N', 'Oe', 'V', 'bbl', 'c/min', 'cP', 'cm', 'cm3', 'cm3/
→h', 'cm3/min', 'cm3/s', 'cu', 'd', 'dAPI', 'dB', 'dB/m', 'deg', 'degC', 'degC/km',
→'degC/m', 'degF', 'degF/ft', 'ft', 'ft/h', 'ft/s2', 'ft3', 'ft3/bbl', 'ft3/ft3', 'g',
→'g/cm3', 'gAPI', 'h', 'in', 'kHz', 'kN.m', 'kPa', 'kPa/h', 'keV', 'kg', 'kg/m', 'kg/m3
→', 'kgf/kgf', 'kohm', 'lbf', 'lbf/lbf', 'lbm', 'lbm/ft', 'lbm/gal', 'lbm/min', 'm', 'm/
→h', 'm/min', 'm/s', 'm/s2', 'm2/g', 'm3', 'm3/m3', 'mD', 'mD.ft', 'mD/cP', 'mPa.s',
→'mS/m', 'mT', 'mV/m', 'min', 'mm', 'mm2', 'mol/kg', 'ms', 'nT', 'nW', 'nm', 'ohm',
→'ohm.m', 'ppk', 'ppm', 'psi', 'psi/h', 'pu', 'rad/ft3', 'rad/m3', 's', 's/m3', 'uA',
→'uV', 'us', 'us/ft', 'us/m']
Missing standard form: ['', '%', '1000 kPa.s/m', '96.487 C/g', 'Mrayl', 'gn', 'mA', 'mV',
→'mbar', 'mgn']

```

TotalDepth.RP66V1.util.XMLReadUnits.FileResult

alias of TotalDepth.RP66V1.util.XMLReadUnits.IndexResult

TotalDepth.RP66V1.util.XMLReadUnits.**read_a_single_file**(*xml_path_in*: str, *accumulator*: TotalDepth.RP66V1.util.XMLReadUnits.AggregateCounts)
→
TotalDepth.RP66V1.util.XMLReadUnits.IndexResult

Reads a single XML index and analyses it.

TotalDepth.RP66V1.util.XMLReadUnits.**check_against_osdd_units**(*acc*: TotalDepth.RP66V1.util.XMLReadUnits.AggregateCounts)
→ None

Checks which units are in our version of the OSDD.

TOTALDEPTH.DAT.DAT_PARSER

7.1 DAT Files

Parses DAT files. DAT is an informal specification (i.e. undefined) with loads of poor implementations.

Here described:

Section 1: Channel Declaration. A set of lines of the form: A B C, whitespace separated, where A is an uppercase word, B is free text, C is a word. There is no guarantee that channels declared here have any data in the subsequent sections. Order of this section is ignored.

Section 2: Channel Header A single line with space separated uppercase words. All words must appear as channel declarations from section 1 but some declarations from section 1 may be missing. The order of the words in the channel header is used to interpret the order of the subsequent values in section 3.

So deciding if we are in section 2 can be done with some (dubious?) heuristics:

- List matches list of section 1. Seems sensible but does not work when channels are declared but not defined.
- Some subset of the declared channels.
- All uppercase?
- Lots of words?
- Starts with 'UTIM DATE TIME ...'

In this implementation we use the latter.

Section 3: Channel values Space separated values. Mostly numeric but date/time conversion can be inferred from section 1. The column order is defined by the order of the Channel Header.

Note many deficiencies here:

DATE Date ddmmyy but value 9Dec06, 09-Dec-06 etc.

TIME Time hhmmss but value 11-50-17

Example, <...> is continuation:

```
UTIM Unix Time sec DATE Date ddmmyy TIME Time hhmmss WAC Wits Activity Code unitless BDIA Bit Diameter inch <...> NPEN n-Pentane ppm EPEN Neo-Pentane ppm UTIM DATE TIME WAC BDIA <...> NPEN EPEN
1165665017 09Dec06 11-50-17 0 8.50 <...> 0 0
```

7.2 Performance

There is no particular effort made here for high performance. DAT files are small, typically <10Mb, so artful coding is not really required.

7.3 API

exception TotalDepth.DAT.DAT_parser.ExceptionDAT

General exception for problems with a DAT object.

exception TotalDepth.DAT.DAT_parser.ExceptionDATRead

Exception for reading a DAT file.

TotalDepth.DAT.DAT_parser.RE_CHANNEL_DEFINITION =

re.compile('^[A-Z0-9]+\s(?:\s(?:\s+))\$')

Matches 'UTIM Unix Time sec' Also need to process: "UTIM Unix Time sec"

TotalDepth.DAT.DAT_parser.RE_DATA_HEADER_DEFINITION =

re.compile('UTIM\s+DATE\s+TIME\s+.\$')

Matches 'UTIM DATE TIME ...'

TotalDepth.DAT.DAT_parser.RE_DATE_STYLE_A =

re.compile('^(\\d+)(Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)(\\d+)\$')

Matches '12Oct20' and '5Oct20'

TotalDepth.DAT.DAT_parser.RE_DATE_STYLE_B =

re.compile('^(\\d+)-(Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)-(\\d+)\$')

Matches '12-Oct-20' and '5O-Oct-20'

TotalDepth.DAT.DAT_parser.NAME_UNITS_TYPE_MAP = {('DATE', 'ddmmyy'): <class 'object'>, ('TIME', 'hhmmss'): <class 'object'>, ('UTIM', 'sec'): <class 'object'>}

Map of numpy dtype from the name/units.

TotalDepth.DAT.DAT_parser.parse_file(file_object: TextIO, ident: str = "", description: str = 'DAT File') →

TotalDepth.common.LogPass.FrameArray

Parse the File object as a DAT file into a FrameArray. Will raise an ExceptionDAT on error.

Parameters

- **file_object** – The file to parse.
- **ident** – Identification of this DAT file.
- **description** – Description of this DAT file.

Returns

TotalDepth.DAT.DAT_parser.parse_path(path: str) → *TotalDepth.common.LogPass.FrameArray*

Parse the DAT file in the given path.

TotalDepth.DAT.DAT_parser.can_parse_file(file_object: TextIO) → bool

Tries to parse the file with just one row of data. On error returns False.

TotalDepth.DAT.DAT_parser.can_parse_path(path: str) → bool

Tries to parse the file at path with just one row of data. On error returns False.

TotalDepth.DAT.DAT_parser.main() → int

Read a file and dump the Log Pass.

TOTALDEPTH.COMMON - COMMON PACKAGE REFERENCE

Contents:

8.1 TotalDepth.common.cmn_cmd_opts (Common Command Line Options)

Common command line options, this is to try and present some degree of interface consistency among command line applications.

Copyright (c) 2010-2019 Paul Ross. All rights reserved.

TotalDepth.common.cmn_cmd_opts.arg_parser(*desc, prog=None, version=None, **kwargs*) → `argparse.ArgumentParser`

Return an command line parser with the standard pre-set options.

Standard options are `-h`, `--version` and:

`-k`: Flag to indicate that we should keep going as far as sensible.

`-l`: Log level.

`-v`: Verbosity.

TotalDepth.common.cmn_cmd_opts.path_in(**args, **kwargs*) → `argparse.ArgumentParser`
Return an command line parser with the standard pre-set options plus an input path as an argument.

TotalDepth.common.cmn_cmd_opts.path_in_out(**args, **kwargs*) → `argparse.ArgumentParser`
Return an command line parser with the standard pre-set options plus an input and output paths as an arguments.

TotalDepth.common.cmn_cmd_opts.path_in_out_required(**args, **kwargs*) → `argparse.ArgumentParser`
Return an command line parser with the standard pre-set options plus an input and output paths as an arguments.

TotalDepth.common.cmn_cmd_opts.DEFAULT_OPT_LOG_LEVEL = 30
Default log level

TotalDepth.common.cmn_cmd_opts.DEFAULT_OPT_LOG_FORMAT = '%(asctime)s %(process)d
%(levelname)-8s %(message)s'
Default log format (terse)

TotalDepth.common.cmn_cmd_opts.DEFAULT_OPT_LOG_FORMAT_VERBOSE = '%(asctime)s -
%(filename)-16s#%(lineno)-4d - %(process)5d - %(threadName)-10s - %(levelname)-8s -
%(message)s'
Default log format (verbose)

`TotalDepth.common.cmn_cmd_opts.add_log_level(parser: argparse.ArgumentParser, level: int = 30) → None`

Adds log level to the argument parser. The value can be either an integer of a string so 20 and 'INFO' are equivalent.

`TotalDepth.common.cmn_cmd_opts.set_log_level(parsed_args, format: str = '%(asctime)s - %(filename)-16s#%(lineno)-4d - %(process)5d - %(threadName)-10s) - %(levelname)-8s - %(message)s') → int`

Initialise logging.

`TotalDepth.common.cmn_cmd_opts.add_multiprocessing(parser: argparse.ArgumentParser) → None`

Adds log level to the argument parser as -jobs.

`TotalDepth.common.cmn_cmd_opts.multiprocessing_requested(parsed_args) → bool`

Returns True if the --jobs= option requires multiprocessing.

`TotalDepth.common.cmn_cmd_opts.number_multiprocessing_jobs(parsed_args) → int`

Returns the number of multiprocessing nodes interpreted from the ``-jobs=`` option.

8.2 TotalDepth.common.colorama

Abstraction over colorama

`TotalDepth.common.colorama.section(title: str, fillchar: str, colour: colorama.ansi.AnsiFore = '\x1b[32m', width: int = 75, out_stream: TextIO = <colorama.ansitowin32.StreamWrapper object>)`

Write a coloured header and trailer.

8.3 TotalDepth.common.data_table (Pretty Printing Tables)

Formats a table as a list of printable strings.

TODO: Support specific styles:

Sphinx style:

+-----+-----+-----+-----+			
Header row, column 1	Header 2	Header 3	Header 4
(header rows optional)			
+=====+=====+=====+=====+			
body row 1, column 1	column 2	column 3	column 4
+-----+-----+-----+-----+			
body row 2	...	...	
+-----+-----+-----+-----+			

Result:

Header row, column 1 (header rows optional)	Header 2	Header 3	Header 4
body row 1, column 1	column 2	column 3	column 4
body row 2	...	...	

Markdown style:

```
| Calculation | Video Time t (s) | Distance from start of Runway (m) | Distance from
↪ start of Runway at t=0 (m) |
| --- | --: | --: | --: |
| Mid speed -10 knots | -30.1 | 537 | 1325 |
| Mid speed | -32.8 | 232 | 1182 |
| Mid speed +10 knots | -35.6 | -87 | 1039 |
| **Range and worst error** | **-32.8 ±2.8** | **232 ±319** | **1182 ±143** |
```

`TotalDepth.common.data_table.format_object(o: Any) → str`

Format a value as a string.

`TotalDepth.common.data_table.format_table(rows: Sequence[Sequence[Any]], pad: str = '', heading_underline: str = "", left_flush: bool = False) → List[str]`

Given a table of strings this formats them as a list of strings.

`TotalDepth.common.data_table.format_table_columns(rows: Sequence[Sequence[Any]], column_formats: List[str], pad: str = '', heading_underline: str = "") → List[str]`

Given a list of objects this formats them as a list of strings.

8.4 TotalDepth.common.LogPass

This provides a file format agnostic representation of a LogPass.

- A LogPass consists of a set of FrameArray(s).
- A FrameArray consists of a set of FrameChannel(s).
- A FrameChannel consists of a set of values in a Numpy array of any shape from a single recorded channel.

exception `TotalDepth.common.LogPass.ExceptionLogPassBase`

General exception for problems with this module.

exception `TotalDepth.common.LogPass.ExceptionLogPass`

General exception for problems with a LogPass object.

exception `TotalDepth.common.LogPass.ExceptionFrameChannel`

General exception for problems with a FrameChannel object.

exception `TotalDepth.common.LogPass.ExceptionFrameArray`

General exception for problems with a FrameArray object.

class `TotalDepth.common.LogPass.FrameChannel`(*ident: Hashable, long_name: Union[str, bytes], units: Union[str, bytes], shape: Tuple[int, ...], np_dtype: numpy.dtype*)

This represents a single channel in a frame. It is file format independent and can be used depending on the source of the information: LIS/LAS/RP66V1 file, XML index, Postgres database etc.

__init__(*ident: Hashable, long_name: Union[str, bytes], units: Union[str, bytes], shape: Tuple[int, ...], np_dtype: numpy.dtype*)

Constructor.

Parameters

- **ident** – Some hashable identity.
- **long_name** – A description of the channel

- **units** – Units of Measure.
- **shape** – A list of dimensions of each value. [1] is a single value per frame. [4, 1024] is a 4 * 1024 matrix such as sonic waveform of 1024 samples with 4 waveforms per frame.
- **np_dtype** – The numpy dtype to use.

property ident: Hashable

Overload this if necessary, for example RP66V1 has an OBNAME.

__str__() → str
Return str(self).

__getitem__(key)
Gets the value in the numpy array where key is a tuple of integers of length self.dimensions. For example this might be from self.numpy_indexes().

__setitem__(key, value)
Sets the value in the numpy array where key is a tuple of integers of length self.dimensions. For example this might be from self.numpy_indexes().

init_array(number_of_frames: int) → None

Initialises an empty Numpy array suitable to fill with <frames> number of frame data for this channel. If an array already exists of the correct length it is reused.

property array_size: int

The number of elements in the numpy array.

property sizeof_array: int

The size of each element of the current array as represented by numpy.

property sizeof_frame: int

The size of a single frame in bytes as represented by numpy.

numpy_indexes(frame_number: int) → itertools.product

Returns a generator of numpy indexes for a particular frame.

Example for a 2 x 3 array, given frame index 7:

```
>>> list(itertools.product([7], [0,1], [0,1,2]))
[(7, 0, 0), (7, 0, 1), (7, 0, 2), (7, 1, 0), (7, 1, 1), (7, 1, 2)]
>>> list(itertools.product([7], range(2), range(3)))
[(7, 0, 0), (7, 0, 1), (7, 0, 2), (7, 1, 0), (7, 1, 1), (7, 1, 2)]
```

Usage, where *function* is a conversion function on the data:

```
for dim in self.numpy_indexes(frame_number):
    # dim is a tuple of length self.rank + 1
    self.array[dim] = some_value
```

mask_array(absent_value: Union[None, int, float]) → None

Masks the absent values.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.common.LogPass.**FrameArray**(ident: Hashable, description: Union[str, bytes])

Represents a set of channels recorded simultaneously. In the olden days we would record this on a single piece of continuous film.

Subclass this depending on the source of the information: LIS/LAS/DLIS file, XML index etc.

__init__(*ident: Hashable, description: Union[str, bytes]*)

append(*channel: TotalDepth.common.LogPass.FrameChannel*) → None

Add a channel to the Array.

__str__() → str

Return str(self).

__len__() → int

The number of channels.

property sizeof_array: int

The total of the current frame array as represented by numpy.

property sizeof_frame: int

The size of the internal representation of a frame as represented by numpy.

property shape: List[Tuple[int, ...]]

The shape of the frame array.

init_arrays(*number_of_frames: int*) → None

Initialises empty Numpy arrays for each channel suitable to fill with <frames> number of frame data.

init_arrays_partial(*number_of_frames: int, channels: Set[Hashable]*) → None

Initialises empty Numpy arrays for each of the specified channels suitable to fill with <frames> number of frame data. The channels parameter limits the initialisation to only those channels. Unknown channels in that parameter are ignored.

mask_array(*absent_value: Union[int, float]*) → None

Mask the absent values in all but the index channels.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.common.LogPass.LogPass

This represents the structure a single run of data acquisition such as 'Repeat Section' or 'Main Log'. These runs have one or more independent simultaneous recordings of different sensors at different depth/time resolutions. Each of these simultaneous recordings is represented as a FrameArray object.

- A LogPass consists of a set of FrameArray(s).
- A FrameArray consists of a set of FrameChannel(s).
- A FrameChannel consists of a set of values in a Numpy array of any shape from a single recorded channel (sensor).

This is a file format independent design. Different file formats use this in different ways:

- LIS79 - The standard allows 2 simultaneous FrameArrays, IFLR type 0, 1. Type 1 has never been seen in the wild.
- LAS (all versions) - The standard excludes simultaneous FrameArrays.
- RP66V1 - The standard allows for any number of simultaneous FrameArrays and this is common.
- DAT - No simultaneous FrameArrays.
- BIT - Custom and practice shows that there can be any number of simultaneous FrameArrays.

__init__()

has(*key: Hashable*) → bool

Returns True if the key is in the Frame Array Map.

keys() → Iterable

The identities of the Frame Arrays.

append(*frame_array*: `TotalDepth.common.LogPass.FrameArray`) → None

Add a channel to the Array.

__str__() → str

Return str(self).

__len__() → int

The number of Frame Arrays.

__getitem__(*item*: `Union[int, str, bytes]`) → `TotalDepth.common.LogPass.FrameArray`

The Frame Array by index or ID.

__weakref__

list of weak references to the object (if defined)

8.5 TotalDepth.common.lookup_mnemonic

8.5.1 Schlumberger Data

<https://www.apps.slb.com/cmd/>

“The Curve Mnemonic Dictionary is the publicly accessible version of the Oilfield Services Data Dictionary (OSDD).”

Mnemonics

URL is of the form:

`https://www.apps.slb.com/cmd/<TYPE>.aspx?code=<NAME>`

Where <TYPE> is:

Data channels: ChannelItem Parameters: ParameterItem Logging Tools: ToolItem Software Products: ProductItem

Examples:

`https://www.apps.slb.com/cmd/ChannelItem.aspx?code=RHOB`
`https://www.apps.slb.com/cmd/ParameterItem.aspx?code=LATI`
`https://www.apps.slb.com/cmd/ToolItem.aspx?code=HDT`

Anything that does not start with [A-Z] is in <https://www.apps.slb.com/cmd/ChannelsList.aspx?start=na>

Example:

`https://www.apps.slb.com/cmd/ChannelItem.aspx?code=A0`

Content contains the key/value table:

```
<table cellpadding="5" cellspacing="1" id="main_DetailsView1" style="width:492px;">
  <tr>
    <td style="font-weight:bold;">Channel</td>
    <td>A0</td>
  </tr>
  <tr>
```

(continues on next page)

(continued from previous page)

```

        <td style="font-weight:bold;">Description</td>
        <td>Analog 0 (Regular)</td>
    </tr>
    <tr>
        <td style="font-weight:bold;">Unit quantity</td>
        <td>
            <a id="main_DetailsView1_HyperLink1" href="UOMDetail.aspx?
↪dim=ElectricPotential">ElectricPotential</a>
        </td>
    </tr>
    <tr>
        <td style="font-weight:bold;">Property</td>
        <td>
            <a id="main_DetailsView1_HyperLink4" href="PropertyItem.aspx?code=Electric_
↪Potential">Electric_Potential</a>
        </td>
    </tr>
</table>

```

Data Channels

Example:

```
https://www.apps.slb.com/cmd/ChannelItem.aspx?code=RHOB
```

Channels have the attributes: Channel, Description, Unit quantity, Property as key/value in a table 'main_DetailsView1'.

In the table 'main_GridView1' there are 'Related tools' as a list of key/value:

```
<th scope="col">Tool</th><th scope="col">Description</th>
```

In the table 'main_GridView2' there are 'Related products' as a list of key/value:

```
<th scope="col">Product</th><th scope="col">Description</th>
```

Parameters

Example:

```
https://www.apps.slb.com/cmd/ParameterItem.aspx?code=LATI
```

Parameters have the attributes: Code, Description, Unit quantity, Property in a key/value table 'main_DetailsView1'.

They also have 'Related products' in the 'main_GridView2' a table with a list of key/values:

```
<th scope="col">Product</th><th scope="col">Description</th>
```

Logging Tools

Example:

```
https://www.apps.slb.com/cmd/ToolItem.aspx?code=HDT
```

In the table 'main_DetailsView1' a ToolItem has the attributes: Code, Technology, Discipline, Method, Description as a key/value table.

Example:

Code	HDT
Technology	Dipmeter
Discipline	Geology
Method	WIRELIN
Description	High Resolution Dipmeter Tool

They also have 'Related Channels' in the 'main_GridView1' a table a list of key/values:

```
<th scope="col">Channel</th><th scope="col">Description</th>
```

They also have 'Related Parameters' in the 'main_GridView2' a table a list of key/values:

```
<th scope="col">Parameter</th><th scope="col">Description</th>
```

Software Products

Example:

```
https://www.apps.slb.com/cmd/ProductItem.aspx?code=CALCULATE_TRAJECTORY
```

In the table 'main_DetailsView1' has the attributes: Code, Name, Discipline, Type, Description as a key/value table.

They also have 'Related Channels' in the 'main_GridView1' a table a list of key/values with links to ChannelItem:

```
<th scope="col">Channel</th><th scope="col">Description</th>
```

Properties

Example:

```
https://www.apps.slb.com/cmd/PropertyItem.aspx?code=Diameter
```

In the table 'main_DetailsView1' has the attributes: Code, Name, Parents, Description as a key/value table.

Parents are as a hierarchy, for example:

```
<td style="font-weight:bold;">Parents</td>
<td>&nbsp;
  <span id="main_DetailsView1_labParents">
    <a href="PropertyItem.aspx?code=Property">Property</a>
    &gt;
    <a href="PropertyItem.aspx?code=Length">Length</a>
  </span>
</td>
```


They also have ‘Related children’ in the ‘main_GridView1’ a table a list of key/values with links to PropertyItem:

```
<th scope="col">Code</th><th scope="col">Name</th>
```

Units

See slb_units()

Unit assignment:

<https://www.apps.slb.com/cmd/unitassignment.aspx>

Example:

```
<table cellpadding="4" cellspacing="0" id="main_GridView1" style="width:940px;border-
collapse:collapse;">
  <tr align="left" style="background-color:#E0E0E0;font-size:X-Small;">
    <th scope="col">Unit</th>
    <th scope="col">Unit System</th>
    <th scope="col">Unit Quantity</th>
    <th scope="col">Dimension</th>
  </tr><tr>
    <td>(bbl/d)/(rev/s)</td>
    <td>ProductionEnglish</td>
    <td>FlowratePerRotationalVelocity</td>
    <td>VolumePerRotation</td>
  </tr><tr>
    <td>(rev/s)/(ft/min)</td>
    <td>ProductionEnglish</td>
    <td>RotationalVelocityPerVelocity</td>
    <td>RotationPerLength</td>
  </tr><tr>
    <td>gn</td>
    <td>Metric</td>
    <td>Gravity</td>
    <td>Acceleration</td>
  </tr>
```

exception TotalDepth.common.lookup_mnemonic.ExceptionLookupMnemonic

exception TotalDepth.common.lookup_mnemonic.ExceptionLookupMnemonicReadURL

exception TotalDepth.common.lookup_mnemonic.ExceptionLookupMnemonicTable

exception TotalDepth.common.lookup_mnemonic.ExceptionLookupMnemonicReadTable

TotalDepth.common.lookup_mnemonic.decompose_table_by_header_row(*parse_tree*: *bs4.BeautifulSoup*,
table_id: *str*) → List[Dict[*str*,
str]]

Return a list of rows from an HTML table of given ID.

class TotalDepth.common.lookup_mnemonic.ProductDescription(*product*, *description*)

product: *str*

Alias for field number 0

description: **str**

Alias for field number 1

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static **__new__**(*_cls, product: str, description: str*)

__repr__()

Return a nicely formatted representation string

class TotalDepth.common.lookup_mnemonic.**ToolDescription**(*tool, description*)

tool: **str**

Alias for field number 0

description: **str**

Alias for field number 1

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static **__new__**(*_cls, tool: str, description: str*)

__repr__()

Return a nicely formatted representation string

class TotalDepth.common.lookup_mnemonic.**Channel**(*channel, description, unit_quantity, property, related_tools, related_products*)

channel: **str**

Alias for field number 0

description: **str**

Alias for field number 1

unit_quantity: **str**

Alias for field number 2

property: **str**

Alias for field number 3

related_tools: **Tuple**[*TotalDepth.common.lookup_mnemonic.ToolDescription, ...*]

Alias for field number 4

related_products: **Tuple**[*TotalDepth.common.lookup_mnemonic.ProductDescription, ...*]

Alias for field number 5

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static **__new__**(*_cls, channel: str, description: str, unit_quantity: str, property: str, related_tools: Tuple[TotalDepth.common.lookup_mnemonic.ToolDescription, ...], related_products: Tuple[TotalDepth.common.lookup_mnemonic.ProductDescription, ...]*)

__repr__()

Return a nicely formatted representation string

TotalDepth.common.lookup_mnemonic.**slb_data_channel**(*name: str*) →

TotalDepth.common.lookup_mnemonic.Channel

Returns the Channel corresponding to the name. This is a cached live lookup.

```
class TotalDepth.common.lookup_mnemonic.Parameter(code, description, unit_quantity, property,
                                                related_products)
```

```

code: str
    Alias for field number 0

description: str
    Alias for field number 1

unit_quantity: str
    Alias for field number 2

property: str
    Alias for field number 3

related_products: Tuple[TotalDepth.common.lookup_mnemonic.ProductDescription]
    Alias for field number 4

__getnewargs__()
    Return self as a plain tuple. Used by copy and pickle.

static __new__(_cls, code: str, description: str, unit_quantity: str, property: str, related_products:
                Tuple[TotalDepth.common.lookup_mnemonic.ProductDescription])

__repr__()
    Return a nicely formatted representation string
```

```
TotalDepth.common.lookup_mnemonic.slb_parameter(name: str) →
                                TotalDepth.common.lookup_mnemonic.Parameter
    Returns the Parameter corresponding to the name. This is a cached live lookup.
```

```
class TotalDepth.common.lookup_mnemonic.ChannelDescription(channel, description)
```

```

channel: str
    Alias for field number 0

description: str
    Alias for field number 1

__getnewargs__()
    Return self as a plain tuple. Used by copy and pickle.

static __new__(_cls, channel: str, description: str)

__repr__()
    Return a nicely formatted representation string
```

```
class TotalDepth.common.lookup_mnemonic.ParameterDescription(parameter, description)
```

```

parameter: str
    Alias for field number 0

description: str
    Alias for field number 1

__getnewargs__()
    Return self as a plain tuple. Used by copy and pickle.

static __new__(_cls, parameter: str, description: str)
```

__repr__()

Return a nicely formatted representation string

```
class TotalDepth.common.lookup_mnemonic.LoggingTool(code, technology, discipline, method,  
description, related_channels,  
related_parameters)
```

code: str

Alias for field number 0

technology: str

Alias for field number 1

discipline: str

Alias for field number 2

method: str

Alias for field number 3

description: str

Alias for field number 4

related_channels: Tuple[TotalDepth.common.lookup_mnemonic.ChannelDescription]

Alias for field number 5

related_parameters: Tuple[TotalDepth.common.lookup_mnemonic.ParameterDescription]

Alias for field number 6

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

```
static __new__(cls, code: str, technology: str, discipline: str, method: str, description: str,  
related_channels: Tuple[TotalDepth.common.lookup_mnemonic.ChannelDescription],  
related_parameters:  
Tuple[TotalDepth.common.lookup_mnemonic.ParameterDescription])
```

__repr__()

Return a nicely formatted representation string

```
TotalDepth.common.lookup_mnemonic.slb_logging_tool(name: str) → TotalDepth.common.lookup_mnemonic.LoggingTool
```

Logging Tools

Example:

<https://www.apps.slb.com/cmd/ToolItem.aspx?code=HDT>

In the table 'main_DetailsView1' a ToolItem has the attributes: Code, Technology, Discipline, Method, Description as a key/value table.

Example:

Code	HDT
Technology	Dipmeter
Discipline	Geology
Method	WIRELINE
Description	High Resolution Dipmeter Tool

They also have 'Related Channels' in the 'main_GridView1' a table a list of key/values:

```
<th scope="col">Channel</th><th scope="col">Description</th>
```

They also have ‘Related Parameters’ in the ‘main_GridView2’ a table a list of key/values:

```
<th scope="col">Parameter</th><th scope="col">Description</th>
```

8.6 TotalDepth.common.process (Monitoring a Process’s CPU and Memory)

A HOWTO is here [TotalDepth-tech-process](#)

Logs process information, such as memory usage, to a log as JSON. Example with (‘memory_info’, ‘cpu_times’):

```
(Thread-7 ) ProcessLoggingThread JSON: {"memory_info": {"rss": 145448960, "vms": 4542902272, "pfaults": 37618, "pageins": 0}, "cpu_times": {"user": 0.28422032, "system": 0.099182912, "children_user": 0.0, "children_system": 0.0}}
```

There are several DoF here:

- Logging interval in seconds. Or by poke()?
- Logging level, DEBUG, INFO etc.
- Logging verbosity, for example just memory? Or everything about the process (self._process.as_dict())

Also need to add a log parser to, well what?

```
TotalDepth.common.process.LOGGER_PREFIX = 'ProcessLoggingThread-JSON'
```

Unique string in the log line

```
TotalDepth.common.process.RE_LOG_LINE =
```

```
re.compile('^.+?ProcessLoggingThread-JSON(-START|-STOP)?\\s*(.+)$')
```

Regex for the unique string in the log line

```
TotalDepth.common.process.DATETIME_NOW_FORMAT = '%Y-%m-%d %H:%M:%S.%f'
```

Regex for timestamp, matches ‘2019-06-07 11:57:58.390921’

```
TotalDepth.common.process.KEY_TIMESTAMP = 'timestamp'
```

The JSON key that is the timestamp

```
TotalDepth.common.process.KEY_ELAPSED_TIME = 'elapsed_time'
```

The JSON key that is elapsed (wall clock) time in seconds. This is `time.time() - self._process.create_time()`

```
TotalDepth.common.process.KEY_LABEL = 'label'
```

The JSON key that is the label

```
TotalDepth.common.process.KEY_PROCESS_ID = 'pid'
```

The JSON key that is the process ID

```
TotalDepth.common.process.PSUTIL_PROCESS_AS_DICT_KEYS = ['cmdline', 'connections', 'cpu_percent', 'cpu_times', 'create_time', 'cwd', 'environ', 'exe', 'gids', 'memory_full_info', 'memory_info', 'memory_percent', 'name', 'nice', 'num_ctx_switches', 'num_fds', 'num_threads', 'open_files', 'pid', 'ppid', 'status', 'terminal', 'threads', 'uids', 'username']
```

`psutil.Process().as_dict()` has the following keys:

```
TotalDepth.common.process.GNUPLOT_PLT = '\nset grid\nset title "Memory and CPU Usage."
font ",14"\nset xlabel "Elapsed Time (s)"\n# set mxtics 5\n# set xrange [0:3000]\n# set
xtics\n# set format x ""\n\n#set logscale y\nset ylabel "Memory Usage (Mb)"\n# set yrange
[0:500]\n# set ytics 20\n# set mytics 2\n# set ytics 8,35,3\n\n#set logscale y2\nset
y2label "CPU Usage (%), Page Faults (10,000/s)"\n# set y2range [0:200]\nset y2tics\n\nset
pointsize 1\nset datafile separator whitespace#\n\t"\nset datafile missing "NaN"\n\nset
terminal svg size 1000,700 # choose the file format\nset output "{name}.svg" # choose the
output device\n\n# set key off\n\n{labels}\n\n#set key title "Window Length"\n# lw 2
pointsize 2\n\nplot "{name}.dat" using 1:($2 / 1024**2) axes x1y1 title "RSS (Mb), left
axis" with lines lt 1 lw 2, \\n "{name}.dat" using 1:($3 / 10000) axes x1y2 title "Page
Faults (10,000/s), right axis" with lines lt 3 lw 1, \\n "{name}.dat" using 1:5 axes
x1y2 title "Mean CPU (%), right axis" with lines lt 2 lw 1, \\n "{name}.dat" using 1:6
axes x1y2 title "Instantaneous CPU (%), right axis" with lines lt 7 lw 1\n\nreset\n'
Usage: GNUPLOT_PLT.format(name=dat_file_name)
```

TotalDepth.common.process.parse_timestamp(s: str) → datetime.datetime
Read a string such as '2019-06-07 11:57:58.390921' and return a datetime.

TotalDepth.common.process.extract_json(istream: TextIO) → List[Dict[str, Any]]
Reads a log file and returns the JSON as a list of dicts. Non-matching lines are ignored.

TotalDepth.common.process.extract_labels_from_json(json_data: List[Dict[str, Any]]) → List[Dict[str, Any]]
Returns a list of dicts of JSON data where 'label' is a key'.

TotalDepth.common.process.extract_json_as_table(json_data: List[Dict[str, Any]]) → Tuple[Dict[int, List[List[str]]], Dict[int, float], Dict[int, float], Dict[int, float], Dict[int, float]]

Create a table from JSON suitable for a Gnuplot .dat file.

Returns:

- { process_id : [rows of data, ...], ... }
- { process_id : t min, ... }
- { process_id : t max, ... }
- { process_id : RSS min, ... }
- { process_id : RSS max, ... }

A row of data is:

time, RSS, PageFaults, User, Mean CPU, Insantanous CPU, Timestamp, PID, Label

TotalDepth.common.process.invoke_gnuplot(log_path: str, gnuplot_dir: str) → int
Reads a log file, extracts the data, writes it out to gnuplot_dir and invokes gnuplot on it.

TotalDepth.common.process.add_message_to_queue(msg: str) → None
Adds a message onto the queue.

class TotalDepth.common.process.ProcessLoggingThread(group=None, target=None, name=None, args=(), kwargs=None, *, daemon=None)

Thread that regularly logs out process parameters.

__init__(group=None, target=None, name=None, args=(), kwargs=None, *, daemon=None)
Constructor. args[0], or interval=... must be the reporting interval in seconds, default 1.0. args[1], or log_level=... must be the log level to report with, default logging.INFO.

run() → None
thread.run(). Write to log then sleep.

join(*args, **kwargs)
thread.join(). Write to log last time.

TotalDepth.common.process.**log_process**(*args, **kwargs)
Context manager to log process data at regular intervals.

TotalDepth.common.process.**add_process_logger_to_argument_parser**(parser:
argparse.ArgumentParser) →
None

Add a --log-process option to the argument parser.

TotalDepth.common.process.**main**() → int
Main CLI entry point. For testing.

8.7 TotalDepth.common.Rle (Run Length Encoding Data Compression)

Code for Run Length Encoding.

```
class TotalDepth.common.Rle.RLEItem(datum: Union[int, float])
    Class that represents a single entry in a Run Length Encoding set. v - The datum value.

    __init__(datum: Union[int, float])
    __str__() → str
        String representation.

    __len__() → int
        Total number of values.

    add(v: Union[int, float]) → bool
        Returns True if v has been absorbed in this entry. False means a new entry is required.

    values() → Sequence[Union[int, float]]
        Generates all values.

    value(i) → Tuple[int, Optional[Union[int, float]]]
        Returns a particular value.

    range() → range
        Returns a range object that has (start, stop, step).

    last()
        Returns the last value.

    __weakref__
        list of weak references to the object (if defined)

class TotalDepth.common.Rle.RLE(theFunc=None)
    Class that represents Run Length Encoding.

    theFunc - optional unary function to convert all values with.

    __init__(theFunc=None)
        Constructor, optionally takes a unary function to convert all values with.

    __str__()
        Return str(self).

    __len__()
        The number of RLEItem(s).
```

__getitem__(*key*)

Returns a RLEItem.

num_values()

Total number of record values.

add(*v*)

Adds a value to this RLE object.

values()

Generates all values entered.

value(*i*)

Indexing; this returns the *i*'th value added.

ranges()

Returns a list of range() objects.

first()

Returns the first value or None if no values added.

last()

Returns the last value or None if no values added.

largest_le(*value: Union[int, float]*) → Union[int, float]

Return the largest value less than or equal to the given value.

__weakref__

list of weak references to the object (if defined)

TotalDepth.common.Rle.**create_rle**(*values: Iterable, fn: Optional[Callable] = None*) →

TotalDepth.common.Rle.RLE

Create a RLE object from an iterable.

8.8 TotalDepth.common.Slice (Data Slicing)

class TotalDepth.common.Slice.SliceABC

abstract first(*length: int*) → int

The index of the first element of a sequence of length.

abstract last(*length: int*) → int

The index of the last element of a sequence of length.

abstract step(*length: int*) → int

The sequence of length step.

abstract count(*length: int*) → int

Returns the number of values that will result if the slice is applied to a sequence of given length.

abstract gen_indices(*length: int*) → range

Generates the indices for the sequence of the given length.

abstract indices(*length: int*) → List[int]

Returns a fully composed list of indices for the sequence of the given length.

abstract __eq__(*other*) → bool

Mostly used for testing.

abstract long_str(length: int) → str

Return a long string.

abstract __str__() → str

String representation.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.common.Slice.**Slice**(start: Union[None, int] = None, stop: Union[None, int] = None, step: Union[None, int] = None)

Class that wraps a builtin slice object for integers and provides some useful APIs. NOTE: The builtin slice object can take non-integer values but raises later, for example:

```
slice(1, 4, 2.0).indices(45)
```

__init__(start: Union[None, int] = None, stop: Union[None, int] = None, step: Union[None, int] = None)

first(length: int) → int

The index of the first element of a sequence of length.

last(length: int) → int

The index of the last element of a sequence of length.

step(length: int) → int

The sequence of length step.

count(length: int) → int

Returns the number of values that will result if the slice is applied to a sequence of given length.

gen_indices(length: int) → range

Generates the indices for the sequence of the given length.

indices(length: int) → List[int]

Returns a fully composed list of indices for the sequence of the given length.

__eq__(other) → bool

Mostly used for testing.

long_str(length: int) → str

Return a long string.

__str__() → str

String representation.

class TotalDepth.common.Slice.**Sample**(sample_size: int)

This has the same API as Slice but takes a single integer.

NOTE: This may not produce a regular sequence. For example sampling 7 items out of a 12 element list gives the indices [0, 1, 3, 5, 6, 8, 10]

__init__(sample_size: int)

first(length) → int

The index of the first element of a sequence of length.

last(length) → int

The index of the last element of a sequence of length.

step(length) → int

The sequence of length step.

count(length: int) → int

Returns the number of values that will result if the slice is applied to a sequence of given length.

gen_indices(*length: int*) → range
Generates the indices for the sequence of the given length.

indices(*length: int*) → List[int]
Returns a fully composed list of indices for the sequence of the given length.

__eq__(*other*) → bool
Mostly used for testing.

long_str(*length: int*) → str
Long descriptive string.

__str__() → str
String representation.

TotalDepth.common.Slice.create_slice_or_sample(*slice_string: str*) →
Union[*TotalDepth.common.Slice.Slice*,
TotalDepth.common.Slice.Sample]
Returns a Slice object from a string such as: ‘, ‘None,72’, ‘None,72,14’

8.9 TotalDepth.common.statistics

class TotalDepth.common.statistics.LengthDict
Provides statistics about a summary of lengths such as file lengths or logical data lengths.

__init__()

__len__() → int
Number of distinct length entries.

__getitem__(*item*) → int
Returns the count of a particular length. Does not write to the dict.

property min: int
Minimum length.

property max: int
Maximum length.

property count: int
Return the total number of entries.

property zero_count: int
Return the number of length zero.

keys() → KeysView[int]
Return the keys (lengths seen).

add(*length: int*)
Add a length to the summary.

reduced_power_2() → DefaultDict[int, int]
Return a histogram with keys reduced to power of 2.

histogram_power_of_2(*width: int = 40, bar_char: str = '+'*) → List[str]
Return the count of zero and a histogram of strings of the lengths to power of 2.

__weakref__
list of weak references to the object (if defined)

8.10 TotalDepth.common.units

This provides Unit conversion information from lookup sources.

The primary source is Schlumberger's Oilfield Services Data Dictionary (OSDD): <https://www.apps.slb.com/cmd/units.aspx>

The fallback, secondary source, is from our static snapshot of that page which lives in `src/TotalDepth/common/data/osdd_units.json`

When running tests with `--runslow` the `tests.integration.common.test_units.test_slb_units_write_to_json` test will re-populate that static data file.

This included currencies that all have zero offset and unit scale. Currencies: https://en.wikipedia.org/wiki/ISO_4217
ISO 4217 official currency codes in XML: https://www.currency-iso.org/dam/downloads/lists/list_one.xml

exception `TotalDepth.common.units.ExceptionUnits`

Base class exception for this module.

exception `TotalDepth.common.units.ExceptionUnitsLookup`

Raised if the unit lookup fails.

exception `TotalDepth.common.units.ExceptionUnitsDimension`

Raised if two units are of different dimensions.

class `TotalDepth.common.units.Unit`(*code: str, name: str, standard_form: str, dimension: str, scale: float, offset: float*)

Represents one row in the table at <https://www.apps.slb.com/cmd/units.aspx>

Examples:

Code	Name	Standard Form	Dimension	Scale	Offset
DEGC	'degree celsius'	degC	Temperature	1	-273.15
DEGF	'degree fahrenheit'	degF	Temperature	0.5555555555555556	-459.67
DEGK	'kelvin'	K	Temperature	1	0
DEGR	'degree rankine'	degR	Temperature	0.5555555555555556	0

There will also be an entry for RP66V1 files:

degF	'degree fahrenheit'	"5/9 degC +32"	Temperature	0.5555555555555556	-459.67
------	---------------------	----------------	-------------	--------------------	---------

So conversion from, say DEGC to DEGF is:

```
((value - DEGC.offset) * DEGC.scale) / DEGF.scale + DEGF.offset
((0.0 - -273.15) * 1.0) / 0.5555555555555556 + -459.67 == 32.0
```

code: str

Alias for field number 0

name: str

Alias for field number 1

standard_form: str

Alias for field number 2

dimension: str

Alias for field number 3

scale: float

Alias for field number 4

offset: float

Alias for field number 5

property is_primary: bool

True if this is looks like a primary unit.

has_offset() → bool

True if this has an offset, for example DEGC. False otherwise, for example metres.

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static __new__(cls, code: str, name: str, standard_form: str, dimension: str, scale: float, offset: float)

__repr__()

Return a nicely formatted representation string

TotalDepth.common.units.osdd_data_file_path() → str

Path to our static snapshot of the OSDD units page.

TotalDepth.common.units.read_osdd_static_data() → Dict[str, TotalDepth.common.units.Unit]

Read our static snapshot of the OSDD units page.

TotalDepth.common.units.slb_load_units()

Eagerly load the units into the cache.

TotalDepth.common.units.has_slb_units(unit_code: str) → bool

Returns True if the Schlumberger Unit exists.

TotalDepth.common.units.slb_units(unit: str) → TotalDepth.common.units.Unit

Returns the Schlumberger Unit corresponding to the unit code.

TotalDepth.common.units.has_slb_standard_form(standard_form: str) → bool

Returns True if an entry for the standard form exists.

TotalDepth.common.units.slb_standard_form_to_unit_code(standard_form: str) →

List[TotalDepth.common.units.Unit]

Returns the unit(s) corresponding to the standard form. Example given 'degC' this returns the Units corresponding to ['DEGC', 'deg C', 'oC'].

TotalDepth.common.units.same_dimension(a: TotalDepth.common.units.Unit, b: TotalDepth.common.units.Unit) → bool

Returns True if both units have the same dimension.

TotalDepth.common.units.convert(value: float, unit_from: TotalDepth.common.units.Unit, unit_to: TotalDepth.common.units.Unit) → float

Converts a value from one unit to another.

Examples:

Code	Name	Standard Form	Dimension	Scale	Offset
DEGC	'degree celsius'	degC	Temperature	1	-273.15
DEGF	'degree fahrenheit'	degF	Temperature	0.5555555555555556	-459.67

So conversion from, say DEGC to DEGF is:

```
((value - DEGC.offset) * DEGC.scale) / DEGF.scale + DEGF.offset
```

```
((0.0 - -273.15) * 1.0) / 0.5555555555555556 + -459.67 == 32.0
```

`TotalDepth.common.units.convert_function`(*unit_from*: `TotalDepth.common.units.Unit`, *unit_to*: `TotalDepth.common.units.Unit`) → Callable

Return a partial function to convert from one units to another.

`TotalDepth.common.units.convert_array`(*array*: `numpy.ndarray`, *unit_from*: `TotalDepth.common.units.Unit`, *unit_to*: `TotalDepth.common.units.Unit`) → `numpy.ndarray`

Convert an array of values.

`TotalDepth.common.units.convert_array_inplace`(*array*: `numpy.ndarray`, *unit_from*: `TotalDepth.common.units.Unit`, *unit_to*: `TotalDepth.common.units.Unit`) → None

Convert an array of values in-place.

8.11 TotalDepth.common.xml (Importing XML Libraries)

Imports the best XML libraries as etree. Modified from: <https://lxml.de/tutorial.html>

8.12 TotalDepth.common.xxd

`TotalDepth.common.xxd.xxd`(*by*: `bytes`, *columns*: `int = 16`, *uppercase*: `bool = False`, *ebcdic*: `bool = False`, *binary*: `bool = False`, *length*: `int = 0`, *offset*: `int = 0`, *seek*: `int = 0`) → str

Returns an xxd style string of the bytes. For example:

```
0084 8000 8400 2647 3546 3239 2020 2020 2020 .....&G5F29
```

`columns` - Number of octets in each row.

`uppercase` - use upper case hex letters.

`ebcdic` - show characters in EBCDIC. Default ASCII.

`binary` - binary digit dump. Default hex.

`length` - stop after <length> octets.

`offset` - add <offset> to the displayed file position.

`seek` - start at <seek> bytes in file offset.

TOTALDEPTH.UTIL - UTILITY PACKAGE REFERENCE

Contents:

9.1 TotalDepth.util.CopyBinFiles

Copies files of a specific type from one file tree to another.

TotalDepth.util.CopyBinFiles.copy_files(*path_in: str, path_out: str, binary_file_types: Set[str], move: bool, nervous: bool*) → List[str]

Copies binary files from *path_in* to *path_out*.

If *move* is True the file is moved, if False the file is copied. Returns a list of destination paths.

TotalDepth.util.CopyBinFiles.main() → int

Main entry point. Copies of moves particular file types from one tree to another

9.2 TotalDepth.util.DictTree (Tree-like Dictionary)

A dictionary that takes a list of hashables as a key and behaves like a tree.

exception TotalDepth.util.DictTree.ExceptionDictTree

Exception when handling a DictTree object.

exception TotalDepth.util.DictTree.ExceptionDictTreeHtmlTable

Exception when handling a DictTreeHtmlTable object.

class TotalDepth.util.DictTree.DictTree(*value_iterable=None*)

A dictionary that takes a list of hashables as a key and behaves like a tree. A node can have multiple values represented as a set or list.

__init__(*value_iterable=None*)

add(*key: Sequence[Hashable], value: Any*) → None

Add a key/value. *k* is a list of hashables.

remove(*key: Sequence[Hashable], value: Optional[Any] = None*) → None

Remove a key/value.

value(*key: Sequence[Hashable]*) → Optional[Any]

Value corresponding to a key or None. *k* is a list of hashables.

values() → List[Any]

Returns a list of all values.

keys() → List[Hashable]

Return a list of keys where each key is a list of hashables.

items() → Sequence[Tuple[Sequence[Hashable], Any]]

Yields a sequence of key, value pairs.

__len__() → int

Returns the number of keys.

depth() → int

Returns the maximum tree depth as an integer.

indented_string() → str

Returns an indented string.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.util.DictTree.DictTreeTableEvent(*branch: List[Any], node: Any, row_span: int, col_span: int*)

POD class that contains the data needed for a HTML table entry. *branch* - the data route to this node. *node* - the columns of the table entry. *row_span* - the HTML rowspan attribute for the <td>. *col_span* - the HTML colspan attribute for the <td>.

branch: List[Any]

Alias for field number 0

node: Any

Alias for field number 1

row_span: int

Alias for field number 2

col_span: int

Alias for field number 3

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

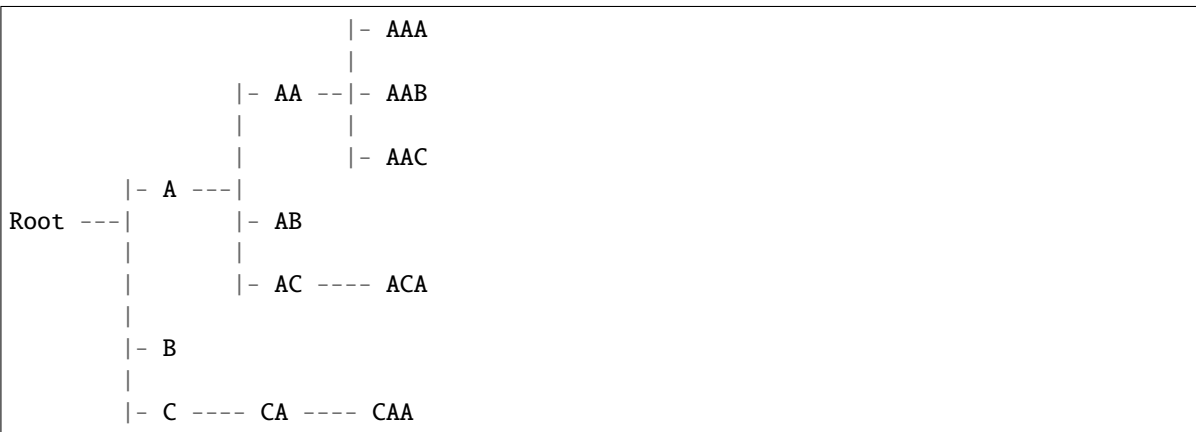
static **__new__**(*_cls, branch: List[Any], node: Any, row_span: int, col_span: int*)

__repr__()

Return a nicely formatted representation string

class TotalDepth.util.DictTree.DictTreeHtmlTable(**args, **kwargs*)

A sub-class of DictTree that helps writing HTML row/col span tables Suppose we have a tree like this:



And we want to represent the tree like this when laid out as an HTML table:

A	AA	AAA

		AAB

		AAC

	AB	

	AC	ACA

B		

C	CA	CAA

In this example the tree is loaded branch by branch thus:

```
myTree = DictTreeHtmlTable()
myTree.add(('A', 'AA', 'AAA'), None)
myTree.add(('A', 'AA', 'AAB'), None)
myTree.add(('A', 'AA', 'AAC'), None)
myTree.add(('A', 'AB',), None)
myTree.add(('A', 'AC', 'ACA'), None)
myTree.add(('B',), None)
myTree.add(('C', 'CA', 'CAA'), None)
```

The HTML code generator can be used like this:

```
# Write: <table border="2" width="100%">
with XmlWrite.Element(xhtml_stream, 'table', {}):
    for event in myTree.genColRowEvents():
        if event == myTree.ROW_OPEN:
            # Write out the '<tr>' element
            xhtml_stream.startElement('tr', {})
        elif event == myTree.ROW_CLOSE:
            # Write out the '</tr>' element
            xhtml_stream.endElement('tr')
        else:
            # Write '<td rowspan="..." colspan="...">...</td>' % (r, c, v)
            with XmlWrite.Element(xhtml_stream, 'td', event.html_attrs()):
                xhtml_stream.characters(str(event.node))
# Write: </table>
```

And the HTML will look like this:

```
<table border="2" width="100%">
  <tr valign="top">
    <td rowspan="5">A</td>
    <td rowspan="3">AA</td>
    <td>AAA</td>
  </tr>
```

(continues on next page)

(continued from previous page)

```

<tr valign="top">
  <td>AAB</td>
</tr>
<tr valign="top">
  <td>AAC</td>
</tr>
<tr valign="top">
  <td colspan="2">AB</td>
</tr>
<tr valign="top">
  <td>AC</td>
  <td>ACA</td>
</tr>
<tr valign="top">
  <td colspan="3">B</td>
</tr>
<tr valign="top">
  <td>C</td>
  <td>CA</td>
  <td>CAA</td>
</tr>
</table>

```

ROW_OPEN = DictTreeTableEvent(branch=[], node=None, row_span=0, col_span=0)

HTML table event: open row with <tr ...>

ROW_CLOSE = DictTreeTableEvent(branch=[], node=None, row_span=-1, col_span=-1)

HTML table event: close row with </tr>

__init__(*args, **kwargs)

is_row_open(event: TotalDepth.util.DictTree.DictTreeTableEvent) → bool

Return True if the event I have generated is a ROW_OPEN event.

is_row_close(event: TotalDepth.util.DictTree.DictTreeTableEvent) → bool

Return True if the event I have generated is a ROW_CLOSE event.

add(key: Sequence[Hashable], value: Any) → None

Add a key/value.

remove(key: Sequence[Hashable], value: Optional[Any] = None)

Remove a key/value.

set_row_column_span() → None

Top level call that sets colspan and rowspan attributes.

gen_row_column_events() → Sequence[TotalDepth.util.DictTree.DictTreeTableEvent]

Returns a set of events that are quadruples. (key_branch, value, rowspan_int, colspan_int) The branch is a list of keys the from the branch of the tree. The rowspan and colspan are both integers. At the start of the a <tr> there will be a ROW_OPEN and at row end (</tr>) a ROW_CLOSE will be yielded

gen_row_column_events_from_branch(key_branch: List[Hashable]) →

Sequence[TotalDepth.util.DictTree.DictTreeTableEvent]

Yields a set of events that are a tuple of quadruples. (key_branch, value, rowspan_integer, colspan_integer) For example: ([‘a’, ‘b’], ‘c’, 3, 7) At the start of the a <tr> there will be a ROW_OPEN and at row end (</tr>) a ROW_CLOSE will be yielded

depth_from_branch(*key_branch: Sequence[Hashable]*) → int

Finds the remainder of depth from the branch.

walk_row_col_span() → str

Return the internal tree as a string.

9.3 TotalDepth.util.DirWalk (Directory Walking)

Provides various ways of walking a directory tree

Created on Jun 9, 2011

class TotalDepth.util.DirWalk.**FileInOut**(*filePathIn, filePathOut*)

filePathIn: str

Alias for field number 0

filePathOut: str

Alias for field number 1

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static **__new__**(*_cls, filePathIn: str, filePathOut: str*)

__repr__()

Return a nicely formatted representation string

exception TotalDepth.util.DirWalk.**ExceptionDirWalk**

Exception class for this module.

TotalDepth.util.DirWalk.**gen_big_first**(*directory*)

Generator that yields the biggest files (name not path) first. This is fairly simple in that it only looks the current directory not only sub-directories. Useful for multiprocessing.

TotalDepth.util.DirWalk.**dirWalk**(*theIn: str, theOut: str = "", theFnMatch: Tuple[str] = (), recursive: bool = False, bigFirst: bool = False*) → Sequence[TotalDepth.util.DirWalk.FileInOut]

Walks a directory tree generating file paths as FileInOut(in, out) objects.

theIn - The input directory.

theOut - The output directory. If an empty string the out path will be an empty string. NOTE: This does not create the output directory structure, it is up to the caller to do that.

theFnMatch - A glob like match pattern for file names (not tested for directory names). TODO: Should be a tuple of fnmatch strings.

recursive - Boolean to recurse or not.

bigFirst - If True then the largest files in directory are given first. If False it is alphabetical.

class TotalDepth.util.DirWalk.**Event**(*value*)

An enumeration.

class TotalDepth.util.DirWalk.**FileEvent**(*event, path_in, path_out*)

event: TotalDepth.util.DirWalk.Event

Alias for field number 0

path_in: str

Alias for field number 1

path_out: str

Alias for field number 2

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static __new__(cls, event: TotalDepth.util.DirWalk.Event, path_in: str, path_out: str)

__repr__()

Return a nicely formatted representation string

TotalDepth.util.DirWalk.**dir_event_walk**(path_in: str, path_out: str = "", fn_match: str = "", recursive: bool = False, big_first: bool = False) → Sequence[TotalDepth.util.DirWalk.FileEvent]

Walks a directory tree generating file paths as FileEvent() objects.

path_in - The input directory.

path_out - The output directory. If an empty string the out path will always be an empty string. NOTE: This does not create the output directory structure, it is up to the caller to do that.

fn_match - A glob like match pattern for file names (not tested for directory names).

recursive - Boolean to recurse or not.

big_first - If True then the largest files in directory are given first. Order is files first followed by directories. If False the order is alphabetical files and directories (with recursion as appropriate).

9.4 TotalDepth.util.EBCDIC

Support for EBCDIC character encoding.

References:

<https://en.wikipedia.org/wiki/EBCDIC>

https://www.ibm.com/support/knowledgecenter/SSGH4D_16.1.0/com.ibm.xlf161.aix.doc/language_ref/asciit.html

From: https://www.ibm.com/support/knowledgecenter/ssw_ibm_i_71/rzaat/rzaate.htm

“EBCDIC single-byte encoding scheme An 8-bit-per-byte structure. The EBCDIC single-byte structure has a valid code-point range for 00 to FF Control characters have a range from 00 to 3F. Graphic characters have a range from 41 to FE. The space character is 40.”

TotalDepth.util.EBCDIC.EBCDIC_ALL = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 63, 64, 74, 75, 76, 77, 78, 79, 80, 90, 91, 92, 93, 94, 95, 96, 97, 106, 107, 108, 109, 110, 111, 121, 122, 123, 124, 125, 126, 127, 129, 130, 131, 132, 133, 134, 135, 136, 137, 143, 145, 146, 147, 148, 149, 150, 151, 152, 153, 161, 162, 163, 164, 165, 166, 167, 168, 169, 176, 186, 187, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 255}

EBCDIC all character points. From <https://en.wikipedia.org/wiki/EBCDIC>

```
TotalDepth.util.EBCDIC.EBCDIC_PRINTABLE = {5, 11, 12, 13, 37, 64, 74, 75, 76, 77, 78, 79,
80, 90, 91, 92, 93, 94, 95, 96, 97, 107, 108, 109, 110, 111, 121, 122, 123, 124, 125,
126, 127, 129, 130, 131, 132, 133, 134, 135, 136, 137, 145, 146, 147, 148, 149, 150, 151,
152, 153, 161, 162, 163, 164, 165, 166, 167, 168, 169, 187, 192, 193, 194, 195, 196, 197,
198, 199, 200, 201, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 224, 226, 227, 228,
229, 230, 231, 232, 233, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249}
```

Printable range. This is obtained by using cp500:

```
string.printable.encode('cp500')
```

Which gives:

```
EBCDIC_PRINTABLE = {5, 11, 12, 13, 37, 64, 74, 75, 76, 77, 78, 79, 80, 90, 91, 92, ↵
↵ 93, 94, 95, 96, 97, 107, 108, 109,
110, 111, 121, 122, 123, 124, 125, 126, 127, 129, 130, 131, 132, 133, 134, 135, 136,
↵ 137, 145, 146,
147, 148, 149, 150, 151, 152, 153, 161, 162, 163, 164, 165, 166, 167, 168, 169, 187,
↵ 192, 193, 194,
195, 196, 197, 198, 199, 200, 201, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217,
↵ 224, 226, 227,
228, 229, 230, 231, 232, 233, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249}
```

```
class TotalDepth.util.EBCDIC.EbcdicAsciiTableEntry(decimal_str: str, hex_str: str, ctrl_char: str, ascii:
str, ascii_meaning: str, ebcdic: str,
ebcdic_meaning: str)
```

A single entry in the EBCDIC table.

decimal_str: str

Alias for field number 0

hex_str: str

Alias for field number 1

ctrl_char: str

Alias for field number 2

ascii: str

Alias for field number 3

ascii_meaning: str

Alias for field number 4

ebcdic: str

Alias for field number 5

ebcdic_meaning: str

Alias for field number 6

property value: int

Returns the decimal value of the entry.

property is_ctrl: bool

Returns True if this is a control character, False otherwise.

property ctrl_symbol: str

The single control character or ‘’. For example decimal value 3 returns ‘C’.

property ebcdic_printable: bool

Returns True if this is a printable character, False otherwise.

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static __new__(*_cls, decimal_str: str, hex_str: str, ctrl_char: str, ascii: str, ascii_meaning: str, ebcdic: str, ebcdic_meaning: str*)

__repr__()

Return a nicely formatted representation string

TotalDepth.util.EBCDIC.**ebcdic_all_printable**(*byt: bytes*) → bool

True if all character codes are printable EBCDIC.

TotalDepth.util.EBCDIC.**ebcdic_ascii_description**(*byt: bytes*) →

TotalDepth.util.EBCDIC.EbcdicAsciiTableEntry

Look up the first character in the given bytes and return the EBCDIC table entry.

TotalDepth.util.EBCDIC.**ebcdic_to_ascii**(*byt: bytes*) → str

Convert EBCDIC to ASCII.

TotalDepth.util.EBCDIC.**ascii_to_ebcdic**(*ascii_str: str*) → bytes

Convert ASCII to EBCDIC.

9.5 TotalDepth.util.ExecTimer (Timing Code Execution)

Has classes for timing execution

exception TotalDepth.util.ExecTimer.**ExceptionExecTimer**

Specialisation of exception for this module.

class TotalDepth.util.ExecTimer.**Timer**(*description: str*)

Records the timing of a single event.

__init__(*description: str*)

stop(*work_done: int = 0*) → None

Stop the timer and record how much work was done.

add_work_done(*work_done: int*) → None

Adds work done.

property elapsed_perf_counter: float

Executions time in seconds as seen by a wall clock.

property elapsed_wall_clock: float

Executions time in seconds as seen by a wall clock.

__str__() → str

Return str(self).

property ms_mb: float

Return the work rate in ms/MB.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.util.ExecTimer.**TimerList**

Maintains a list of execution time objects

__init__()

Constructor

```

__len__() → int
    Number of task timers.

add_timer(description: str) → None
    Load a new task timer starting right now.

property timer: TotalDepth.util.ExecTimer.Timer
    The current timer.

property has_active_timer
    True if there is a running timer, False if there are either no timers or the latest timer is halted.

stop(work_done=0) → None
    Stop current timer.

__str__()
    Return str(self).

__weakref__
    list of weak references to the object (if defined)

```

9.6 TotalDepth.util.FileBuffer (Look-ahead File Buffer)

Provides a ‘look ahead’ file buffer where the caller can inspect bytes ahead of the current position.

Created on Oct 26, 2011

```

exception TotalDepth.util.FileBuffer.ExceptionFileBuffer
    Specialisation of Exception for the FileBuffer module.

__weakref__
    list of weak references to the object (if defined)

exception TotalDepth.util.FileBuffer.ExceptionFileBufferEOF
    Specialisation of Exception for the FileBuffer EOF.

class TotalDepth.util.FileBuffer.FileBuffer(f)
    Provides a buffer interface to a file where the user can look ahead any distance from the current position.

    __init__(f)

    tell()
        Current file position.

    step()
        Increment the file position by one byte, returns the byte just read.

    __getitem__(i)
        Get an arbitrary byte or slice.

    __weakref__
        list of weak references to the object (if defined)

```

9.7 TotalDepth.util.FileStatus

Created on 28 Jun 2010

exception TotalDepth.util.FileStatus.**ExceptionFileStatus**

__weakref__

list of weak references to the object (if defined)

class TotalDepth.util.FileStatus.**FileInfo**(*thePath*)

Obtains the status of a file, hash SLOC etc.

__init__(*thePath*)

writeHeader(*theS*=<*colorama.ansitowin32.StreamWrapper* object>)

Write the summary header.

write(*theS*=<*colorama.ansitowin32.StreamWrapper* object>, *incHash*=True)

Write the summary.

property **sloc**

SLOC

property **size**

Size in bytes.

property **count**

Number of files, 0 or 1.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.util.FileStatus.**FileInfoSet**(*thePath*, *glob*=None, *isRecursive*=False, *isTestOnly*=False)

Represents a set of files from a directory tree.

__init__(*thePath*, *glob*=None, *isRecursive*=False, *isTestOnly*=False)

processPath(*theP*, *glob*=None, *isRecursive*=False, *isTestOnly*=False)

Process a file or directory.

processDir(*theDir*, *glob*, *isRecursive*, *isTestOnly*)

Read a directory and return a map of {path : class FileInfo, ... }

write(*theS*=<*colorama.ansitowin32.StreamWrapper* object>)

Write out the summary.

__weakref__

list of weak references to the object (if defined)

TotalDepth.util.FileStatus.**main**()

Process a path and write out the file summary.

9.8 TotalDepth.util.Histogram

Produces histograms.

Created on Nov 29, 2011

class TotalDepth.util.Histogram.Histogram(*pre_load=None*)

A histogram class.

__init__(*pre_load=None*)

add(*x, count=1*)

Increments the count of value x by count (default 1).

__getitem__(*x*)

Returns the current count of x.

strRep(*width=75, chr='+', valTitle='', inclCount=False*)

Returns a string representation of the histogram in ASCII.

width - The maximum width to use.

chr - The character to use in the plot.

valTitle - The title to use for values.

inclCount - Include the actual count for each value?

__weakref__

list of weak references to the object (if defined)

9.8.1 Testing

Tests are in test/TestHistogram.py

Running Tests

```
$ python3 test/testHistogram.py
```

Test Coverage

```
$ coverage run test/testHistogram.py
...
$ coverage report -m
```

9.8.2 Examples

```
from TotalDepth.util import Histogram

myH = Histogram.Histogram()
for x in range(1, 12):
    self._hist.add(x, x)
print(self._hist.strRep())
# Prints """ 1 | ++++++
2 | ++++++
3 | ++++++
4 | ++++++
5 | ++++++
6 | ++++++
7 | ++++++
8 | ++++++
9 | ++++++
10 | ++++++
11 | ++++++ """
```

9.9 TotalDepth.util.HtmlUtils (HTML Utilities)

HTML utility functions.

TotalDepth.util.HtmlUtils.retHtmlFileName(*thePath*)

Creates a unique, short, human readable file name base on the input file path.

TotalDepth.util.HtmlUtils.retHtmlFileLink(*theSrcPath*, *theLineNum*)

Returns a link to a file/line.

TotalDepth.util.HtmlUtils.writeHtmlFileLink(*theS*, *theSrcPath*, *theLineNum*, *theText*="", *theClass*=None)

Writes a link to another HTML file that represents source code. *theS* is an XHTML stream. *theSrcPath* is the path of the original source, which will be encoded with `retHtmlFileName()`. *theLineNum* is an integer line number in the target. *theText* is optional navigation text. *theClass* is optional CSS class for the navigation text.

TotalDepth.util.HtmlUtils.writeHtmlFileAnchor(*theS*, *theLineNum*, *theText*="", *theClass*=None)

Write an anchor to the stream.

TotalDepth.util.HtmlUtils.pathSplit(*p*)

Split a path into its components.

TotalDepth.util.HtmlUtils.writeFileListAsTable(*theS*, *theFileLinkS*, *tableAttrs*, *includeKeyTail*)

Writes a list of file names as an HTML table looking like a directory structure. *theFileLinkS* is a list of pairs (*file_path*, *href*). The navigation text in the cell will be the basename of the *file_path*.

TotalDepth.util.HtmlUtils.writeFileListTrippleAsTable(*theS*, *theFileLinkS*, *tableAttrs*, *includeKeyTail*)

Writes a list of file names as an HTML table looking like a directory structure. *theFileLinkS* is a list of triples (*file_name*, *href*, *nav_text*).

TotalDepth.util.HtmlUtils.writeDictTreeAsTable(*theS*, *theDt*, *tableAttrs*, *includeKeyTail*)

Writes a DictTreeHtmlTable object as a table, for example as a directory structure.

The key list in the DictTreeHtmlTable object is the path to the file i.e. `os.path.abspath(p).split(os.sep)` and the value is expected to be a pair of (*link*, *nav_text*) or None.

`TotalDepth.util.HtmlUtils.writeFilePathsAsTable(valueType, theS, theKvS, tableStyle, fnTd)`

Writes file paths as a table, for example as a directory structure.

`valueType` - The type of the value: `None` | `'list'` | `'set'`

`theKvS` - A list of pairs (`file_path`, `value`).

`tableStyle` - The style used for the table.

`fnTd` - A callback function that is executed for a `<td>` element when there is a non-`None` value. This is called with the following arguments:

- `theS` - The HTML stream.
- `attrs` - A map of attrs that include the `rowspan/colspan` for the `<td>`
- `k` - The key as a list of path components.
- `v` - The value given by the caller.

9.10 TotalDepth.util.PatternSearch (Search for Patterns in Binary Files)

Searches for runs of data in binary files.

Created on Oct 24, 2011

@author: paulross

`TotalDepth.util.PatternSearch.reportAll(theFile, theS=<colorama.ansitowin32.StreamWrapper object>)`

Print a histogram of byte values.

`TotalDepth.util.PatternSearch.report0x80(theFile, theS=<colorama.ansitowin32.StreamWrapper object>, theMin=1, showTell=False)`

Print a for 0x80.

`TotalDepth.util.PatternSearch.reportIDENT(theFile, theS=<colorama.ansitowin32.StreamWrapper object>, theMin=2, showTell=False)`

IDENT rep code: The valid character subset consists of null (0) plus the codes 33 0x21 (!) to 96 0x60 (`) and from 123 0x7b ({) to 126 0x7e (~) inclusive. This excludes all control characters, all “white space”, and the lower-case alphabet.

9.11 TotalDepth.util.RemoveDupeFiles (Remove Duplicate Files)

Scans a directory and removes duplicate files based on their SHA.

`TotalDepth.util.RemoveDupeFiles.remove_dupes(path: str, nervous: bool) → Tuple[int, int]`

Scans a directory tree removing duplicate files detected by their SHA512.

`TotalDepth.util.RemoveDupeFiles.main() → int`

Main CLI entry point.

9.12 TotalDepth.util.XmlWrite (XML/XHTML/HTML Writer)

Writes XML and XHTML.

exception TotalDepth.util.XmlWrite.**ExceptionXml**

Exception specialisation for the XML writer.

exception TotalDepth.util.XmlWrite.**ExceptionXmlElement**

Exception specialisation for end of element.

TotalDepth.util.XmlWrite.**encodeString**(*theS*, *theCharPrefix*='_')

Returns a string that is the argument encoded. RFC3548:

Table 1: The Base 64 Alphabet							
Value	Encoding	Value	Encoding	Value	Encoding	Value	Encoding
0	A	17	R	34	i	51	z
1	B	18	S	35	j	52	0
2	C	19	T	36	k	53	1
3	D	20	U	37	l	54	2
4	E	21	V	38	m	55	3
5	F	22	W	39	n	56	4
6	G	23	X	40	o	57	5
7	H	24	Y	41	p	58	6
8	I	25	Z	42	q	59	7
9	J	26	a	43	r	60	8
10	K	27	b	44	s	61	9
11	L	28	c	45	t	62	+
12	M	29	d	46	u	63	/
13	N	30	e	47	v		
14	O	31	f	48	w	(pad)	=
15	P	32	g	49	x		
16	Q	33	h	50	y		

See section 3 of : <http://www.faqs.org/rfcs/rfc3548.html>

TotalDepth.util.XmlWrite.**decodeString**(*theS*)

Returns a string that is the argument decoded. May raise a TypeError.

TotalDepth.util.XmlWrite.**nameFromString**(*theStr*)

Returns a name from a string.

See <http://www.w3.org/TR/1999/REC-html401-19991224/types.html#type-cdata>

“ID and NAME tokens must begin with a letter ([A-Za-z]) and may be followed by any number of letters, digits ([0-9]), hyphens (“-”), underscores (“_”), colons (“:”), and periods (“.”).

This also works for in namespaces as ‘:’ is not used in the encoding.

class TotalDepth.util.XmlWrite.**XmlStream**(*theFout*, *theEnc*='utf-8', *theDtdLocal*=None, *theId*=0)

Creates and maintains an XML output stream.

__init__(*theFout*, *theEnc*='utf-8', *theDtdLocal*=None, *theId*=0)

Initialise with a writable file like object or a file path.

theFout - The file-like object or a string. If the latter it will be closed on **__exit__**.

theEnc - The encoding to be used.

theDtdLocal - Any local DTD as a string.

`id` - An integer value to use as an ID string.

property `id`

A unique ID in this stream. The ID is incremented on each call.

`xmlSpacePreserve()`

Suspends indentation for this element and its descendants.

`characters(theString)`

Encodes the string and writes it to the output.

`literal(theString)`

Writes `theString` to the output without encoding.

`comment(theS)`

Writes a comment to the output stream.

`pI(theS)`

Writes a Processing Instruction to the output stream.

`endElement(name)`

Ends an element.

`writeECMAScript(theScript)`

Writes the ECMA script.

Example:

```
<script type="text/ecmascript">
//<![CDATA[
...
// ]]>
</script>
```

`__enter__()`

Context manager support.

`__exit__(exc_type, exc_value, traceback)`

Context manager support.

`__weakref__`

list of weak references to the object (if defined)

class `TotalDepth.util.XmlWrite.XhtmlStream(theFout, theEnc='utf-8', theDtdLocal=None, theId=0)`

`__enter__()`

Context manager support.

`charactersWithBr(sIn)`

Writes the string replacing any `n` characters with `<br/>` elements.

class `TotalDepth.util.XmlWrite.Element(theXmlStream, theElemName, theAttrs=None)`

Represents an element in a markup stream.

`__init__(theXmlStream, theElemName, theAttrs=None)`

`__enter__()`

Context manager support.

`__weakref__`

list of weak references to the object (if defined)

```
__exit__(excType, excValue, tb)
    Context manager support.
```

9.13 TotalDepth.util.archive (Archive Management)

Extract summary data from archives of log files.

This is a bit hacked together to help create a good archive of test data. It is not production code.

```
class TotalDepth.util.archive.FileBase(path: str)
    Base class to represent a file, either on-disc or a ZIP file.

    __init__(path: str)

    __str__()
        Return str(self).

    __eq__(other)
        Return self==value.

    __lt__(other)
        Return self<value.

    __weakref__
        list of weak references to the object (if defined)

class TotalDepth.util.archive.FileOnDisc(path: str)
    Represents an on-disc file.

    __init__(path: str)

class TotalDepth.util.archive.FileInMemory(path: str, size: int, binary_type: str, mod_date:
                                         datetime.datetime, by: bytes)
    Represents an in-memory file, for example contained in a ZIP. We need to be given the file data as we can't read
    it from disc.

    __init__(path: str, size: int, binary_type: str, mod_date: datetime.datetime, by: bytes)

class TotalDepth.util.archive.FileMembers(archive_path: str, depth: int)
    Represents a tree of files for example a ZIP might contain a ZIP.

    __init__(archive_path: str, depth: int)

    __str__()
        Return str(self).

    __weakref__
        list of weak references to the object (if defined)

class TotalDepth.util.archive.FileArchive(archive_path: str, depth: int)
    Represents a file that is an archive of other files.

    __init__(archive_path: str, depth: int)

class TotalDepth.util.archive.FileZip(archive_path: str)
    Represents an on-disc file that is a ZIP file.

    __init__(archive_path: str)

    __str__()
        Return str(self).
```

`TotalDepth.util.archive.analyse_archive(files: List[TotalDepth.util.archive.FileBase], file_types: List[str], num_bytes: int, include_size_histogram: bool) → None`

Take a list of FileBase and write out the analysis.

`TotalDepth.util.archive.explore_tree_single_process(path: str, recurse: bool) → List[TotalDepth.util.archive.FileBase]`

Single process code to scan an archive.

`TotalDepth.util.archive.explore_tree_multi_process(path: str, recurse: bool, jobs: int) → List[TotalDepth.util.archive.FileBase]`

Multiprocessing code to scan an archive.

`TotalDepth.util.archive.copy_tree(path_from: str, path_to: str, recurse: bool, file_types: List[str], nervous: bool, over_write: bool) → Tuple[Dict[str, int], int]`

Copies particular binary file types from one directory structure to another.

`TotalDepth.util.archive.expand_and_delete_archives(target_dir: str, nervous: bool) → Tuple[int, int]`
 Recursively searches for archives in target_dir, expands them and deletes the original archive. This repeatedly scans the target_dir until no more archive paths are found.

9.14 TotalDepth.util.bin_file_type (Binary File Type Classification)

Identifies the type of file as a string such as “PDF”, “RP66V1” by an analysis (mostly) of the initial bytes of the file.

```
TotalDepth.util.bin_file_type.RE_COMPILED = {'RP66V1': {'Comment_1': re.compile(b'^[0-9]*$'), 'Comment_2': re.compile(b'^V1.(\d\d\d)$'), 'Comment_3': re.compile(b'^RECORD$'), 'Comment_4': re.compile(b'^[0-9]*$')}, 'RP66V2': {'Comment_1': re.compile(b'^[0-9]*$'), 'Comment_2': re.compile(b'^V1.(\d\d\d)$'), 'Comment_3': re.compile(b'^RECORD$'), 'Comment_4': re.compile(b'^B([0-9]+)[ ]*$'), 'Comment_5': re.compile(b'^[ ]*[0-9]*$'), 'Comment_6': re.compile(b'^[ ]*[0-9]*?$', 'Comment_7': re.compile(b'^\\d\\d-[A-Z]{3}-\\d\\d\\d\\d$'), 'Comment_9': re.compile(b'^ $')}}

```

Regular expressions for RP66 files. Keys refer to the documentation.

```
TotalDepth.util.bin_file_type.ASCII_PRINTABLE_BYTES = {9, 10, 11, 12, 13, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126}

```

string.printable contains tab, backspace etc. which is undesirable: '0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz;=>?@[\\]^_`{|}~tnrx0bx0c'

```
TotalDepth.util.bin_file_type.RE_LAS_VERSION_LINE = re.compile(b'^\\s*VERS\\s*\\.\\.\\s+([\\d.]+)\\s*:\\s*(.+)\\s*$')

```

Regex for extracting the LAS version.

```
TotalDepth.util.bin_file_type.LAS_BINARY_FILE_TYPES = {'LAS1.2', 'LAS2.0'}

```

LAS binary file types we support

```
TotalDepth.util.bin_file_type.FUNCTION_ID_MAP: Tuple[Tuple[Callable, str], ...] =
(((<function _rcd>, 'RCD'), (<function _stk>, 'STK'), (<function _bit>, 'BIT'), (<function
_cfbf>, 'CFBF'), (<function _pds>, 'PDS'), (<function _xml>, 'XML'), (<function _pdf>,
'PDF'), (<function _ps>, 'PS'), (<function _zip>, 'ZIP'), (<function _tiff>, 'TIFF'),
(<function _jpeg>, 'JPEG'), (<function _lasv12>, 'LAS1.2'), (<function _lasv20>,
'LAS2.0'), (<function _lasv30>, 'LAS3.0'), (<function _rp66v1>, 'RP66V1'), (<function
_rp66v1_tif>, 'RP66V1t'), (<function _rp66v1_tif_r>, 'RP66V1tr'), (<function _rp66v2>,
'RP66V2'), (<function _dat>, 'DAT'), (<function _segv>, 'SEGY'), (<function _lis_ver>,
'LISVER'), (<function _ascii>, 'ASCII'), (<function _lis>, 'LIS'), (<function _lis>,
'LISStr'), (<function _lis>, 'LIS'))
```

Ordered so that more specific files are earlier in the list, more general ones later. Also, as an optimisation, the more common file formats appear earlier.

```
TotalDepth.util.bin_file_type.BINARY_FILE_TYPE_CODE_WIDTH: int = 8
```

Length of the longest binary file type supported.

```
TotalDepth.util.bin_file_type.BINARY_FILE_TYPES_SUPPORTED: Set[str] = {'ASCII', 'BIT',
'CFBF', 'DAT', 'JPEG', 'LAS1.2', 'LAS2.0', 'LAS3.0', 'LIS', 'LISVER', 'LISStr', 'LISStr',
'PDF', 'PDS', 'PS', 'RCD', 'RP66V1', 'RP66V1t', 'RP66V1tr', 'RP66V2', 'SEGY', 'STK',
'TIFF', 'XML', 'ZIP'}
```

Set of binary file types supported.

```
TotalDepth.util.bin_file_type.LIS_BINARY_FILE_TYPES = {'LIS', 'LISStr', 'LISStr'}
```

LIS binary file types we support

```
TotalDepth.util.bin_file_type.is_lis_file_type(file_type: str) → bool
```

True if the binary file type is supported by the LIS parser.

```
TotalDepth.util.bin_file_type.summary_file_types_supported(short: bool) → str
```

Returns a string of the supported file types. If short is False this is a multi-line string.

```
TotalDepth.util.bin_file_type.binary_file_type(fobj: BinaryIO) → str
```

Function that takes a file object that supports read() and seek() and returns a file type based on the analysis of the contents of the file. On success fobj will be at the start of file. On failure fobj will be in an indeterminate state.

```
TotalDepth.util.bin_file_type.binary_file_type_from_path(path: str) → str
```

Returns a file type based on the analysis of the contents of the file.

```
TotalDepth.util.bin_file_type.format_bytes(by: bytes) → str
```

Formats bytes with xxd.

9.15 TotalDepth.util.gnuplot (Gnuplot Graphs Support)

A HOWTO is here: [TotalDepth-tech-gnuplot](#)

Provides gnuplot support to command line tools.

```
TotalDepth.util.gnuplot.add_gnuplot_to_argument_parser(parser: argparse.ArgumentParser) → None
```

Adds --gnuplot=<DIRECTORY_FOR_GNUPLOT_OUTPUT> to the argument parser as args.gnuplot.

```
TotalDepth.util.gnuplot.version() → bytes
```

For example: b'gnuplot 5.2 patchlevel 6'

```
TotalDepth.util.gnuplot.create_gnuplot_dat(table: Sequence[Sequence[Any]]) → str
```

Returns a pretty formatted string of the data in the given table suitable for use as a gnuplot .dat file.

`TotalDepth.util.gnuplot.invoke_gnuplot`(*path*: str, *name*: str, *table*: Sequence[Sequence[Any]], *plt*: str) → int

Create the plot for name. path - the directory to write the data and plot files to. name - the name of those files. table - the table of values to write to the data file.

Returns the gnuplot error code.

`TotalDepth.util.gnuplot.write_test_file`(*path*: str, *typ*: str) → int

Writes out a Gnuplot test file.

9.16 Plot Package Reference

Contents:

9.16.1 TotalDepth.util.plot.AREACfg (AREA Plotting Configuration)

Module for plotting areas of particular patterns with well log data.

Created on 1 Apr 2011

Provides access to patterns either using the Data URI Scheme (https://en.wikipedia.org/wiki/Data_URI_scheme) or as PNG files. Both can be in monochrome or RGB

`TotalDepth.util.plot.AREACfg.png_location`(*pattern*: str, *is_monochrome*: bool) → str

Returns the absolute path of the location of a PNG file for the pattern.

9.16.2 TotalDepth.util.plot.Coord (Plot Coordinates)

Main Classes

Most classes in this module are `collections.namedtuple` objects.

Class	Description	Attributes
<code>Dim</code>	Linear dimension	value units
<code>Box</code>	A Box	width depth
<code>Pad</code>	Padding around a tree object	prev next, parent child
<code>Margin</code>	Padding around an object	left right top bottom
<code>Pt</code>	A point in Cartesian space	x y

Reference

Provides a fairly basic two dimensional coordinate system.

exception `TotalDepth.util.plot.Coord.ExceptionCoord`

Exception class for representing Coordinates.

exception `TotalDepth.util.plot.Coord.ExceptionCoordUnitConvert`

Exception raised when converting units.

`TotalDepth.util.plot.Coord.BASE_UNITS = 'px'`

Base units for dimensions

```
TotalDepth.util.plot.Coord.UNIT_MAP = {None: 1.0, 'px': 1.0, 'pt': 1.0, 'pc': 12.0,
'in': 72.0, 'cm': 28.346456692913385, 'mm': 2.834645669291339}
```

Map of {unit name : conversion factor to base units, ...}

```
TotalDepth.util.plot.Coord.exactConversion(units_a, units_b='px')
```

Returns True if the two dimension can be converted exactly. This is the case where the units are the same or the factors are exact multiples.

```
TotalDepth.util.plot.Coord.UNIT_MAP_DEFAULT_FORMAT = {None: '%.4f', 'px': '%d', 'pt':
'd', 'pc': '%.2f', 'cm': '%.2f', 'mm': '%.1f', 'in': '%.3f', 'm': '%.4f', 'ft':
'%.4f', 'NM': '%.6f'}
```

Formatting strings for writing attributes. We are trying not to write 3.999999999mm here!

```
TotalDepth.util.plot.Coord.UNIT_MAP_DEFAULT_FORMAT_WITH_UNITS = {None: '%.4f%s', 'px':
'd%s', 'pt': '%d%s', 'pc': '%.2f%s', 'cm': '%.2f%s', 'mm': '%.1f%s', 'in':
'%.3f%s', 'm': '%.4f%s', 'ft': '%.4f%s', 'NM': '%.6f%s'}
```

Map of formatting strings for value and units e.g. to create '0.667in' from (2.0 / 3.0, 'in')

```
TotalDepth.util.plot.Coord.units()
```

Returns the unsorted list of acceptable units.

```
TotalDepth.util.plot.Coord.convert(val, unitFrom, unitTo)
```

Convert a value from one set of units to another.

```
class TotalDepth.util.plot.Coord.Dim(value, units)
```

Represents a dimension as an engineering value i.e. a number and units.

```
scale(factor)
```

Returns a new Dim() multiplied by a factor, units are unchanged.

```
divide(factor)
```

Returns a new Dim() divided by a factor, units are unchanged.

```
convert(u)
```

Returns a new Dim() with units changed and value converted.

```
__str__()
```

Return str(self).

```
__repr__()
```

Return a nicely formatted representation string

```
__format__(format_spec)
```

Default object formatter.

```
__add__(other)
```

Overload self+other, returned result has the sum of self and other. The units chosen are self's unless self's units are None in which case other's units are used (if not None).

```
__sub__(other)
```

Overload self-other, returned result has the difference of self and other. The units chosen are self's unless self's units are None in which case other's units are used (if not None).

```
__iadd__(other)
```

Addition in place, value of other is converted to my units and added.

```
__isub__(other)
```

Subtraction in place, value of other is subtracted.

```
__mul__(other)
```

Multiply by a factor that is a number.

```

__truediv__(other)
    Divide by a factor that is a number.

__imul__(other)
    Indirect multiply by a factor that is a number.

__itruediv__(other)
    Indirect divide by a factor that is a number.

__lt__(other)
    Return self<value.

__le__(other)
    Return self<=value.

__eq__(other)
    Return self==value.

__ne__(other)
    Return self!=value.

__gt__(other)
    Return self>value.

__ge__(other)
    Return self>=value.

```

TotalDepth.util.plot.Coord.**dimIn**(v)
Returns a Dim object with the value in inches.

class TotalDepth.util.plot.Coord.**Box**(width, depth)

```

__str__()
    Return str(self).

__repr__()
    Return a nicely formatted representation string

__format__(format_spec)
    Default object formatter.

```

class TotalDepth.util.plot.Coord.**Pad**(prev, next, parent, child)
Padding around another object that forms the Bounding Box. All 4 attributes are Dim() objects

```

__str__()
    Return str(self).

__repr__()
    Return a nicely formatted representation string

__format__(format_spec)
    Default object formatter.

```

class TotalDepth.util.plot.Coord.**Margin**(left, right, top, bottom)
Margin padding around another object. All 4 attributes are Coord.Dim() objects.

```

__str__()
    Return str(self).

__repr__()
    Return a nicely formatted representation string

```

__format__(*format_spec*)
Default object formatter.

class TotalDepth.util.plot.Coord.**Pt**(*x*, *y*)
A point, an absolute x/y position on the plot area. Members are Coord.Dim().

__eq__(*other*)
Comparison.

__str__()
Return str(self).

__repr__()
Return a nicely formatted representation string

__format__(*format_spec*)
Default object formatter.

convert(*u*)
Returns a new Pt() with units changed and value converted.

scale(*factor*)
Returns a new Pt() scaled by a factor, units are unchanged.

normalise_units(*units=None*)
Returns a point with both x and y with the same units. If units is given then x and y will be in those units.
This may return self or a new point.

TotalDepth.util.plot.Coord.**to_cartesian**(*origin*: TotalDepth.util.plot.Coord.Pt, *radius*:
TotalDepth.util.plot.Coord.Dim, *angle*: float) →
TotalDepth.util.plot.Coord.Pt

Displaces a point by radius in direction angle in radians which is an x to y rotation. dx is cos(angle) and dy is sin(angle). For example in a SVG coordinate system where +x is right and +y down an angle of less than pi/2 will move the point to the right and down. For use in a mapping system where +x is northing/Latitude N and +y easting/Longitude E an angle of less then pi/2 will move the point up and to the right.

TotalDepth.util.plot.Coord.**to_polar**(*pt_from*: TotalDepth.util.plot.Coord.Pt, *pt_to*:
TotalDepth.util.plot.Coord.Pt) →
Tuple[TotalDepth.util.plot.Coord.Dim, float]

Returns a radius as a Dim and angle in radians. NOTE: This uses math.atan2() so returns the result is between -pi and pi.

Will raise if the given points are identical.

TotalDepth.util.plot.Coord.**baseUnitsDim**(*theLen*)
Returns a Coord.Dim() of length and units BASE_UNITS.

Parameters *theLen* (float, int) – Length.

Returns cpip.plot.Coord.Dim([float, str]) – A new dimension of theLen in base units.

TotalDepth.util.plot.Coord.**zeroBaseUnitsDim**()
Returns a Coord.Dim() of zero length and units BASE_UNITS.

Returns cpip.plot.Coord.Dim([float, str]) – A new dimension of zero.

TotalDepth.util.plot.Coord.**zeroBaseUnitsBox**()
Returns a Coord.Box() of zero dimensions and units BASE_UNITS.

TotalDepth.util.plot.Coord.**zeroBaseUnitsPad**()
Returns a Coord.Pad() of zero dimensions and units BASE_UNITS.

TotalDepth.util.plot.Coord.**zeroBaseUnitsPt**()

Returns a Coord.Dim() of zero length and units BASE_UNITS.

Returns cpip.plot.Coord.Pt([cpip.plot.Coord.Dim([float, str]), cpip.plot.Coord.Dim([float, str])]) – A new point with the values [0, 0].

TotalDepth.util.plot.Coord.**pxUnitsDim**(*theLen*)

Returns a Coord.Dim() of length and units 'px'.

Parameters *theLen* (float, int) – Length.

Returns cpip.plot.Coord.Dim([float, str]) – A new dimension of theLen in base units.

TotalDepth.util.plot.Coord.**pxBaseUnitsDim**()

Returns a Coord.Dim() of zero length and units 'px'.

Returns cpip.plot.Coord.Dim([float, str]) – A new dimension of zero.

TotalDepth.util.plot.Coord.**pxBaseUnitsBox**()

Returns a Coord.Box() of zero dimensions and units 'px'.

TotalDepth.util.plot.Coord.**pxBaseUnitsPad**()

Returns a Coord.Pad() of zero dimensions and units 'px'.

TotalDepth.util.plot.Coord.**pxBaseUnitsPt**()

Returns a Coord.Dim() of zero length and units 'px'.

Returns cpip.plot.Coord.Pt([cpip.plot.Coord.Dim([float, str]), cpip.plot.Coord.Dim([float, str])]) – A new point with the values [0, 0].

TotalDepth.util.plot.Coord.**newPt**(*theP*, *incX=None*, *incY=None*)

Returns a new Pt object by incrementing existing point incX, incY that are both Dim() objects or None.

Parameters

- **theP** (cpip.plot.Coord.Pt([cpip.plot.Coord.Dim([float, str]), cpip.plot.Coord.Pt([cpip.plot.Coord.Dim([float, str])])]) – The initial point.
- **incX** (NoneType, cpip.plot.Coord.Dim([float, str]), cpip.plot.Coord.Dim([int, str])) – Distance to move in the x axis.
- **incY** (NoneType, cpip.plot.Coord.Dim([float, str]), cpip.plot.Coord.Dim([int, str])) – Distance to move in the y axis.

Returns cpip.plot.Coord.Pt([cpip.plot.Coord.Dim([float, str]), cpip.plot.Coord.Dim([float, str])]) – The new point.

TotalDepth.util.plot.Coord.**convertPt**(*theP*, *theUnits*)

Returns a new point with the dimensions of theP converted to theUnits.

TODO: Deprecate this.

TotalDepth.util.plot.Coord.**mirrorPt**(*start*: TotalDepth.util.plot.Coord.Pt, *finish*:

TotalDepth.util.plot.Coord.Pt) → TotalDepth.util.plot.Coord.Pt

Returns a new point that is the mirror of the finish point, 180 degrees from start to finish.

Examples

Coord.Dim()

Creation, addition and subtraction:

```
d = Coord.Dim(1, 'in') + Coord.Dim(18, 'px')
# d is 1.25 inches
d = Coord.Dim(1, 'in') - Coord.Dim(18, 'px')
# d is 0.75 inches
d += Coord.Dim(25.4, 'mm')
# d is 1.75 inches
```

Scaling and unit conversion returns a new object:

```
a = Coord.Dim(12, 'px')
b = myObj.scale(6.0)
# b is 72 pixels
c = b.convert('in')
# 1 is 1 inch
```

Comparison:

```
assert(Coord.Dim(1, 'in') == Coord.Dim(72, 'px'))
assert(Coord.Dim(1, 'in') >= Coord.Dim(72, 'px'))
assert(Coord.Dim(1, 'in') <= Coord.Dim(72, 'px'))
assert(Coord.Dim(1, 'in') > Coord.Dim(71, 'px'))
assert(Coord.Dim(1, 'in') < Coord.Dim(73, 'px'))
```

Coord.Pt()

Creation:

```
p = Coord.Pt(
    Coord.Dim(12, 'px'),
    Coord.Dim(24, 'px'),
)
print(p)
# Prints: 'Pt(x=Dim(12px), y=Dim(24px))'
p.x # Coord.Dim(12, 'px')
p.y # Coord.Dim(24, 'px')
# Scale up by 6 and convert units
pIn = p.scale(6).convert('in')
# pIn now 'Pt(x=Dim(1in), y=Dim(2in))'
```

Testing

The unit tests are in `test/TestCoord.py`.

9.16.3 TotalDepth.util.plot.FILMCfg (FILM Plotting Configuration)

Represents the part of a plot configuration that, typically, can be obtained from a LIS FILM table.

Created on 21 Mar 2011

Example of the data in a film table:

```
Table record (type 34) type: FILM
MNEM  GCOD  GDEC  DEST  DSCA
-----
1      EEE   ----  PF2   D200
2      EEE   ----  PF1   DM
```

Other FILM Table Examples:

```
Table record (type 34) type: FILM
MNEM  GCOD  GDEC  DEST  DSCA
-----
1      EEB   ----  PF1   D200
2      EEB   ----  PF2   DM

Table record (type 34) type: FILM
MNEM  GCOD  GDEC  DEST  DSCA
-----
1      E20   -4--  PF1   D200
2      EEE   ----  PF2   D200

MNEM  GCOD  GDEC  DEST  DSCA
-----
1      EEE   ----  PF1   D200
2      E1E   -4-   PF2   D200

Table record (type 34) type: FILM
MNEM  GCOD  GDEC  DEST  DSCA
-----
D      E3E   -3-   PFD   D200
E      E3E   -3-   PFE   D500
5      EB0   ---   PF5   D200
6      EEB   ---   PF6   D200

Table record (type 34) type: FILM
MNEM  GCOD  GDEC  DEST  DSCA
```

(continues on next page)

(continued from previous page)

```

-----
8      EB0    ---   PF8   D200
A      LLLL  1111  PFA    DM
E      E4E   -4-   PFE    D200
K      E4E   -4-   PFK    D500

```

Table record (type 34) type: FILM

MNEM	CINT	GCOD	GDEC	DEST	DSCA
1	0.00000	E2E	-2-	PF1	D200
2	0.00000	E2E	-1-	PF2	D500
3	0.00000	EEE	----	NEIT	S5
4	0.00000	EEE	----	NEIT	S5

Table record (type 34) type: FILM

MNEM	CINT	GCOD	GDEC	DEST	DSCA
1	300.000	E2E	-2-	PF1	D200
2	300.000	E2E	-2-	PF2	D500
3	300.000	EEE	----	NEIT	S5
4	300.000	EEE	----	NEIT	S5

Table record (type 34) type: FILM

MNEM	CINT	GCOD	GDEC	DEST	DSCA
1	50.0000	EEE	----	PF1	D200
2	0.00000	EEE	----	PF2	D200
3	0.00000	EEE	----	NEIT	D200
4	0.00000	EEE	----	NEIT	D200

Minimal, but not complete interpretation:

Ignore GDEC as dupe.

- E - Equi-spaced (linear).
- n - Log with number of decades.
- B - Blank.
- L - ?

What to do with 0 (continuation?). Examples: E20 -4- means 4 decades over track 23.

Four tracks, from 200099.S07:

```

Table record (type 34) type: FILM
MNEM  GCOD  GDEC  DEST  DSCA
-----

```

(continues on next page)

(continued from previous page)

```

...
A      LLLL  1111  PFA   DM
...

```

Table record (type 34) type: PRES

MNEM	OUTP	STAT	TRAC	CODI	DEST	MODE	FILT	LEDG	REDG	COLO
...										
AX1A	AX	ALLO	F2	LLIN	A	SHIF	0.500000	-9.000000	9.000000	400
AY1A	AY	ALLO	F2	LSPO	A	SHIF	0.500000	-9.000000	9.000000	400
AZ1A	AZ	ALLO	F2	LGAP	A	NB	0.500000	-9.000000	9.000000	400
AN1A	ANOR	ALLO	F4	HDAS	A	NB	0.500000	9.000000	11.0000	420
CS1A	CS	DISA	F1	LDAS	A	NB	0.500000	0.000000	150000.	000
FX1A	FX	ALLO	F3	LLIN	A	NB	0.500000	-0.700000	0.700000	003
FY1A	FY	ALLO	F3	LGAP	A	NB	0.500000	-0.700000	0.700000	003
FZ1A	FZ	ALLO	F3	LSPO	A	NB	0.500000	-0.700000	0.700000	003
FN1A	FNOR	ALLO	F4	LLIN	A	NB	0.500000	0.200000	0.700000	420
FI1A	FINC	ALLO	F4	LGAP	A	NB	1.000000	0.000000	90.0000	420
GA1A	GADZ	DISA	F1	LLIN	A	NB	1.000000	-1.000000	1.000000	020
GP1A	GPV	DISA	F1	LGAP	A	NB	1.000000	14.0000	16.0000	020
GN1A	GNV	DISA	F1	LDAS	A	NB	1.000000	-16.0000	-14.0000	020
GM1A	GMT	DISA	F3	HDAS	A	SHIF	1.000000	80.0000	130.000	020
GA2A	GAT	DISA	F2	HDAS	A	SHIF	1.000000	80.0000	130.000	020
SI1A	SILO	DISA	FD	HLIN	A	NB	1.000000	0.000000	20.0000	000
ST1A	STIT	DISA	FD	LLIN	A	NB	1.000000	0.000000	20.0000	000
ST2A	STIA	ALLO	FD	LLIN	A	NB	1.000000	0.000000	20.0000	000
TE1A	TENS	DISA	FD	LDAS	A	WRAP	0.500000	2000.00	7000.00	000
...										

TODO: What about CINT and FORM headings?

2011-05-27 Frequency Analysis done on:

```

$ python3 TableHistogram.py -k --name=FILM -l40 --col=DEST ../../pLogicTestData/LIS/
Cmd: TableHistogram.py -k --name=FILM -l40 --col=DEST ../../pLogicTestData/LIS/
2011-05-27 09:23:23,870 ERROR      Can not read LIS file ../../pLogicTestData/LIS/13576.
↳S1 with error: Can not fit integer number of frames length 120 into LR length 824,
↳modulo 104 [indirect size 0].
2011-05-27 09:23:24,649 ERROR      Can not read LIS file ../../pLogicTestData/LIS/13610.
↳S1 with error: Can not fit integer number of frames length 7176 into LR length 13354,
↳modulo 6178 [indirect size 0].
...

```

GCOD:

```

===== Count of all table entries =====
{
"(34, b'FILM', b'GCOD', b'BBB ')": 2,
"(34, b'FILM', b'GCOD', b'E1E ')": 5,
"(34, b'FILM', b'GCOD', b'E20 ')": 26,
"(34, b'FILM', b'GCOD', b'E2E ')": 40,
"(34, b'FILM', b'GCOD', b'E3E ')": 3,
"(34, b'FILM', b'GCOD', b'E40 ')": 2,

```

(continues on next page)

(continued from previous page)

```

"(34, b'FILM', b'GCOD', b'E4E '): 3,
"(34, b'FILM', b'GCOD', b'EB0 '): 2,
"(34, b'FILM', b'GCOD', b'EEB '): 22,
"(34, b'FILM', b'GCOD', b'EEE '): 225,
"(34, b'FILM', b'GCOD', b'LLLL'): 1,
}
===== Count of all table entries END =====

```

GDEC:

```

===== Count of all table entries =====
{
"(34, b'FILM', b'GDEC', b'--- '): 10,
"(34, b'FILM', b'GDEC', b'----'): 227,
"(34, b'FILM', b'GDEC', b'-1- '): 2,
"(34, b'FILM', b'GDEC', b'-2- '): 6,
"(34, b'FILM', b'GDEC', b'-2--'): 32,
"(34, b'FILM', b'GDEC', b'-3- '): 3,
"(34, b'FILM', b'GDEC', b'-4- '): 10,
"(34, b'FILM', b'GDEC', b'-4--'): 26,
"(34, b'FILM', b'GDEC', b'1111'): 1,
"(34, b'FILM', b'GDEC', b'EEE '): 14,
}
===== Count of all table entries END =====

```

DEST:

```

===== Count of all table entries =====
{
"(34, b'FILM', b'DEST', b'NEIT'): 66,
"(34, b'FILM', b'DEST', b'PF1 '): 124,
"(34, b'FILM', b'DEST', b'PF2 '): 125,
"(34, b'FILM', b'DEST', b'PF5 '): 1,
"(34, b'FILM', b'DEST', b'PF6 '): 5,
"(34, b'FILM', b'DEST', b'PF8 '): 1,
"(34, b'FILM', b'DEST', b'PFA '): 1,
"(34, b'FILM', b'DEST', b'PFD '): 2,
"(34, b'FILM', b'DEST', b'PFE '): 3,
"(34, b'FILM', b'DEST', b'PFJ '): 2,
"(34, b'FILM', b'DEST', b'PFK '): 1,
}
===== Count of all table entries END =====

```

DSCA:

```

===== Count of all table entries =====
{
"(34, b'FILM', b'DSCA', b'D200'): 156,
"(34, b'FILM', b'DSCA', b'D500'): 17,
"(34, b'FILM', b'DSCA', b'DM '): 76,
"(34, b'FILM', b'DSCA', b'S5 '): 82,
}
===== Count of all table entries END =====

```

exception TotalDepth.util.plot.FILMCfg.ExceptionFILMCfg

Specialisation of exception for this module.

exception TotalDepth.util.plot.FILMCfg.ExceptionPhysFilmCfg

Specialisation of exception for PhysFilmCfg.

exception TotalDepth.util.plot.FILMCfg.ExceptionFilmCfgLISRead

Specialisation of exception for FilmCfgLISRead in this module.

class TotalDepth.util.plot.FILMCfg.PhysFilmCfg(*theName, theTracks, theDest, theX*)

Contains the configuration equivalent to a single line in a FILM table.

theName is a hashable.

theTracks is a list of Track.Track objects.

theX is an integer scale.

__init__(*theName, theTracks, theDest, theX*)

Constructor. theName is a hashable. theTracks is a list of Track.Track objects. theX is an integer scale.

property name

Name of the FILM.

property xScale

The FILM X axis scale as a number.

__len__()

Number of Track.Track objects.

__getitem__(*i*)

Returns the Track.Track object at position i.

genTracks()

Generate all tracks.

interpretTrac(*theTracStr*)

Turns TRAC information into left/right positions as Coord.Dim() objects and the number of half-tracks of the start and half-tracks covered. The later two values are used for stacking and packing the plot header and footer scales so that they take the minimum space.

e.g. b'T23 ' returns: (the left position of T2, right of T3, 4, 4).

e.g. b'T2 ' returns: (the left position of T2, right of T2, 4, 2).

Note: There is some fudging going on here

__weakref__

list of weak references to the object (if defined)

class TotalDepth.util.plot.FILMCfg.PhysFilmCfgLISRead(*theRow*)

Tracks from a LIS FILM table, essentially the pair of GCOD and GDEC defines the left-to-right layout of the plot.

Grid codes from analysis above: b'BBB ', b'E1E ', b'E20 ', b'E2E ', b'E3E ', b'E40 ', b'E4E ', b'EB0 ', b'EEB ', b'EEE ', b'LLLL'

The DSCA defines the top-to-bottom nature of the plot. DSCA codes from analysis above: b'D200', b'D500', b'DM ', b'S5 '

But we can guess some others...

```
DSCA_MAP = {b'D20 ': 20, b'D200': 200, b'D240 ': 240, b'D40 ': 40, b'D500':  
500, b'DM ': 1000, b'S2 ': 600, b'S5 ': 240}
```

X axis scale from a LIS FILM table DSCA codes from analysis above: b'D200', b'D500', b'DM ', b'S5 '
But we can guess some others...

__init__(*theRow*)

Reads a LogiRec.TableRow object and populates a CurveCfg.

Example:

MNEM	GCOD	GDEC	DEST	DSCA
1	E20	-4--	PF1	D200

supportedFilmTracks()

A list of supported film (name, decade) pairs.

class TotalDepth.util.plot.FILMCfg.FilmCfg

Contains the configuration equivalent to a complete FILM table.

__init__()

add(*k, thePfc*)

Add a PhysFilmCfg object to the map with key *k*, typically a FILM mnemonic in bytes such as Mnem.Mnem(b'1 ') or some other ID such as a string, the filename of an XML file.

keys()

All FILM Mnemonics.

__len__()

Number of unique film destination names.

__getitem__(*name*)

Returns the PhysFilmCfg object corresponding to *name* - a Mnem() object. Will raise KeyError if not exact match. See retFilmDest() for an API that can handle curve destinations of BOTH, ALL etc.

__contains__(*name*)

Membership test.

retAllFILMDestS(*curveDestID*)

Returns an unordered list of FILM destinations for a curve destination. For example if curveDestID is b'BOTH' this might return [b'2 ', b'1 ']

retFILMDest(*filmDestID, curveDestID*)

Returns a PhysFilmCfg object by matching curveDestID to the filmDestID. Returns None on failure. For LIS curveDestID can be 1, BOTH, ALL, NEIT etc. This is commonly used by the PRESCfg module so that interpretTrac() can be called on the result and thus build up a map of track positions for all possible logical film outputs.

interpretTrac(*filmDestID, curveDestID, trackStr*)

Given a film destination ID and a curve destination (which could be b'ALL') and a track string (e.g. b'T23') this returns the left/right positions as Coord.Dim() objects and the number of half-tracks of the start and half-tracks covered (used for plot header and footer scales). Returns None if there is no match for the filmDestID/curveDestID (for example if curveDestID is b'NEIT'). e.g. (b'1 ', b'BOTH', b'T23 ') returns: (the left position of T2, right of T3, 4, 4).

__weakref__

list of weak references to the object (if defined)

class TotalDepth.util.plot.FILMCfg.FilmCfgLISRead(*theLr*)

Interprets a FILM table from a LIS Logical Record.

__init__(theLr)

Reads a LogiRec.Table object and creates a PhysFilmCfgLISRead for each row.

Typical FILM table:

MNEM	GCOD	GDEC	DEST	DSCA

1	E20	-4--	PF1	D200
2	EEE	----	PF2	D200

retAllFILMDestS(curveDestID)

Returns an unordered list of FILM destinations for a curve destination.

For example if curveDestID is b'BOTH' this might return [b'2 ', b'1 ']

retFILMDest(filmDestID, curveDestID)

Returns a PhysFilmCfg object by matching curveDestID to the filmDestID. Returns None on failure. curveDestID can be 1, BOTH, ALL, NEIT etc.

This is commonly used by the PRESCfg module so that interpretTrac() can be called on the result and thus build up a map of track positions for all possible logical film outputs.

9.16.4 TotalDepth.util.plot.FILMCfgXML (FILM Plotting Configuration from XML files)

Creates FILM configurations from LgFormat XML files.

Created on Dec 14, 2011

exception TotalDepth.util.plot.FILMCfgXML.ExceptionFILMCfgXML

Specialisation of exception for FILMCfgXML module.

exception TotalDepth.util.plot.FILMCfgXML.ExceptionFILMCfgXMLRead

Specialisation of exception for FILMCfgXMLRead module.

exception TotalDepth.util.plot.FILMCfgXML.ExceptionFILMCfgXMLReadLookup

Specialisation of exception for FILMCfgXMLRead module when a lookup fails that would normally raise a KeyError.

class TotalDepth.util.plot.FILMCfgXML.PhysFilmCfgXMLRead(root)

Extracts FILM information from a single XML file root element.

TRAC_STANDARD_SPAN = 2

Default number of half-tracks to span

TRAC_UNIQUEID_TO_NON_STANDARD_SPAN = {'TrackFC1': 1, 'TrackFC2': 1, 'TrackFC3': 1, 'TrackFC4': 1, 'track12': 4, 'track23': 4, 'trackLHT1': 1, 'trackRHT1': 1}

Map of track IDs to start track location in half-tracks

TRAC_UNIQUEID_TO_NON_STANDARD_START = {'TrackFC1': 4, 'TrackFC2': 5, 'TrackFC3': 6, 'TrackFC4': 7, 'track12': 2, 'track23': 4, 'trackLHT1': 0, 'trackRHT1': 1}

Number of half-tracks of the start of the track, this is a fudge around deciding this from the order of the LgTrack elements

__init__(root)

Constructor. theName is a hashable. theTracks is a list of Track.Track objects. theX is an integer scale.

interpretTrac(theTracStr)

Turns TRAC information into left/right positions as Coord.Dim() objects and the number of half-tracks of the start and half-tracks covered. The later two values are used for stacking and packing the plot header

and footer scales so that they take the minimum space. e.g. b'T23 ' returns: (the left position of T2, right of T3, 4, 4). e.g. b'T2 ' returns: (the left position of T2, right of T2, 4, 2). Note: There is some fudging going on here

class TotalDepth.util.plot.FILMCfgXML.FilmCfgXMLRead(*directory=None*)

Contains the configuration equivalent to a complete FILM table from a set of XML files.

__init__(*directory=None*)

Constructor with a directory, all files in the directory (non-recursive) are read as LGFormat XML files. If dir is None then the "formats/" directory relative to this module is searched. If dir is an empty string then no search will be done - useful for testing with addXMLRoot(etree.fromstring(.)).

readDir(*d=None*)

Read a directory of XML files (not recursive).

addXMLRoot(*theRoot*)

Adds a parsed LgFormat XML document to the IR.

chOutpMnemInFilmId(*chOutp, filmID*)

Returns True if this curve appears in the film.

longStr(*verbose=1*)

Returns a long string of the XML UniqueIds and their description.

uniqueIds()

Returns the UniqueId values that I know about.

description(*theUID*)

Returns the XML description corresponding to the Unique ID or None if unknown.

rootNode(*theUID*)

Returns the XML root node corresponding to the Unique ID.

retAllFILMDestS(*curveDestID*)

Returns an unordered list of FILM destinations for a curve destination. For example if curveDestID is b'BOTH' this might return [b'2 ', b'1 ']

retFILMDest(*filmDestID, curveDestID*)

Returns a PhysFilmCfgXMLRead object by matching curveDestID to the filmDestID. Returns None on failure. filmDestID and curveDestID are implementation specific e.g. for LIS the curveDestID can be 1, BOTH, ALL, NEIT etc. This is commonly used by the PRESCfg module so that interpretTrac() can be called on the result and thus build up a map of track positions for all possible logical film outputs.

9.16.5 TotalDepth.util.plot.LogHeader (Plotting Well Log API headers)

Plots various headers as SVG. Of note is the API header

Created on Dec 30, 2011

exception TotalDepth.util.plot.LogHeader.ExceptionLogHeader

Exception for plotting Log Headers.

exception TotalDepth.util.plot.LogHeader.ExceptionLogHeaderLIS

Exception for plotting Log Headers from LIS data.

exception TotalDepth.util.plot.LogHeader.ExceptionLogHeaderLAS

Exception for plotting Log Headers from LAS data.

class TotalDepth.util.plot.LogHeader.Static(*x, y, w, d, text, font, size, tAttr, rAttr, mnem, xMnem*)

Tuple to describe static data locations If text is None then no text will be plotted If rAttr is None then no box will be plotted If mnem is non-None then a Mmem can be plotted at xMenm increment, a Coord.Dim()

__getnewargs__()
Return self as a plain tuple. Used by copy and pickle.

static __new__(_cls, x, y, w, d, text, font, size, tAttr, rAttr, mnem, xMnem)

__repr__()
Return a nicely formatted representation string

d
Alias for field number 3

font
Alias for field number 5

mnem
Alias for field number 9

rAttr
Alias for field number 8

size
Alias for field number 6

tAttr
Alias for field number 7

text
Alias for field number 4

w
Alias for field number 2

x
Alias for field number 0

xMnem
Alias for field number 10

y
Alias for field number 1

TotalDepth.util.plot.LogHeader.FONT_PROP = 'Verdana'
Default font

TotalDepth.util.plot.LogHeader.RECT_ATTRS_FINE = {'fill': 'none', 'stroke': 'black',
'stroke-width': '.5'}
SVG attributes for a fine lined rectangle

TotalDepth.util.plot.LogHeader.TEXT_ATTRS_FINE = {'text-anchor': 'start'}
SVG attributes for a fine text

TotalDepth.util.plot.LogHeader.TEXT_ATTRS_LARGE = {'font-weight': 'bold', 'text-anchor':
'start'}
SVG attributes for a bold text

TotalDepth.util.plot.LogHeader.TEXT_ATTRS_LARGE_WOB = {'fill': 'white', 'font-weight':
'bold', 'text-anchor': 'start'}
SVG attributes for a large WOB text

TotalDepth.util.plot.LogHeader.HEADER_PLOT_UNITS = 'in'
Standard plot units used in the layout definition

class TotalDepth.util.plot.LogHeader.APIHeaderBase(*isTopOfLog=False*)
Base class to be used by APIHeaderLIS or APIHeaderLAS.

If isTopOfLog is True plot is rotated 90 deg as if to fit on top of a traditional log.

__init__(*isTopOfLog=False*)

Constructor with flag that controls plot orientation. If True plot is rotated 90 deg as if to fit on top of a traditional log.

missingFields(*theWsd*)

Returns two sets: A set of mnemonics that could be plotted but are not in the Logical Record(s). A set of mnemonics that are in the Logical Record(s) but could not be plotted.

size()

Returns a Coord.Box for my size, currently a single page on fan folded paper.

viewPort(*theTl*)

The SVG viewport.

plot(*xS, theTl, theWsdS=None*)

Write the header to the SVG stream at position offset top left. theWsd is a list of records that contain well site data. Will raise ExceptionLogHeader is wrong type of Logical Record.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.util.plot.LogHeader.**APIHeaderLIS**(*isTopOfLog=False*)

Can lay out an API header from LIS information, specifically a type 34 CONS record.

If isTopOfLog is True plot is rotated 90 deg as if to fit on top of a traditional log.

__init__(*isTopOfLog=False*)

Constructor with a Logical Record.

missingFields(*theWsd*)

Well site data (theWsd) in LIS is a list of CONS Logical Records.

Returns two sets:

A set of mnemonics that could be plotted but are not in the Logical Record(s).

A set of mnemonics that are in the Logical Record(s) but could not be plotted.

lrDataCount(*theLrS*)

Returns the number of Mnem's in MNEM_SET that could be plotted in a header that are found in all the Logical Records.

class TotalDepth.util.plot.LogHeader.**APIHeaderLAS**(*isTopOfLog=False*)

Can lay out an API header from a representation of a LAS file.

LIS_MNEM_TO_LAS_MNEM = {'BLI': 'STRT', 'CN': 'COMP', 'DFD': 'FD', 'FL': 'LOC',
'FL1': 'PROV', 'FL2': 'UWI', 'FN': 'FLD', 'LATI': 'LAT', 'LONG': 'LON', 'LUL':
'SRVC', 'MRT': 'BHT', 'TDL': 'DL', 'TLI': 'STOP', 'WN': 'WELL'}

Some conversions from LIS standard to LAS standard All as truncated ascii strings i.e. using
pStr(strip=True)

__init__(*isTopOfLog=False*)

Constructor with a Logical Record.

missingFields(*theLasFile*)

Returns two sets:

A set of mnemonics that could be plotted but are not in the Logical Record(s).

A set of mnemonics that are in the Logical Record(s) but could not be plotted.

9.16.6 TotalDepth.util.plot.Plot (Plotting Well Logs)

Created on 28 Feb 2011

Plotting LIS data requires these components:

- A PlotConfig object
- A data set (e.g. a LogPass with a FrameSet).
- An output driver (e.g. screen, print PDF, web SVG).

User creates a PlotConfig (this reflects a PRES table). This is reusable.

User specifies a data set (from, to, channels etc.). e.g. `LogPass.setFrameSet(File, theFrameSlice=theFrameSlice, theChList=theChList)`

User says 'plot this data set with this configuration to this output device'. e.g. `Plot(PlotConfig, LogPass, PlotDevice)` Plot uses `LogPass.genChScValues(ch, sc)` to plot individual curves.

Lacunae

Area plotting. Caching (e.g. SVG fragments - is this worth it?)

PlotConfig

PlotTracks

Typically a three track (+depth) have these dimensions in inches:

Track	Left	Right	Width
1	0	2.4	2.4
Depth	2.4	3.2	0.8
2	3.2	5.6	2.4
3	5.6	8.0	2.4

Track names can be split (e.g. LHT1 is left hand track 1) or merged (T23 is spread across tracks two and three).

Examples of PRES and FILM records:

Table record (type 34) type: PRES									
MNEM	OUTP	STAT	TRAC	CODI	DEST	MODE	FILT	LEDG	REDG
SP	SP	ALLO	T1	LLIN	1	SHIF	0.500000	-80.0000	20.0000
CALI	CALI	ALLO	T1	LDAS	1	SHIF	0.500000	5.00000	15.0000
MINV	MINV	DISA	T1	LLIN	1	SHIF	0.500000	30.0000	0.00000
MNOR	MNOR	DISA	T1	LDAS	1	SHIF	0.500000	30.0000	0.00000
LLD	LLD	ALLO	T23	LDAS	1	GRAD	0.500000	0.200000	2000.00
LLDB	LLD	ALLO	T2	HDAS	1	GRAD	0.500000	2000.00	200000.
LLG	LLG	DISA	T23	LDAS	1	GRAD	0.500000	0.200000	2000.00
LLGB	LLG	DISA	T2	HDAS	1	GRAD	0.500000	2000.00	200000.
LLS	LLS	ALLO	T23	LSPO	1	GRAD	0.500000	0.200000	2000.00
LLSB	LLS	ALLO	T2	HSPO	1	GRAD	0.500000	2000.00	200000.

(continues on next page)

(continued from previous page)

MSFL	MSFL	ALLO	T23	LLIN	1	GRAD	0.500000	0.200000	2000.00
Table record (type 34) type: FILM									
MNEM	GCOD	GDEC	DEST	DSCA					

1	E20	-4--	PF1	D200					
2	EEE	----	PF2	D200					
Table record (type 34) type: PRES									
MNEM	OUTP	STAT	TRAC	CODI	DEST	MODE	FILT	LEDG	REDG

NPHI	NPHI	ALLO	T23	LDAS	1	SHIF	0.500000	0.450000	-0.150000
DRHO	DRHO	ALLO	T3	LSPO	1	NB	0.500000	-0.250000	0.250000
PEF	PEF	ALLO	T23	LGAP	1	SHIF	0.500000	0.000000	10.0000
SGR		DISA	T1	LLIN	1	SHIF	0.500000	0.000000	300.000
CGR		DISA	T1	LGAP	1	SHIF	0.500000	0.000000	300.000
TENS	TENS	DISA	T3	LGAP	1	SHIF	0.500000	14000.0	4000.00
CAL	CALI	ALLO	T1	LSPO	1	SHIF	0.500000	5.000000	15.0000
BS	BS	DISA	T1	LGAP	1	SHIF	0.500000	5.000000	15.0000
FFLS	FFLS	DISA	T1	LLIN	2	NB	0.500000	-0.150000	0.150000
FFSS	FFSS	DISA	T1	LDAS	2	NB	0.500000	-0.150000	0.150000
LSHV	LSHV	DISA	T3	LLIN	2	WRAP	0.500000	2150.00	2250.00
SSHV	SSHV	DISA	T3	LDAS	2	WRAP	0.500000	1950.00	2050.00
FLS	FLS	DISA	T2	LLIN	2	SHIF	0.500000	0.000000	150.000
FSS	FSS	DISA	T2	LDAS	2	SHIF	0.500000	0.000000	150.000
RHOB	RHOB	ALLO	T23	LLIN	1	SHIF	0.500000	1.950000	2.950000
PHIX	PHIX	ALLO	T1	LLIN	1	NB	0.500000	0.500000	0.000000

TotalDepth.util.plot.Plot.COMMENTS_IN_SVG_TRACE = False

Allows detailed trace comments to appear in the SVG

TotalDepth.util.plot.Plot.COMMENTS_IN_SVG_SECTION = True

A few comments in SVG to delinialte header, Grid, Xaxis, curves etc

TotalDepth.util.plot.Plot.COMMENTS_IN_SVG_SECTION_WIDTH = 40

Width of comment for sections

TotalDepth.util.plot.Plot.COMMENTS_IN_SVG_SECTION_LEVEL_TUPLE = ('=', '.', '^')

Map of section level to comment padding character

exception TotalDepth.util.plot.Plot.ExceptionTotalDepthLISPlot

Exception for plotting.

exception TotalDepth.util.plot.Plot.ExceptionTotalDepthPlotRoll

Exception for plotting.

class TotalDepth.util.plot.Plot.CurvePlotScale(name, halfTrackStart, halfTracks)

Holds a minimal amount of curve plot scale and so on for the layout of the scale pane at each end of the log.

halfTrackStart is the start of the curve for a standard three track log T1=0, TD=2, T2=4, T3=6

halfTracks is the curve span as an integer so T23=4 LHT1=1 and so on.

__lt__(*other*)

Slightly weird sort order, larger halfTracks come first then names.

class TotalDepth.util.plot.Plot.**ScaleSliceCurve**(*slice, curveName, start, span*)

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static **__new__**(*_cls, slice, curveName, start, span*)

__repr__()

Return a nicely formatted representation string

curveName

Alias for field number 1

slice

Alias for field number 0

span

Alias for field number 3

start

Alias for field number 2

class TotalDepth.util.plot.Plot.**CurvePlotScaleSlotMap**(*theCpsS*)

Keeps track of which slots are available for putting the curve scales in at the top and bottom of the log. This scale are is divided into slices that span the plot from left to right. These slices are subdivided into slots that correspond to a half-track in that slice.

__init__(*theCpsS*)

Ctor with a list of CurvePlotScale objects (can be unsorted).

reset()

Clears the slot map for the current slice.

canFit(*theCps*)

Returns True if I can fit the CurvePlotScale object in the current slice.

fit(*theCps*)

Populates slots from a CurvePlotScale, caller should call canFit() first.

genScaleSliceCurve()

Generates a ordered list of ScaleSliceCurve objects laid out in a 'nice' fashion.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.util.plot.Plot.**PlotRoll**(*theXStart, theXStop, theScale, theLegendDepth, theHeadDepth=Dim(0, 'in'), theTailDepth=Dim(0, 'in'), plotUp=True, theWidth=Dim(8.5, 'in'), theMargin=Margin(left=Dim(0.25, 'in'), right=Dim(0.25, 'in'), top=Dim(0.25, 'in'), bottom=Dim(0.25, 'in'))*)

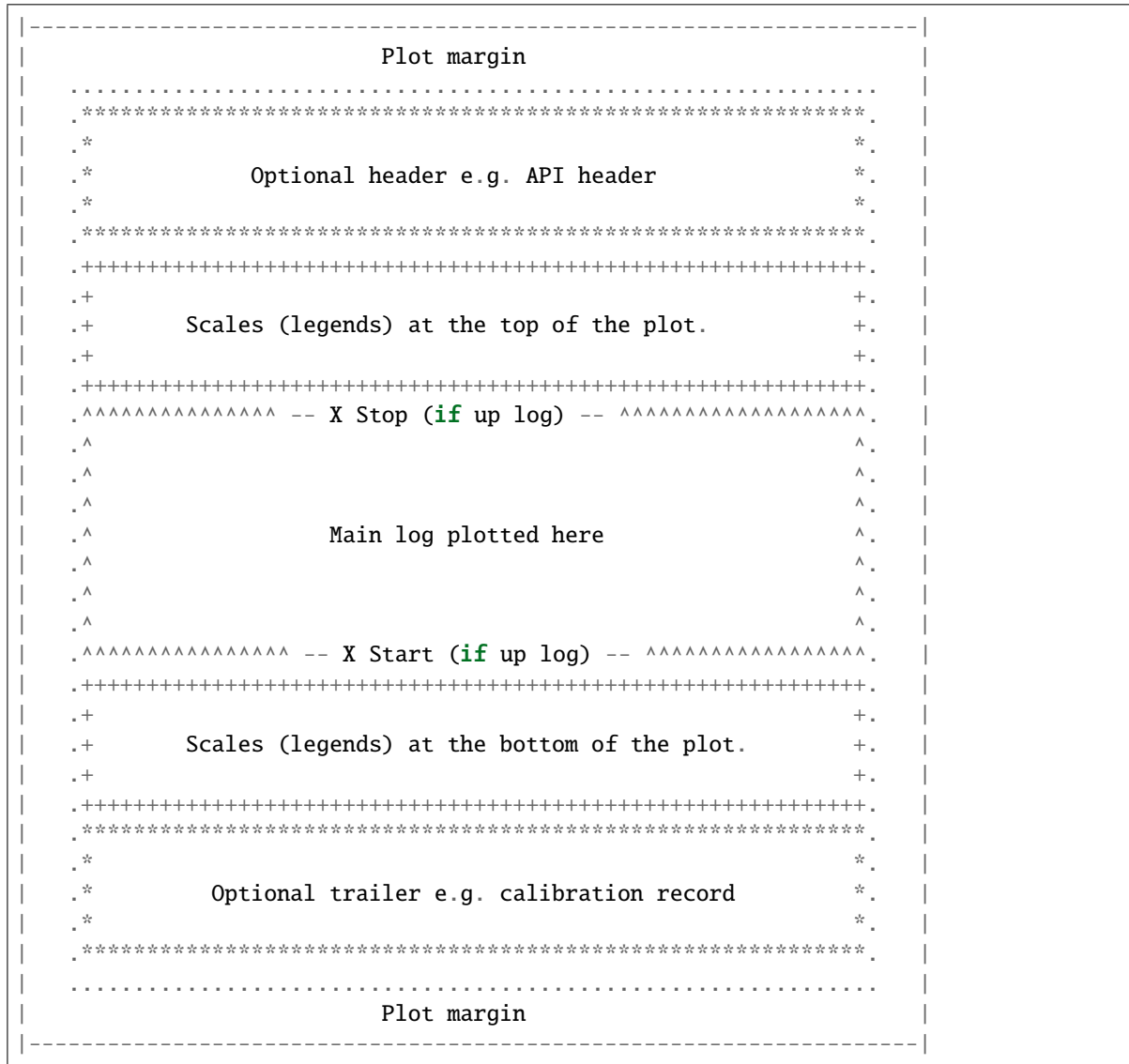
Describes the plot canvas as if it were a roll of paper. This can compute the various dimensions and positions of plot panes:

Legend:

- ... - The plot within margins.
- *** - Optional headers and trailers.
- +++ - The upper and lower headers for scales (legends).

- `^^^` - The main plotting area.

In this diagram adjacent lines overlay:



`__init__(theXStart, theXStop, theScale, theLegendDepth, theHeadDepth=Dim(0, 'in'), theTailDepth=Dim(0, 'in'), plotUp=True, theWidth=Dim(8.5, 'in'), theMargin=Margin(left=Dim(0.25, 'in'), right=Dim(0.25, 'in'), top=Dim(0.25, 'in'), bottom=Dim(0.25, 'in'))`

Initialise with:

theXStart The X start position as an EngVal.

theXStop The X stop position as an EngVal.

theScale The plot scale as a number.

theLegendDepth A Coord.Dim() that is the depth of the scales used in the header and footer.

theHeadDepth A Coord.Dim() that is the depth of any log header.

theTailDepth A Coord.Dim() that is the depth of any log trailer.

plotUp, bool True if X start is at the bottom of the main pane.

theWidth The absolute width of the plot as a `Coord.Dim()`.

theMargin A `Coord.Margin()` that describes the untouchable edges of the plot.

property viewBox

The overall size of the plot.

property width

The overall width as a number in `PlotConstants.DEFAULT_PLOT_UNITS`.

property depth

The overall width as a number in `PlotConstants.DEFAULT_PLOT_UNITS`.

property widthDim

The overall width as a `Coord.Dim()`.

property depthDim

The overall width as a `Coord.Dim()`.

property trackTopLeft

“A `Coord.Pt()` that is the top left of the pane that tracks are plotted within.

property mainPanePlotDepth

The depth of the plot of the main pane as a `Coord.Dim()`.

property availableWidth

The available width inside the margins.

retHeadPane()

Returns a pair of top-left `Coord.Pt()`, `Coord.Box()` for the top header where the header goes.

retLegendPane(*isTop*)

Returns a pair of top-left `Coord.Pt()`, `Coord.Box()` for the top or bottom legend pane where the scales go.

retMainPane()

Returns a pair of top-left `Coord.Pt()`, `Coord.Box()` for the pane where the main log goes.

retTailPane()

Returns a pair of top-left `Coord.Pt()`, `Coord.Box()` for the pane where the trailer goes.

xDepth(*theX*)

Returns a `Coord.Dim()` on the main pane that corresponds to *theX* as a number or an `EngVal`. If this is a number it is expected to be in the units of the `xStart/xStop` in the constructor.

polyLinePt(*theX*, *theTracPos*)

Returns a `Coord.Pt` from the *X* axis value (or `EngVal`) and the *TracPos* that is a value in `DEFAULT_PLOT_UNITS`, for example given by a `tracValueFunction`. The `Coord.Pt()` will be scaled by `VIEW_BOX_UNITS_PER_PLOT_UNITS`.

retMainPaneStart()

Returns the start `Coord.Pt()` for the pane where the main log goes. For and `upPlot` this will be pane-bottom-left, for a `downPlot` this will be pane-top-left.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.util.plot.Plot.Plot(*theFilmCfg*, *thePresCfg*, *theScale=0*)

Defines a plot configuration. The basic architecture follows the data. The constructor takes all the static data, typically this can be obtained from the `PRES` and `FILE` tables. The dynamic (or user selected data) is passed in to `plotLogPassLIS()`. This is intended as a single `Plot` object might be used multiple times (e.g. on ‘scroll up’).

TITLE_FONT_FAMILY = 'Verdana'

Title font

TITLE_FONT_SIZE = 10

Title font size

LEGEND_DEPTH_PER_CURVE = Dim(0.5, 'in')

How much depth to give each legend (curve scale) at the top and bottom of the log

LEGEND_DEPTH_SPARE = Dim(0.5, 'in')

How much spare depth to give over the legend (curve scale) sections at the top and bottom of the log

LEGEND_HORIZONTAL_LINE_DEPTH_PROPORTION = 0.625

Where the curve line in the legend section appears as a proportion of LEGEND_DEPTH_PER_CURVE

LEGEND_ARROW_DISPLAY = True

Arrow heads on legend scales

LEGEND_ARROW_WIDTH = Dim(0.08, 'in')

Arrow head width on legend scales

LEGEND_ARROW_DEPTH = Dim(0.04, 'in')

Arrow head depth on legend scales

LEGEND_ARROW_WIDTH_PX = 0.75

Arrow head line width on legend scales

CURVE_LEGEND_FONT_FAMILY = 'Courier'

Legend font.

CURVE_LEGEND_FONT_SIZE = 9

Legend font family.

MICRO_MARGIN = Dim(0.04, 'in')

This is just a very small margin to provide a tiny bit of whitespace between elements in certain places

MAX_BACKUP_TRACK_CROSSING_LINES = 4

Maximum number of backup lines that can cross a single track in a single X step See the source of `_filterCrossLineList()` for an explanation.

__init__(*theFilmCfg, thePresCfg, theScale=0*)

xScale(*theFilmID*)

Returns the X axis scale as a number given the FILM ID.

filmIdS()

Returns an unordered list of FILM IDs.

hasDataToPlotLIS(*theLogPass, theFilmId*)

Returns True if a call to `plotLogPassLIS()` is likely to lead to some plot data being produced.

plotLogPassLIS(*theLisFile, theLogPass, theXStart, theXStop, theFilmId, theFpOut, frameStep=1, title="", lrCONS=None, timerS=None*)

Plot a part of a LogPass and returns a list of Channel IDs plotted.

theLisFile The LIS File object.

theLogPass A LogPass object, the FrameSet will be populated here.

theXStart The start X axis position as an EngVal.

theXStop The stop X axis position as an EngVal.

theFilmId The ID of the output device from the film table

theFpOut A file path for the output SVG.

frameStep Integer number of frame steps, 1 is all frames.

title A string for the title that will appear in LEGEND_DEPTH_SPARE

lrCONS A CONS Logical Record that will be used to plot an API header in SVG.

timerS Optional ExecTimer.ExecTimerList for performance measurement.

TODO: If title is empty do not use the space self.LEGEND_DEPTH_SPARE

hasDataToPlotLAS(*theLasFile, theFilmId*)

Returns True if a call to plotLogPassLIS() is likely to lead to some plot data being produced.

plotLogPassLAS(*theLasFile, theXStart, theXStop, theFilmId, theFpOut, frameStep=1, title="", plotHeader=False, timerS=None*)

Plot a part of a LogPass and returns a list of Channel IDs plotted.

theLisFile - The LIS File object.

theLogPass - A LogPass object, the FrameSet will be populated here.

theXStart - The start X axis position as an EngVal.

theXStop - The stop X axis position as an EngVal.

theFilmId - The ID of the output device from the film table

theFpOut - A file path for the output SVG.

frameStep - Integer number of frame steps, 1 is all frames.

title - A string for the title that will appear in LEGEND_DEPTH_SPARE

lrCONS - A CONS Logical Record that will be used to plot an API header in SVG.

timerS - Optional ExecTimer.ExecTimerList for performance measurement.

TODO: If title is empty do not use the space self.LEGEND_DEPTH_SPARE

__weakref__

list of weak references to the object (if defined)

class TotalDepth.util.plot.Plot.PlotReadLIS(*lrFILM, lrPRES, lrAREA=None, lrPIP=None, theScale=0*)

A subclass of Plot that is configured from FILM, PRES and (optionally) AREA, PIP Logical Records.

__init__(*lrFILM, lrPRES, lrAREA=None, lrPIP=None, theScale=0*)

class TotalDepth.util.plot.Plot.PlotReadXML(*uniqueId, theScale=0*)

A subclass of Plot that is configured from XML file(s) using LgFormat.

__init__(*uniqueId, theScale=0*)

9.16.7 TotalDepth.util.plot.PlotConstants (Plot Constants for Wireline Logs)

Some basic constants used in plotting.

Created on Jan 5, 2012

TotalDepth.util.plot.PlotConstants.DEFAULT_PLOT_UNITS = 'in'

Default SVG plot units

TotalDepth.util.plot.PlotConstants.DEFAULT_PLOT_LIS_UNITS = b'IN '

Units that we can convert to in LIS terms

TotalDepth.util.plot.PlotConstants.VIEW_BOX_UNITS_PER_PLOT_UNITS = 96.0

Number of pixels per standard plot unit

```
TotalDepth.util.plot.PlotConstants.MarginQtrInch = Margin(left=Dim(0.25, 'in'),
right=Dim(0.25, 'in'), top=Dim(0.25, 'in'), bottom=Dim(0.25, 'in'))
```

Definition of a quarter inch margin

```
TotalDepth.util.plot.PlotConstants.STANDARD_PAPER_WIDTH = Dim(8.5, 'in')
```

Standard wireline log fan paper width i.e. long side.

```
TotalDepth.util.plot.PlotConstants.STANDARD_PAPER_DEPTH = Dim(6.25, 'in')
```

Standard wireline log fan paper depth i.e. short side.

```
TotalDepth.util.plot.PlotConstants.STANDARD_PAPER_MARGIN = Margin(left=Dim(0.25, 'in'),
right=Dim(0.25, 'in'), top=Dim(0.25, 'in'), bottom=Dim(0.25, 'in'))
```

Standard wireline log fan paper margin.

9.16.8 TotalDepth.util.plot.PRESCfg (PRES Plotting Configuration)

Holds all the information to draw a curve on a plot.

TODO: Split this to:

- Generic: PRESCfg
- LIS specific: PRESCfgLIS
- XML specific: PRESCfgXML

Created on 20 Mar 2011

Examples of PRES records summaries:

Table record (type 34) type: PRES									
MNEM	OUTP	STAT	TRAC	CODI	DEST	MODE	FILT	LEDG	REDG

SP	SP	ALLO	T1	LLIN	1	SHIF	0.500000	-80.0000	20.0000
CALI	CALI	ALLO	T1	LDAS	1	SHIF	0.500000	5.00000	15.0000
MINV	MINV	DISA	T1	LLIN	1	SHIF	0.500000	30.0000	0.00000
MNOR	MNOR	DISA	T1	LDAS	1	SHIF	0.500000	30.0000	0.00000
LLD	LLD	ALLO	T23	LDAS	1	GRAD	0.500000	0.200000	2000.00
LLDB	LLD	ALLO	T2	HDAS	1	GRAD	0.500000	2000.00	200000.
LLG	LLG	DISA	T23	LDAS	1	GRAD	0.500000	0.200000	2000.00
LLGB	LLG	DISA	T2	HDAS	1	GRAD	0.500000	2000.00	200000.
LLS	LLS	ALLO	T23	LSPO	1	GRAD	0.500000	0.200000	2000.00
LLSB	LLS	ALLO	T2	HSPO	1	GRAD	0.500000	2000.00	200000.
MSFL	MSFL	ALLO	T23	LLIN	1	GRAD	0.500000	0.200000	2000.00
11	DUMM	DISA	T1	LLIN	NEIT	NB	0.500000	0.00000	1.00000
12	DUMM	DISA	T1	LLIN	NEIT	NB	0.500000	0.00000	1.00000
13	DUMM	DISA	T1	LLIN	NEIT	NB	0.500000	0.00000	1.00000
14	DUMM	DISA	T1	LLIN	NEIT	NB	0.500000	0.00000	1.00000
15	DUMM	DISA	T1	LLIN	NEIT	NB	0.500000	0.00000	1.00000
16	DUMM	DISA	T1	LLIN	NEIT	NB	0.500000	0.00000	1.00000
17	DUMM	DISA	T1	LLIN	NEIT	NB	0.500000	0.00000	1.00000
18	DUMM	DISA	T1	LLIN	NEIT	NB	0.500000	0.00000	1.00000
19	DUMM	DISA	T1	LLIN	NEIT	NB	0.500000	0.00000	1.00000
Table record (type 34) type: PRES									

(continues on next page)

(continued from previous page)

MNEM	OUTP	STAT	TRAC	CODI	DEST	MODE	FILT	LEDG	REDG
NPHI	NPHI	ALLO	T23	LDAS	1	SHIF	0.500000	0.450000	-0.150000
DRHO	DRHO	ALLO	T3	LSPO	1	NB	0.500000	-0.250000	0.250000
PEF	PEF	ALLO	T23	LGAP	1	SHIF	0.500000	0.000000	10.0000
SGR		DISA	T1	LLIN	1	SHIF	0.500000	0.000000	300.000
CGR		DISA	T1	LGAP	1	SHIF	0.500000	0.000000	300.000
TENS	TENS	DISA	T3	LGAP	1	SHIF	0.500000	14000.0	4000.00
CAL	CALI	ALLO	T1	LSPO	1	SHIF	0.500000	5.000000	15.0000
BS	BS	DISA	T1	LGAP	1	SHIF	0.500000	5.000000	15.0000
FFLS	FFLS	DISA	T1	LLIN	2	NB	0.500000	-0.150000	0.150000
FFSS	FFSS	DISA	T1	LDAS	2	NB	0.500000	-0.150000	0.150000
LSHV	LSHV	DISA	T3	LLIN	2	WRAP	0.500000	2150.00	2250.00
SSHV	SSHV	DISA	T3	LDAS	2	WRAP	0.500000	1950.00	2050.00
FLS	FLS	DISA	T2	LLIN	2	SHIF	0.500000	0.000000	150.000
FSS	FSS	DISA	T2	LDAS	2	SHIF	0.500000	0.000000	150.000
RHOB	RHOB	ALLO	T23	LLIN	1	SHIF	0.500000	1.950000	2.95000
PHIX	PHIX	ALLO	T1	LLIN	1	NB	0.500000	0.500000	0.00000

TRAC nomenclature:

<LH | RH> T n <m>

Or as a regex: `re.compile(r'^(LH|RH)*T(\d)(\d)*\s*$')`

But this is handled by FILMCfg.

Example:

```
$ python3 TableHistogram.py -k --name=PRES -l40 --col=TRAC ../../pLogicTestData/LIS/
Cmd: TableHistogram.py -k --name=PRES -l40 --col=TRAC ../../pLogicTestData/LIS/
2011-05-27 09:26:12,324 ERROR    Can not create Logical Record, error: can't convert
↳negative value to unsigned int
2011-05-27 09:26:12,335 ERROR    Can not create Logical Record, error: can't convert
↳negative value to unsigned int
2011-05-27 09:26:12,346 ERROR    Can not create Logical Record, error: can't convert
↳negative value to unsigned int
2011-05-27 09:26:13,086 ERROR    Can not read LIS file ../../pLogicTestData/LIS/13576.
↳S1 with error: Can not fit integer number of frames length 120 into LR length 824,
↳modulo 104 [indirect size 0].
2011-05-27 09:26:13,907 ERROR    Can not read LIS file ../../pLogicTestData/LIS/13610.
↳S1 with error: Can not fit integer number of frames length 7176 into LR length 13354,
↳modulo 6178 [indirect size 0].
===== Count of all table entries =====
{'(34, b'PRES', b'TRAC', b'F1  ')': 4,
 '(34, b'PRES', b'TRAC', b'F2  ')': 4,
 '(34, b'PRES', b'TRAC', b'F3  ')': 4,
 '(34, b'PRES', b'TRAC', b'F4  ')': 3,
 '(34, b'PRES', b'TRAC', b'FD  ')': 4,
 '(34, b'PRES', b'TRAC', b'LHT1')': 4,
 '(34, b'PRES', b'TRAC', b'LHT2')': 8,
 '(34, b'PRES', b'TRAC', b'LHT3')': 59,
 '(34, b'PRES', b'TRAC', b'RHT1')': 4,
```

(continues on next page)

(continued from previous page)

```

"(34, b'PRES', b'TRAC', b'RHT2')": 22,
"(34, b'PRES', b'TRAC', b'RHT3')": 63,
"(34, b'PRES', b'TRAC', b'T1  ')": 2363,
"(34, b'PRES', b'TRAC', b'T2  ')": 354,
"(34, b'PRES', b'TRAC', b'T23 ')": 192,
"(34, b'PRES', b'TRAC', b'T3  ')": 178,
"(34, b'PRES', b'TRAC', b'TD  ')": 93,
"(34, b'PRES', b'TRAC', b'XXXX')": 18}
===== Count of all table entries END =====

```

DEST:

```

===== Count of all table entries =====
{"(34, b'PRES', b'DEST', b'1  ')": 457,
 "(34, b'PRES', b'DEST', b'123 ')": 2,
 "(34, b'PRES', b'DEST', b'134 ')": 20,
 "(34, b'PRES', b'DEST', b'2  ')": 139,
 "(34, b'PRES', b'DEST', b'5  ')": 26,
 "(34, b'PRES', b'DEST', b'6  ')": 122,
 "(34, b'PRES', b'DEST', b'8  ')": 16,
 "(34, b'PRES', b'DEST', b'A  ')": 19,
 "(34, b'PRES', b'DEST', b'ALL ')": 4,
 "(34, b'PRES', b'DEST', b'BOTH')": 774,
 "(34, b'PRES', b'DEST', b'D  ')": 63,
 "(34, b'PRES', b'DEST', b'E  ')": 60,
 "(34, b'PRES', b'DEST', b'J  ')": 52,
 "(34, b'PRES', b'DEST', b'K  ')": 13,
 "(34, b'PRES', b'DEST', b'NEIT')": 1610}
===== Count of all table entries END =====

```

MODE:

```

$ python3 TableHistogram.py -k --name=PRES -l40 --col=MODE ../../pLogicTestData/LIS/
Cmd: TableHistogram.py -k --name=PRES -l40 --col=MODE ../../pLogicTestData/LIS/
2011-06-02 08:22:54,839 ERROR      Can not read LIS file ../../pLogicTestData/LIS/13576.
↳S1 with error: Can not fit integer number of frames length 120 into LR length 824,
↳modulo 104 [indirect size 0].
2011-06-02 08:22:55,671 ERROR      Can not read LIS file ../../pLogicTestData/LIS/13610.
↳S1 with error: Can not fit integer number of frames length 7176 into LR length 13354,
↳modulo 6178 [indirect size 0].
===== Count of all table entries =====
{"(34, b'PRES', b'MODE', b'GRAD')": 245,
 "(34, b'PRES', b'MODE', b'NB  ')": 2329,
 "(34, b'PRES', b'MODE', b'SHIF')": 597,
 "(34, b'PRES', b'MODE', b'SWF  ')": 3,
 "(34, b'PRES', b'MODE', b'VDLN')": 3,
 "(34, b'PRES', b'MODE', b'WRAP')": 186,
 "(34, b'PRES', b'MODE', b'X10 ')": 10}
===== Count of all table entries END =====

```

COLO:

```

===== Count of all table entries =====
{"(34, b'PRES', b'COLO', b'000 ')": 173,
 "(34, b'PRES', b'COLO', b'002 ')": 3,
 "(34, b'PRES', b'COLO', b'003 ')": 6,
 "(34, b'PRES', b'COLO', b'004 ')": 33,
 "(34, b'PRES', b'COLO', b'014 ')": 6,
 "(34, b'PRES', b'COLO', b'020 ')": 9,
 "(34, b'PRES', b'COLO', b'021 ')": 8,
 "(34, b'PRES', b'COLO', b'030 ')": 34,
 "(34, b'PRES', b'COLO', b'034 ')": 2,
 "(34, b'PRES', b'COLO', b'044 ')": 6,
 "(34, b'PRES', b'COLO', b'104 ')": 3,
 "(34, b'PRES', b'COLO', b'134 ')": 14,
 "(34, b'PRES', b'COLO', b'203 ')": 3,
 "(34, b'PRES', b'COLO', b'221 ')": 3,
 "(34, b'PRES', b'COLO', b'312 ')": 10,
 "(34, b'PRES', b'COLO', b'400 ')": 44,
 "(34, b'PRES', b'COLO', b'404 ')": 6,
 "(34, b'PRES', b'COLO', b'420 ')": 9,
 "(34, b'PRES', b'COLO', b'430 ')": 17,
 "(34, b'PRES', b'COLO', b'AQUA')": 3,
 "(34, b'PRES', b'COLO', b'BLAC')": 1260,
 "(34, b'PRES', b'COLO', b'BLUE')": 8,
 "(34, b'PRES', b'COLO', b'GREE')": 37,
 "(34, b'PRES', b'COLO', b'RED ')": 12}
===== Count of all table entries END =====

```

What are these numbers, RGB Base 5?

Track names F1 to F4 and FD.

This seems to be the nomenclature for four track plots. For example:

Table record (type 34) type: FILM

MNEM	GCOD	GDEC	DEST	DSCA

8	EB0	---	PF8	D200
A	LLLL	1111	PFA	DM
E	E4E	-4-	PFE	D200
K	E4E	-4-	PFK	D500

Table record (type 34) type: PRES

MNEM	OUTP	STAT	TRAC	CODI	DEST	MODE	FILT	LEDG	REDG	COLO

...										
AX1A	AX	ALLO	F2	LLIN	A	SHIF	0.500000	-9.000000	9.000000	400
AY1A	AY	ALLO	F2	LSPO	A	SHIF	0.500000	-9.000000	9.000000	400
AZ1A	AZ	ALLO	F2	LGAP	A	NB	0.500000	-9.000000	9.000000	400
AN1A	ANOR	ALLO	F4	HDAS	A	NB	0.500000	9.000000	11.0000	420
CS1A	CS	DISA	F1	LDAS	A	NB	0.500000	0.000000	150000.	000
FX1A	FX	ALLO	F3	LLIN	A	NB	0.500000	-0.700000	0.700000	003

(continues on next page)

(continued from previous page)

FY1A	FY	ALLO	F3	LGAP	A	NB	0.500000	-0.700000	0.700000	003
FZ1A	FZ	ALLO	F3	LSPO	A	NB	0.500000	-0.700000	0.700000	003
FN1A	FNOR	ALLO	F4	LLIN	A	NB	0.500000	0.200000	0.700000	420
FI1A	FINC	ALLO	F4	LGAP	A	NB	1.000000	0.000000	90.0000	420
GA1A	GADZ	DISA	F1	LLIN	A	NB	1.000000	-1.000000	1.000000	020
GP1A	GPV	DISA	F1	LGAP	A	NB	1.000000	14.0000	16.0000	020
GN1A	GNV	DISA	F1	LDAS	A	NB	1.000000	-16.0000	-14.0000	020
GM1A	GMT	DISA	F3	HDAS	A	SHIF	1.000000	80.0000	130.000	020
GA2A	GAT	DISA	F2	HDAS	A	SHIF	1.000000	80.0000	130.000	020
SI1A	SILO	DISA	FD	HLIN	A	NB	1.000000	0.000000	20.0000	000
ST1A	STIT	DISA	FD	LLIN	A	NB	1.000000	0.000000	20.0000	000
ST2A	STIA	ALLO	FD	LLIN	A	NB	1.000000	0.000000	20.0000	000
TE1A	TENS	DISA	FD	LDAS	A	WRAP	0.500000	2000.00	7000.00	000
...										

exception TotalDepth.util.plot.PRESCfg.ExceptionPRESCfg

Specialisation of exception for this module.

exception TotalDepth.util.plot.PRESCfg.ExceptionLineTransBase

Specialisation of exception for LineTransBase and descendants.

exception TotalDepth.util.plot.PRESCfg.ExceptionLineTransBaseMath

For LineTransBase and descendants where math errors occur.

exception TotalDepth.util.plot.PRESCfg.ExceptionPRESCfgLISRead

Specialisation of exception for this module.

exception TotalDepth.util.plot.PRESCfg.ExceptionPresCfg

Specialisation of exception for PresCfg.

exception TotalDepth.util.plot.PRESCfg.ExceptionCurveCfg

Specialisation of exception for CurveCfg.

exception TotalDepth.util.plot.PRESCfg.ExceptionCurveCfgCtor

Construction exception when making a CurveCfg object or descendant.

exception TotalDepth.util.plot.PRESCfg.ExceptionCurveCfgLISRead

Specialisation of exception for CurveCfgLISRead and its travails.

TotalDepth.util.plot.PRESCfg.DEFAULT_LLINE_WIDTH_PX = 0.5

Default light line width in pixels

TotalDepth.util.plot.PRESCfg.DEFAULT_HLINE_WIDTH_PX = 1.5

Default heavy line width in pixels

```
TotalDepth.util.plot.PRESCfg.LIS_CODI_MAP = {None: Stroke(width=0.5, colour='black',
coding=None, opacity=1.0), b'LLIN': Stroke(width=0.5, colour='black', coding=None,
opacity=1.0), b'LSPO': Stroke(width=0.5, colour='black', coding=(2, 2), opacity=1.0),
b'LDAS': Stroke(width=0.5, colour='black', coding=(4, 4), opacity=1.0), b'LGAP':
Stroke(width=0.5, colour='black', coding=(6, 2), opacity=1.0), b'HLIN': Stroke(width=1.5,
colour='black', coding=None, opacity=1.0), b'HSP0': Stroke(width=1.5, colour='black',
coding=(2, 2), opacity=1.0), b'HDAS': Stroke(width=1.5, colour='black', coding=(4, 4),
opacity=1.0), b'HGAP': Stroke(width=1.5, colour='black', coding=(6, 2), opacity=1.0),
b'SFLN': Stroke(width=0.0625, colour='black', coding=None, opacity=1.0), b'SFDA':
Stroke(width=0.0625, colour='black', coding=(4, 4), opacity=1.0)}
```

Maps LIS CODI mnemonics to a Stroke object: If either value is None an SVG attribute is not needed i.e. default SVG behaviour

```
TotalDepth.util.plot.PRESCfg.LIS_COLO_MAP = {Mnem(b'AQUA'): 'aqua', Mnem(b'BLAC'):  
'black', Mnem(b'BLUE'): 'blue', Mnem(b'CYAN'): 'cyan', Mnem(b'GREE'): 'green',  
Mnem(b'GREY'): 'grey', Mnem(b'MAGE'): 'magenta', Mnem(b'RED\x00'): 'red', Mnem(b'YELL'):  
'yellow'}
```

Maps LIS COLO mnemonics to CSS/SVG style colour specifications.

```
TotalDepth.util.plot.PRESCfg.lisColo(theMnem)
```

Returns a SVG colour as a string from a Mnem or None on failure.

```
TotalDepth.util.plot.PRESCfg.coloStroke(theSt, theCm)
```

Takes a Stroke object and returns a new stroke object with the colour replaced with the Color Mnem looked up, or determined from, the LIS_COLO_MAP.

```
TotalDepth.util.plot.PRESCfg.BACKUP_NONE = (1, -1)
```

No backup

```
TotalDepth.util.plot.PRESCfg.BACKUP_ALL = (0, 0)
```

Every backup i.e. 'wrap' Note: Plot.py has a way of limiting ludicrous backup lines to a sensible number; say 8

```
TotalDepth.util.plot.PRESCfg.BACKUP_ONCE = (-1, 1)
```

Single backup to left or right

```
TotalDepth.util.plot.PRESCfg.BACKUP_TWICE = (-2, 2)
```

Two backups to left or right

```
TotalDepth.util.plot.PRESCfg.BACKUP_LEFT = (0, -1)
```

Single backup to left only

```
TotalDepth.util.plot.PRESCfg.BACKUP_RIGHT = (1, 0)
```

Single backup to right only

```
TotalDepth.util.plot.PRESCfg.BACKUP_FROM_MODE_MAP = {None: (0, 0), b'SHIF': (-1, 1),  
b'GRAD': (1, -1), b'NB ': (1, -1), b'WRAP': (0, 0)}
```

Map of backup mode to internal representation

```
class TotalDepth.util.plot.PRESCfg.LineTransBase(leftP, rightP, leftL, rightL, backup)
```

Base class for line generators.

```
__init__(leftP, rightP, leftL, rightL, backup)
```

Ctor with values; leftP, rightP are physical positions as numbers. leftL, rightL are logical scales as numbers. backup is a pair (left, right). Will raise a ExceptionLineTransBase if leftP >= rightP.

property leftL

The left value of the curve scale as a number.

property rightL

The right value of the curve scale as a number.

```
__str__()
```

Return str(self).

```
L2P(val)
```

Scale a given value to a dimension.

```
wrapPos(val)
```

For a given value returns a pair (wrap, pos). wrap is an integer number of times the curve is wrapped. pos is a float that is the physical plot position of the value. TODO: Benchmark this, it could be slow.

```
offScale(w)
```

Returns 0 if wrap integer is on scale depending on the backup setting. Returns -1 if off scale low, +1 if off scale high.

isOffScaleLeft(*w*)

True is wrap integer is off-scale low according to the backup setting.

isOffScaleRight(*w*)

True is wrap integer is off-scale high according to the backup setting.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.util.plot.PRESCfg.**LineTransLin**(*leftP*, *rightP*, *leftL*, *rightL*, *backup*=(0, 0))

Linear grid.

__init__(*leftP*, *rightP*, *leftL*, *rightL*, *backup*=(0, 0))

Ctor with values; leftP, rightP are physical positions as numbers. leftL, rightL are logical scales as numbers. backup is a pair (left, right).

L2P(*val*)

Scale a given value to a dimension.

wrapPos(*val*)

For a given value returns a pair (wrap, pos). wrap is an integer number of times the curve is wrapped. pos is a float that is the physical plot position of the value. TODO: Benchmark this, it could be slow.

class TotalDepth.util.plot.PRESCfg.**LineTransLog10**(*leftP*, *rightP*, *leftL*, *rightR*, *backup*=(0, 0))

Logarithmic grid.

__init__(*leftP*, *rightP*, *leftL*, *rightR*, *backup*=(0, 0))

Ctor with values; leftP, rightP are physical positions as numbers. leftL, rightL are logical scales as numbers. backup is a pair (left, right).

L2P(*val*)

Scale a given value to a dimension.

wrapPos(*val*)

For a given value returns a pair (wrap, pos). wrap is an integer number of times the curve is wrapped. pos is a float that is the physical plot position of the value. TODO: Benchmark this, it could be slow.

class TotalDepth.util.plot.PRESCfg.**TrackWidthData**(*leftP*, *rightP*, *halfTrackStart*, *halfTracks*)

Used to record the physical width data of a track leftP, rightP are Coord.Dim objects. halfTrackStart, halfTracks are integers

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static **__new__**(*_cls*, *leftP*, *rightP*, *halfTrackStart*, *halfTracks*)

__repr__()

Return a nicely formatted representation string

halfTrackStart

Alias for field number 2

halfTracks

Alias for field number 3

leftP

Alias for field number 0

rightP

Alias for field number 1

class TotalDepth.util.plot.PRESCfg.**CurveCfg**

Contains the configuration of a single curve.

__init__()

Populate attribute with reasonable default values. Second stage is to set: self.mnem, self.outp, self.trac, self.dest. Optionally other properties as well.

longStr()

Returns a long descriptive string of the internal state.

tracWidthData(*theFilmID*)

Returns a TrackWidthData object for the film ID.

tracValueFunction(*theFilmID*)

Given a FILM ID (a Mnem() object) this returns a LineTransBase or derivation that describes how this curve is plotted on that film. In particular the return value will have a function wrapPos() for generating track positions from a value.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.util.plot.PRESCfg.CurveCfgLISRead(*theRow, theFILMCfg*)

DEFAULT_MODE = b'WRAP'

Default backup mode

__init__(*theRow, theFILMCfg*)

Reads a LogiRec.TableRow object from a PRES table and populates a CurveCfg. theFILMCfg is expected to be a FILMCfgLISRead object. Typical row: MNEM OUTP STAT TRAC CODI DEST MODE FILT LEDG REDG _____ NPHI NPHI ALLO T23 LDAS 1 SHIF 0.500000 0.450000 -0.150000 May raise an ExceptionCurveCfg or an IndexError. TODO: Not raise IndexError but normalise to ExceptionCurveCfg? For example have an API missing-Cols(...) that returns None if all present or a tuple of those missing.

class TotalDepth.util.plot.PRESCfg.PresCfg

Contains the configuration equivalent to a complete PRES table.

__init__()

add(*theCurveCfg, theFilmDestS*)

Adds to the IR. theCurveCfg is a CurveCfg object, theFilmDestS is a list of film destinations expanded from the FILM table e.g. if the destination is b'ALL' then all FILM destination mnemonics should be in the list.

keys()

Returns the curve mnemonics.

__len__()

Number of curves in this table.

__getitem__(*theCurvID*)

Returns the CurveCfg object corresponding to curve ID, a Mnem.Mnem object.

__contains__(*theCurvID*)

Returns True if I have an entry for the curve ID, a Mnem.Mnem object.

hasCurvesForDest(*theDest*)

Returns True if there are curves that go to theDest i.e. FILM ID.

outpChIDs(*theDest*)

Returns a list of output channel IDs for a given film destination, a Mnem.Mnem object.

outpCurveIDs(*theDest, theOutp*)

Returns a list of channel IDs for a given film destination and output that feeds those curves. The curve data is accessible by __getitem__(). Arguments should be Mnem.Mnem objects

usesOutpChannel(*theDest, theOutp*)

Returns True if this PRES table + FILM destination uses theOutp ID. Arguments should be , a Mnem.Mnem objects.

__weakref__

list of weak references to the object (if defined)

class TotalDepth.util.plot.PRESCfg.PresCfgLISRead(*theLr, theFILMCfg*)

Information from a complete LIS PRES table.

__init__(*theLr, theFILMCfg*)

Reads a LogiRec.Table object of type PRES and creates a CurveCfgLISRead for each row. theFILMCfg is expected to be a FILMCfgLISRead object.

Typical PRES table:

MNEM	OUTP	STAT	TRAC	CODI	DEST	MODE	FILT	LEDG	REDG

↪---									
NPHI	NPHI	ALLO	T23	LDAS	1	SHIF	0.500000	0.450000	-0.
↪150000									
DRHO	DRHO	ALLO	T3	LSPO	1	NB	0.500000	-0.250000	0.
↪250000									
PEF	PEF	ALLO	T23	LGAP	1	SHIF	0.500000	0.000000	10.
↪00000									

9.16.9 TotalDepth.util.plot.PRESCfgXML (PRES Plotting Configuration from XML files)

Creates PRES configurations from LgFormat XML files.

Created on Dec 16, 2011

exception TotalDepth.util.plot.PRESCfgXML.ExceptionPRESCfgXML

Specialisation of exception for PRESCfgXML module.

exception TotalDepth.util.plot.PRESCfgXML.ExceptionCurveCfgXMLRead

Specialisation of exception for CurveCfgXMLRead module.

exception TotalDepth.util.plot.PRESCfgXML.ExceptionPresCfgXMLRead

Specialisation of exception for PresCfgXMLRead module.

```
TotalDepth.util.plot.PRESCfgXML.XML_CODI_MAP = {None: Stroke(width=0.5, colour='black',
coding=None, opacity=1.0), 'LG_SOLID_LINE': Stroke(width=0.5, colour='black',
coding=None, opacity=1.0), 'LG_DOT_LINE': Stroke(width=0.5, colour='black', coding=(2,
2), opacity=1.0), 'LG_DASH_LINE': Stroke(width=0.5, colour='black', coding=(4, 4),
opacity=1.0), 'LG_LONG_DASH_LINE': Stroke(width=0.5, colour='black', coding=(6, 2),
opacity=1.0), 'LG_SOLID_HEAVY_LINE': Stroke(width=1.5, colour='black', coding=None,
opacity=1.0), 'LG_SOLID_HEAVY_DOT': Stroke(width=1.5, colour='black', coding=(2, 2),
opacity=1.0), 'LG_SOLID_HEAVY_DASH': Stroke(width=1.5, colour='black', coding=(4, 4),
opacity=1.0), 'LG_SOLID_HEAVY_LONG_DASH': Stroke(width=1.5, colour='black', coding=(6,
2), opacity=1.0)}
```

Maps LgFormat mnemonics to a Stroke object: If either value is None an SVG attribute is not needed i.e. default SVG behaviour


```
TotalDepth.util.plot.PRESCfgXML.BACKUP_FROM_MODE_MAP = {None: (0, 0), 'LG_LEFT_WRAPPED': (0, -1), 'LG_RIGHT_WRAPPED': (1, 0), 'LG_WRAPPED': (0, 0), 'LG_X10': (0, 0), '1': (-1, 1), '2': (-2, 2)}
```

Maps LgFormat backup specifications to an internal representation Taken from the <WrapMode> element.

```
TotalDepth.util.plot.PRESCfgXML.BACKUP_FROM_COUNT_MAP = {None: (0, 0), '-1': (0, -1), '0': (1, -1), '1': (-1, 1), '2': (-2, 2)}
```

Fallback mapping LgFormat backup specifications to an internal representation Taken from the <WrapCount> element.

```
class TotalDepth.util.plot.PRESCfgXML.CurveCfgXMLRead(e, theTrac, theFILMCfg)
```

Represents a single curve from an XML file specification

```
TRAC_XML_UNIQUEID_TO_PRES = {'DepthTrack': b'TD ', 'Track2': b'T2 ', 'Track3': b'T3 ', 'TrackFC0': b'LHT2', 'TrackFC1': b'LHT2', 'TrackFC2': b'RHT2', 'TrackFC3': b'LHT3', 'TrackFC4': b'RHT2', 'depthTrack': b'TD ', 'track1': b'T1 ', 'track12': b'T12 ', 'track2': b'T2 ', 'track23': b'T23 ', 'track3': b'T3 ', 'track4': b'T4 ', 'trackLHT1': b'LHT1', 'trackRHT1': b'RHT1'}
```

First column is observed tracks in the XML, note capitalisation inconsistencies. Second column is LIS DEST equivalent

```
__init__(e, theTrac, theFILMCfg)
```

Creates a single CurveCfg object from an XML LgCurve element and populates a CurveCfg. e is the root LgCurve element. theTrac is the track name that this is being plotted on. theFILMCfg is expected to be a FilmCfgXMLRead object.

Example:

```
<LgCurve UniqueId="ROP5">
  <ChannelName>ROP5</ChannelName>
  <Color>0000FF</Color>
  <LeftLimit>500</LeftLimit>
  <RightLimit>0</RightLimit>
  <LineStyle>LG_DASH_LINE</LineStyle>
  <Thickness>1.75</Thickness>
  <WrapMode>LG_LEFT_WRAPPED</WrapMode>
</LgCurve>
Or:
<LgCurve UniqueId="PSR">
  <ChannelName>PSR</ChannelName>
  <Color>00C000</Color>
  <LeftLimit>0.2</LeftLimit>
  <RightLimit>2000</RightLimit>
  <Thickness>2</Thickness>
  <Transform>LG_LOGARITHMIC</Transform>
  <WrapCount>0</WrapCount>
</LgCurve>
```

```
class TotalDepth.util.plot.PRESCfgXML.PresCfgXMLRead(theFILMCfg, theUniqueId)
```

Extracts all curve presentation information from a single XML file.

```
__init__(theFILMCfg, theUniqueId)
```

Reads a XML and creates a CurveCfgXMLRead for each LgFormat/LgTrack/LgCurve element.

theFILMCfg is expected to be a FILMCfgXMLRead object.

9.16.10 TotalDepth.util.plot.Stroke (Plot Pen Stroke)

Defines how the stroke of a pen is represented on a plot.

Created on 7 Mar 2011

class TotalDepth.util.plot.Stroke.**Stroke**(*width, colour, coding, opacity*)

Stroke basic properties

__getnewargs__()

Return self as a plain tuple. Used by copy and pickle.

static **__new__**(*_cls, width, colour, coding, opacity*)

__repr__()

Return a nicely formatted representation string

coding

Alias for field number 2

colour

Alias for field number 1

opacity

Alias for field number 3

width

Alias for field number 0

TotalDepth.util.plot.Stroke.**StrokeBlackSolid** = **Stroke**(width='1', colour='black', coding=None, opacity=1.0)

Prototypical black solid stroke Usage: StrokeBlackSolid._replace(width=2.0)

TotalDepth.util.plot.Stroke.**retSVGAttrsFromStroke**(*stroke*)

Returns SVG attributes as a dictionary from a Stroke() object.

9.16.11 TotalDepth.util.plot.SVGWriter (SVG Writer Module)

An SVG writer.

TODO: Add a float format to reduce the size of the SVG file.

exception TotalDepth.util.plot.SVGWriter.**ExceptionSVGWriter**

Exception class for SVGWriter.

TotalDepth.util.plot.SVGWriter.**DEFAULT_VALUE_FORMAT** = '{:.3f}'

Defaults format for points that are specified in inches or such like

TotalDepth.util.plot.SVGWriter.**DEFAULT_VALUE_FORMAT_POINTS** = '{:.1f}'

Defaults format for points that are specified in pixels

TotalDepth.util.plot.SVGWriter.**dimToTxt**(*theDim*)

Converts a Coord.Dim() object to text for SVG units.

class TotalDepth.util.plot.SVGWriter.**SVGWriter**(*theFile, theViewPort, rootAttrs=None*)

__init__(*theFile, theViewPort, rootAttrs=None*)

Initialise the stream with a file and Coord.Box() object. The view port units must be the same for width and depth.

__enter__()

Context manager support.

class TotalDepth.util.plot.SVGWriter.SVGGroup(*theXmlStream*, *attrs=None*)

A group element in SVG.

__init__(*theXmlStream*, *attrs=None*)

Initialise the group with a stream.

See: <http://www.w3.org/TR/2003/REC-SVG11-20030114/struct.html#GElement>

Sadly we can't use ****kwargs** because of Python restrictions on keyword names. **stroke-width** is not a valid keyword argument (although **stroke_width** would be). So instead we pass in an optional dictionary {string : string, ...}

class TotalDepth.util.plot.SVGWriter.SVGRect(*theXmlStream*, *thePoint*, *theBox*, *attrs=None*)

A rectangle in SVG. Initialise the rectangle with a stream, a Coord.Pt() and a Coord.Box() objects.

See: <http://www.w3.org/TR/2003/REC-SVG11-20030114/shapes.html#RectElement>

Typical attributes:

```
{'fill' : "blue", 'stroke' : "black", 'stroke-width' : "2", }
```

__init__(*theXmlStream*, *thePoint*, *theBox*, *attrs=None*)

Initialise the rectangle with a stream, a Coord.Pt() and a Coord.Box() objects.

See: <http://www.w3.org/TR/2003/REC-SVG11-20030114/shapes.html#RectElement>

Typical attributes:

```
{'fill' : "blue", 'stroke' : "black", 'stroke-width' : "2", }
```

class TotalDepth.util.plot.SVGWriter.SVGCircle(*theXmlStream*, *thePoint*, *theRadius*, *attrs=None*)

A circle in SVG. Initialise the circle with a stream, a Coord.Pt() and a Coord.Dim() objects.

See: <http://www.w3.org/TR/2003/REC-SVG11-20030114/shapes.html#CircleElement>

__init__(*theXmlStream*, *thePoint*, *theRadius*, *attrs=None*)

Initialise the circle with a stream, a Coord.Pt() and a Coord.Dim() objects.

class TotalDepth.util.plot.SVGWriter.SVGEllipse(*theXmlStream*, *ptFrom*, *theRadX*, *theRadY*, *attrs=None*)

An ellipse in SVG. Initialise the circle with a stream, a Coord.Pt() and a Coord.Dim() objects.

See: <http://www.w3.org/TR/2003/REC-SVG11-20030114/shapes.html#EllipseElement>

__init__(*theXmlStream*, *ptFrom*, *theRadX*, *theRadY*, *attrs=None*)

Initialise the circle with a stream, a Coord.Pt() and a Coord.Dim() objects.

class TotalDepth.util.plot.SVGWriter.SVGLine(*theXmlStream*, *ptFrom*, *ptTo*, *attrs=None*)

A rectangle in SVG. Initialise the line with a stream, and two Coord.Pt() objects.

See: <http://www.w3.org/TR/2003/REC-SVG11-20030114/shapes.html#LineElement>

__init__(*theXmlStream*, *ptFrom*, *ptTo*, *attrs=None*)

Initialise the line with a stream, and two Coord.Pt() objects.

class TotalDepth.util.plot.SVGWriter.SVGPointList(*theXmlStream*, *name*, *pointS*, *attrs*)

An abstract class that takes a list of points, derived by polyline and polygon.

Initialise the element with a stream, a name, and a list of Coord.Pt() objects.

NOTE: The units of the points are ignored, it is up to the caller to convert them to the User Coordinate System.

TODO: Make the caller supply points as numbers not Coord.Pt as this may be faster??? e.g.:

```
FMT_PAIR_STR = FMT_STR + ',' + FMT_STR
' '.join([self.FMT_PAIR_STR.format(x, y) for x,y in pointS])
```

__init__(*theXmlStream, name, pointS, attrs*)

Initialise the element with a stream, a name, and a list of Coord.Pt() objects. NOTE: The units of the points are ignored, it is up to the caller to convert them to the User Coordinate System. TODO: Make the caller supply points as numbers not Coord.Pt as this may be faster??? e.g. FMT_PAIR_STR = FMT_STR + ',' + FMT_STR ' '.join([self.FMT_PAIR_STR.format(x, y) for x,y in pointS])

class TotalDepth.util.plot.SVGWriter.SVGPolyline(*theXmlStream, pointS, attrs=None*)

A polyline in SVG. Initialise the polyline with a stream, and a list of Coord.Pt() objects.

NOTE: The units of the points are ignored, it is up to the caller to convert them to the User Coordinate System.

See: <http://www.w3.org/TR/2003/REC-SVG11-20030114/shapes.html#PolylineElement>

__init__(*theXmlStream, pointS, attrs=None*)

Initialise the polyline with a stream, and a list of Coord.Pt() objects. NOTE: The units of the points are ignored, it is up to the caller to convert them to the User Coordinate System.

class TotalDepth.util.plot.SVGWriter.SVGPolygon(*theXmlStream, pointS, attrs=None*)

A polygon in SVG. Initialise the polygon with a stream, and a list of Coord.Pt() objects.

NOTE: The units of the points are ignored, it is up to the caller to convert them to the User Coordinate System.

See: <http://www.w3.org/TR/2003/REC-SVG11-20030114/shapes.html#PolygonElement>

__init__(*theXmlStream, pointS, attrs=None*)

Initialise the polygon with a stream, and a list of Coord.Pt() objects. NOTE: The units of the points are ignored, it is up to the caller to convert them to the User Coordinate System.

class TotalDepth.util.plot.SVGWriter.SVGText(*theXmlStream, thePoint, theFont, theSize, attrs=None*)

Text in SVG. Initialise the text with a stream, a Coord.Pt() and font as a string and size as an integer. If thePoint is None then no location will be specified (for example for use inside a <defs> element).

See: <http://www.w3.org/TR/2003/REC-SVG11-20030114/text.html#TextElement>

__init__(*theXmlStream, thePoint, theFont, theSize, attrs=None*)

Initialise the text with a stream, a Coord.Pt() and font as a string and size as an integer. If thePoint is None then no location will be specified (for example for use inside a <defs> element).

Examples

All these examples assume these imports:

```
import io
from TotalDepth.util import XmlWrite
from TotalDepth.util.plot import SVGWriter
from TotalDepth.util.plot import Coord
```

Construction

Writing to an in-memory file:

```
f = io.StringIO()
vp = Coord.Box(
    Coord.Dim(100, 'mm'),
    Coord.Dim(20, 'mm'),
)
with SVGWriter.SVGWriter(myF, myViewPort):
    pass
print(myF.getvalue())
# Prints:
<?xml version='1.0' encoding="utf-8"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN" "http://www.w3.org/Graphics/SVG/1.1/DTD/
↪svg11.dtd">
<svg height="20.000mm" version="1.1" width="100.000mm" xmlns="http://www.w3.org/2000/svg
↪"/>
```

Writing Objects to SVG

Writing a rectangles to a stream:

```
myF = io.StringIO()
vp = Coord.Box(Coord.Dim(5, 'cm'), Coord.Dim(4, 'cm'))
with SVGWriter.SVGWriter(myF, vp) as xS:
    with XmlWrite.Element(xS, 'desc'):
        xS.characters('A couple of rectangles')
    myPt = Coord.Pt(Coord.Dim(0.5, 'cm'), Coord.Dim(0.5, 'cm'))
    myBx = Coord.Box(Coord.Dim(2.0, 'cm'), Coord.Dim(1.0, 'cm'))
    with SVGWriter.SVGRect(xS, myPt, myBx):
        pass
    myPt = Coord.Pt(Coord.Dim(0.01, 'cm'), Coord.Dim(0.01, 'cm'))
    myBx = Coord.Box(Coord.Dim(4.98, 'cm'), Coord.Dim(3.98, 'cm'))
    with SVGWriter.SVGRect(xS, myPt, myBx, attrs= {'fill' : "none", 'stroke' : "blue",
↪'stroke-width' : ".02cm"}):
        pass
print(myF.getvalue())
# Prints:
<?xml version='1.0' encoding="utf-8"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN" "http://www.w3.org/Graphics/SVG/1.1/DTD/
↪svg11.dtd">
<svg height="4.000cm" version="1.1" width="5.000cm" xmlns="http://www.w3.org/2000/svg">
    <desc>A couple of rectangles</desc>
    <rect height="1.000cm" width="2.000cm" x="0.500cm" y="0.500cm"/>
    <rect fill="none" height="3.980cm" stroke="blue" stroke-width=".02cm" width="4.980cm
↪" x="0.010cm" y="0.010cm"/>
</svg>
```

9.16.12 TotalDepth.util.plot.Track (Track Module)

Created on 28 Feb 2011

Plotting LIS data requires these components: * A PlotConfig object * A data set (e.g. a LogPass with a FrameSet). * An output driver (e.g. screen, print PDF, web SVG).

User creates a PlotConfig (this reflects a PRES table). This is reusable.

User specifies a data set (from, to, channels etc.). e.g. `Invoke LogPass.setFrameSet(File, theFrameSlice=theFrameSlice, theChList=theChList)`

User says 'plot this data set with this configuration to this output device'. e.g. `Plot(PlotConfig, LogPass, PlotDevice)` Plot uses `LogPass.frameSet.genChScValues(ch, sc)` to plot individual curves.

Lacunae

Area plotting. Caching (e.g. SVG fragments - is this worth it?)

PlotConfig

PlotTracks

Typically a three track (+depth) have these dimensions in inches:

Track	Left	Right	Width
1	0	2.4	2.4
Depth	2.4	3.2	0.8
2	3.2	5.6	2.4
3	5.6	8.0	2.4

Track names can be split (e.g. LHT1 is left hand track 1) or merged (T23 is spread across tracks two and three).

Examples of PRES and FILM records:

Table record (type 34) type: PRES									
MNEM	OUTP	STAT	TRAC	CODI	DEST	MODE	FILT	LEDG	REDG
SP	SP	ALLO	T1	LLIN	1	SHIF	0.500000	-80.0000	20.0000
CALI	CALI	ALLO	T1	LDAS	1	SHIF	0.500000	5.00000	15.0000
MINV	MINV	DISA	T1	LLIN	1	SHIF	0.500000	30.0000	0.00000
MNOR	MNOR	DISA	T1	LDAS	1	SHIF	0.500000	30.0000	0.00000
LLD	LLD	ALLO	T23	LDAS	1	GRAD	0.500000	0.200000	2000.00
LLDB	LLD	ALLO	T2	HDAS	1	GRAD	0.500000	2000.00	200000.
LLG	LLG	DISA	T23	LDAS	1	GRAD	0.500000	0.200000	2000.00
LLGB	LLG	DISA	T2	HDAS	1	GRAD	0.500000	2000.00	200000.
LLS	LLS	ALLO	T23	LSP0	1	GRAD	0.500000	0.200000	2000.00
LLSB	LLS	ALLO	T2	HSP0	1	GRAD	0.500000	2000.00	200000.
MSFL	MSFL	ALLO	T23	LLIN	1	GRAD	0.500000	0.200000	2000.00
Table record (type 34) type: FILM									

(continues on next page)

(continued from previous page)

MNEM	GCOD	GDEC	DEST	DSCA

1	E20	-4--	PF1	D200
2	EEE	----	PF2	D200

Table record (type 34) type: PRES

MNEM	OUTP	STAT	TRAC	CODI	DEST	MODE	FILT	LEDG	REDG

NPHI	NPHI	ALLO	T23	LDAS	1	SHIF	0.500000	0.450000	-0.150000
DRHO	DRHO	ALLO	T3	LSPO	1	NB	0.500000	-0.250000	0.250000
PEF	PEF	ALLO	T23	LGAP	1	SHIF	0.500000	0.000000	10.0000
SGR		DISA	T1	LLIN	1	SHIF	0.500000	0.000000	300.000
CGR		DISA	T1	LGAP	1	SHIF	0.500000	0.000000	300.000
TENS	TENS	DISA	T3	LGAP	1	SHIF	0.500000	14000.0	4000.00
CAL	CALI	ALLO	T1	LSPO	1	SHIF	0.500000	5.000000	15.0000
BS	BS	DISA	T1	LGAP	1	SHIF	0.500000	5.000000	15.0000
FFLS	FFLS	DISA	T1	LLIN	2	NB	0.500000	-0.150000	0.150000
FFSS	FFSS	DISA	T1	LDAS	2	NB	0.500000	-0.150000	0.150000
LSHV	LSHV	DISA	T3	LLIN	2	WRAP	0.500000	2150.00	2250.00
SSHV	SSHV	DISA	T3	LDAS	2	WRAP	0.500000	1950.00	2050.00
FLS	FLS	DISA	T2	LLIN	2	SHIF	0.500000	0.000000	150.000
FSS	FSS	DISA	T2	LDAS	2	SHIF	0.500000	0.000000	150.000
RHOB	RHOB	ALLO	T23	LLIN	1	SHIF	0.500000	1.95000	2.95000
PHIX	PHIX	ALLO	T1	LLIN	1	NB	0.500000	0.500000	0.00000

Table record (type 34) type: FILM

MNEM	GCOD	GDEC	DEST	DSCA

1	EEE	----	PF2	D200
2	EEE	----	PF1	DM

Other FILM Table Examples:

Table record (type 34) type: FILM				
MNEM	GCOD	GDEC	DEST	DSCA

1	EEB	----	PF1	D200
2	EEB	----	PF2	DM

Table record (type 34) type: FILM

MNEM	GCOD	GDEC	DEST	DSCA

1	E20	-4--	PF1	D200
2	EEE	----	PF2	D200

(continues on next page)

(continued from previous page)

MNEM	GCOD	GDEC	DEST	DSCA

1	EEE	----	PF1	D200
2	E1E	-4-	PF2	D200

Table record (type 34) type: FILM

MNEM	GCOD	GDEC	DEST	DSCA

D	E3E	-3-	PFD	D200
E	E3E	-3-	PFE	D500
5	EB0	---	PF5	D200
6	EEB	---	PF6	D200

Table record (type 34) type: FILM

MNEM	GCOD	GDEC	DEST	DSCA

8	EB0	---	PF8	D200
A	LLLL	1111	PFA	DM
E	E4E	-4-	PFE	D200
K	E4E	-4-	PFK	D500

Table record (type 34) type: FILM

MNEM	CINT	GCOD	GDEC	DEST	DSCA

1	0.00000	E2E	-2-	PF1	D200
2	0.00000	E2E	-1-	PF2	D500
3	0.00000	EEE	----	NEIT	S5
4	0.00000	EEE	----	NEIT	S5

Table record (type 34) type: FILM

MNEM	CINT	GCOD	GDEC	DEST	DSCA

1	300.000	E2E	-2-	PF1	D200
2	300.000	E2E	-2-	PF2	D500
3	300.000	EEE	----	NEIT	S5
4	300.000	EEE	----	NEIT	S5

Table record (type 34) type: FILM

MNEM	CINT	GCOD	GDEC	DEST	DSCA

1	50.0000	EEE	----	PF1	D200
2	0.00000	EEE	----	PF2	D200

(continues on next page)

(continued from previous page)

3	0.00000	EEE	----	NEIT	D200
4	0.00000	EEE	----	NEIT	D200

Minimal, but not complete interpretation:

Ignore GDEC as dupe.
 E - Equi-spaced (linear).
 n - Log with number of decades.
 B - Blank.
 L - ?

What to do with 0 (continuation?). Examples: E20 -4- means 4 decades over track 23.

exception TotalDepth.util.plot.Track.ExceptionTotalDepthLISPlotTrack

Exception for plotting tracks.

TotalDepth.util.plot.Track.genLinear10(*l, r*)

Generate a linear series of 10 track grid lines as (Stroke, position).

TotalDepth.util.plot.Track.genLog10(*l, r, cycles=1, start=1*)

Generate a log10 series of track grid lines as (Stroke, position). *cycles* is the number of log cycles to split the track up into. *start* is the start position of the scale e.g. 2 for common resistivity curves. So *cycles=2, start=2* would give log grid 2 to 20000 i.e. to plot resistivity in the range 0.2 to 2000.

TotalDepth.util.plot.Track.genLog10Decade2(*l, r, *, cycles=2, start=1*)

Generator for 2 decades of log base 10

TotalDepth.util.plot.Track.genLog10Decade3(*l, r, *, cycles=3, start=1*)

Generator for 3 decades of log base 10

TotalDepth.util.plot.Track.genLog10Decade4(*l, r, *, cycles=4, start=1*)

Generator for 4 decades of log base 10

TotalDepth.util.plot.Track.genLog10Decade5(*l, r, *, cycles=5, start=1*)

Generator for 5 decades of log base 10

TotalDepth.util.plot.Track.genLog10Decade1Start2(*l, r, *, cycles=1, start=2*)

Generator for 1 decade of log base 10 starting at 2

TotalDepth.util.plot.Track.genLog10Decade2Start2(*l, r, *, cycles=2, start=2*)

Generator for 2 decades of log base 10 starting at 2

class TotalDepth.util.plot.Track.Track(*leftPos, rightPos, gridGn, plotXLines=True, plotXAlpha=False*)

Class that represents a single track. The track, as a structural graphical element, is merely a grid. The actual curves are plotted on panes that are independent from a single track (can span multiple tracks for example).

leftPos - A Coord.Dim() object that is the left edge.

rightPos - A Coord.Dim() object that is the right edge.

gridGn - A generator of line positions, None for blank track.

plotXLines - If True then plot X Axis information (depth lines) in this track.

plotXAlpha - If True then plot X Axis information (depth numbers) in this track.

DEPTH_PER_CH = Dim(0.25, 'in')

Space for plotting the scale for each curve

__init__(*leftPos, rightPos, gridGn, plotXLines=True, plotXAlpha=False*)

The track, as a structural graphical element, is merely a grid. The actual curves are plotted on panes that are independent from a single track (can span multiple tracks for example). *leftPos* - A `Coord.Dim()` object that is the left edge. *rightPos* - A `Coord.Dim()` object that is the right edge. *gridGn* - A generator of line positions, `None` for blank track. *plotXLines* - If `True` then plot X Axis information (depth lines) in this track. *plotXAlpha* - If `True` then plot X Axis information (depth numbers) in this track.

__str__()

Return `str(self)`.

property left

The left edge as a `Coord.Dim()`.

property right

The left edge as a `Coord.Dim()`.

property hasGrid

True if there is a grid to be generated for this track.

plotSVG(*topLeft, depth, theSVGWriter*)

Plot the track gridlines. *topLeft* - A `Coord.Pt()` object that is the top left of the canvas. *depth* - A `Coord.Dim()` object that is the total depth of the grid below *topLeft*. *drive* - The plot driver.

__weakref__

list of weak references to the object (if defined)

9.16.13 TotalDepth.util.plot.XGrid (X Axis Grid Line Module)

Provides plotting support for the X axis grid i.e. Depth grid lines and text.

Created on 28 Feb 2011

TODO: Remove APIs that are not used by Plot or anything. Plot only appears to use `genXAxisRange()` and `genXAxisTextRange()`.

exception `TotalDepth.util.plot.XGrid.ExceptionPlotXGrid`

Exception for plotting.

`TotalDepth.util.plot.XGrid.StrokeGreySolid = Stroke(width='1', colour='grey', coding=None, opacity=1.0)`

Definition of a grey solid stroke Usage: `StrokeBlackGrey._replace(width=2.0)`

class `TotalDepth.util.plot.XGrid.XGrid(scale)`

Class that can generate depth line grid and alphanumeric values. Constructed with integer scale.

`DEFAULT_INTERVAL_MAP = {1: Stroke(width=0.5, colour='black', coding=None, opacity=1.0), 10: Stroke(width=1.0, colour='black', coding=None, opacity=1.0)}`

Default for unknown units and scale, this is basically like simple graph paper

`DEFAULT_INTERVAL_TEXT = 100`

Default position for text on the X axis

__init__(*scale*)

Constructor with integer scale. We make *scale* a constructor argument as we know that up front. We don't necessarily know the X units.

genXAxisRange(*evFrom, evTo*)

Generates a bounded series of X axis line plot positions as (`Dim()`, `Stroke()`). *evFrom* and *evTo* and `EngVal` objects.

genXPosStroke(*xFrom*, *xInc*, *units*)

Generates unbounded series of X line positions as (Dim(), Stroke()).

xFrom - The starting value as a float, positions may not include this if fractional. First x position generated will be `math.ceil()` if *xInc*, `math.floor()` otherwise.

xInc - A boolean, True if X increases.

units - Units of X axis e.g. b'FEET'.

genEventsUnits(*xFrom*, *xInc*, *units*)

Generates events from/to in units.

genXAxisTextRange(*evFrom*, *evTo*)

Generates a bounded series of X axis line plot positions as (Dim(), value). *evFrom* and *evTo* and *EngVal* objects.

__weakref__

list of weak references to the object (if defined)

9.16.14 TotalDepth.util.plot.XMLCfg (XML Plot Description Low-level support)

Provides low level support for XML plot configurations

Created on Dec 13, 2011

exception TotalDepth.util.plot.XMLCfg.ExceptionXMLCfg

Exception class for this module.

exception TotalDepth.util.plot.XMLCfg.ExceptionXMLCfgWrongRootElem

Exception when the root element is wrong.

exception TotalDepth.util.plot.XMLCfg.ExceptionXMLCfgMissingElem

Exception when the is a missing mandatory element.

exception TotalDepth.util.plot.XMLCfg.ExceptionXMLCfgNoContent

Exception when the is missing content.

class TotalDepth.util.plot.XMLCfg.LgXMLBase

Base class for XML functionality that can be used by both FILM and PRES XML classes.

str(*e*, *name*, *default=None*)

Returns the text in a single child element or None.

bool(*e*, *name*, *default=False*)

Returns the text in a single child element converting 1/0 to True/False.

int(*e*, *name*, *default=0*)

Returns the text in a single child element as an integer defaulting to 0.

float(*e*, *name*, *default=0.0*)

Returns the text in a single child element as a float.

__weakref__

list of weak references to the object (if defined)

9.16.15 TotalDepth.util.plot.XMLMatches (XML Plot Description Discovery)

This looks at the available XML LgFormats and works out what curves of a LogPass or LASFile can be plotted by each one.

Created on Jan 21, 2012

TotalDepth.util.plot.XMLMatches.fileCurveMap(*theLpOrLasFile*, *directory=None*)

Returns a map of {FilmID : [OUTP, ...], ...} which is a list of OUTP in theLpOrLasFile that could be plotted with that film ID.

TotalDepth.util.plot.XMLMatches.fileCurveMapFromFILM(*theLpOrLasFile*, *theFilmCfg*)

Returns a map of {FilmID : [OUTP, ...], ...} which is a list of OUTP in theLpOrLasFile that could be plotted with that film ID.

GLOSSARY

Note: Terms here starting with “RP66V1.” are taken directly from the [RP66V1 glossary](#). They are lightly edited, corrected and enhanced before being reproduced here for the convenience of the reader.

Absent Value A recorded value from a sensor that is not a true recorded value and should be ignored. A point of difficulty arises for a conformant application; within the [LIS-79](#) format this is specified in the [DFSR](#) so can be per frame array. For [RP66V1](#) the absent value is supposed to be represented by having an attribute count of zero. In practice this never happens, instead a bunch of ad-hoc values need to be presumed such as -999, -999.25 based on their [RepCode](#). See also [RP66V1.Absent Value](#)

Backup Mode A means of specifying what happens to plotted lines when they go off scale. Typical examples are None (all intermediate data is lost) and ‘wrap’ (all data is plotted with lines at modulo scale).

Data Format Specification Record [[LIS](#)] An EFLR that describes type 0 or type 1 binary IFLRs containing log data. A DFSR consists of a set of Entry Blocks followed by a list of Datum Specification Blocks.

NOTE: The LIS specification associates a particular DFSR type 0 | 1 with binary IFLRs of type 0 | 1. These collections will be independent of each other and thus permits the simultaneous recording of entirely different data sets. In practice there is no evidence that any LIS implementations supports this.

Datum Specification Block [[LIS](#)] A fixed format data block that defines the characteristics of a single, independent, data channel in a DFSR.

DFSR See [Data Format Specification Record](#)

DLIS This adds schema specific semantics to [RP66V1](#) or [RP66V2](#).

DSB See [Datum Specification Block](#).

EFLR See [Explicitly Formatted Logical Record](#).

Engineering Value A numeric value together with its units of measure.

Entry Block [[LIS](#)] A variable length block of data that describes a static value in DFSR. This value is local to a Log Pass. For example and Entry Block might describe the NULL or absent value for any channel in a Log Pass.

Explicitly Formatted Logical Record This is an internally complete, self-describing record. See also [RP66V1.Explicitly Formatted Logical Record](#)

FFLR See [Fixed Format Logical Record](#).

Fixed Format Logical Record This is an internally complete record whose format is described by a standard. Does not occur in [RP66V1](#).

Frame An array of values for each channel at a particular depth (or time).

Frame Set

Frame Array A set of frames representing multi-channel data that is typically depth or time series based.

IFLR

Indirectly Formatted Logical Record This is a Logical Record whose format is described by another EFLR. The EFLR that describes an IFLR might be identified formally; for example by a specific reference to an EFLR (as in RP66) or informally; by some heuristic (as in LIS) such as “the immediately prior Logical Record that is type 64 i.e. a *Data Format Specification Record*. See also *RP66V1.Indirectly Formatted Logical Record*

LAS Text based flog file format managed by the [Canadian Well Logging Society](#)

LIS

LIS-79 Log Information Standard. The original file format used by Schlumberger from 1979 onwards. It is a *proprietary, de-facto* standard.

LIS-A *Enhanced* Log Information Standard. This adds somewhat to the [\[LIS-79\]](#) standard.

Log Pass A TotalDepth term that describes a continuous body of logging data recorded simultaneously and independent of any other recording. Examples might be “Repeat Section” or “Main Log”. A Log Pass contains one or more *Frame Array* (s). The number of allowable Frame Arrays within a Log Pass depends on the log format:

LAS can only support a single Frame Array within a Log Pass.

In the *LIS* format the Log Pass is defined by a single *DFSR* Logical Record. This can describe up to two Log Passes (type 0 or type 1) Logical Records. In practice only type 0 exists so LIS has just one Frame Array per Log Pass.

RP66V1 supports any number of Frame Arrays within a Log Pass and usually does.

Logical Record [\[LIS\]](#) A formal record from a LIS file. Logical Records consist of a header and optional payload. The Logical Records *type* is identified in the header. The interpretation of the payload of (some) Logical Records types is defined in the LIS standard. Logical Records consist of the payloads of one or more Physical Records. Logical Records are either EFLR or IFLR records.

LRH Logical Record header. The bytes that describe the type and attributes of a Logical Record.

Mnem See *Mnemonic*.

Mnemonic [\[LIS\]](#) A four byte identifier that is human readable e.g. RHOB for Bulk Density.

Physical Record [\[LIS\]](#) A formal record in a LIS file. Physical Records consist of a header, optional payload and optional trailer. Logical Records consist of the payloads of one or more Physical Records.

RepCode

Rep Code See *Representation Code*.

Representation Code A code, usually an integer, that describes how a particular run of bytes should be interpreted as a value (number, string etc.). For example in the LIS standard representation code 68 describes a 32 bit floating point format.

RP66 *Recommended Practice 66* is an API standard that describes a more advanced file format for, among other things, wireline logs. Comes in two flavours version 1 and version 2. Often (and incorrectly) referred to as *DLIS*.

RP66V1 *Recommended Practice 66* version 1 is an API standard first published on MAY 1, 1991 that describes a (mostly) more advanced file format compared with *LIS*. The specification can be found online from the Petrotechnical Open Software Corporation here [RP66V1 standard](#) See also the [RP66V1 glossary](#)

RP66V1.Absent Value The Value of an Attribute is an Absent Value and is undefined when the Attribute Count is zero. A Channel Sample Value is an Absent Value and is undefined when its Dimension Count is zero. Under DLIS, Absent Values are explicitly absent and are not represented by specially-designated numeric quantities.

Warning: Unfortunately RP66V1 does not allow this to be set *per frame* (i.e. per [RP66V1.IFLR](#)) but only *per channel*. So data providers use a fixed and undeclared value such as -999 for integers and -999.25 for floats. This allows per-frame *and* per-channel absent values which means that RP66V1 files from all providers should be treated with some care in this area.

RP66V1.Attribute One of possibly many specific named items of information or data associated with an Object. An Attribute is similar in function to a column value of a row in a table or to a field of a record in a database relation. Its information content can be more general, however.

RP66V1.Channel A measured or computed quantity that occurs as a sequence of samples indexed against time, depth, or some other physical dimension of a well. Also a Set Type.

RP66V1.Characteristic A descriptive feature of a Set, an Object, or an Attribute. The Characteristics of a Set are its Name and Type, of an Object its Name, and of an Attribute its Label, Count, Representation Code, Units, and Value.

RP66V1.Company Code A numeric code assigned to a company that writes information under the DLIS format. Each company (or major division) is assigned a unique Company Code by the API.

RP66V1.Component The basic syntactic unit of an Explicitly Formatted Logical Record (EFLR). A Component consists of a Component Descriptor (one byte), followed by zero or more fields that contain Characteristics associated with a Set, Object, or Attribute.

RP66V1.Component Descriptor The first byte of a Component. It has a Role field (bits 1-3), which specifies whether the Component describes a Set, Object, or Attribute, and a Format field (bits 4-8), which indicate which Characteristics of the thing described are present in the remainder of the Component.

RP66V1.Component Format See [RP66V1.Component Descriptor](#).

RP66V1.Component Role See [RP66V1.Component Descriptor](#).

RP66V1.Compound Representation Code A Representation Code that is defined in terms of other simpler Representation Codes.

RP66V1.Consumer The system or application program or company that reads information recorded under the DLIS Logical format. The Consumer reads what the Producer writes.

RP66V1.Copy Number A number, having meaning only within the context of a Logical File, that is used to distinguish two Objects of the same Type that have the same Identifier and Origin. The Name of an Object consists

RP66V1.Count One of the five Characteristics of an Attribute. The Count specifies how many Elements are in the Value of the Attribute. When the Count is zero, the Attribute has an Absent Value.

RP66V1.Data Descriptor Reference The first field of an Indirectly Formatted Logical Record (IFLR). The Data Descriptor Reference is the Name of an Object that identifies and describes a sequence of IFLRs. Each IFLR Type is associated with a specific Set Type to which such Objects belong.

RP66V1.Defining Origin The first Origin Object in a Logical File. The Defining Origin describes the environment under which the Logical File was created.

RP66V1.Descriptor See [RP66V1.Component Descriptor](#).

RP66V1.Dictionary A Dictionary is a database in which Identifiers used under DLIS are administered. The standard does not specify the mechanisms for designing, creating, or managing Dictionaries. However, it does specify for which Set Types Identifiers should be managed. The statement, "Names of Set Type X are dictionary-controlled" means that Identifiers for such Objects have a persistent meaning in all Logical Files in which they occur (by a given Producer). Identifiers of Objects for Set Types that are not dictionary-controlled are considered void of meaning and are expected to be computer-generated.

RP66V1.Dimension This is a vector of integers which describe the form and size of a rectangular array that is represented elsewhere, for example as a Channel Sample. The first integer specifies the number of remaining integers

and the dimensionality of the array (i.e., 1-d, 2-d, etc.). The remaining integers specify the number of elements along each dimension (or coordinate) of the array. The Dimension of an array is typically contained in the Dimension Attribute of some Object that is associated with the array.

RP66V1.EFLR See *RP66V1.Explicitly Formatted Logical Record*

RP66V1.Element One of a vector of homogeneous quantities that make up the Value of an Attribute or of a Channel Sample. A Value or Sample may consist of one or more Elements. All Elements have the same Representation Code and Units. The number of Elements of an Attribute Value is specified by the Attribute Count. The number of Elements of a Channel Sample is specified by its associated Dimension.

RP66V1.Encryption Packet An optional sequence of bytes that follows the Logical Record Segment Header and precedes the Logical Record Segment Body and that contains information used to decrypt the Logical Record. The first two bytes of the Encryption Packet specify its length, and the next two bytes specify the Producer's Company Code. The remaining bytes are meaningful only to the Producer.

RP66V1.Explicitly Formatted Logical Record One of two kinds of Logical Record defined under DLIS. The Body of an EFLR is a sequence of Components that combine to describe a single Set of Objects. An EFLR is self-describing and can be interpreted without the use or knowledge of any other Logical Records. More simply put, an Explicitly Formatted Logical Record is a table of rows and columns. Each row/column contains a *RP66V1.Attribute*.

RP66V1.Format Version A two-byte field immediately following the Visible Record Length in Visible Records on Record Storage Units (e.g., standard 9-track tapes). This field is used to distinguish DLIS from other formats and to distinguish DLIS Version 1 from later major versions. The first byte of the Format Version contains the value FF (hex), and the second byte is the major version number of the standard (in the current case, 1).

RP66V1.Frame The Indirectly Formatted Data of an IFLR of Type FDATA (see Appendix A) is called a Frame. A Frame is made of a Frame Number, followed by a set of Channel sample values, one sample per Channel, all sampled at the same index value. One of the Channels may serve as an index. When this is the case, it is always the first Channel in the Frame. When there is no Channel index, then the Frame Number serves as an index.

RP66V1.Frame Data Information recorded in Frames is called Frame Data. This consists of Channel samples, one sample per Channel per Frame.

RP66V1.Frame Number A positive integer recorded at the beginning of each Frame. The Frame Number is a sequential index of the Frames of the same Frame Type. Frame *n* precedes Frame *n*+1, although other Logical Records may fall between.

RP66V1.Frame Type The Name (Origin, Copy Number, Identifier) of a Frame Object used to group Frames that have the same organization. This Name is also used as the Data Descriptor Reference in the Frames, and the Frames are known to be of the given Frame Type. Frames of a given Frame Type all contain samples of the same set of Channels and all in the same order. The Representation Code and Units used to record a Channel are the same in all Frames of a given Frame Type but may be different in another Frame Type. A Channel sample may change size (number of Elements) from Frame to Frame and may even become Absent when its number of Elements reduces to zero.

RP66V1.Header (Refer to Logical Record Segment Header) See *RP66V1.Logical Record Segment Header*.

RP66V1.Identifier That part of an Object Name that is textual. The Identifier is what commonly distinguishes one Object from another. Two Objects of the same Type may have the same Identifier, in which case the other Subfields of the Name are used to distinguish the Objects. Identifiers of certain Types of Objects uniquely identify the type of data represented in the Object, and such Identifiers (typically mnemonic in nature) are dictionary-controlled.

RP66V1.IFLR See *RP66V1.Indirectly Formatted Logical Record*

RP66V1.Index Channel The first Channel in a Frame, when the Frame has an Index Channel. A Frame may be indexed by Frame Number only and need not have an Index Channel. Whether or not a Frame has an Index

Channel is specified in the associated Frame Object. When a Frame has an Index Channel, then all Channel values in the Frame are considered to be sampled at the index indicated by the value of the Index Channel.

RP66V1.Indirectly Formatted Data That part of the Body of an Indirectly Formatted Logical Record (IFLR) that follows the Data Descriptor Reference.

RP66V1.Indirectly Formatted Logical Record One of two kinds of Logical Record defined under DLIS. The Body of an IFLR consists of a Data Descriptor Reference, followed by an arbitrary number of bytes of Indirectly Formatted Data. This data is not self-descriptive. Instead, its format is determined from information contained in the Object named by the Data Descriptor Reference and possibly related Objects. For example, the format of a Frame Data IFLR is specified by a Frame Object and by one or more Channel Objects referenced by the Frame Object.

RP66V1.Invisible Envelope Data recorded on the physical medium and used as a control interface by the processor I/O subsystem, but not visible through normal application read/write requests, for example tape marks on magnetic tape.

RP66V1.Lexicon A list of dictionary-controlled words or phrases applicable as Name Part Values for a particular Name Part Type. For example, each Producer manages a Lexicon of Entity names and another Lexicon of Quantity names.

RP66V1.Locus A sequence of distinct points in space and time, each of which has a three-dimensional Position coordinate and a Time coordinate.

RP66V1.Logical File A sequence of two or more contiguous Logical Records in a Storage Set that begins with a File Header Logical Record and contains no other File Header Logical Records. A Logical File must have at least one OLR (Origin) Logical Record immediately following the File Header Logical Record. A Logical File supports user-level organization of data.

RP66V1.Logical Format The view of DLIS data that is completely independent of any physical mapping. The DLIS Logical Format consists of a sequence of Logical Records organized into one or more Logical Files. This format is the same for any physical representation of the data.

RP66V1.Logical Record A sequence of one or more contiguous Logical Record Segments. A Logical Record supports application-level organization of data.

RP66V1.Logical Record Body The sequential concatenation of the Logical Record Segment Bodies from the Logical Record Segments that make up the Logical Record.

RP66V1.Logical Record Segment A sequence of contiguous 8-bit bytes organized to have a Logical Record Segment Header, followed (optionally) by an Encryption Packet, followed by a Logical Record Segment Body, followed (optionally) by a Logical Record Segment Trailer. Logical Record Segments are used to bind the Logical Format to a physical format.

RP66V1.Logical Record Segment Attributes Eight bits of binary data that describe the attributes of a Logical Record Segment.

RP66V1.Logical Record Segment Body The part of a Logical Record Segment that contains some or all of the data belonging to a Logical Record. The intersection of a Logical Record and one of its Logical Record Segments is the Logical Record Segment Body.

RP66V1.Logical Record Segment Encryption Packet An optional packet of information, following the Logical Record Segment Header, that contains encryption/decryption information for the Logical Record Segment. The Encryption Packet begins with its size in bytes and the Company Code of the Producer. Any additional data in the Encryption Packet is meaningful only to the Producer's organization.

RP66V1.Logical Record Segment Header The first part of a Logical Record Segment. It contains the Logical Record Segment Length, the Logical Record Segment Attributes, and the Logical Record Type.

RP66V1.Logical Record Segment Length A two-byte unsigned integer that specifies the combined length of all parts of the Logical Record Segment.

RP66V1.Logical Record Segment Trailer The last part of a Logical Record Segment. It contains three fields, all of which are optional: the Padding, the Checksum, and the Logical Record Segment Trailing Length.

RP66V1.Logical Record Structure One of the attributes specified in the Logical Record Segment Attributes. It specifies whether the Logical Record is an EFLR or an IFLR.

RP66V1.Logical Record Type A one-byte unsigned integer that indicates the general semantic content of the Logical Record.

RP66V1.Long Name A structured textual description that provides an understanding, to humans, of the named item, with enough detail to distinguish it from similar items that have different meanings. It is not a unique identifier. A Long Name is represented in a Long-Name Object.

RP66V1.Maximum Visible Record Length The maximum permitted length of a Visible on a Record Storage Unit. Its current value is 16,384 bytes.

RP66V1.Minimum Visible Record Length The minimum permitted length of a Visible Record on a Record Storage Unit. Its current value is 20 bytes, which is based on the minimum Logical Record Segment Length (16 bytes) plus the Visible Record Length (2 bytes) and the Format Version (2 bytes).

RP66V1.Name Used to refer to the Name Characteristic of a Set, Object, or Attribute. The Name of a Set or Attribute is a character string (Representation Code IDENT). The Name of an Object is an aggregate consisting of an integer Origin, an integer Copy Number, and a character Identifier.

RP66V1.Name Part Type A classification of the words or phrases that apply to a particular part of the Long Name structure, for example Entity or Quantity.

RP66V1.Name Part Value A word or phrase that applies to a particular part (Name Part Type) of a Long Name structure. For example, “Density” and “Porosity” are Name Part Values that apply to the Name Part Type “Quantity”.

RP66V1.Object A data entity that has a Name and a number of Attributes. An Object is like a row in a table of information. Its Attributes are like the column values in the row. Objects are recorded in EFLRs.

RP66V1.Object Component An Object Component indicates the beginning of a new Object in a Set and is followed by zero or more Attribute Components. The Attributes of an Object that has no Attribute Components are completely specified in the Template. The Object Component contains a single Characteristic, the Object Name, which is mandatory.

RP66V1.Origin As an Object in a Logical File, an Origin contains information describing the original circumstances under which that or another Logical File was created. Only one Origin Object in a Logical File, namely the first one, describes that Logical File. Additional Origin Objects describe other Logical Files from which data has been copied. Other Objects in a Logical File are keyed to their appropriate Origin Object by means of an integer Subfield in their Names, namely the Origin Subfield. This integer value matches the Origin Subfield of the appropriate Origin Object. This integer value is also commonly referred to as the Object’s Origin.

RP66V1.Pad Bytes Pad Bytes are part of the Logical Record Segment Trailer and are used to alter the size of a Logical Record Segment to satisfy minimum size requirements or more commonly to make the Logical Record Segment Length divisible by some integer. In all cases, the Logical Record Segment Length must be divisible by two. Additionally, certain encryption methods may require the length of the Logical Record Segment Body plus the Pad Bytes to be divisible by some other factor.

RP66V1.Pad Count This is the first byte of the Pad Bytes and indicates how many Pad Bytes there are. The maximum number of Pad Bytes may therefore not exceed 255.

RP66V1.Padding An informal reference to Pad Bytes.

RP66V1.Parent File The Logical File in which data are originally created. Some data in a Logical File may have been copied from other Logical Files.

RP66V1.Path A sequence of space-time coordinates, where space is typically represented by depth, radial distance from a vertical line, and angular displacement about the same vertical line. The vertical line used is the one

that goes through a well's Well Reference Point, a point used to identify the location of a well. A Path may be represented by a combination of Channels, each of which represents one of the above-mentioned coordinates.

RP66V1.Physical Format The way in which recorded data is located and organized on a particular physical medium such as a magnetic tape or disk file. With some I/O systems more than one organization and view of data is supported on the same medium. Each such view corresponds to a different physical format. For example, disk files may be viewed as having variable-length record structures, block structures, byte stream structures, etc., depending on the I/O facility that is used. The physical format determines the way in which Logical Record Segments are used but generally has no effect on Logical Records.

RP66V1.Predecessor Used to indicate the relation between successive Logical Record Segments. If two Logical Record Segments belong to the same Logical Record, then one of them — the one that comes first — is a Predecessor of the other. The first Logical Record Segment of a Logical Record has no Predecessor.

RP66V1.Private A Logical Record with Type ≥ 128 is said to be Private. In particular, the semantic content of such a Logical Record is decided upon solely by the Producer and not via any public standardization process. Private Logical Records are available to consumers in general, unless encrypted. The fact that a Logical Record is Private does not imply that it is also encrypted.

RP66V1.Producer The system or application program or company that records information under the DLIS Logical Format. The Producer writes what the Consumer reads.

RP66V1.Public A Logical Record with Type < 128 is said to be Public. In particular, the semantic content of such a Logical Record is agreed to by the all users via a standardization process administered by the API. Such Logical Records may not be used except in accordance with the standard definition. Public Logical Records may be encrypted or not, according to the needs of the Producer.

RP66V1.Radial Drift Radial Drift is the perpendicular distance of a point from a vertical line that passes through the Well Reference Point of a well.

RP66V1.Record Structure One of possibly many different physical formats. A Storage Unit is said to have Record Structure if data is written and read in sequential, variable-length records. For all Record Structure Storage Units each record must begin with a two-byte unsigned integer Visible Record Length, followed by a two-byte Format Version, followed by an integer number of Logical Record Segments. Other requirements, for example use of Tape Marks, depend on the particular physical medium.

RP66V1.Representation Code Each distinct piece of information in the Logical Format has a well-defined representation that extends across one or more bytes. Each different representation is identified by a one-byte Representation Code. Representation Codes are defined in Appendix B and identify the various floating point, integer, and text representations permitted under the DLIS.

RP66V1.Sample (of a Channel) A Channel Sample is one of a sequence of evaluations of a Channel. A Channel Sample may be a scalar sensor reading (i.e., a single number), or it may be an array representing a waveform or some other multi-dimensional data.

RP66V1.Semantics Semantics is the definition of what data means and how it is used. Whereas syntax provides rules for recording Objects in Sets, semantics defines the Objects that may be recorded, e.g., the File-Header, Origin, Channel, Frame, etc. Objects.

RP66V1.Set A data entity that has a Type and optionally a Name, and contains a number of Objects. A Set is like a table of information in which the Objects are the rows of the table. Each Set is contained in an EFLR (exactly one Set per EFLR), and there may be more than one Set with the same Type in a Logical File.

RP66V1.Set Component A Set Component indicates the beginning of a Set and is followed by one or more Template Attribute Components. A Set Component always contains a Type Characteristic and may contain a Name Characteristic.

RP66V1.Set Type A textual identifier of the type of Objects contained in a Set. The Objects in a Set are characterized by the Attributes in the Template of the Set. The Attributes associated with each given Set Type are specified in the standard.

RP66V1.Slot A data entity that has a Type and optionally a Name, and contains a number of Objects. A Set is like a table of information in which the Objects are the rows of the table. Each Set is contained in an EFLR (exactly one Set per EFLR), and there may be more than one Set with the same Type in a Logical File. One of a fixed number of positions in a Frame for recording a single Channel Sample value. Channels are assigned to Slots in a Frame in a specific order, and all Slots follow the Frame Number. The Index Channel, if there is one, is in the first Slot of a Frame.

RP66V1.Splice A Splice is the result of concatenating two or more instances of a Channel (e.g., from different runs) to obtain a resultant Channel defined over a larger domain or interval. The information associated with a Splice is represented in a Splice Object.

RP66V1.Static Information Static Information consists of Objects typically used to describe Channels and Frames, and information about Channels and Frames. Static Information is typically required by an application prior to the processing of Frames.

RP66V1.Storage Set A group of Storage Units that contain a common DLIS Logical Format (e.g., a sequence of Logical Files) and for which at least two Storage Units are spanned by a single Logical File.

RP66V1.Storage Set Identifier A 60-character ASCII field in the Storage Unit Label used to identify a Storage Set.

RP66V1.Storage Unit Something that contains DLIS data and is manageable as a unit at the human level, (e.g., a tape or disk file).

RP66V1.Storage Unit Label The first 80 bytes of the Visible Envelope of a Storage Unit. The Storage Unit Label consists of five fixed-length ASCII fields used to identify the Storage Unit and the Storage Set of which it is a part.

RP66V1.Storage Unit Sequence Number A positive integer (its ASCII representation) in the Storage Unit Label that indicates the order in which a Storage Unit occurs in a Storage Set.

RP66V1.Storage Unit Structure An ASCII keyword in the Storage Unit Label that reflects the Physical Format of the Storage Unit and indicates the binding mechanism between the Physical Format and the DLIS Logical Format.

RP66V1.Subfield A part of a datum for which the representation is described by a simple (not compound) Representation Code. For example, the Subfields of a datum having Representation Code OBNAME are, in order, an integer (UVARI), another integer (USHORT), and a string (IDENT).

RP66V1.Successor Used to indicate the relation between successive Logical Record Segments. If two Logical Record Segments belong to the same Logical Record, then one of them — the one that comes second — is a Successor of the other. The last Logical Record Segment of a Logical Record has no Successor.

RP66V1.Syntax Syntax is the definition of the rules for how to record data but not for what the data means (at the application level) or how it is to be used. Syntax does convey meaning of data, but at a level below applications. For example, the rules of syntax tell when a Component has a Type Characteristic and how to get it, but syntax provides no information on the meaning or use of the values the Type Characteristic may have.

RP66V1.Template A sequence of Attributes at the beginning of a Set that specify defaults for the Objects in the Set. Attributes in the Template must have Labels. Objects in the Set have no Attributes other than those identified in the Template.

RP66V1.Tool Zero Point A fixed point on the tool string (usually the bottom of the bottom tool) that stands opposite the Well Reference Point when Borehole Depth is zero.

RP66V1.Trailing Length The optional last field in the Logical Record Segment Trailer that contains a copy of the Logical Record Segment Length.

RP66V1.Transient Information Transient Information consists of Objects that correspond to events that occur during the processing of Frames. These events can affect Objects in the Static Information or can correspond to messages between the operator and the system.

RP66V1.Unzoned A Parameter or Computation Object is said to be Unzoned when it has the same value everywhere. This is the case when the Zones Attribute of the Object is absent.

RP66V1.Update An Update is a change made to data represented by an Object (e.g., a Parameter) previously recorded in a Logical File. The change and information related to the change are represented in an Update Object recorded in the same Logical File.

RP66V1.Value (of an Attribute) The Value of an Attribute is the data contained in its Value Characteristic. A Value may consist of one or more Elements, each of which has the same Units and Representation Code.

RP66V1.Vertical Depth Depth measured along the Vertical Generatrix from the Well Reference Point. Vertical depth increases in a downward direction and is negative above the Well Reference Point.

RP66V1.Vertical Generatrix A vertical line that passes through the Well Reference Point.

RP66V1.Visible Envelope Information on a Storage Unit that is provided to applications as normal data by the processor's I/O subsystem, but which is not part of the DLIS Logical Format. Information in the Visible Envelope includes the Storage Unit Label. Other information in the Visible Envelope may be used to define or enhance the binding of Logical Record Segments to the Physical Format.

RP66V1.Visible Record A Visible Record is a term that applies to Record Structure Storage Units. It consists of all the data bytes accessed by means of a record read operation from the system-specific file access subsystem.

RP66V1.Visible Record Length When DLIS information is recorded in variable length physical records, each Visible Record begins with a two-byte unsigned integer length of the Visible Record called the Visible Record Length. This length is considered to be external the DLIS Logical Format.

RP66V1.Well Reference Point A unique point that is the origin of a well's spatial coordinate system for information in a Logical File. This point is defined relative to some permanent vertical structure, such as ground level or mean sea level, and to three independent geographical coordinates, which typically include Latitude and Longitude. The same well may have different Well Reference Points in different Logical Files.

RP66V1.Zone A Zone is a single interval in depth or time. The depth coordinate may be either Vertical Depth or Borehole Depth.

RP66V1.Zoned A Parameter or Computation Object is said to be Zoned when it has different values in different intervals along a depth or time domain or is undefined in some interval of a depth or time domain. This is the case when the Zones Attribute of the Object is not absent.

RP66V2 *Recommended Practice 66* version 2 is an API standard first published in June 1996 that describes a (mostly) more advanced file format compared with [RP66V1](#). It is unused by the industry and will not be referenced here. The specification can be found online from the Petrotechnical Open Software Corporation here [RP66V2 standard](#) See also the [RP66V2 glossary](#)

UOM *Unit of Measure* for engineering values. In [LIS](#) these are a set of fixed terms organised into several categories, such as *Linear Length*. Values can only be converted between units of in the same category. In [RP66V1](#) these are composed by a BNF parseable string.

Xaxis

X Axis The index channel in an array, for example an array of frames. Typically depth or time.

LICENCE

B.1 OpenSource Licence

TotalDepth is released under GPL 2.:

GNU GENERAL PUBLIC LICENSE
Version 2, June 1991

Copyright (C) 1989, 1991 Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA
Everyone is permitted to copy and distribute verbatim copies
of this license document, but changing it is not allowed.

Preamble

The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change free software--to make sure the software is free for all its users. This General Public License applies to most of the Free Software Foundation's software and to any other program whose authors commit to using it. (Some other Free Software Foundation software is covered by the GNU Lesser General Public License instead.) You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs; and that you know you can do these things.

To protect your rights, we need to make restrictions that forbid anyone to deny you these rights or to ask you to surrender the rights. These restrictions translate to certain responsibilities for you if you distribute copies of the software, or if you modify it.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must give the recipients all the rights that you have. You must make sure that they, too, receive or can get the

(continues on next page)

(continued from previous page)

source code. And you must show them these terms so they know their rights.

We protect your rights with two steps: (1) copyright the software, and (2) offer you this license which gives you legal permission to copy, distribute and/or modify the software.

Also, for each author's protection and ours, we want to make certain that everyone understands that there is no warranty for this free software. If the software is modified by someone else and passed on, we want its recipients to know that what they have is not the original, so that any problems introduced by others will not reflect on the original authors' reputations.

Finally, any free program is threatened constantly by software patents. We wish to avoid the danger that redistributors of a free program will individually obtain patent licenses, in effect making the program proprietary. To prevent this, we have made it clear that any patent must be licensed for everyone's free use or not licensed at all.

The precise terms and conditions for copying, distribution and modification follow.

GNU GENERAL PUBLIC LICENSE
TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This License applies to any program or other work which contains a notice placed by the copyright holder saying it may be distributed under the terms of this General Public License. The "Program", below, refers to any such program or work, and a "work based on the Program" means either the Program or any derivative work under copyright law: that is to say, a work containing the Program or a portion of it, either verbatim or with modifications and/or translated into another language. (Hereinafter, translation is included without limitation in the term "modification".) Each licensee is addressed as "you".

Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running the Program is not restricted, and the output from the Program is covered only if its contents constitute a work based on the Program (independent of having been made by running the Program). Whether that is true depends on what the Program does.

1. You may copy and distribute verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and give any other recipients of the Program a copy of this License along with the Program.

You may charge a fee for the physical act of transferring a copy, and

(continues on next page)

(continued from previous page)

you may at your option offer warranty protection in exchange for a fee.

2. You may modify your copy or copies of the Program or any portion of it, thus forming a work based on the Program, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

- a) You must cause the modified files to carry prominent notices stating that you changed the files and the date of any change.
- b) You must cause any work that you distribute or publish, that in whole or in part contains or is derived from the Program or any part thereof, to be licensed as a whole at no charge to all third parties under the terms of this License.
- c) If the modified program normally reads commands interactively when run, you must cause it, when started running for such interactive use in the most ordinary way, to print or display an announcement including an appropriate copyright notice and a notice that there is no warranty (or else, saying that you provide a warranty) and that users may redistribute the program under these conditions, and telling the user how to view a copy of this License. (Exception: if the Program itself is interactive but does not normally print such an announcement, your work based on the Program is not required to print an announcement.)

These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Program, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Program, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Program.

In addition, mere aggregation of another work not based on the Program with the Program (or with a work based on the Program) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You may copy and distribute the Program (or a work based on it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you also do one of the following:

- a) Accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections

(continues on next page)

(continued from previous page)

1 and 2 above on a medium customarily used for software interchange; or,

b) Accompany it with a written offer, valid for at least three years, to give any third party, for a charge no more than your cost of physically performing source distribution, a complete machine-readable copy of the corresponding source code, to be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or,

c) Accompany it with the information you received as to the offer to distribute corresponding source code. (This alternative is allowed only for noncommercial distribution and only if you received the program in object code or executable form with such an offer, in accord with Subsection b above.)

The source code for a work means the preferred form of the work for making modifications to it. For an executable work, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the executable. However, as a special exception, the source code distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

If distribution of executable or object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place counts as distribution of the source code, even though third parties are not compelled to copy the source along with the object code.

4. You may not copy, modify, sublicense, or distribute the Program except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense or distribute the Program is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

5. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Program or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Program (or any work based on the Program), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Program or works based on it.

6. Each time you redistribute the Program (or any work based on the Program), the recipient automatically receives a license from the original licensor to copy, distribute or modify the Program subject to

(continues on next page)

(continued from previous page)

these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties to this License.

7. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Program at all. For example, if a patent license would not permit royalty-free redistribution of the Program by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Program.

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system, which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

8. If the distribution and/or use of the Program is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Program under this License may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

9. The Free Software Foundation may publish revised and/or new versions of the General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies a version number of this License which applies to it and "any

(continues on next page)

(continued from previous page)

later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of this License, you may choose any version ever published by the Free Software Foundation.

10. If you wish to incorporate parts of the Program into other free programs whose distribution conditions are different, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

11. BECAUSE THE PROGRAM IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

12. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

END OF TERMS AND CONDITIONS

How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively convey the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

<one line to give the program's name and a brief idea of what it does.>
Copyright (C) <year> <name of author>

(continues on next page)

(continued from previous page)

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA.

Also add information on how to contact you by electronic and paper mail.

If the program is interactive, make it output a short notice like this when it starts in an interactive mode:

```
Gnomovision version 69, Copyright (C) year name of author
Gnomovision comes with ABSOLUTELY NO WARRANTY; for details type `show w'.
This is free software, and you are welcome to redistribute it
under certain conditions; type `show c' for details.
```

The hypothetical commands `show w' and `show c' should show the appropriate parts of the General Public License. Of course, the commands you use may be called something other than `show w' and `show c'; they could even be mouse-clicks or menu items--whatever suits your program.

You should also get your employer (if you work as a programmer) or your school, if any, to sign a "copyright disclaimer" for the program, if necessary. Here is a sample; alter the names:

```
Yoyodyne, Inc., hereby disclaims all copyright interest in the program
`Gnomovision' (which makes passes at compilers) written by James Hacker.
```

```
<signature of Ty Coon>, 1 April 1989
Ty Coon, President of Vice
```

This General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License.

B.2 Other Licences

For other licences contact: apaulross@gmail.com

PYTHON MODULE INDEX

t

TotalDepth.BIT.ReadBIT, 7
TotalDepth.BIT.ToLAS, 13
TotalDepth.common.cmn_cmd_opts, 139
TotalDepth.common.colorama, 140
TotalDepth.common.data_table, 140
TotalDepth.common.LogPass, 141
TotalDepth.common.lookup_mnemonic, 144
TotalDepth.common.process, 151
TotalDepth.common.Rle, 153
TotalDepth.common.Slice, 154
TotalDepth.common.statistics, 156
TotalDepth.common.units, 157
TotalDepth.common.xml, 159
TotalDepth.common.xxd, 159
TotalDepth.DAT.DAT_parser, 137
TotalDepth.DeTif, 3
TotalDepth.LAS.core.LASRead, 15
TotalDepth.LAS.ReadLASFiles, 15
TotalDepth.LIS.core.cRepCode, 68
TotalDepth.LIS.core.EngVal, 21
TotalDepth.LIS.core.File, 24
TotalDepth.LIS.core.FileIndexer, 27
TotalDepth.LIS.core.FrameSet, 32
TotalDepth.LIS.core.LogiRec, 38
TotalDepth.LIS.core.LogPass, 50
TotalDepth.LIS.core.Mnem, 57
TotalDepth.LIS.core.PhysRec, 58
TotalDepth.LIS.core.pRepCode, 66
TotalDepth.LIS.core.RawStream, 62
TotalDepth.LIS.core.RepCode, 64
TotalDepth.LIS.core.Rle, 69
TotalDepth.LIS.core.TifMarker, 71
TotalDepth.LIS.core.Type01Plan, 73
TotalDepth.LIS.core.Units, 75
TotalDepth.LIS.DumpFrameSet, 78
TotalDepth.LIS.Index, 78
TotalDepth.LIS.lis_cmn_cmd_opts, 84
TotalDepth.LIS.LisToHtml, 79
TotalDepth.LIS.PlotLogPasses, 80
TotalDepth.LIS.ProcLISPath, 81
TotalDepth.LIS.RandomFrameSetRead, 82
TotalDepth.LIS.ScanLogiData, 82
TotalDepth.LIS.ScanLogiRec, 82
TotalDepth.LIS.ScanPhysRec, 82
TotalDepth.LIS.TableHistogram, 83
TotalDepth.LIS.ToLAS, 84
TotalDepth.PlotLogs, 5
TotalDepth.RP66V1.core.AbsentValue, 85
TotalDepth.RP66V1.core.LogicalFile, 94
TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor, 102
TotalDepth.RP66V1.core.LogicalRecord.EFLR, 104
TotalDepth.RP66V1.core.LogicalRecord.Encryption, 108
TotalDepth.RP66V1.core.LogicalRecord.IFLR, 108
TotalDepth.RP66V1.core.LogicalRecord.Semantics, 109
TotalDepth.RP66V1.core.LogicalRecord.Types, 109
TotalDepth.RP66V1.core.LogPass, 93
TotalDepth.RP66V1.core.pFile, 85
TotalDepth.RP66V1.core.pIndex, 91
TotalDepth.RP66V1.core.RepCode, 97
TotalDepth.RP66V1.core.stringify, 102
TotalDepth.RP66V1.core.Units, 97
TotalDepth.RP66V1.core.XAxis, 101
TotalDepth.RP66V1.IndexPickle, 110
TotalDepth.RP66V1.IndexXML, 111
TotalDepth.RP66V1.LogRecIndex, 113
TotalDepth.RP66V1.Scan, 114
TotalDepth.RP66V1.ScanHTML, 116
TotalDepth.RP66V1.SearchFF01, 117
TotalDepth.RP66V1.ToLAS, 117
TotalDepth.RP66V1.util.IndexXMLRead, 119
TotalDepth.RP66V1.util.XMLReadUnits, 120
TotalDepth.util.archive, 176
TotalDepth.util.bin_file_type, 177
TotalDepth.util.CopyBinFiles, 161
TotalDepth.util.DictTree, 161
TotalDepth.util.DirWalk, 165
TotalDepth.util.EBCDIC, 166

TotalDepth.util.ExecTimer, 168
TotalDepth.util.FileBuffer, 169
TotalDepth.util.FileStatus, 170
TotalDepth.util.gnuplot, 178
TotalDepth.util.Histogram, 171
TotalDepth.util.HtmlUtils, 172
TotalDepth.util.PatternSearch, 173
TotalDepth.util.plot.AREACfg, 179
TotalDepth.util.plot.Coord, 179
TotalDepth.util.plot.FILMCfg, 185
TotalDepth.util.plot.FILMCfgXML, 191
TotalDepth.util.plot.LogHeader, 192
TotalDepth.util.plot.Plot, 195
TotalDepth.util.plot.PlotConstants, 201
TotalDepth.util.plot.PRESCfg, 202
TotalDepth.util.plot.PRESCfgXML, 210
TotalDepth.util.plot.Stroke, 212
TotalDepth.util.plot.SVGWriter, 212
TotalDepth.util.plot.Track, 216
TotalDepth.util.plot.XGrid, 220
TotalDepth.util.plot.XMLCfg, 221
TotalDepth.util.plot.XMLMatches, 222
TotalDepth.util.RemoveDupeFiles, 173
TotalDepth.util.XmlWrite, 174

Symbols

__add__() (TotalDepth.LIS.core.EngVal.EngVal method), 22
 __add__() (TotalDepth.util.plot.Coord.Dim method), 180
 __bool__() (TotalDepth.RP66V1.core.pFile.LogicalData method), 89
 __contains__() (TotalDepth.LAS.core.LASRead.LASSection method), 17
 __contains__() (TotalDepth.LIS.core.LogiRec.LrTable method), 47
 __contains__() (TotalDepth.LIS.core.LogiRec.TableRow method), 46
 __contains__() (TotalDepth.util.plot.FILMCfg.FilmCfg method), 190
 __contains__() (TotalDepth.util.plot.PRESCfg.PresCfg method), 209
 __enter__() (TotalDepth.LIS.core.RawStream.RawStream method), 62
 __enter__() (TotalDepth.RP66V1.core.LogicalFile.LogicalIndex method), 97
 __enter__() (TotalDepth.util.XmlWrite.Element method), 175
 __enter__() (TotalDepth.util.XmlWrite.XhtmlStream method), 175
 __enter__() (TotalDepth.util.XmlWrite.XmlStream method), 175
 __enter__() (TotalDepth.util.plot.SVGWriter.SVGWriter method), 212
 __eq__() (TotalDepth.LIS.core.EngVal.EngVal method), 23
 __eq__() (TotalDepth.LIS.core.Mnem.Mnem method), 57
 __eq__() (TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.ComponentDescriptor method), 103
 __eq__() (TotalDepth.RP66V1.core.LogicalRecord.EFLR.AttributeBase method), 105
 __eq__() (TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormattedLogicalRecord method), 107
 __eq__() (TotalDepth.RP66V1.core.LogicalRecord.EFLR.Object method), 106
 __eq__() (TotalDepth.RP66V1.core.LogicalRecord.EFLR.Set method), 105
 __eq__() (TotalDepth.RP66V1.core.LogicalRecord.EFLR.Template method), 106
 __eq__() (TotalDepth.RP66V1.core.pFile.LogicalRecordPosition method), 88
 __eq__() (TotalDepth.RP66V1.core.pFile.VisibleRecord method), 87
 __eq__() (TotalDepth.common.Slice.Sample method), 156
 __eq__() (TotalDepth.common.Slice.Slice method), 155
 __eq__() (TotalDepth.common.Slice.SliceABC method), 154
 __eq__() (TotalDepth.util.archive.FileBase method), 176
 __eq__() (TotalDepth.util.plot.Coord.Dim method), 181
 __eq__() (TotalDepth.util.plot.Coord.Pt method), 182
 __exit__() (TotalDepth.LIS.core.RawStream.RawStream method), 62
 __exit__() (TotalDepth.RP66V1.core.LogicalFile.LogicalIndex method), 97
 __exit__() (TotalDepth.util.XmlWrite.Element method), 175
 __exit__() (TotalDepth.util.XmlWrite.XmlStream method), 175
 __format__() (TotalDepth.RP66V1.core.pFile.VisibleRecord method), 87
 __format__() (TotalDepth.util.plot.Coord.Box method), 181
 __format__() (TotalDepth.util.plot.Coord.Dim method), 180
 __format__() (TotalDepth.util.plot.Coord.Margin method), 181
 __format__() (TotalDepth.util.plot.Coord.Pad method), 181
 __format__() (TotalDepth.util.plot.Coord.Pt method), 182
 __ge__() (TotalDepth.LIS.core.EngVal.EngVal method), 23
 __ge__() (TotalDepth.util.plot.Coord.Dim method), 181
 __getattr__() (TotalDepth.LIS.core.LogiRec.EntryBlockSet method), 48
 __getitem__() (TotalDepth.LAS.core.LASRead.LASBase

method), 18
 __getitem__(TotalDepth.LAS.core.LASRead.LASSection_getnewargs__ method), 17
 __getitem__(TotalDepth.LAS.core.LASRead.LASSectionArray method), 18
 __getitem__(TotalDepth.LIS.core.LogiRec.EntryBlockSet method), 48
 __getitem__(TotalDepth.LIS.core.LogiRec.LrTable method), 46
 __getitem__(TotalDepth.LIS.core.LogiRec.TableRow method), 46
 __getitem__(TotalDepth.RP66V1.core.LogicalFile.LogicalIndex method), 83
 __getitem__(TotalDepth.RP66V1.core.LogicalFile.LogicalRecord.EFLR.ExplicitFormListScanPhysRecPhysRecScanResultWithPad method), 107
 __getitem__(TotalDepth.RP66V1.core.LogicalRecord.EFLR.Object method), 106
 __getitem__(TotalDepth.RP66V1.core.LogicalRecord.EFLR.Template method), 106
 __getitem__(TotalDepth.RP66V1.core.XAxis.XAxis method), 101
 __getitem__(TotalDepth.RP66V1.core.pFile.LogicalData_getnewargs__ method), 90
 __getitem__(TotalDepth.common.LogPass.FrameChannel method), 142
 __getitem__(TotalDepth.common.LogPass.LogPass method), 144
 __getitem__(TotalDepth.common.Rle.RLE method), 153
 __getitem__(TotalDepth.common.statistics.LengthDict method), 156
 __getitem__(TotalDepth.util.FileBuffer.FileBuffer method), 169
 __getitem__(TotalDepth.util.Histogram.Histogram method), 171
 __getitem__(TotalDepth.util.plot.FILMCfg.FilmCfg method), 190
 __getitem__(TotalDepth.util.plot.FILMCfg.PhysFilmCfg method), 189
 __getitem__(TotalDepth.util.plot.PRESCfg.PresCfg method), 209
 __getnewargs__(TotalDepth.BIT.ReadBIT.FileSizeTime method), 13
 __getnewargs__(TotalDepth.BIT.ReadBIT.LogPassRange method), 11
 __getnewargs__(TotalDepth.BIT.ReadBIT.TifMarkedBytes method), 11
 __getnewargs__(TotalDepth.BIT.ReadBIT.TifMarker method), 10
 __getnewargs__(TotalDepth.DeTif.TifMarker method), 3
 __getnewargs__(TotalDepth.LAS.core.LASRead.SectLine method), 17
 __getnewargs__(TotalDepth.LIS.LisToHtml.FileInfo method), 79
 __getnewargs__(TotalDepth.LIS.PlotLogPasses.IndexTableValue method), 80
 __getnewargs__(TotalDepth.LIS.ScanPhysRec.PhysRecScanResult method), 83
 __getnewargs__(TotalDepth.LIS.ScanPhysRec.PhysRecScanResultWithPad method), 83
 __getnewargs__(TotalDepth.LIS.TableHistogram.TableMatcher method), 84
 __getnewargs__(TotalDepth.LIS.core.File.PhysicalRecordSettings method), 26
 __getnewargs__(TotalDepth.PlotLogs.IndexTableValue method), 5
 __getnewargs__(TotalDepth.RP66V1.IndexPickle.IndexResult method), 110
 __getnewargs__(TotalDepth.RP66V1.IndexXML.IndexResult method), 113
 __getnewargs__(TotalDepth.RP66V1.LogRecIndex.IndexResult method), 113
 __getnewargs__(TotalDepth.RP66V1.Scan.IndexResult method), 115
 __getnewargs__(TotalDepth.RP66V1.Scan.LRSHSummary method), 114
 __getnewargs__(TotalDepth.RP66V1.ScanHTML.FileNameLinkHref method), 116
 __getnewargs__(TotalDepth.RP66V1.core.LogicalFile.PositionEFLR method), 95
 __getnewargs__(TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.Chan method), 103
 __getnewargs__(TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.Role method), 102
 __getnewargs__(TotalDepth.RP66V1.core.LogicalRecord.EFLR.PublicEFLRType method), 107

<code>__getnewargs__()</code> <i>talDepth.RP66V1.core.LogicalRecord.Types.TypeDescription</i> <i>method), 110</i>	(To- <code>__getnewargs__()</code> <i>talDepth.RP66V1.core.XAxis.IFLRReference</i> <i>method), 101</i>	(To- <code>__getnewargs__()</code> <i>talDepth.RP66V1.core.pFile.LRPosDesc</i> <i>method), 89</i>	(To- <code>__getnewargs__()</code> <i>talDepth.RP66V1.core.pFile.LogicalDataDescription</i> <i>method), 88</i>	(To- <code>__getnewargs__()</code> <i>talDepth.RP66V1.util.IndexXMLRead.IndexResult</i> <i>method), 119</i>	(To- <code>__getnewargs__()</code> <i>talDepth.common.lookup_mnemonic.Channel</i> <i>method), 148</i>	(To- <code>__getnewargs__()</code> <i>talDepth.common.lookup_mnemonic.ChannelDescription</i> <i>method), 149</i>	(To- <code>__getnewargs__()</code> <i>talDepth.common.lookup_mnemonic.LoggingTool</i> <i>method), 150</i>	(To- <code>__getnewargs__()</code> <i>talDepth.common.lookup_mnemonic.Parameter</i> <i>method), 149</i>	(To- <code>__getnewargs__()</code> <i>talDepth.common.lookup_mnemonic.ParameterDescription</i> <i>method), 149</i>	(To- <code>__getnewargs__()</code> <i>talDepth.common.lookup_mnemonic.ProductDescription</i> <i>method), 148</i>	(To- <code>__getnewargs__()</code> <i>talDepth.common.lookup_mnemonic.ToolDescription</i> <i>method), 148</i>	(TotalDepth.common.units.Unit <i>method), 158</i>	(To- <code>__getnewargs__()</code> <i>talDepth.util.DictTree.DictTreeTableEvent</i> <i>method), 162</i>	(TotalDepth.util.DirWalk.FileEvent <i>method), 166</i>	(TotalDepth.util.DirWalk.FileInOut <i>method), 165</i>	(To- <code>__getnewargs__()</code> <i>talDepth.util.EBCDIC.EbcdicAsciiTableEntry</i> <i>method), 167</i>	(To- <code>__getnewargs__()</code> <i>talDepth.util.plot.LogHeader.Static</i> <i>method), 192</i>	(To- <code>__getnewargs__()</code> <i>talDepth.util.plot.PRESCfg.TrackWidthData</i> <i>method), 208</i>	(TotalDepth.util.plot.ScaleSliceCurve <i>method), 197</i>	(TotalDepth.util.plot.Stroke.Stroke <i>method), 212</i>	(TotalDepth.LIS.core.EngVal.EngVal <i>method), 23</i>	(TotalDepth.util.plot.Coord.Dim <i>method), 181</i>	(TotalDepth.LIS.core.Mnem.Mnem <i>method), 57</i>	(TotalDepth.LIS.core.EngVal.EngVal <i>method), 22</i>	(TotalDepth.util.plot.Coord.Dim <i>method), 180</i>	(TotalDepth.LIS.core.EngVal.EngVal <i>method), 23</i>	(TotalDepth.util.plot.Coord.Dim <i>method), 181</i>	(TotalDepth.LIS.core.EngVal.EngVal <i>method), 12</i>	(TotalDepth.LAS.core.LASRead.LASBase <i>method), 18</i>	(TotalDepth.LAS.core.LASRead.LASRead <i>method), 19</i>	(TotalDepth.LAS.core.LASRead.LASSection <i>method), 17</i>	(TotalDepth.LAS.core.LASRead.LASSectionArray <i>method), 18</i>	(TotalDepth.LIS.LisToHtml.LisToHtml <i>method), 79</i>	(TotalDepth.LIS.PlotLogPasses.PlotLogInfo <i>method), 80</i>	(TotalDepth.LIS.PlotLogPasses.PlotLogPasses <i>method), 81</i>	(TotalDepth.LIS.ProcLISPath.ProcLISPathBase <i>method), 81</i>	(TotalDepth.LIS.ToLAS.LisLogicalFile <i>method), 84</i>	(TotalDepth.LIS.core.EngVal.EngVal <i>method), 22</i>	(TotalDepth.LIS.core.EngVal.EngValRc <i>method), 23</i>	(TotalDepth.LIS.core.File.FileBase <i>method), 24</i>	(TotalDepth.LIS.core.File.FileRead <i>method), 24</i>	(TotalDepth.LIS.core.File.FileWrite <i>method), 25</i>	(TotalDepth.LIS.core.FileIndexer.FileIndex <i>method), 30</i>	(TotalDepth.LIS.core.FileIndexer.IndexFileHead <i>method), 28</i>	(TotalDepth.LIS.core.FileIndexer.IndexFileHeadTail <i>method), 28</i>
---	---	--	---	--	--	---	--	--	---	---	--	--	--	---	---	---	--	--	--	--	--	--	--	--	--	--	--	--	--	--	---	--	---	---	---	---	--	--	--	--	--	---	--	--	--

<code>__init__()</code> (<i>TotalDepth.LIS.core.FileIndexer.IndexFileTail</i> <code>__init__()</code> method), 28	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.CbEngVal</i> <code>__init__()</code> method), 45
<code>__init__()</code> (<i>TotalDepth.LIS.core.FileIndexer.IndexLogPass</i> <code>__init__()</code> method), 29	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.CbEngValRead</i> <code>__init__()</code> method), 46
<code>__init__()</code> (<i>TotalDepth.LIS.core.FileIndexer.IndexLrFull</i> <code>__init__()</code> method), 28	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.CbEngValWrite</i> <code>__init__()</code> method), 46
<code>__init__()</code> (<i>TotalDepth.LIS.core.FileIndexer.IndexNone</i> <code>__init__()</code> method), 27	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.DatumSpecBlock</i> <code>__init__()</code> method), 49
<code>__init__()</code> (<i>TotalDepth.LIS.core.FileIndexer.IndexObjBase</i> <code>__init__()</code> method), 27	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.DatumSpecBlockRead</i> <code>__init__()</code> method), 49
<code>__init__()</code> (<i>TotalDepth.LIS.core.FileIndexer.IndexReelHead</i> <code>__init__()</code> method), 28	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.EntryBlockSet</i> <code>__init__()</code> method), 48
<code>__init__()</code> (<i>TotalDepth.LIS.core.FileIndexer.IndexReelHeadTail</i> <code>__init__()</code> method), 28	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrBOT</i> <code>__init__()</code> method), 42
<code>__init__()</code> (<i>TotalDepth.LIS.core.FileIndexer.IndexReelTail</i> <code>__init__()</code> method), 28	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrBOTRead</i> <code>__init__()</code> method), 42
<code>__init__()</code> (<i>TotalDepth.LIS.core.FileIndexer.IndexTable</i> <code>__init__()</code> method), 29	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrBase</i> <code>__init__()</code> method), 41
<code>__init__()</code> (<i>TotalDepth.LIS.core.FileIndexer.IndexTapeHead</i> <code>__init__()</code> method), 28	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrDFSR</i> <code>__init__()</code> method), 49
<code>__init__()</code> (<i>TotalDepth.LIS.core.FileIndexer.IndexTapeHeadTail</i> <code>__init__()</code> method), 28	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrDFSRRRead</i> <code>__init__()</code> method), 49
<code>__init__()</code> (<i>TotalDepth.LIS.core.FileIndexer.IndexTapeTail</i> <code>__init__()</code> method), 28	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrEOF</i> <code>__init__()</code> method), 41
<code>__init__()</code> (<i>TotalDepth.LIS.core.FileIndexer.IndexUnknownFormat</i> <code>__init__()</code> method), 27	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrEOFRead</i> <code>__init__()</code> method), 42
<code>__init__()</code> (<i>TotalDepth.LIS.core.FileIndexer.PlotRecordSet</i> <code>__init__()</code> method), 29	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrEOM</i> <code>__init__()</code> method), 42
<code>__init__()</code> (<i>TotalDepth.LIS.core.FrameSet.AccActivity</i> <code>__init__()</code> method), 38	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrEOMRead</i> <code>__init__()</code> method), 42
<code>__init__()</code> (<i>TotalDepth.LIS.core.FrameSet.AccBias</i> <code>__init__()</code> method), 37	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrEOT</i> <code>__init__()</code> method), 42
<code>__init__()</code> (<i>TotalDepth.LIS.core.FrameSet.AccCount</i> <code>__init__()</code> method), 37	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrEOTRead</i> <code>__init__()</code> method), 42
<code>__init__()</code> (<i>TotalDepth.LIS.core.FrameSet.AccDelta</i> <code>__init__()</code> method), 37	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrFactory</i> <code>__init__()</code> method), 50
<code>__init__()</code> (<i>TotalDepth.LIS.core.FrameSet.AccDrift</i> <code>__init__()</code> method), 38	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrFactoryRead</i> <code>__init__()</code> method), 50
<code>__init__()</code> (<i>TotalDepth.LIS.core.FrameSet.AccMax</i> <code>__init__()</code> method), 36	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrFileHead</i> <code>__init__()</code> method), 43
<code>__init__()</code> (<i>TotalDepth.LIS.core.FrameSet.AccMean</i> <code>__init__()</code> method), 36	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrFileHeadRead</i> <code>__init__()</code> method), 43
<code>__init__()</code> (<i>TotalDepth.LIS.core.FrameSet.AccMin</i> <code>__init__()</code> method), 36	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrFileHeadTail</i> <code>__init__()</code> method), 42
<code>__init__()</code> (<i>TotalDepth.LIS.core.FrameSet.AccStDev</i> <code>__init__()</code> method), 36	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrFileTail</i> <code>__init__()</code> method), 43
<code>__init__()</code> (<i>TotalDepth.LIS.core.FrameSet.ChArTe</i> <code>__init__()</code> method), 33	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrFileTailRead</i> <code>__init__()</code> method), 43
<code>__init__()</code> (<i>TotalDepth.LIS.core.FrameSet.DataSeqBase</i> <code>__init__()</code> method), 32	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrMarker</i> <code>__init__()</code> method), 41
<code>__init__()</code> (<i>TotalDepth.LIS.core.FrameSet.FrameSet</i> <code>__init__()</code> method), 34	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrMisc</i> <code>__init__()</code> method), 44
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogPass.LogPass</i> <code>__init__()</code> method), 51	<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrMiscRead</i> <code>__init__()</code> method), 44

<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrNormalAlternateData</i> method), 49	<code>__init__()</code> (<i>TotalDepth.LIS.core.Rle.RLEType01</i> method), 70
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrNormalAlternateDataRecd</i> method), 50	<code>__init__()</code> (<i>TotalDepth.LIS.core.TifMarker.TifMarkerBase</i> method), 72
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrReelHead</i> method), 44	<code>__init__()</code> (<i>TotalDepth.LIS.core.TifMarker.TifMarkerRead</i> method), 73
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrReelHeadRead</i> method), 44	<code>__init__()</code> (<i>TotalDepth.LIS.core.TifMarker.TifMarkerWrite</i> method), 73
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrReelHeadTail</i> method), 44	<code>__init__()</code> (<i>TotalDepth.LIS.core.Type01Plan.FrameSetPlan</i> method), 74
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrReelTail</i> method), 44	<code>__init__()</code> (<i>TotalDepth.LIS.core.Units.UnitConvert</i> method), 76
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrReelTailRead</i> method), 44	<code>__init__()</code> (<i>TotalDepth.LIS.core.Units.UnitConvertCategory</i> method), 76
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrReelTapeHeadTail</i> method), 43	<code>__init__()</code> (<i>TotalDepth.PlotLogs.PlotLogInfo</i> method), 5
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrTable</i> method), 46	<code>__init__()</code> (<i>TotalDepth.PlotLogs.PlotLogPasses</i> method), 6
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrTableRead</i> method), 47	<code>__init__()</code> (<i>TotalDepth.RP66V1.core.LogPass.RP66V1FrameChannel</i> method), 93
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrTableWrite</i> method), 47	<code>__init__()</code> (<i>TotalDepth.RP66V1.core.LogicalFile.LogicalFile</i> method), 95
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrTapeHead</i> method), 43	<code>__init__()</code> (<i>TotalDepth.RP66V1.core.LogicalFile.LogicalIndex</i> method), 96
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrTapeHeadRead</i> method), 43	<code>__init__()</code> (<i>TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor</i> method), 103
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrTapeHeadTail</i> method), 43	<code>__init__()</code> (<i>TotalDepth.RP66V1.core.LogicalRecord.EFLR.Attribute</i> method), 105
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrTapeTail</i> method), 43	<code>__init__()</code> (<i>TotalDepth.RP66V1.core.LogicalRecord.EFLR.AttributeBase</i> method), 105
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrTapeTailRead</i> method), 44	<code>__init__()</code> (<i>TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyForm</i> method), 107
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrUnknown</i> method), 44	<code>__init__()</code> (<i>TotalDepth.RP66V1.core.LogicalRecord.EFLR.Object</i> method), 106
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrUnkownRead</i> method), 45	<code>__init__()</code> (<i>TotalDepth.RP66V1.core.LogicalRecord.EFLR.Set</i> method), 105
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrWithDateField</i> method), 42	<code>__init__()</code> (<i>TotalDepth.RP66V1.core.LogicalRecord.EFLR.Template</i> method), 105
<code>__init__()</code> (<i>TotalDepth.LIS.core.LogiRec.TableRow</i> method), 46	<code>__init__()</code> (<i>TotalDepth.RP66V1.core.LogicalRecord.EFLR.TemplateAttri</i> method), 105
<code>__init__()</code> (<i>TotalDepth.LIS.core.Mnem.Mnem</i> method), 57	<code>__init__()</code> (<i>TotalDepth.RP66V1.core.LogicalRecord.IFLR.IndirectlyForm</i> method), 108
<code>__init__()</code> (<i>TotalDepth.LIS.core.PhysRec.PhysRecBase</i> method), 60	<code>__init__()</code> (<i>TotalDepth.RP66V1.core.LogicalRecord.Semantics.Restriction</i> method), 109
<code>__init__()</code> (<i>TotalDepth.LIS.core.PhysRec.PhysRecRead</i> method), 60	<code>__init__()</code> (<i>TotalDepth.RP66V1.core.XAxis.XAxis</i> method), 101
<code>__init__()</code> (<i>TotalDepth.LIS.core.PhysRec.PhysRecTail</i> method), 61	<code>__init__()</code> (<i>TotalDepth.RP66V1.core.pFile.FileLogicalData</i> method), 90
<code>__init__()</code> (<i>TotalDepth.LIS.core.PhysRec.PhysRecWrite</i> method), 62	<code>__init__()</code> (<i>TotalDepth.RP66V1.core.pFile.FileRead</i> method), 90
<code>__init__()</code> (<i>TotalDepth.LIS.core.RawStream.RawStream</i> method), 62	<code>__init__()</code> (<i>TotalDepth.RP66V1.core.pFile.LogicalData</i> method), 89
<code>__init__()</code> (<i>TotalDepth.LIS.core.Rle.RLEItemType01</i> method), 69	<code>__init__()</code> (<i>TotalDepth.RP66V1.core.pFile.LogicalRecordPosition</i> method), 88

`__init__()` (*TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHeader* method), 176
`__init__()` (*TotalDepth.RP66V1.core.pFile.StorageUnitLabel* method), 86
`__init__()` (*TotalDepth.RP66V1.core.pFile.VisibleRecord* method), 87
`__init__()` (*TotalDepth.RP66V1.core.pIndex.LogicalRecordIndex* method), 92
`__init__()` (*TotalDepth.common.LogPass.FrameArray* method), 142
`__init__()` (*TotalDepth.common.LogPass.FrameChannel* method), 141
`__init__()` (*TotalDepth.common.LogPass.LogPass* method), 143
`__init__()` (*TotalDepth.common.Rle.RLE* method), 153
`__init__()` (*TotalDepth.common.Rle.RLEItem* method), 153
`__init__()` (*TotalDepth.common.Slice.Sample* method), 155
`__init__()` (*TotalDepth.common.Slice.Slice* method), 155
`__init__()` (*TotalDepth.common.process.ProcessLoggingThread* method), 152
`__init__()` (*TotalDepth.common.statistics.LengthDict* method), 156
`__init__()` (*TotalDepth.util.DictTree.DictTree* method), 161
`__init__()` (*TotalDepth.util.DictTree.DictTreeHtmlTable* method), 164
`__init__()` (*TotalDepth.util.ExecTimer.Timer* method), 168
`__init__()` (*TotalDepth.util.ExecTimer.TimerList* method), 168
`__init__()` (*TotalDepth.util.FileBuffer.FileBuffer* method), 169
`__init__()` (*TotalDepth.util.FileStatus.FileInfo* method), 170
`__init__()` (*TotalDepth.util.FileStatus.FileInfoSet* method), 170
`__init__()` (*TotalDepth.util.Histogram.Histogram* method), 171
`__init__()` (*TotalDepth.util.XmlWrite.Element* method), 175
`__init__()` (*TotalDepth.util.XmlWrite.XmlStream* method), 174
`__init__()` (*TotalDepth.util.archive.FileArchive* method), 176
`__init__()` (*TotalDepth.util.archive.FileBase* method), 176
`__init__()` (*TotalDepth.util.archive.FileInMemory* method), 176
`__init__()` (*TotalDepth.util.archive.FileMembers* method), 176
`__init__()` (*TotalDepth.util.archive.FileOnDisc* method), 176
`__init__()` (*TotalDepth.util.archive.FileZip* method), 176
`__init__()` (*TotalDepth.util.plot.FILMCfg.FilmCfg* method), 190
`__init__()` (*TotalDepth.util.plot.FILMCfg.FilmCfgLISRead* method), 190
`__init__()` (*TotalDepth.util.plot.FILMCfg.PhysFilmCfg* method), 189
`__init__()` (*TotalDepth.util.plot.FILMCfg.PhysFilmCfgLISRead* method), 190
`__init__()` (*TotalDepth.util.plot.FILMCfgXML.FilmCfgXMLRead* method), 192
`__init__()` (*TotalDepth.util.plot.FILMCfgXML.PhysFilmCfgXMLRead* method), 191
`__init__()` (*TotalDepth.util.plot.LogHeader.APIHeaderBase* method), 194
`__init__()` (*TotalDepth.util.plot.LogHeader.APIHeaderLAS* method), 194
`__init__()` (*TotalDepth.util.plot.LogHeader.APIHeaderLIS* method), 194
`__init__()` (*TotalDepth.util.plot.PRESCfg.CurveCfg* method), 208
`__init__()` (*TotalDepth.util.plot.PRESCfg.CurveCfgLISRead* method), 209
`__init__()` (*TotalDepth.util.plot.PRESCfg.LineTransBase* method), 207
`__init__()` (*TotalDepth.util.plot.PRESCfg.LineTransLin* method), 208
`__init__()` (*TotalDepth.util.plot.PRESCfg.LineTransLog10* method), 208
`__init__()` (*TotalDepth.util.plot.PRESCfg.PresCfg* method), 209
`__init__()` (*TotalDepth.util.plot.PRESCfg.PresCfgLISRead* method), 210
`__init__()` (*TotalDepth.util.plot.PRESCfgXML.CurveCfgXMLRead* method), 211
`__init__()` (*TotalDepth.util.plot.PRESCfgXML.PresCfgXMLRead* method), 211
`__init__()` (*TotalDepth.util.plot.Plot.CurvePlotScaleSlotMap* method), 197
`__init__()` (*TotalDepth.util.plot.Plot.Plot* method), 200
`__init__()` (*TotalDepth.util.plot.Plot.PlotReadLIS* method), 201
`__init__()` (*TotalDepth.util.plot.Plot.PlotReadXML* method), 201
`__init__()` (*TotalDepth.util.plot.Plot.PlotRoll* method), 198
`__init__()` (*TotalDepth.util.plot.SVGWriter.SVGCircle* method), 213
`__init__()` (*TotalDepth.util.plot.SVGWriter.SVGElipse* method), 213
`__init__()` (*TotalDepth.util.plot.SVGWriter.SVGGroup* method), 213

<code>__init__()</code> (<i>TotalDepth.util.plot.SVGWriter.SVGLine</i> method), 213	<code>__len__()</code> (<i>TotalDepth.common.LogPass.FrameArray</i> method), 143
<code>__init__()</code> (<i>TotalDepth.util.plot.SVGWriter.SVGPointList</i> method), 214	<code>__len__()</code> (<i>TotalDepth.common.LogPass.LogPass</i> method), 144
<code>__init__()</code> (<i>TotalDepth.util.plot.SVGWriter.SVGPolygon</i> method), 214	<code>__len__()</code> (<i>TotalDepth.common.Rle.RLE</i> method), 153
<code>__init__()</code> (<i>TotalDepth.util.plot.SVGWriter.SVGPolyline</i> method), 214	<code>__len__()</code> (<i>TotalDepth.common.Rle.RLEItem</i> method), 153
<code>__init__()</code> (<i>TotalDepth.util.plot.SVGWriter.SVGRect</i> method), 213	<code>__len__()</code> (<i>TotalDepth.common.statistics.LengthDict</i> method), 156
<code>__init__()</code> (<i>TotalDepth.util.plot.SVGWriter.SVGText</i> method), 214	<code>__len__()</code> (<i>TotalDepth.util.DictTree.DictTree</i> method), 162
<code>__init__()</code> (<i>TotalDepth.util.plot.SVGWriter.SVGWriter</i> method), 212	<code>__len__()</code> (<i>TotalDepth.util.ExecTimer.TimerList</i> method), 168
<code>__init__()</code> (<i>TotalDepth.util.plot.Track.Track</i> method), 219	<code>__len__()</code> (<i>TotalDepth.util.plot.FILMCfg.FilmCfg</i> method), 190
<code>__init__()</code> (<i>TotalDepth.util.plot.XGrid.XGrid</i> method), 220	<code>__len__()</code> (<i>TotalDepth.util.plot.FILMCfg.PhysFilmCfg</i> method), 189
<code>__isub__()</code> (<i>TotalDepth.LIS.core.EngVal.EngVal</i> method), 23	<code>__len__()</code> (<i>TotalDepth.util.plot.PRESCfg.PresCfg</i> method), 209
<code>__isub__()</code> (<i>TotalDepth.util.plot.Coord.Dim</i> method), 180	<code>__lt__()</code> (<i>TotalDepth.LIS.core.EngVal.EngVal</i> method), 23
<code>__iter__()</code> (<i>TotalDepth.LIS.core.Mnem.Mnem</i> method), 57	<code>__lt__()</code> (<i>TotalDepth.LIS.core.Mnem.Mnem</i> method), 57
<code>__itruediv__()</code> (<i>TotalDepth.LIS.core.EngVal.EngVal</i> method), 23	<code>__lt__()</code> (<i>TotalDepth.util.archive.FileBase</i> method), 176
<code>__itruediv__()</code> (<i>TotalDepth.util.plot.Coord.Dim</i> method), 181	<code>__lt__()</code> (<i>TotalDepth.util.plot.Coord.Dim</i> method), 181
<code>__le__()</code> (<i>TotalDepth.LIS.core.EngVal.EngVal</i> method), 23	<code>__lt__()</code> (<i>TotalDepth.util.plot.Plot.CurvePlotScale</i> method), 196
<code>__le__()</code> (<i>TotalDepth.util.plot.Coord.Dim</i> method), 181	<code>__mul__()</code> (<i>TotalDepth.LIS.core.EngVal.EngVal</i> method), 22
<code>__len__()</code> (<i>TotalDepth.LAS.core.LASRead.LASBase</i> method), 18	<code>__mul__()</code> (<i>TotalDepth.util.plot.Coord.Dim</i> method), 180
<code>__len__()</code> (<i>TotalDepth.LAS.core.LASRead.LASSection</i> method), 17	<code>__ne__()</code> (<i>TotalDepth.LIS.core.EngVal.EngVal</i> method), 23
<code>__len__()</code> (<i>TotalDepth.LAS.core.LASRead.LASSectionArray</i> method), 18	<code>__ne__()</code> (<i>TotalDepth.LIS.core.Mnem.Mnem</i> method), 57
<code>__len__()</code> (<i>TotalDepth.LIS.core.LogiRec.LrTable</i> method), 47	<code>__ne__()</code> (<i>TotalDepth.util.plot.Coord.Dim</i> method), 181
<code>__len__()</code> (<i>TotalDepth.LIS.core.LogiRec.TableRow</i> method), 46	<code>__new__()</code> (<i>TotalDepth.BIT.ReadBIT.FileSizeTime</i> static method), 13
<code>__len__()</code> (<i>TotalDepth.RP66V1.core.LogicalFile.LogicalIndex</i> method), 96	<code>__new__()</code> (<i>TotalDepth.BIT.ReadBIT.LogPassRange</i> static method), 11
<code>__len__()</code> (<i>TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyMarkedLogicalRecord</i> method), 107	<code>__new__()</code> (<i>TotalDepth.BIT.ReadBIT.TifMarkedBytes</i> static method), 11
<code>__len__()</code> (<i>TotalDepth.RP66V1.core.LogicalRecord.EFLR.Object</i> method), 106	<code>__new__()</code> (<i>TotalDepth.BIT.ReadBIT.TifMarker</i> static method), 11
<code>__len__()</code> (<i>TotalDepth.RP66V1.core.LogicalRecord.EFLR.Template</i> method), 106	<code>__new__()</code> (<i>TotalDepth.DeTif.TifMarker</i> static method), 11
<code>__len__()</code> (<i>TotalDepth.RP66V1.core.XAxis.XAxis</i> method), 101	<code>__new__()</code> (<i>TotalDepth.LAS.core.LASRead.SectLine</i> static method), 17
<code>__len__()</code> (<i>TotalDepth.RP66V1.core.pFile.FileLogicalData</i> method), 90	<code>__new__()</code> (<i>TotalDepth.LIS.LisToHtml.FileInfo</i> static method), 79
<code>__len__()</code> (<i>TotalDepth.RP66V1.core.pFile.LogicalData</i> method), 90	<code>__new__()</code> (<i>TotalDepth.LIS.PlotLogPasses.IndexTableValue</i> static method), 80

__new__ () (TotalDepth.LIS.ScanPhysRec.PhysRecScanResultWithPad static method), 83	__new__ () (TotalDepth.common.lookup_mnemonic.ToolDescription static method), 148
__new__ () (TotalDepth.LIS.ScanPhysRec.PhysRecScanResultWithPad static method), 83	__new__ () (TotalDepth.common.units.Unit static method), 158
__new__ () (TotalDepth.LIS.TableHistogram.TableMatcher static method), 84	__new__ () (TotalDepth.util.DictTree.DictTreeTableEvent static method), 162
__new__ () (TotalDepth.LIS.core.File.PhysicalRecordSettings static method), 26	__new__ () (TotalDepth.util.DirWalk.FileEvent static method), 166
__new__ () (TotalDepth.LIS.core.LogiRec.EntryBlockRead static method), 48	__new__ () (TotalDepth.util.DirWalk.FileInOut static method), 165
__new__ () (TotalDepth.PlotLogs.IndexTableValue static method), 5	__new__ () (TotalDepth.util.EBCDIC.EbcdicAsciiTableEntry static method), 168
__new__ () (TotalDepth.RP66V1.IndexPickle.IndexResult static method), 110	__new__ () (TotalDepth.util.plot.LogHeader.Static static method), 193
__new__ () (TotalDepth.RP66V1.IndexXML.IndexResult static method), 113	__new__ () (TotalDepth.util.plot.PRESCfg.TrackWidthData static method), 208
__new__ () (TotalDepth.RP66V1.LogRecIndex.IndexResult static method), 113	__new__ () (TotalDepth.util.plot.Plot.ScaleSliceCurve static method), 197
__new__ () (TotalDepth.RP66V1.Scan.IndexResult static method), 115	__new__ () (TotalDepth.util.plot.Stroke.Stroke static method), 212
__new__ () (TotalDepth.RP66V1.Scan.LRSHSummary static method), 114	__radd__ () (TotalDepth.LIS.core.EngVal.EngVal static method), 22
__new__ () (TotalDepth.RP66V1.ScanHTML.FileNameLinkHref static method), 116	__repr__ () (TotalDepth.BIT.ReadBIT.FileSizeTime static method), 13
__new__ () (TotalDepth.RP66V1.core.LogicalFile.PositionEFLR static method), 95	__repr__ () (TotalDepth.BIT.ReadBIT.LogPassRange static method), 11
__new__ () (TotalDepth.RP66V1.core.LogicalRecord.ComponentDescription (TotalDepth.BIT.RepCodeBETToMDefBy static method), 103	__repr__ () (TotalDepth.BIT.RepCodeBETToMDefBy static method), 11
__new__ () (TotalDepth.RP66V1.core.LogicalRecord.ComponentDescription (TotalDepth.BIT.RepCodeBETToMDefBy static method), 102	__repr__ () (TotalDepth.BIT.RepCodeBETToMDefBy static method), 11
__new__ () (TotalDepth.RP66V1.core.LogicalRecord.EFLR.PredefEFLRType static method), 107	__repr__ () (TotalDepth.DeTif.TifMarker static method), 3
__new__ () (TotalDepth.RP66V1.core.LogicalRecord.Types.TypeDescriptionAllowableSetTypes static method), 110	__repr__ () (TotalDepth.LAS.core.LASRead.SectLine static method), 7
__new__ () (TotalDepth.RP66V1.core.XAxis.IFLRReference static method), 101	__repr__ () (TotalDepth.LIS.LisToHtml.FileInfo static method), 79
__new__ () (TotalDepth.RP66V1.core.pFile.LRPosDesc static method), 89	__repr__ () (TotalDepth.LIS.PlotLogPasses.IndexTableValue static method), 80
__new__ () (TotalDepth.RP66V1.core.pFile.LogicalDataDescription static method), 88	__repr__ () (TotalDepth.LIS.ScanPhysRec.PhysRecScanResult static method), 83
__new__ () (TotalDepth.RP66V1.util.IndexXMLRead.IndexResult static method), 119	__repr__ () (TotalDepth.LIS.ScanPhysRec.PhysRecScanResultWithPad static method), 83
__new__ () (TotalDepth.common.lookup_mnemonic.Channel static method), 148	__repr__ () (TotalDepth.LIS.TableHistogram.TableMatcher static method), 84
__new__ () (TotalDepth.common.lookup_mnemonic.ChannelDescription static method), 149	__repr__ () (TotalDepth.LIS.core.File.PhysicalRecordSettings static method), 25
__new__ () (TotalDepth.common.lookup_mnemonic.LoggingTool static method), 150	__repr__ () (TotalDepth.LIS.core.Mnem.Mnem static method), 57
__new__ () (TotalDepth.common.lookup_mnemonic.Parameter static method), 149	__repr__ () (TotalDepth.PlotLogs.IndexTableValue static method), 5
__new__ () (TotalDepth.common.lookup_mnemonic.ParameterDescription static method), 149	__repr__ () (TotalDepth.RP66V1.IndexPickle.IndexResult static method), 110
__new__ () (TotalDepth.common.lookup_mnemonic.ProductDescription static method), 148	__repr__ () (TotalDepth.RP66V1.IndexXML.IndexResult static method), 113
	__repr__ () (TotalDepth.RP66V1.LogRecIndex.IndexResult static method), 113

Index 251

method), 51

__str__() (TotalDepth.LIS.core.LogiRec.CbEngVal method), 45

__str__() (TotalDepth.LIS.core.LogiRec.EntryBlockSet method), 48

__str__() (TotalDepth.LIS.core.LogiRec.LrBase method), 41

__str__() (TotalDepth.LIS.core.Mnem.Mnem method), 57

__str__() (TotalDepth.LIS.core.PhysRec.PhysRecBase method), 60

__str__() (TotalDepth.LIS.core.PhysRec.PhysRecRead method), 60

__str__() (TotalDepth.LIS.core.PhysRec.PhysRecTail method), 61

__str__() (TotalDepth.LIS.core.Rle.RLEItemType01 method), 69

__str__() (TotalDepth.LIS.core.Rle.RLEType01 method), 70

__str__() (TotalDepth.LIS.core.TifMarker.TifMarkerBase method), 72

__str__() (TotalDepth.LIS.core.Type01Plan.FrameSetPlan method), 74

__str__() (TotalDepth.PlotLogs.PlotLogInfo method), 5

__str__() (TotalDepth.RP66V1.core.LogicalRecord.ComponentsDescrToComponentDescr method), 103

__str__() (TotalDepth.RP66V1.core.LogicalRecord.EFLR.AttributeBase method), 105

__str__() (TotalDepth.RP66V1.core.LogicalRecord.EFLR.Extrinsic method), 107

__str__() (TotalDepth.RP66V1.core.LogicalRecord.EFLR.Object method), 106

__str__() (TotalDepth.RP66V1.core.LogicalRecord.EFLR.Section method), 105

__str__() (TotalDepth.RP66V1.core.LogicalRecord.EFLR.Template method), 106

__str__() (TotalDepth.RP66V1.core.LogicalRecord.IFLR.IndirectlyFormattedLogicalRecord method), 108

__str__() (TotalDepth.RP66V1.core.pFile.FileLogicalData method), 90

__str__() (TotalDepth.RP66V1.core.pFile.LRPosDesc method), 89

__str__() (TotalDepth.RP66V1.core.pFile.LogicalData method), 90

__str__() (TotalDepth.RP66V1.core.pFile.LogicalDataDescription method), 88

__str__() (TotalDepth.RP66V1.core.pFile.LogicalRecordPosition method), 88

__str__() (TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHeader method), 87

__str__() (TotalDepth.RP66V1.core.pFile.StorageUnitLabel method), 86

__str__() (TotalDepth.RP66V1.core.pFile.VisibleRecord method), 87

__str__() (TotalDepth.common.LogPass.FrameArray method), 143

__str__() (TotalDepth.common.LogPass.FrameChannel method), 142

__str__() (TotalDepth.common.LogPass.LogPass method), 144

__str__() (TotalDepth.common.Rle.RLE method), 153

__str__() (TotalDepth.common.Rle.RLEItem method), 153

__str__() (TotalDepth.common.Slice.Sample method), 156

__str__() (TotalDepth.common.Slice.Slice method), 155

__str__() (TotalDepth.common.Slice.SliceABC method), 155

__str__() (TotalDepth.util.ExecTimer.Timer method), 168

__str__() (TotalDepth.util.ExecTimer.TimerList method), 169

__str__() (TotalDepth.util.archive.FileBase method), 176

__str__() (TotalDepth.util.archive.FileMembers method), 176

__str__() (TotalDepth.util.archive.FileZip method), 176

__str__() (TotalDepth.util.plot.Coord.Box method), 181

__str__() (TotalDepth.util.plot.Coord.Coord method), 180

__str__() (TotalDepth.util.plot.Coord.Coord.Margin method), 181

__str__() (TotalDepth.util.plot.Coord.Pad method), 181

__str__() (TotalDepth.util.plot.Coord.Pt method), 182

__str__() (TotalDepth.util.plot.PRESCfg.LineTransBase method), 207

__str__() (TotalDepth.util.plot.Track.Track method), 207

__sub__() (TotalDepth.LIS.core.EngVal.EngVal method), 22

__sub__() (TotalDepth.util.plot.Coord.Coord method), 180

__truediv__() (TotalDepth.LIS.core.EngVal.EngVal method), 22

__truediv__() (TotalDepth.util.plot.Coord.Coord method), 180

__weakref__ (TotalDepth.BIT.ReadBIT.BITFrameArray attribute), 12

__weakref__ (TotalDepth.DeTif.DeTifException attribute), 3

__weakref__ (TotalDepth.LAS.core.LASRead.LASBase attribute), 19

__weakref__ (TotalDepth.LAS.core.LASRead.LASSection attribute), 18

__weakref__ (TotalDepth.LIS.PlotLogPasses.PlotLogInfo attribute), 18

<i>attribute</i>), 81	<i>attribute</i>), 63
__weakref__ (TotalDepth.LIS.PlotLogPasses.PlotLogPasses_ <i>attribute</i>), 81	__weakref__ (TotalDepth.LIS.core.TifMarker.TifMarkerBase <i>attribute</i>), 72
__weakref__ (TotalDepth.LIS.ProcLISPath.ProcLISPathBase_ <i>attribute</i>), 81	__weakref__ (TotalDepth.LIS.core.Type01Plan.FrameSetPlan <i>attribute</i>), 74
__weakref__ (TotalDepth.LIS.ToLAS.LisLogicalFile_ <i>attribute</i>), 84	__weakref__ (TotalDepth.LIS.core.Units.UnitConvert <i>attribute</i>), 76
__weakref__ (TotalDepth.LIS.core.EngVal.EngVal_ <i>attribute</i>), 23	__weakref__ (TotalDepth.LIS.core.Units.UnitConvertCategory <i>attribute</i>), 76
__weakref__ (TotalDepth.LIS.core.File.FileBase_ <i>attribute</i>), 24	__weakref__ (TotalDepth.PlotLogs.PlotLogInfo_ <i>attribute</i>), 6
__weakref__ (TotalDepth.LIS.core.FileIndexer.FileIndex_ <i>attribute</i>), 30	__weakref__ (TotalDepth.PlotLogs.PlotLogPasses_ <i>attribute</i>), 6
__weakref__ (TotalDepth.LIS.core.FileIndexer.IndexObjBase_ <i>attribute</i>), 27	__weakref__ (TotalDepth.RP66V1.core.LogicalFile.LogicalFile <i>attribute</i>), 96
__weakref__ (TotalDepth.LIS.core.FileIndexer.PlotRecordSet_ <i>attribute</i>), 30	__weakref__ (TotalDepth.RP66V1.core.LogicalFile.LogicalIndex <i>attribute</i>), 97
__weakref__ (TotalDepth.LIS.core.FrameSet.AccCount_ <i>attribute</i>), 37	__weakref__ (TotalDepth.RP66V1.core.LogicalRecord.ComponentDescrip_ <i>attribute</i>), 104
__weakref__ (TotalDepth.LIS.core.FrameSet.AccDelta_ <i>attribute</i>), 37	__weakref__ (TotalDepth.RP66V1.core.LogicalRecord.EFLR.AttributeBas_ <i>attribute</i>), 105
__weakref__ (TotalDepth.LIS.core.FrameSet.AccMax_ <i>attribute</i>), 36	__weakref__ (TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFo_ <i>attribute</i>), 108
__weakref__ (TotalDepth.LIS.core.FrameSet.AccMean_ <i>attribute</i>), 36	__weakref__ (TotalDepth.RP66V1.core.LogicalRecord.EFLR.Object <i>attribute</i>), 106
__weakref__ (TotalDepth.LIS.core.FrameSet.AccMin_ <i>attribute</i>), 36	__weakref__ (TotalDepth.RP66V1.core.LogicalRecord.EFLR.Set <i>attribute</i>), 105
__weakref__ (TotalDepth.LIS.core.FrameSet.AccStDev_ <i>attribute</i>), 37	__weakref__ (TotalDepth.RP66V1.core.LogicalRecord.EFLR.Template <i>attribute</i>), 106
__weakref__ (TotalDepth.LIS.core.FrameSet.DataSeqBase_ <i>attribute</i>), 32	__weakref__ (TotalDepth.RP66V1.core.LogicalRecord.IFLR.IndirectlyFor_ <i>attribute</i>), 108
__weakref__ (TotalDepth.LIS.core.FrameSet.FrameSet_ <i>attribute</i>), 36	__weakref__ (TotalDepth.RP66V1.core.LogicalRecord.Semantics.Restrict_ <i>attribute</i>), 109
__weakref__ (TotalDepth.LIS.core.LogPass.LogPass_ <i>attribute</i>), 53	__weakref__ (TotalDepth.RP66V1.core.XAxis.XAxis_ <i>attribute</i>), 101
__weakref__ (TotalDepth.LIS.core.LogiRec.CbEngVal_ <i>attribute</i>), 45	__weakref__ (TotalDepth.RP66V1.core.pFile.FileLogicalData <i>attribute</i>), 90
__weakref__ (TotalDepth.LIS.core.LogiRec.DatumSpecBlock_ <i>attribute</i>), 49	__weakref__ (TotalDepth.RP66V1.core.pFile.FileRead <i>attribute</i>), 91
__weakref__ (TotalDepth.LIS.core.LogiRec.EntryBlockSet_ <i>attribute</i>), 49	__weakref__ (TotalDepth.RP66V1.core.pFile.LogicalData <i>attribute</i>), 90
__weakref__ (TotalDepth.LIS.core.LogiRec.LrBase_ <i>attribute</i>), 41	__weakref__ (TotalDepth.RP66V1.core.pFile.LogicalRecordPosition <i>attribute</i>), 88
__weakref__ (TotalDepth.LIS.core.LogiRec.LrFactory_ <i>attribute</i>), 50	__weakref__ (TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHea_ <i>attribute</i>), 88
__weakref__ (TotalDepth.LIS.core.LogiRec.TableRow_ <i>attribute</i>), 46	__weakref__ (TotalDepth.RP66V1.core.pFile.StorageUnitLabel <i>attribute</i>), 86
__weakref__ (TotalDepth.LIS.core.Mnem.Mnem_ <i>attribute</i>), 57	__weakref__ (TotalDepth.RP66V1.core.pFile.VisibleRecord <i>attribute</i>), 87
__weakref__ (TotalDepth.LIS.core.PhysRec.PhysRecBase_ <i>attribute</i>), 60	__weakref__ (TotalDepth.RP66V1.core.pIndex.LogicalRecordIndex <i>attribute</i>), 92
__weakref__ (TotalDepth.LIS.core.PhysRec.PhysRecTail_ <i>attribute</i>), 61	__weakref__ (TotalDepth.common.LogPass.FrameArray <i>attribute</i>), 143
__weakref__ (TotalDepth.LIS.core.RawStream.RawStream_ <i>attribute</i>), 63	__weakref__ (TotalDepth.common.LogPass.FrameChannel <i>attribute</i>), 143

- attribute), 142
- __weakref__ (TotalDepth.common.LogPass.LogPass attribute), 144
- __weakref__ (TotalDepth.common.Rle.RLE attribute), 154
- __weakref__ (TotalDepth.common.Rle.RLEItem attribute), 153
- __weakref__ (TotalDepth.common.Slice.SliceABC attribute), 155
- __weakref__ (TotalDepth.common.statistics.LengthDict attribute), 156
- __weakref__ (TotalDepth.util.DictTree.DictTree attribute), 162
- __weakref__ (TotalDepth.util.ExecTimer.Timer attribute), 168
- __weakref__ (TotalDepth.util.ExecTimer.TimerList attribute), 169
- __weakref__ (TotalDepth.util.FileBuffer.ExceptionFileBuffer attribute), 169
- __weakref__ (TotalDepth.util.FileBuffer.FileBuffer attribute), 169
- __weakref__ (TotalDepth.util.FileStatus.ExceptionFileStatus attribute), 170
- __weakref__ (TotalDepth.util.FileStatus.FileInfo attribute), 170
- __weakref__ (TotalDepth.util.FileStatus.FileInfoSet attribute), 170
- __weakref__ (TotalDepth.util.Histogram.Histogram attribute), 171
- __weakref__ (TotalDepth.util.XmlWrite.Element attribute), 175
- __weakref__ (TotalDepth.util.XmlWrite.XmlStream attribute), 175
- __weakref__ (TotalDepth.util.archive.FileBase attribute), 176
- __weakref__ (TotalDepth.util.archive.FileMembers attribute), 176
- __weakref__ (TotalDepth.util.plot.FILMCfg.FilmCfg attribute), 190
- __weakref__ (TotalDepth.util.plot.FILMCfg.PhysFilmCfg attribute), 189
- __weakref__ (TotalDepth.util.plot.LogHeader.APIHeaderBase attribute), 194
- __weakref__ (TotalDepth.util.plot.PRESCfg.CurveCfg attribute), 209
- __weakref__ (TotalDepth.util.plot.PRESCfg.LineTransBase attribute), 208
- __weakref__ (TotalDepth.util.plot.PRESCfg.PresCfg attribute), 210
- __weakref__ (TotalDepth.util.plot.Plot.CurvePlotScaleSlotManager attribute), 197
- __weakref__ (TotalDepth.util.plot.Plot.Plot attribute), 201
- __weakref__ (TotalDepth.util.plot.Plot.PlotRoll attribute), 199
- __weakref__ (TotalDepth.util.plot.Track.Track attribute), 220
- __weakref__ (TotalDepth.util.plot.XGrid.XGrid attribute), 221
- __weakref__ (TotalDepth.util.plot.XMLCfgy.LgXMLBase attribute), 221
- ## A
- Absent Value, 223
- absent_value_from_rep_code() (in module TotalDepth.RP66V1.core.AbsentValue), 85
- AccActivity (class in TotalDepth.LIS.core.FrameSet), 38
- AccBias (class in TotalDepth.LIS.core.FrameSet), 37
- AccCount (class in TotalDepth.LIS.core.FrameSet), 37
- AccDec (class in TotalDepth.LIS.core.FrameSet), 37
- AccDelta (class in TotalDepth.LIS.core.FrameSet), 37
- AccDrift (class in TotalDepth.LIS.core.FrameSet), 38
- AccEq (class in TotalDepth.LIS.core.FrameSet), 37
- AccInc (class in TotalDepth.LIS.core.FrameSet), 37
- AccMax (class in TotalDepth.LIS.core.FrameSet), 36
- AccMean (class in TotalDepth.LIS.core.FrameSet), 36
- AccMin (class in TotalDepth.LIS.core.FrameSet), 36
- AccStDev (class in TotalDepth.LIS.core.FrameSet), 36
- accumulate() (TotalDepth.LIS.core.FrameSet.FrameSet method), 36
- add() (TotalDepth.common.Rle.RLE method), 154
- add() (TotalDepth.common.Rle.RLEItem method), 153
- add() (TotalDepth.common.statistics.LengthDict method), 156
- add() (TotalDepth.LIS.core.FileIndexer.IndexLogPass method), 29
- add() (TotalDepth.LIS.core.FrameSet.AccActivity method), 38
- add() (TotalDepth.LIS.core.FrameSet.AccBias method), 37
- add() (TotalDepth.LIS.core.FrameSet.AccCount method), 37
- add() (TotalDepth.LIS.core.FrameSet.AccDec method), 37
- add() (TotalDepth.LIS.core.FrameSet.AccDelta method), 37
- add() (TotalDepth.LIS.core.FrameSet.AccDrift method), 38
- add() (TotalDepth.LIS.core.FrameSet.AccEq method), 37
- add() (TotalDepth.LIS.core.FrameSet.AccInc method), 37
- add() (TotalDepth.LIS.core.FrameSet.AccMax method), 36
- add() (TotalDepth.LIS.core.FrameSet.AccMean method), 36

add() (*TotalDepth.LIS.core.FrameSet.AccMin method*), 36
 add() (*TotalDepth.LIS.core.FrameSet.AccStDev method*), 37
 add() (*TotalDepth.LIS.core.Rle.RLEItemType01 method*), 69
 add() (*TotalDepth.LIS.core.Rle.RLEType01 method*), 70
 add() (*TotalDepth.util.DictTree.DictTree method*), 161
 add() (*TotalDepth.util.DictTree.DictTreeHtmlTable method*), 164
 add() (*TotalDepth.util.Histogram.Histogram method*), 171
 add() (*TotalDepth.util.plot.FILMCfg.FilmCfg method*), 190
 add() (*TotalDepth.util.plot.PRESCfg.PresCfg method*), 209
 add_block() (*TotalDepth.BIT.ReadBIT.BITFrameArray method*), 12
 add_bytes() (*TotalDepth.RP66V1.core.pFile.FileLogicalData method*), 90
 add_eflr() (*TotalDepth.RP66V1.core.LogicalFile.LogicalFile method*), 96
 add_gnuplot_to_argument_parser() (*in module TotalDepth.util.gnuplot*), 178
 add_iflr() (*TotalDepth.RP66V1.core.LogicalFile.LogicalFile method*), 96
 add_index() (*TotalDepth.LIS.ToLAS.LisLogicalFile method*), 84
 add_log_level() (*in module TotalDepth.common.cmn_cmd_opts*), 139
 add_member_line() (*TotalDepth.LAS.core.LASRead.LASSection method*), 17
 add_member_line() (*TotalDepth.LAS.core.LASRead.LASSectionArray method*), 18
 add_message_to_queue() (*in module TotalDepth.common.process*), 152
 add_multiprocessing() (*in module TotalDepth.common.cmn_cmd_opts*), 140
 add_process_logger_to_argument_parser() (*in module TotalDepth.common.process*), 153
 add_timer() (*TotalDepth.util.ExecTimer.TimerList method*), 169
 add_work_done() (*TotalDepth.util.ExecTimer.Timer method*), 168
 addCb() (*TotalDepth.LIS.core.LogiRec.TableRow method*), 46
 addDatumBlock() (*TotalDepth.LIS.core.LogiRec.LrTable method*), 47
 addPlotResult() (*TotalDepth.LIS.PlotLogPasses.PlotLogInfo method*), 80
 addPlotResult() (*TotalDepth.PlotLogs.PlotLogInfo method*), 5
 addType01Data() (*TotalDepth.LIS.core.LogPass.LogPass method*), 53
 addXMLRoot() (*TotalDepth.util.plot.FILMCfgXML.FilmCfgXMLRead method*), 192
 allowable_set_types (*TotalDepth.RP66V1.core.LogicalRecord.EFLR.PublicEFLRType attribute*), 106
 allowable_set_types (*TotalDepth.RP66V1.core.LogicalRecord.Types.TypeDescriptionAll attribute*), 110
 analyse_archive() (*in module TotalDepth.util.archive*), 176
 analysis (*TotalDepth.RP66V1.util.IndexXMLRead.IndexResult attribute*), 119
 APIHeaderBase (*class in TotalDepth.util.plot.LogHeader*), 193
 APIHeaderLAS (*class in TotalDepth.util.plot.LogHeader*), 194
 APIHeaderLIS (*class in TotalDepth.util.plot.LogHeader*), 194
 append() (*TotalDepth.common.LogPass.FrameArray method*), 143
 append() (*TotalDepth.common.LogPass.LogPass method*), 144
 append() (*TotalDepth.RP66V1.core.XAxis.XAxis method*), 101
 arg_parser() (*in module TotalDepth.common.cmn_cmd_opts*), 139
 array_size (*TotalDepth.common.LogPass.FrameChannel property*), 142
 as_bytes() (*TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHeader method*), 87
 as_bytes() (*TotalDepth.RP66V1.core.pFile.StorageUnitLabel method*), 86
 as_bytes() (*TotalDepth.RP66V1.core.pFile.VisibleRecord method*), 87
 ascii (*TotalDepth.util.EBCDIC.EbcdicAsciiTableEntry attribute*), 167
 ascii_meaning (*TotalDepth.util.EBCDIC.EbcdicAsciiTableEntry attribute*), 167
 ASCII_PRINTABLE_BYTES (*in module TotalDepth.util.bin_file_type*), 177
 ascii_to_ebcdic() (*in module TotalDepth.util.EBCDIC*), 168
 ATTR_MAP (*TotalDepth.LIS.core.LogiRec.EntryBlockSet attribute*), 48
 Attribute (*class in TotalDepth.RP66V1.core.LogicalRecord.EFLR*), 105
 attribute_str() (*TotalDepth.LIS.core.PhysRec.PhysRecBase*

method), 60
 attribute_str_short() (TotalDepth.LIS.core.PhysRec.PhysRecBase method), 60
 AttributeBase (class in TotalDepth.RP66V1.core.LogicalRecord.EFLR), 105
 attributes (TotalDepth.RP66V1.core.pFile.LogicalDataDescription attribute), 88
 availableWidth (TotalDepth.util.plot.Plot.PlotRoll property), 199

B

Backup Mode, 223
 BACKUP_ALL (in module TotalDepth.util.plot.PRESCfg), 207
 BACKUP_FROM_COUNT_MAP (in module TotalDepth.util.plot.PRESCfgXML), 211
 BACKUP_FROM_MODE_MAP (in module TotalDepth.util.plot.PRESCfg), 207
 BACKUP_FROM_MODE_MAP (in module TotalDepth.util.plot.PRESCfgXML), 210
 BACKUP_LEFT (in module TotalDepth.util.plot.PRESCfg), 207
 BACKUP_NONE (in module TotalDepth.util.plot.PRESCfg), 207
 BACKUP_ONCE (in module TotalDepth.util.plot.PRESCfg), 207
 BACKUP_RIGHT (in module TotalDepth.util.plot.PRESCfg), 207
 BACKUP_TWICE (in module TotalDepth.util.plot.PRESCfg), 207
 BASE_UNITS (in module TotalDepth.util.plot.Coord), 179
 baseUnitsDim() (in module TotalDepth.util.plot.Coord), 182
 best_physical_record_pad_settings() (in module TotalDepth.LIS.core.File), 26
 binary_file_type() (in module TotalDepth.util.bin_file_type), 178
 BINARY_FILE_TYPE_CODE_WIDTH (in module TotalDepth.util.bin_file_type), 178
 binary_file_type_from_path() (in module TotalDepth.util.bin_file_type), 178
 BINARY_FILE_TYPES_SUPPORTED (in module TotalDepth.util.bin_file_type), 178
 bit_frame_array_to_las_file() (in module TotalDepth.BIT.ToLAS), 13
 BITFrameArray (class in TotalDepth.BIT.ReadBIT), 12
 BLOCKS_TO_SKIP (TotalDepth.LIS.core.LogiRec.EntryBlockSet attribute), 48
 bool() (TotalDepth.util.plot.XMLCfg.LgXMLBase method), 221
 Box (class in TotalDepth.util.plot.Coord), 181
 branch (TotalDepth.util.DictTree.DictTreeTableEvent attribute), 162
 bursts() (TotalDepth.LIS.core.LogiRec.DatumSpecBlock method), 49
 bytes_to_float() (in module TotalDepth.BIT.ReadBIT), 10

C

can_parse_file() (in module TotalDepth.DAT.DAT_parser), 138
 can_parse_path() (in module TotalDepth.DAT.DAT_parser), 138
 canAdd() (TotalDepth.LIS.core.FileIndexer.IndexLogPass method), 29
 canAdd() (TotalDepth.LIS.core.FileIndexer.IndexObjBase method), 27
 canFit() (TotalDepth.util.plot.Plot.CurvePlotScaleSlotMap method), 197
 canPlotFromExternalRecords() (TotalDepth.LIS.core.FileIndexer.PlotRecordSet method), 29
 canPlotFromInternalRecords() (TotalDepth.LIS.core.FileIndexer.PlotRecordSet method), 29
 category() (in module TotalDepth.LIS.core.Units), 76
 categoryDescription() (in module TotalDepth.LIS.core.Units), 76
 CB_TYPES (TotalDepth.LIS.core.LogiRec.CbEngVal attribute), 45
 CbEngVal (class in TotalDepth.LIS.core.LogiRec), 45
 CbEngValRead (class in TotalDepth.LIS.core.LogiRec), 45
 CbEngValWrite (class in TotalDepth.LIS.core.LogiRec), 46
 Channel (class in TotalDepth.common.lookup_mnemonic), 148
 channel (TotalDepth.common.lookup_mnemonic.Channel attribute), 148
 channel (TotalDepth.common.lookup_mnemonic.ChannelDescription attribute), 149
 ChannelDescription (class in TotalDepth.common.lookup_mnemonic), 149
 channelSize() (TotalDepth.LIS.core.Type01Plan.FrameSetPlan method), 74
 characteristic (TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor attribute), 102
 CharacteristicRepCodeGlobalDefault (class in TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor), 102
 characters() (TotalDepth.util.XmlWrite.XmlStream method), 175
 charactersWithBr() (TotalDepth.util.XmlWrite.XhtmlStream method), 175

- ChArTe (class in TotalDepth.LIS.core.FrameSet), 33
- check_against_osdd_units() (in module TotalDepth.RP66V1.util.XMLReadUnits), 136
- chkIdx() (in module TotalDepth.LIS.core.FrameSet), 32
- chOffset() (TotalDepth.LIS.core.Type01Plan.FrameSetPlan method), 74
- chOutpMnemInFilmId() (TotalDepth.util.plot.FILMCfgXML.FilmCfgXMLRead method), 192
- chunk() (TotalDepth.RP66V1.core.pFile.LogicalData method), 89
- close() (TotalDepth.LIS.core.File.FileWrite method), 25
- close() (TotalDepth.LIS.core.PhysRec.PhysRecBase method), 60
- close() (TotalDepth.LIS.core.PhysRec.PhysRecWrite method), 62
- close() (TotalDepth.LIS.core.RawStream.RawStream method), 63
- close() (TotalDepth.LIS.core.TifMarker.TifMarkerWrite method), 73
- code (TotalDepth.common.lookup_mnemonic.LoggingTool attribute), 150
- code (TotalDepth.common.lookup_mnemonic.Parameter attribute), 149
- code (TotalDepth.common.units.Unit attribute), 157
- code (TotalDepth.RP66V1.core.LogicalRecord.EFLR.PublicEFLRTyp attribute), 106
- coding (TotalDepth.util.plot.Stroke.Stroke attribute), 212
- col_span (TotalDepth.util.DictTree.DictTreeTableEvent attribute), 162
- collabels() (TotalDepth.LIS.core.LogiRec.LrTable method), 47
- coloStroke() (in module TotalDepth.util.plot.PRESCfg), 207
- colour (TotalDepth.util.plot.Stroke.Stroke attribute), 212
- comment() (TotalDepth.util.XmlWrite.XmlStream method), 175
- COMMENTS_IN_SVG_SECTION (in module TotalDepth.util.plot.Plot), 196
- COMMENTS_IN_SVG_SECTION_LEVEL_TUPLE (in module TotalDepth.util.plot.Plot), 196
- COMMENTS_IN_SVG_SECTION_WIDTH (in module TotalDepth.util.plot.Plot), 196
- COMMENTS_IN_SVG_TRACE (in module TotalDepth.util.plot.Plot), 196
- compare_next() (in module TotalDepth.DeTif), 3
- complete() (TotalDepth.BIT.ReadBIT.BITFrameArray method), 12
- COMPONENT_BLOCK_DATUM_BLOCK_ENTRY (in module TotalDepth.LIS.core.LogiRec), 45
- COMPONENT_BLOCK_DATUM_BLOCK_START (in module TotalDepth.LIS.core.LogiRec), 45
- COMPONENT_BLOCK_TABLE (in module TotalDepth.LIS.core.LogiRec), 45
- ComponentDescriptor (class in TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor), 103
- computeChecksum() (TotalDepth.LIS.core.PhysRec.PhysRecTail method), 61
- contFileName (TotalDepth.LIS.core.LogiRec.LrFileHeadTail property), 42
- convert() (in module TotalDepth.common.units), 158
- convert() (in module TotalDepth.LIS.core.Units), 77
- convert() (in module TotalDepth.RP66V1.core.Units), 100
- convert() (in module TotalDepth.util.plot.Coord), 180
- convert() (TotalDepth.LIS.core.EngVal.EngVal method), 23
- convert() (TotalDepth.LIS.core.Units.UnitConvert method), 76
- convert() (TotalDepth.LIS.core.Units.UnitConvertCategory method), 76
- convert() (TotalDepth.util.plot.Coord.Dim method), 180
- convert() (TotalDepth.util.plot.Coord.Pt method), 182
- convert_array() (in module TotalDepth.common.units), 159
- convert_array_inplace() (in module TotalDepth.common.units), 159
- convert_function() (in module TotalDepth.common.units), 158
- convert_function() (in module TotalDepth.RP66V1.core.Units), 100
- convertPt() (in module TotalDepth.util.plot.Coord), 183
- copy_files() (in module TotalDepth.util.CopyBinFiles), 161
- copy_tree() (in module TotalDepth.util.archive), 177
- count (TotalDepth.common.statistics.LengthDict property), 156
- count (TotalDepth.util.FileStatus.FileInfo property), 170
- count() (TotalDepth.common.Slice.Sample method), 155
- count() (TotalDepth.common.Slice.Slice method), 155
- count() (TotalDepth.common.Slice.SliceABC method), 154
- cpuTime (TotalDepth.LIS.LisToHtml.FileInfo attribute), 79
- create_bit_frame_array_from_file() (in module TotalDepth.BIT.ReadBIT), 12
- create_bit_frame_array_from_path() (in module TotalDepth.BIT.ReadBIT), 12
- create_gnuplot_dat() (in module TotalDepth.util.gnuplot), 178
- create_index() (TotalDepth.LAS.core.LASRead.LASSection method), 17
- create_index() (TotalDepth.LAS.core.LASRead.LASSectionArray

- method), 18
- create_rle() (in module *TotalDepth.common.Rle*), 154
- create_slice_or_sample() (in module *TotalDepth.common.Slice*), 156
- create_storage_unit_label() (in module *TotalDepth.RP66V1.core.pFile*), 86
- ctrl_char (*TotalDepth.util.EBCDIC.EbcdicAsciiTableEntry* attribute), 167
- ctrl_symbol (*TotalDepth.util.EBCDIC.EbcdicAsciiTableEntry* property), 167
- CURVE_LEGEND_FONT_FAMILY (TotalDepth.util.plot.Plot.Plot attribute), 200
- CURVE_LEGEND_FONT_SIZE (TotalDepth.util.plot.Plot.Plot attribute), 200
- curve_mnemonics() (TotalDepth.LAS.core.LASRead.LASBase method), 19
- curve_units_as_str() (TotalDepth.LAS.core.LASRead.LASBase method), 19
- CurveCfg (class in *TotalDepth.util.plot.PRESCfg*), 208
- CurveCfgLISRead (class in *TotalDepth.util.plot.PRESCfg*), 209
- CurveCfgXMLRead (class in *TotalDepth.util.plot.PRESCfgXML*), 211
- curveName (TotalDepth.util.plot.Plot.ScaleSliceCurve attribute), 197
- CurvePlotScale (class in *TotalDepth.util.plot.Plot*), 196
- CurvePlotScaleSlotMap (class in *TotalDepth.util.plot.Plot*), 197
- curves (TotalDepth.LIS.PlotLogPasses.IndexTableValue attribute), 80
- curves (TotalDepth.PlotLogs.IndexTableValue attribute), 5
- curveUnits() (TotalDepth.LIS.core.LogPass.LogPass method), 52
- curveUnitsAsStr() (TotalDepth.LIS.core.LogPass.LogPass method), 52
- ## D
- d (TotalDepth.util.plot.LogHeader.Static attribute), 193
- Data Format Specification Record, 223
- DataSeqBase (class in *TotalDepth.LIS.core.FrameSet*), 32
- DATETIME_NOW_FORMAT (in module *TotalDepth.common.process*), 151
- Datum Specification Block, 223
- DatumSpecBlock (class in *TotalDepth.LIS.core.LogiRec*), 49
- DatumSpecBlockRead (class in *TotalDepth.LIS.core.LogiRec*), 49
- de_tif_file() (in module *TotalDepth.DeTif*), 4
- DEBUG_LINE_BY_LINE (in module *TotalDepth.LAS.core.LASRead*), 16
- decimal_str (TotalDepth.util.EBCDIC.EbcdicAsciiTableEntry attribute), 167
- decodeString() (in module *TotalDepth.util.XmlWrite*), 174
- decompose_table_by_header_row() (in module *TotalDepth.common.lookup_mnemonic*), 147
- DEFAULT_HLINE_WIDTH_PX (in module *TotalDepth.util.plot.PRESCfg*), 206
- DEFAULT_INTERVAL_MAP (TotalDepth.util.plot.XGrid.XGrid attribute), 220
- DEFAULT_INTERVAL_TEXT (TotalDepth.util.plot.XGrid.XGrid attribute), 220
- DEFAULT_LLINE_WIDTH_PX (in module *TotalDepth.util.plot.PRESCfg*), 206
- DEFAULT_MODE (TotalDepth.util.plot.PRESCfg.CurveCfgLISRead attribute), 209
- DEFAULT_OPT_LOG_FORMAT (in module *TotalDepth.common.cmn_cmd_opts*), 139
- DEFAULT_OPT_LOG_FORMAT_VERBOSE (in module *TotalDepth.common.cmn_cmd_opts*), 139
- DEFAULT_OPT_LOG_LEVEL (in module *TotalDepth.common.cmn_cmd_opts*), 139
- DEFAULT_PLOT_LIS_UNITS (in module *TotalDepth.util.plot.PlotConstants*), 201
- DEFAULT_PLOT_UNITS (in module *TotalDepth.util.plot.PlotConstants*), 201
- DEFAULT_VALUE_FORMAT (in module *TotalDepth.util.plot.SVGWriter*), 212
- DEFAULT_VALUE_FORMAT_POINTS (in module *TotalDepth.util.plot.SVGWriter*), 212
- defining_origin (TotalDepth.RP66V1.core.LogicalFile.LogicalFile property), 96
- depth (TotalDepth.util.plot.Plot.PlotRoll property), 199
- depth() (TotalDepth.util.DictTree.DictTree method), 162
- depth_from (TotalDepth.BIT.ReadBIT.LogPassRange attribute), 11
- depth_from_branch() (TotalDepth.util.DictTree.DictTreeHtmlTable method), 164
- DEPTH_PER_CH (TotalDepth.util.plot.Track.Track attribute), 219
- depth_to (TotalDepth.BIT.ReadBIT.LogPassRange attribute), 11
- depthDim (TotalDepth.util.plot.Plot.PlotRoll property), 199
- desc (TotalDepth.LAS.core.LASRead.SectLine attribute), 17
- desc (TotalDepth.LIS.core.LogiRec.LrBase property), 41
- description (TotalDepth.common.lookup_mnemonic.Channel

- attribute), 148
- description (TotalDepth.common.lookup_mnemonic.ChannelDescription attribute), 149
- description (TotalDepth.common.lookup_mnemonic.LoggingTool attribute), 150
- description (TotalDepth.common.lookup_mnemonic.Parameter attribute), 149
- description (TotalDepth.common.lookup_mnemonic.ProductDescription attribute), 147
- description (TotalDepth.common.lookup_mnemonic.ToolDescription attribute), 148
- description (TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormatted attribute), 106
- description (TotalDepth.RP66V1.core.LogicalRecord.Types.TypeDescription attribute), 110
- description (TotalDepth.RP66V1.core.pFile.LRPosDesc attribute), 89
- description() (TotalDepth.util.plot.FILMCfgXML.FilmCfgXMLReader method), 192
- DeTifException, 3
- DeTifExceptionCompare, 3
- DeTifExceptionRead, 3
- DFSR, 223
- dfsr (TotalDepth.LIS.core.LogPass.LogPass property), 51
- DictTree (class in TotalDepth.util.DictTree), 161
- DictTreeHtmlTable (class in TotalDepth.util.DictTree), 162
- DictTreeTableEvent (class in TotalDepth.util.DictTree), 162
- Dim (class in TotalDepth.util.plot.Coord), 180
- dimension (TotalDepth.common.units.Unit attribute), 157
- dimensionless() (TotalDepth.LIS.core.EngVal.EngVal method), 22
- dimIn() (in module TotalDepth.util.plot.Coord), 181
- dimToTxt() (in module TotalDepth.util.plot.SVGWriter), 212
- dipmeterIndex() (TotalDepth.LIS.core.FrameSet.ChArTe method), 33
- dir_event_walk() (in module TotalDepth.util.DirWalk), 166
- dirWalk() (in module TotalDepth.util.DirWalk), 165
- discipline (TotalDepth.common.lookup_mnemonic.LoggingTool attribute), 150
- divide() (TotalDepth.util.plot.Coord.Dim method), 180
- DLIS, 223
- DLIS_TO_WELL_INFORMATION_LAS_EFLR_MAPPING (in module TotalDepth.RP66V1.ToLAS), 117
- DSB, 223
- DSCA_MAP (TotalDepth.util.plot.FILMCfg.PhysFilmCfgLISRead attribute), 189
- dumpFrameSets() (in module TotalDepth.LIS.DumpFrameSet), 78
- dumpToRP66V1_test_data() (in module TotalDepth.RP66V1.Scan), 115
- dumpFrames() (TotalDepth.LIS.core.FrameSet.FrameSet method), 34
- DUPE_DESCRIPTOR_CHANNEL_LOGGER() (TotalDepth.RP66V1.core.LogicalFile.LogicalFile method), 95
- DUPE_OBJECT_LOGGER() (TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormatted method), 107
- DUPE_OBJECT_STRATEGY (TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormatted attribute), 107
- allowableSetTypes
- ## E
- EB_DOC (TotalDepth.LIS.core.LogiRec.EntryBlockSet attribute), 48
- ebcdic (TotalDepth.util.EBCDIC.EbcdicAsciiTableEntry attribute), 167
- EBCDIC_ALL (in module TotalDepth.util.EBCDIC), 166
- ebcdic_all_printable() (in module TotalDepth.util.EBCDIC), 168
- ebcdic_ascii_description() (in module TotalDepth.util.EBCDIC), 168
- ebcdic_meaning (TotalDepth.util.EBCDIC.EbcdicAsciiTableEntry attribute), 167
- EBCDIC_PRINTABLE (in module TotalDepth.util.EBCDIC), 166
- ebcdic_printable (TotalDepth.util.EBCDIC.EbcdicAsciiTableEntry property), 167
- ebcdic_to_ascii() (in module TotalDepth.util.EBCDIC), 168
- EbcdicAsciiTableEntry (class in TotalDepth.util.EBCDIC), 167
- EFLR, 223
- eflr (TotalDepth.RP66V1.core.LogicalFile.PositionEFLR attribute), 95
- elapsed_perf_counter (TotalDepth.util.ExecTimer.Timer property), 168
- elapsed_wall_clock (TotalDepth.util.ExecTimer.Timer property), 168
- Element (class in TotalDepth.util.XmlWrite), 175
- encode() (TotalDepth.LIS.core.EngVal.EngValRc method), 23
- encodeString() (in module TotalDepth.util.XmlWrite), 174
- endElement() (TotalDepth.util.XmlWrite.XmlStream method), 175

Engineering Value, [223](#)
EngVal (class in TotalDepth.LIS.core.EngVal), [22](#)
EngValRc (class in TotalDepth.LIS.core.EngVal), [23](#)
Entry Block, [223](#)
EntryBlock (class in TotalDepth.LIS.core.LogiRec), [47](#)
EntryBlockRead (class in TotalDepth.LIS.core.LogiRec), [47](#)
EntryBlockSet (class in TotalDepth.LIS.core.LogiRec), [48](#)
eof (TotalDepth.LIS.core.TifMarker.TifMarkerBase property), [72](#)
error (TotalDepth.LIS.ScanPhysRec.PhysRecScanResult attribute), [83](#)
Event (class in TotalDepth.util.DirWalk), [165](#)
event (TotalDepth.util.DirWalk.FileEvent attribute), [165](#)
EVENT_EXTRAPOLATE (in module TotalDepth.LIS.core.LogPass), [51](#)
EVENT_EXTRAPOLATE (in module TotalDepth.LIS.core.Type01Plan), [73](#)
EVENT_READ (in module TotalDepth.LIS.core.LogPass), [50](#)
EVENT_READ (in module TotalDepth.LIS.core.Type01Plan), [73](#)
EVENT_SEEK_LR (in module TotalDepth.LIS.core.LogPass), [50](#)
EVENT_SKIP (in module TotalDepth.LIS.core.LogPass), [51](#)
EVENT_SKIP (in module TotalDepth.LIS.core.Type01Plan), [73](#)
evFirst (TotalDepth.LIS.PlotLogPasses.IndexTableValue attribute), [80](#)
evFirst (TotalDepth.PlotLogs.IndexTableValue attribute), [5](#)
evInterval (TotalDepth.PlotLogs.IndexTableValue attribute), [5](#)
evLast (TotalDepth.LIS.PlotLogPasses.IndexTableValue attribute), [80](#)
evLast (TotalDepth.PlotLogs.IndexTableValue attribute), [5](#)
exactConversion() (in module TotalDepth.util.plot.Coord), [180](#)
exception (TotalDepth.LIS.LisToHtml.FileInfo attribute), [79](#)
exception (TotalDepth.RP66V1.IndexPickle.IndexResult attribute), [110](#)
exception (TotalDepth.RP66V1.IndexXML.IndexResult attribute), [112](#)
exception (TotalDepth.RP66V1.LogRecIndex.IndexResult attribute), [113](#)
exception (TotalDepth.RP66V1.Scan.IndexResult attribute), [115](#)
exception (TotalDepth.RP66V1.util.IndexXMLRead.IndexResult attribute), [119](#)
ExceptionCbEngValInit, [45](#)
ExceptionCbWrite, [38](#)
ExceptionComponentDescriptor, [102](#)
ExceptionComponentDescriptorAccessError, [102](#)
ExceptionComponentDescriptorInit, [102](#)
ExceptionCoord, [179](#)
ExceptionCoordUnitConvert, [179](#)
ExceptionCurveCfg, [206](#)
ExceptionCurveCfgCtor, [206](#)
ExceptionCurveCfgLISRead, [206](#)
ExceptionCurveCfgXMLRead, [210](#)
ExceptionDAT, [138](#)
ExceptionDATRead, [138](#)
ExceptionDatumSpecBlock, [49](#)
ExceptionDictTree, [161](#)
ExceptionDictTreeHtmlTable, [161](#)
ExceptionDirWalk, [165](#)
ExceptionEFLR, [104](#)
ExceptionEFLRAttribute, [104](#)
ExceptionEFLRObject, [104](#)
ExceptionEFLRObjectDuplicateLabel, [104](#)
ExceptionEFLRSet, [104](#)
ExceptionEFLRSetDuplicateObjectNames, [104](#)
ExceptionEFLRTemplate, [104](#)
ExceptionEFLRTemplateDuplicateLabel, [104](#)
ExceptionEngVal, [22](#)
ExceptionEntryBlock, [47](#)
ExceptionEntryBlockSetInit, [47](#)
ExceptionEOF, [85](#)
ExceptionExecTimer, [168](#)
ExceptionFile, [24](#), [85](#)
ExceptionFileBuffer, [169](#)
ExceptionFileBufferEOF, [169](#)
ExceptionFileIndex, [27](#)
ExceptionFileRead, [24](#), [85](#)
ExceptionFileReadEOF, [85](#)
ExceptionFileReadPositionsInconsistent, [85](#)
ExceptionFileStatus, [170](#)
ExceptionFileWrite, [24](#)
ExceptionFILMCfg, [188](#)
ExceptionFilmCfgLISRead, [189](#)
ExceptionFILMCfgXML, [191](#)
ExceptionFILMCfgXMLRead, [191](#)
ExceptionFILMCfgXMLReadLookUp, [191](#)
ExceptionFrameArray, [93](#), [141](#)
ExceptionFrameArrayInit, [93](#)
ExceptionFrameChannel, [93](#), [141](#)
ExceptionFrameSet, [32](#)
ExceptionFrameSetEmpty, [32](#)
ExceptionFrameSetMixedChannels, [32](#)
ExceptionFrameSetNULLSpacing, [32](#)
ExceptionFrameSetPlan, [73](#)
ExceptionFrameSetPlanNegLen, [73](#)
ExceptionFrameSetPlanOverrun, [73](#)
ExceptionIFLR, [108](#)

ExceptionIndex, 91
 ExceptionIndexXML, 111
 ExceptionIndexXMLRead, 111
 ExceptionLASKeyError, 16
 ExceptionLASRead, 15
 ExceptionLASReadData, 16
 ExceptionLASReadSection, 15
 ExceptionLASReadSectionArray, 16
 ExceptionLineTransBase, 206
 ExceptionLineTransBaseMath, 206
 ExceptionLogHeader, 192
 ExceptionLogHeaderLAS, 192
 ExceptionLogHeaderLIS, 192
 ExceptionLogicalFile, 94
 ExceptionLogicalFileAdd, 94
 ExceptionLogicalFileCtor, 94
 ExceptionLogicalFileMissingData, 94
 ExceptionLogicalIndex, 94
 ExceptionLogicalIndexCtor, 94
 ExceptionLogicalRecordSegmentHeader, 85
 ExceptionLogicalRecordSegmentHeaderAttributes, 85
 ExceptionLogicalRecordSegmentHeaderEOF, 85
 ExceptionLogicalRecordSegmentHeaderSequence, 85
 ExceptionLogPass, 50, 93, 141
 ExceptionLogPassBase, 141
 ExceptionLogPassCtor, 50
 ExceptionLogPassInit, 93
 ExceptionLogPassKeyError, 50
 ExceptionLogPassNoFrameSet, 50
 ExceptionLogPassNoType01Data, 50
 ExceptionLogPassProcessIFLR, 93
 ExceptionLogPassXML, 111
 ExceptionLookupMnemonic, 147
 ExceptionLookupMnemonicReadTable, 147
 ExceptionLookupMnemonicReadURL, 147
 ExceptionLookupMnemonicTable, 147
 ExceptionLr, 38
 ExceptionLrNotImplemented, 38
 ExceptionLrTable, 45
 ExceptionLrTableCompose, 45
 ExceptionLrTableInit, 45
 ExceptionLrTableInternaStructuresCorrupt, 45
 ExceptionLrTableRow, 45
 ExceptionLrTableRowInit, 45
 ExceptionPhysFilmCfg, 189
 ExceptionPhysRec, 58
 ExceptionPhysRecEOF, 58
 ExceptionPhysRecTail, 58
 ExceptionPhysRecUndefinedChecksum, 58
 ExceptionPhysRecUnknownType, 58
 ExceptionPhysRecWrite, 58
 ExceptionPlotXGrid, 220
 ExceptionPRESCfg, 206
 ExceptionPresCfg, 206
 ExceptionPRESCfgLISRead, 206
 ExceptionPRESCfgXML, 210
 ExceptionPresCfgXMLRead, 210
 ExceptionProcLisPath, 81
 ExceptionRawStream, 62
 ExceptionRawStreamEOF, 62
 ExceptionRepCode, 66
 ExceptionRepCodeNoLength, 64
 ExceptionRepCodeRead, 64
 ExceptionRepCodeUnknown, 66
 ExceptionRepCodeWrite, 64
 ExceptionRP66V1IndexXMLRead, 111
 ExceptionRP66V1Units, 100
 ExceptionStorageUnitLabel, 85
 ExceptionSVGWriter, 212
 ExceptionTIF, 85
 ExceptionTIFEOF, 85
 ExceptionTifMarker, 72
 ExceptionTotalDepthBIT, 10
 ExceptionTotalDepthBIT_TIF, 10
 ExceptionTotalDepthBITDataBlocks, 10
 ExceptionTotalDepthBITFirstBlock, 10
 ExceptionTotalDepthLISPlot, 196
 ExceptionTotalDepthLISPlotTrack, 219
 ExceptionTotalDepthPlotRoll, 196
 ExceptionUnits, 75, 157
 ExceptionUnitsDimension, 157
 ExceptionUnitsLookup, 157
 ExceptionUnitsMismatchedCategory, 76
 ExceptionUnitsNoUnitInCategory, 76
 ExceptionUnitsUnknownCategory, 76
 ExceptionUnitsUnknownUnit, 76
 ExceptionVisibleRecord, 85
 ExceptionVisibleRecordEOF, 85
 ExceptionXml, 174
 ExceptionXMLCfg, 221
 ExceptionXMLCfgMissingElem, 221
 ExceptionXMLCfgNoContent, 221
 ExceptionXMLCfgWrongRootElem, 221
 ExceptionXmlEndElement, 174
 expand_and_delete_archives() (in module *TotalDepth.util.archive*), 177
 Explicitly Formatted Logical Record, 223
 ExplicitlyFormattedLogicalRecord (class in *TotalDepth.RP66V1.core.LogicalRecord.EFLR*), 107
 explore_tree_multi_process() (in module *TotalDepth.util.archive*), 177
 explore_tree_single_process() (in module *TotalDepth.util.archive*), 177
 extFrameNum() (*TotalDepth.LIS.core.FrameSet.FrameSet* method), 34

extract_json() (in module *TotalDepth.common.process*), 152
 extract_json_as_table() (in module *TotalDepth.common.process*), 152
 extract_labels_from_json() (in module *TotalDepth.common.process*), 152
 extract_well_information_from_origin() (in module *TotalDepth.RP66V1.ToLAS*), 118

F

FFLR, 223
 file_header_logical_record (TotalDepth.RP66V1.core.LogicalFile.LogicalFile property), 95
 file_name (TotalDepth.RP66V1.ScanHTML.FileNameLinkHref attribute), 116
 file_read_with_best_physical_record_pad_settings() (in module *TotalDepth.LIS.core.File*), 26
 FileArchive (class in *TotalDepth.util.archive*), 176
 FileBase (class in *TotalDepth.LIS.core.File*), 24
 FileBase (class in *TotalDepth.util.archive*), 176
 FileBuffer (class in *TotalDepth.util.FileBuffer*), 169
 fileCurveMap() (in module *TotalDepth.util.plot.XMLMatches*), 222
 fileCurveMapFromFILM() (in module *TotalDepth.util.plot.XMLMatches*), 222
 FileEvent (class in *TotalDepth.util.DirWalk*), 165
 FileIndex (class in *TotalDepth.LIS.core.FileIndexer*), 30
 FileInfo (class in *TotalDepth.LIS.LisToHtml*), 79
 FileInfo (class in *TotalDepth.util.FileStatus*), 170
 FileInfoSet (class in *TotalDepth.util.FileStatus*), 170
 FileInMemory (class in *TotalDepth.util.archive*), 176
 FileInOut (class in *TotalDepth.util.DirWalk*), 165
 FileLogicalData (class in *TotalDepth.RP66V1.core.pFile*), 90
 FileMembers (class in *TotalDepth.util.archive*), 176
 FileNameLinkHref (class in *TotalDepth.RP66V1.ScanHTML*), 116
 FileOnDisc (class in *TotalDepth.util.archive*), 176
 filePathIn (TotalDepth.util.DirWalk.FileInOut attribute), 165
 filePathOut (TotalDepth.util.DirWalk.FileInOut attribute), 165
 FileRead (class in *TotalDepth.LIS.core.File*), 24
 FileRead (class in *TotalDepth.RP66V1.core.pFile*), 90
 FileResult (in module *TotalDepth.RP66V1.util.XMLReadUnits*), 136
 FileSizeTime (class in *TotalDepth.BIT.ReadBIT*), 12
 FileWrite (class in *TotalDepth.LIS.core.File*), 25
 FileZip (class in *TotalDepth.util.archive*), 176
 FilmCfg (class in *TotalDepth.util.plot.FILMCfg*), 190
 FilmCfgLISRead (class in *TotalDepth.util.plot.FILMCfg*), 190
 FilmCfgXMLRead (class in *TotalDepth.util.plot.FILMCfgXML*), 192
 filmIdS() (TotalDepth.util.plot.Plot.Plot method), 200
 finalise() (TotalDepth.LAS.core.LASRead.LASSection method), 17
 finalise() (TotalDepth.LAS.core.LASRead.LASSectionArray method), 18
 find() (TotalDepth.LAS.core.LASRead.LASSection method), 17
 first() (TotalDepth.common.Rle.RLE method), 154
 first() (TotalDepth.common.Slice.Sample method), 155
 first() (TotalDepth.common.Slice.Slice method), 155
 first() (TotalDepth.common.Slice.SliceABC method), 154
 fit() (TotalDepth.util.plot.Plot.CurvePlotScaleSlotMap method), 197
 Fixed Format Logical Record, 223
 float() (TotalDepth.util.plot.XMLCfg.LgXMLBase method), 221
 float_to_bytes() (in module *TotalDepth.BIT.ReadBIT*), 10
 font (TotalDepth.util.plot.LogHeader.Static attribute), 193
 FONT_PROP (in module *TotalDepth.util.plot.LogHeader*), 193
 format_bytes() (in module *TotalDepth.util.bin_file_type*), 178
 format_object() (in module *TotalDepth.common.data_table*), 141
 format_table() (in module *TotalDepth.common.data_table*), 141
 format_table_columns() (in module *TotalDepth.common.data_table*), 141
 Frame, 223
 Frame Array, 224
 Frame Set, 223
 frame() (TotalDepth.LIS.core.FrameSet.FrameSet method), 34
 frame_array_from_RP66V1() (in module *TotalDepth.RP66V1.core.LogPass*), 94
 frame_array_to_XML() (in module *TotalDepth.RP66V1.IndexXML*), 111
 frame_channel_from_RP66V1() (in module *TotalDepth.RP66V1.core.LogPass*), 94
 frame_channel_sub_channel_values() (TotalDepth.LIS.core.FrameSet.FrameSet method), 35
 frame_channel_to_XML() (in module *TotalDepth.RP66V1.IndexXML*), 111
 frame_number (TotalDepth.RP66V1.core.XAxis.IFLRReference attribute), 101
 frame_size() (TotalDepth.LAS.core.LASRead.LASSectionArray method), 18

- FrameArray (class in *TotalDepth.common.LogPass*), 142
 FrameChannel (class in *TotalDepth.common.LogPass*), 141
 frameFromX() (*TotalDepth.LIS.core.LogPass.LogPass* method), 52
 frames (*TotalDepth.LIS.core.FrameSet.FrameSet* property), 34
 FrameSet (class in *TotalDepth.LIS.core.FrameSet*), 33
 frameSet (*TotalDepth.LIS.core.LogPass.LogPass* property), 51
 frameSetLongStr() (*TotalDepth.LIS.core.LogPass.LogPass* method), 51
 FrameSetPlan (class in *TotalDepth.LIS.core.Type01Plan*), 73
 frameSize (*TotalDepth.LIS.core.Type01Plan.FrameSetPlan* property), 74
 frameSpacing() (*TotalDepth.LIS.core.Rle.RLEType01* method), 70
 frameView() (*TotalDepth.LIS.core.FrameSet.FrameSet* method), 35
 FREQUENTLY_USED_ATTRIBUTES (in module *TotalDepth.RP66V1.core.LogicalRecord.Semantics*), 109
 from49() (in module *TotalDepth.LIS.core.pRepCode*), 66
 from50() (in module *TotalDepth.LIS.core.cRepCode*), 68
 from50() (in module *TotalDepth.LIS.core.pRepCode*), 66
 from56() (in module *TotalDepth.LIS.core.cRepCode*), 68
 from56() (in module *TotalDepth.LIS.core.pRepCode*), 66
 from66() (in module *TotalDepth.LIS.core.cRepCode*), 68
 from66() (in module *TotalDepth.LIS.core.pRepCode*), 67
 from68() (in module *TotalDepth.LIS.core.cRepCode*), 68
 from68() (in module *TotalDepth.LIS.core.pRepCode*), 67
 from70() (in module *TotalDepth.LIS.core.pRepCode*), 67
 from73() (in module *TotalDepth.LIS.core.cRepCode*), 68
 from73() (in module *TotalDepth.LIS.core.pRepCode*), 67
 from77() (in module *TotalDepth.LIS.core.cRepCode*), 68
 from77() (in module *TotalDepth.LIS.core.pRepCode*), 67
 from79() (in module *TotalDepth.LIS.core.cRepCode*), 68
 from79() (in module *TotalDepth.LIS.core.pRepCode*), 67
 FUNCTION_ID_MAP (in module *TotalDepth.util.bin_file_type*), 177
G
 gen_big_first() (in module *TotalDepth.util.DirWalk*), 165
 gen_floats() (in module *TotalDepth.BIT.ReadBIT*), 10
 gen_indices() (*TotalDepth.common.Slice.Sample* method), 156
 gen_indices() (*TotalDepth.common.Slice.Slice* method), 155
 gen_indices() (*TotalDepth.common.Slice.SliceABC* method), 154
 gen_mnemonic_units() (*TotalDepth.LIS.core.LogPass.LogPass* method), 53
 gen_row_column_events() (*TotalDepth.util.DictTree.DictTreeHtmlTable* method), 164
 gen_row_column_events_from_branch() (*TotalDepth.util.DictTree.DictTreeHtmlTable* method), 164
 genAll() (*TotalDepth.LIS.core.FileIndexer.FileIndex* method), 30
 genAll() (*TotalDepth.LIS.core.FrameSet.FrameSet* method), 36
 genCells() (*TotalDepth.LIS.core.LogiRec.TableRow* method), 46
 genChScPoints() (*TotalDepth.LIS.core.FrameSet.FrameSet* method), 35
 genChScValues() (*TotalDepth.LIS.core.FrameSet.FrameSet* method), 35
 generate_file_paths() (in module *TotalDepth.LIS.Index*), 79
 generate_lines() (in module *TotalDepth.LAS.core.LASRead*), 16
 generate_sections() (*TotalDepth.LAS.core.LASRead.LASBase* method), 18
 genEvents() (*TotalDepth.LIS.core.Type01Plan.FrameSetPlan* method), 74
 genEventsUnits() (*TotalDepth.util.plot.XGrid.XGrid* method), 221
 genExtChIndexes() (*TotalDepth.LIS.core.FrameSet.FrameSet* method), 35
 genFrameSetChIndexScNameUnit() (*TotalDepth.LIS.core.LogPass.LogPass* method), 53
 genFrameSetHeadings() (*TotalDepth.LIS.core.LogPass.LogPass* method), 52
 genFrameSetScNameUnit() (*TotalDepth.LIS.core.LogPass.LogPass* method), 52
 genLd() (*TotalDepth.LIS.core.PhysRec.PhysRecRead* method), 61

genLinear10() (in module TotalDepth.util.plot.Track), 219
 genLisBytes() (TotalDepth.LIS.core.LogiRec.LrTable method), 47
 genLog10() (in module TotalDepth.util.plot.Track), 219
 genLog10Decade1Start2() (in module TotalDepth.util.plot.Track), 219
 genLog10Decade2() (in module TotalDepth.util.plot.Track), 219
 genLog10Decade2Start2() (in module TotalDepth.util.plot.Track), 219
 genLog10Decade3() (in module TotalDepth.util.plot.Track), 219
 genLog10Decade4() (in module TotalDepth.util.plot.Track), 219
 genLog10Decade5() (in module TotalDepth.util.plot.Track), 219
 genLogPasses() (TotalDepth.LIS.core.FileIndexer.FileIndex method), 30
 genMultipleChScPoints() (TotalDepth.LIS.core.FrameSet.FrameSet method), 36
 genOffsets() (TotalDepth.LIS.core.Type01Plan.FrameSetPlan method), 74
 genOutpPoints() (TotalDepth.LIS.core.LogPass.LogPass method), 53
 genPlotRecords() (TotalDepth.LIS.core.FileIndexer.FileIndex method), 30
 genPr() (TotalDepth.LIS.core.PhysRec.PhysRecRead method), 61
 genRowNames() (TotalDepth.LIS.core.LogiRec.LrTable method), 46
 genRows() (TotalDepth.LIS.core.LogiRec.LrTable method), 46
 genRowValuesInColOrder() (TotalDepth.LIS.core.LogiRec.LrTable method), 47
 genScaleSliceCurve() (TotalDepth.util.plot.Plot.CurvePlotScaleSlotMap method), 197
 genTracks() (TotalDepth.util.plot.FILMCfg.PhysFilmCfg method), 189
 genXAxisRange() (TotalDepth.util.plot.XGrid.XGrid method), 220
 genXAxisTextRange() (TotalDepth.util.plot.XGrid.XGrid method), 221
 genXPosStroke() (TotalDepth.util.plot.XGrid.XGrid method), 220
 get_all_wsd_mnemonics() (TotalDepth.LAS.core.LASRead.LASBase method), 19
 get_errors() (in module TotalDepth.DeTif), 4
 get_file_logical_data() (TotalDepth.RP66V1.core.pFile.FileRead method), 91
 get_file_logical_data() (TotalDepth.RP66V1.core.pIndex.LogicalRecordIndex method), 92
 get_file_logical_data_at_position() (TotalDepth.RP66V1.core.pIndex.LogicalRecordIndex method), 92
 get_wsd_mnemonic() (TotalDepth.LAS.core.LASRead.LASBase method), 19
 getInUnits() (TotalDepth.LIS.core.EngVal.EngVal method), 23
 global_default (TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor attribute), 102
 GNUPLOT_PLT (in module TotalDepth.common.process), 151

H

halfTracks (TotalDepth.util.plot.PRESCfg.TrackWidthData attribute), 208
 halfTrackStart (TotalDepth.util.plot.PRESCfg.TrackWidthData attribute), 208
 has() (TotalDepth.common.LogPass.LogPass method), 143
 has_active_timer (TotalDepth.util.ExecTimer.TimerList property), 169
 has_attribute_C (TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.ComponentDescriptor property), 104
 has_attribute_L (TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.ComponentDescriptor property), 104
 has_attribute_R (TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.ComponentDescriptor property), 104
 has_attribute_U (TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.ComponentDescriptor property), 104
 has_attribute_V (TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.ComponentDescriptor property), 104
 has_las_extension() (in module TotalDepth.LAS.core.LASRead), 16
 has_log_pass (TotalDepth.RP66V1.core.LogicalFile.LogicalFile property), 96
 has_object_N (TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.ComponentDescriptor property), 104
 has_offset() (TotalDepth.common.units.Unit method), 158
 has_output_mnemonic() (TotalDepth.LAS.core.LASRead.LASBase method), 19

`talDepth.LAS.core.LASRead.LASBase`
`method)`, 19
`has_set_N` (`TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor` property), 103
`has_set_T` (`TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor` property), 103
`has_slb_standard_form` (in module `TotalDepth.common.units`), 158
`has_slb_units` (in module `TotalDepth.common.units`), 158
`has_tif` (in module `TotalDepth.DeTif`), 3
`has_tif_file` (in module `TotalDepth.DeTif`), 3
`hasCurvesForDest` (`TotalDepth.util.plot.PRESCfg.PresCfg` method), 209
`hasDataToPlotLAS` (`TotalDepth.util.plot.Plot.Plot` method), 201
`hasDataToPlotLIS` (`TotalDepth.util.plot.Plot.Plot` method), 200
`hasGrid` (`TotalDepth.util.plot.Track.Track` property), 220
`hasLd` (`TotalDepth.LIS.core.File.FileRead` method), 25
`hasLd` (`TotalDepth.LIS.core.PhysRec.PhysRecRead` method), 61
`hasOutpMnem` (`TotalDepth.LIS.core.LogPass.LogPass` method), 52
`hasPrevious` (`TotalDepth.LIS.core.TifMarker.TifMarkerRead` property), 73
`hasTail` (`TotalDepth.LIS.core.PhysRec.PhysRecTail` method), 61
`hasUnit` (in module `TotalDepth.LIS.core.Units`), 76
`hasUnitCategory` (in module `TotalDepth.LIS.core.Units`), 76
`hasXaxisData` (`TotalDepth.LIS.core.Rle.RLEType01` property), 70
`header_as_strings` (`TotalDepth.RP66V1.core.LogicalRecord.EFLR.Template` method), 106
`HEADER_PLOT_UNITS` (in module `TotalDepth.util.plot.LogHeader`), 193
`hex_str` (`TotalDepth.util.EBCDIC.EbcdicAsciiTableEntry` attribute), 167
`Histogram` (class in `TotalDepth.util.Histogram`), 171
`histogram_power_of_2` (`TotalDepth.common.statistics.LengthDict` method), 156
`href` (`TotalDepth.RP66V1.ScanHTML.FileNameLinkHref` attribute), 116
`html_scan_RP66V1_file_data_content` (in module `TotalDepth.RP66V1.ScanHTML`), 116
`html_write_body` (in module `TotalDepth.RP66V1.ScanHTML`), 116
`html_write_table_of_contents` (in module `TotalDepth.RP66V1.ScanHTML`), 116
`id` (`TotalDepth.util.XmlWrite.XmlStream` property), 175
`ident` (`TotalDepth.common.LogPass.FrameChannel` property), 142
`ident` (`TotalDepth.RP66V1.core.LogPass.RP66V1FrameChannel` property), 93
`IFLR`, 224
`IFLRReference` (class in `TotalDepth.RP66V1.core.XAxis`), 101
`iflrType` (`TotalDepth.LIS.core.LogPass.LogPass` property), 51
`iflrType` (`TotalDepth.LIS.core.FileIndexer.IndexLogPass` method), 29
`iflrType` (`TotalDepth.LIS.core.FileIndexer.IndexObjBase` method), 27
`ignored` (`TotalDepth.RP66V1.IndexPickle.IndexResult` attribute), 110
`ignored` (`TotalDepth.RP66V1.IndexXML.IndexResult` attribute), 112
`ignored` (`TotalDepth.RP66V1.LogRecIndex.IndexResult` attribute), 113
`ignored` (`TotalDepth.RP66V1.Scan.IndexResult` attribute), 115
`ignored` (`TotalDepth.RP66V1.util.IndexXMLRead.IndexResult` attribute), 119
`indented_string` (`TotalDepth.util.DictTree.DictTree` method), 162
`index` (`TotalDepth.LIS.core.FrameSet.ChArTe` method), 33
`index` (`TotalDepth.LIS.core.FrameSet.SuChArTe` method), 33
`index_a_single_file` (in module `TotalDepth.RP66V1.LogRecIndex`), 114
`index_dir_multiprocessing` (in module `TotalDepth.RP66V1.IndexPickle`), 111
`index_dir_multiprocessing` (in module `TotalDepth.RP66V1.IndexXML`), 113
`index_dir_multiprocessing` (in module `TotalDepth.RP66V1.LogRecIndex`), 113
`index_dir_or_file` (in module `TotalDepth.RP66V1.LogRecIndex`), 114
`index_dir_single_process` (in module `TotalDepth.LIS.Index`), 79
`index_time` (`TotalDepth.RP66V1.LogRecIndex.IndexResult` attribute), 113
`IndexFileHead` (class in `TotalDepth.LIS.core.FileIndexer`), 28
`IndexFileHeadTail` (class in `TotalDepth.LIS.core.FileIndexer`), 28
`IndexFileTail` (class in `TotalDepth.LIS.core.FileIndexer`), 28
`IndexLogPass` (class in `TotalDepth.LIS.core.FileIndexer`), 29
`IndexLrFull` (class in `TotalDepth.LIS.core.FileIndexer`),

28					
IndexNone	(class in TotalDepth.LIS.core.FileIndexer),			init_x_axis_array()	(TotalDepth.RP66V1.core.LogPass.RP66V1FrameArray method), 94
27				int()	(TotalDepth.util.plot.XMLCfg.LgXMLBase method), 221
IndexObjBase	(class in TotalDepth.LIS.core.FileIndexer),	27		internalChIdx()	(TotalDepth.LIS.core.FrameSet.FrameSet method), 35
IndexReelHead	(class in TotalDepth.LIS.core.FileIndexer),	28		interpretTrac()	(TotalDepth.util.plot.FILMCfg.FilmCfg method), 190
IndexReelHeadTail	(class in TotalDepth.LIS.core.FileIndexer),	28		interpretTrac()	(TotalDepth.util.plot.FILMCfg.PhysFilmCfg method), 189
IndexReelTail	(class in TotalDepth.LIS.core.FileIndexer),	28		interpretTrac()	(TotalDepth.util.plot.FILMCfgXML.PhysFilmCfgXMLRead method), 191
IndexResult	(class in TotalDepth.RP66V1.IndexPickle),	110		intFrameNum()	(TotalDepth.LIS.core.FrameSet.FrameSet method), 34
IndexResult	(class in TotalDepth.RP66V1.IndexXML),	112		invoke_gnuplot()	(in module TotalDepth.common.process), 152
IndexResult	(class in TotalDepth.RP66V1.LogRecIndex),	113		invoke_gnuplot()	(in module TotalDepth.util.gnuplot), 178
IndexResult	(class in TotalDepth.RP66V1.Scan),	115		is_absent_attribute	(TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.ComponentDescriptor property), 103
IndexResult	(class in TotalDepth.RP66V1.util.IndexXMLRead),	119		is_attribute	(TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.ComponentDescriptor property), 103
IndexTable	(class in TotalDepth.LIS.core.FileIndexer),	29		is_attribute_group	(TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.ComponentDescriptor property), 103
IndexTableValue	(class in TotalDepth.LIS.PlotLogPasses),	80		is_bit_file()	(in module TotalDepth.BIT.ReadBIT), 10
IndexTableValue	(class in TotalDepth.PlotLogs),	5		is_bit_file_from_path()	(in module TotalDepth.BIT.ReadBIT), 10
IndexTapeHead	(class in TotalDepth.LIS.core.FileIndexer),	28		is_ctrl	(TotalDepth.util.EBCDIC.EbcdicAsciiTableEntry property), 167
IndexTapeHeadTail	(class in TotalDepth.LIS.core.FileIndexer),	28		is_end()	(TotalDepth.LIS.ToLAS.LisLogicalFile method), 84
IndexTapeTail	(class in TotalDepth.LIS.core.FileIndexer),	28		is_invariant_attribute	(TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.ComponentDescriptor property), 103
IndexUnknownInternalFormat	(class in TotalDepth.LIS.core.FileIndexer),	27		is_key_value()	(TotalDepth.RP66V1.core.LogicalRecord.EFLR.Explicit property), 108
indices()	(TotalDepth.common.Slice.Sample method),	156		is_lis_file_type()	(in module TotalDepth.util.bin_file_type), 178
indices()	(TotalDepth.common.Slice.Slice method),	155		is_log_down()	(TotalDepth.LAS.core.LASRead.LASBase method), 19
indices()	(TotalDepth.common.Slice.SliceABC method),	154		is_next()	(TotalDepth.RP66V1.core.LogicalFile.LogicalFile static method), 96
Indirectly Formatted Logical Record,	224			is_object	(TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.ComponentDescriptor property), 103
IndirectlyFormattedLogicalRecord	(class in TotalDepth.RP66V1.core.LogicalRecord.IFLR),	108		is_primary	(TotalDepth.common.units.Unit property), 158
indirectSize	(TotalDepth.LIS.core.Type01Plan.FrameSetPlan property),	74			
init()	(TotalDepth.LIS.core.LogiRec.LrBase method),	41			
init_array()	(TotalDepth.common.LogPass.FrameChannel method),	142			
init_arrays()	(TotalDepth.common.LogPass.FrameArray method),	143			
init_arrays_partial()	(TotalDepth.common.LogPass.FrameArray method),	143			

is_redundant_set (TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor (TotalDepth.RP66V1.core.pFile.FileRead method), 90
 property), 103
is_replacement_set (TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor (TotalDepth.RP66V1.core.pFile.FileRead method), 90
 property), 103
is_row_close() (TotalDepth.util.DictTree.DictTreeHtmlTable (TotalDepth.common.process.ProcessLoggingThread method), 164
 method), 164
is_row_open() (TotalDepth.util.DictTree.DictTreeHtmlTable (TotalDepth.LIS.core.FileIndexer.FileIndex method), 164
 method), 164
is_sealed() (TotalDepth.RP66V1.core.pFile.FileLogicalDictionary (TotalDepth.LIS.core.FileIndexer.IndexLogPass method), 90
 method), 29
is_set (TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor (TotalDepth.LIS.core.FileIndexer.IndexObjBase method), 103
 property), 103
is_set_group (TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor (TotalDepth.LIS.core.FileIndexer.IndexTable method), 103
 property), 103
isDelimiter (TotalDepth.LIS.core.FileIndexer.IndexObjBase (TotalDepth.LIS.core.LogPass.LogPass method), 27
 property), 27
isDelimiter() (in module TotalDepth.LIS.core.LogiRec), 40
isEOF (TotalDepth.LIS.core.File.FileRead property), 25
isIndirectX (TotalDepth.LIS.core.FrameSet.FrameSet property), 35
isIndirectX (TotalDepth.LIS.core.FrameSet.XAxisDecl property), 33
isIndirectX (TotalDepth.LIS.core.LogPass.LogPass property), 51
isInt() (in module TotalDepth.LIS.core.pRepCode), 66
isLogDown (TotalDepth.LIS.core.FrameSet.XAxisDecl property), 33
isLogUp (TotalDepth.LIS.core.FrameSet.XAxisDecl property), 33
isNull (TotalDepth.LIS.core.LogiRec.DatumSpecBlock property), 49
isOffScaleLeft() (TotalDepth.util.plot.PRESCfg.LineTransBase method), 207
isOffScaleRight() (TotalDepth.util.plot.PRESCfg.LineTransBase method), 208
isSingleParam (TotalDepth.LIS.core.LogiRec.LrTable property), 46
items() (TotalDepth.util.DictTree.DictTree method), 162
iter_logical_record_positions() (TotalDepth.RP66V1.core.pFile.FileRead method), 91
iter_logical_records() (TotalDepth.RP66V1.core.pFile.FileRead method), 91
iter_LRSHs_for_visible_record() (TotalDepth.RP66V1.core.pFile.FileRead method), 90
iter_LRSHs_for_visible_record_and_logical_data_fragment() (TotalDepth.RP66V1.core.pFile.FileRead method), 90

K
KEY_ELAPSED_TIME (in module TotalDepth.common.process), 151
KEY_LABEL (in module TotalDepth.common.process), 151
KEY_PROCESS_ID (in module TotalDepth.common.process), 151
KEY_TIMESTAMP (in module TotalDepth.common.process), 151
key_values() (TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyF method), 108
keys() (TotalDepth.common.LogPass.LogPass method), 143
keys() (TotalDepth.common.statistics.LengthDict method), 156
keys() (TotalDepth.LAS.core.LASRead.LASSection method), 17
keys() (TotalDepth.util.DictTree.DictTree method), 161
keys() (TotalDepth.util.plot.FILMCfg.FilmCfg method), 190
keys() (TotalDepth.util.plot.PRESCfg.PresCfg method), 209

L
L2P() (TotalDepth.util.plot.PRESCfg.LineTransBase method), 207
L2P() (TotalDepth.util.plot.PRESCfg.LineTransLin method), 208
L2P() (TotalDepth.util.plot.PRESCfg.LineTransLog10 method), 208
largest_le() (TotalDepth.common.Rle.RLE method), 154
LAS, 224
LAS_BINARY_FILE_TYPES (in module TotalDepth.util.bin_file_type), 177

LAS_FILE_EXT (in module <i>TotalDepth.LAS.core.LASRead</i>), 16	LineTransLog10 (class in <i>TotalDepth.util.plot.PRESCfg</i>), 208
LAS_FILE_EXT_LOWER (in module <i>TotalDepth.LAS.core.LASRead</i>), 16	link_text (<i>TotalDepth.RP66V1.ScanHTML.FileNameLinkHref</i> attribute), 116
las_file_name() (in module <i>TotalDepth.RP66V1.ToLAS</i>), 119	LIS, 224
LASBase (class in <i>TotalDepth.LAS.core.LASRead</i>), 18	LIS_BINARY_FILE_TYPES (in module <i>TotalDepth.util.bin_file_type</i>), 178
LASRead (class in <i>TotalDepth.LAS.core.LASRead</i>), 19	LIS_CODI_MAP (in module <i>TotalDepth.util.plot.PRESCfg</i>), 206
LASSection (class in <i>TotalDepth.LAS.core.LASRead</i>), 17	LIS_COLO_MAP (in module <i>TotalDepth.util.plot.PRESCfg</i>), 206
LASSectionArray (class in <i>TotalDepth.LAS.core.LASRead</i>), 18	LIS_MNEM_TO_LAS_MNEM (in <i>TotalDepth.util.plot.LogHeader.APIHeaderLAS</i> attribute), 194
last() (<i>TotalDepth.common.Rle.RLE</i> method), 154	LIS-79, 224
last() (<i>TotalDepth.common.Rle.RLEItem</i> method), 153	LIS-A, 224
last() (<i>TotalDepth.common.Slice.Sample</i> method), 155	lisByteList() (<i>TotalDepth.LIS.core.LogiRec.EntryBlockSet</i> method), 49
last() (<i>TotalDepth.common.Slice.Slice</i> method), 155	lisBytes() (<i>TotalDepth.LIS.core.LogiRec.CbEngVal</i> method), 45
last() (<i>TotalDepth.common.Slice.SliceABC</i> method), 154	lisBytes() (<i>TotalDepth.LIS.core.LogiRec.EntryBlock</i> method), 47
ld_length (<i>TotalDepth.RP66V1.core.pFile.LogicalDataDeletion</i> attribute), 88	lisBytes() (<i>TotalDepth.LIS.core.LogiRec.EntryBlockSet</i> method), 49
ldIndex() (<i>TotalDepth.LIS.core.File.FileRead</i> method), 24	lisColo() (in module <i>TotalDepth.util.plot.PRESCfg</i>), 207
ldRemaingInPr() (<i>TotalDepth.LIS.core.PhysRec.PhysRecRead</i> method), 61	LisLogicalFile (class in <i>TotalDepth.LIS.ToLAS</i>), 84
left (<i>TotalDepth.util.plot.Track.Track</i> property), 220	lisSize (<i>TotalDepth.LIS.core.FrameSet.ChArTe</i> property), 33
leftL (<i>TotalDepth.util.plot.PRESCfg.LineTransBase</i> property), 207	lisSize (<i>TotalDepth.LIS.core.FrameSet.FrameSet</i> property), 34
leftP (<i>TotalDepth.util.plot.PRESCfg.TrackWidthData</i> attribute), 208	lisSize (<i>TotalDepth.LIS.LisToHtml.FileInfo</i> attribute), 79
LEGEND_ARROW_DEPTH (<i>TotalDepth.util.plot.Plot.Plot</i> attribute), 200	lisSize() (in module <i>TotalDepth.LIS.core.pRepCode</i>), 66
LEGEND_ARROW_DISPLAY (<i>TotalDepth.util.plot.Plot.Plot</i> attribute), 200	lisSize() (<i>TotalDepth.LIS.core.LogiRec.EntryBlockSet</i> method), 48
LEGEND_ARROW_WIDTH (<i>TotalDepth.util.plot.Plot.Plot</i> attribute), 200	LisToHtml (class in <i>TotalDepth.LIS.LisToHtml</i>), 79
LEGEND_ARROW_WIDTH_PX (<i>TotalDepth.util.plot.Plot.Plot</i> attribute), 200	literal() (<i>TotalDepth.util.XmlWrite.XmlStream</i> method), 175
LEGEND_DEPTH_PER_CURVE (<i>TotalDepth.util.plot.Plot.Plot</i> attribute), 200	Log Pass, 224
LEGEND_DEPTH_SPARE (<i>TotalDepth.util.plot.Plot.Plot</i> attribute), 200	log_pass_from_RP66V1() (in module <i>TotalDepth.RP66V1.core.LogPass</i>), 94
LEGEND_HORIZONTAL_LINE_DEPTH_PROPORTION (<i>TotalDepth.util.plot.Plot.Plot</i> attribute), 200	log_pass_to_XML() (in module <i>TotalDepth.RP66V1.IndexXML</i>), 112
len_input_bytes (<i>TotalDepth.RP66V1.core.LogPass.RP66V1FrameChannel</i> property), 93	log_process() (in module <i>TotalDepth.common.process</i>), 153
LEN_MNEM (in module <i>TotalDepth.LIS.core.Mnem</i>), 57	logDown (<i>TotalDepth.LIS.core.LogiRec.EntryBlockSet</i> property), 48
LengthDict (class in <i>TotalDepth.common.statistics</i>), 156	LOGGER_PREFIX (in module <i>TotalDepth.common.process</i>), 151
LgXMLBase (class in <i>TotalDepth.util.plot.XMLCf</i>), 221	LoggingTool (class in <i>TotalDepth.common.lookup_mnemonic</i>), 150
LineTransBase (class in <i>TotalDepth.util.plot.PRESCfg</i>), 207	Logical Record, 224
LineTransLin (class in <i>TotalDepth.util.plot.PRESCfg</i>), 208	

logical_data_length	(TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHeader property), 88	LR_FIXED_FORMAT (in module TotalDepth.LIS.core.LogiRec), 40	
logical_data_position	(TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHeader property), 88	lr_type (TotalDepth.RP66V1.core.pFile.LogicalDataDescription attribute), 88	
logical_record_position	(TotalDepth.RP66V1.core.XAxis.IFLRReference attribute), 101	LR_TYPE_ALL (in module TotalDepth.LIS.core.LogiRec), 40	
LogicalData (class in TotalDepth.RP66V1.core.pFile), 89		LR_TYPE_ALTERNATE_DATA (in module TotalDepth.LIS.core.LogiRec), 38	
LogicalDataDescription (class in TotalDepth.RP66V1.core.pFile), 88		LR_TYPE_BLANK_RECORD (in module TotalDepth.LIS.core.LogiRec), 40	
LogicalFile (class in TotalDepth.RP66V1.core.LogicalFile), 95		LR_TYPE_BOOTSTRAP_LOADER (in module TotalDepth.LIS.core.LogiRec), 39	
LogicalIndex (class in TotalDepth.RP66V1.core.LogicalFile), 96		LR_TYPE_BOT (in module TotalDepth.LIS.core.LogiRec), 39	
logicalRecord (TotalDepth.LIS.core.FileIndexer.IndexObjBase property), 27		LR_TYPE_CP_KERNEL (in module TotalDepth.LIS.core.LogiRec), 39	
LogicalRecordIndex (class in TotalDepth.RP66V1.core.pIndex), 92		LR_TYPE_DATA_DESCRIPTOR (in module TotalDepth.LIS.core.LogiRec), 39	
LogicalRecordPosition (class in TotalDepth.RP66V1.core.pFile), 88		LR_TYPE_DATA_FORMAT (in module TotalDepth.LIS.core.LogiRec), 39	
LogicalRecordSegmentHeader (class in TotalDepth.RP66V1.core.pFile), 87		LR_TYPE_DELIMITER (in module TotalDepth.LIS.core.LogiRec), 40	
LogPass (class in TotalDepth.common.LogPass), 143		LR_TYPE_DELIMITER_END (in module TotalDepth.LIS.core.LogiRec), 40	
LogPass (class in TotalDepth.LIS.core.LogPass), 51		LR_TYPE_DELIMITER_MARKER (in module TotalDepth.LIS.core.LogiRec), 40	
logPass (TotalDepth.LIS.core.FileIndexer.IndexLogPass property), 29		LR_TYPE_DELIMITER_START (in module TotalDepth.LIS.core.LogiRec), 40	
LogPassRange (class in TotalDepth.BIT.ReadBIT), 11		LR_TYPE_ENCRYPTED_TABLE (in module TotalDepth.LIS.core.LogiRec), 38	
logUp (TotalDepth.LIS.core.LogiRec.EntryBlockSet property), 48		LR_TYPE_EOF (in module TotalDepth.LIS.core.LogiRec), 39	
long_str() (TotalDepth.BIT.ReadBIT.BITFrameArray method), 12		LR_TYPE_EOM (in module TotalDepth.LIS.core.LogiRec), 39	
long_str() (TotalDepth.common.Slice.Sample method), 156		LR_TYPE_EOT (in module TotalDepth.LIS.core.LogiRec), 39	
long_str() (TotalDepth.common.Slice.Slice method), 155		LR_TYPE_FILE_HEAD (in module TotalDepth.LIS.core.LogiRec), 39	
long_str() (TotalDepth.common.Slice.SliceABC method), 154		LR_TYPE_FILE_TAIL (in module TotalDepth.LIS.core.LogiRec), 39	
longDesc() (TotalDepth.LIS.core.FileIndexer.FileIndex method), 30		LR_TYPE_FLIC_COMMENT (in module TotalDepth.LIS.core.LogiRec), 40	
longStr() (TotalDepth.LIS.core.FrameSet.FrameSet method), 34		LR_TYPE_IMAGE (in module TotalDepth.LIS.core.LogiRec), 40	
longStr() (TotalDepth.LIS.core.LogPass.LogPass method), 51		LR_TYPE_JOB_ID (in module TotalDepth.LIS.core.LogiRec), 38	
longStr() (TotalDepth.util.plot.FILMCfgXML.FilmCfgXML method), 192		LR_TYPE_LOG_DATA (in module TotalDepth.LIS.core.LogiRec), 40	
longStr() (TotalDepth.util.plot.PRESCfg.CurveCfg method), 209		LR_TYPE_MARKER (in module TotalDepth.LIS.core.LogiRec), 40	
LR_DESCRIPTION_MAP (in module TotalDepth.LIS.core.LogiRec), 40		LR_TYPE_NORMAL_DATA (in module TotalDepth.LIS.core.LogiRec), 38	
LR_DESCRIPTION_UNKNOWN (in module TotalDepth.LIS.core.LogiRec), 41		LR_TYPE_OPERATOR_INPUT (in module TotalDepth.LIS.core.LogiRec), 39	

LR_TYPE_OPERATOR_RESPONSE (in module <i>TotalDepth.LIS.core.LogiRec</i>), 39	To- LrFileTailRead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 43
LR_TYPE_PICTURE (in module <i>TotalDepth.LIS.core.LogiRec</i>), 40	To- LRH, 224
LR_TYPE_PROGRAM_FILE_HEAD (in module <i>TotalDepth.LIS.core.LogiRec</i>), 39	LrMarker (class in <i>TotalDepth.LIS.core.LogiRec</i>), 41
LR_TYPE_PROGRAM_OVER_HEAD (in module <i>TotalDepth.LIS.core.LogiRec</i>), 39	LrMisc (class in <i>TotalDepth.LIS.core.LogiRec</i>), 44
LR_TYPE_PROGRAM_OVER_LOAD (in module <i>TotalDepth.LIS.core.LogiRec</i>), 39	LrMiscRead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 44
LR_TYPE_REEL_HEAD (in module <i>TotalDepth.LIS.core.LogiRec</i>), 39	To- LrNormalAlternateData (class in <i>TotalDepth.LIS.core.LogiRec</i>), 49
LR_TYPE_REEL_TAIL (in module <i>TotalDepth.LIS.core.LogiRec</i>), 39	To- LrNormalAlternateDataRead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 49
LR_TYPE_SYSTEM_OUTPUT (in module <i>TotalDepth.LIS.core.LogiRec</i>), 39	To- LRPosDesc (class in <i>TotalDepth.RP66V1.core.pFile</i>), 88
LR_TYPE_TABLE_DATA (in module <i>TotalDepth.LIS.core.LogiRec</i>), 40	LrReelHead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 44
LR_TYPE_TABLE_DUMP (in module <i>TotalDepth.LIS.core.LogiRec</i>), 38	To- LrReelHeadRead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 44
LR_TYPE_TAPE_HEAD (in module <i>TotalDepth.LIS.core.LogiRec</i>), 39	To- LrReelHeadTail (class in <i>TotalDepth.LIS.core.LogiRec</i>), 44
LR_TYPE_TAPE_TAIL (in module <i>TotalDepth.LIS.core.LogiRec</i>), 39	To- LrReelTail (class in <i>TotalDepth.LIS.core.LogiRec</i>), 44
LR_TYPE_TOOL_INFO (in module <i>TotalDepth.LIS.core.LogiRec</i>), 38	LrReelTailRead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 44
LR_TYPE_TU10_BOOT (in module <i>TotalDepth.LIS.core.LogiRec</i>), 39	To- LrReelTapeHeadTail (class in <i>TotalDepth.LIS.core.LogiRec</i>), 43
LR_TYPE_UNKNOWN_INTERNAL_FORMAT (in module <i>TotalDepth.LIS.core.LogiRec</i>), 40	To- lrsh_attributes (TotalDepth.RP66V1.Scan.LRSHSummary attribute), 114
LR_TYPE_WELL_DATA (in module <i>TotalDepth.LIS.core.LogiRec</i>), 38	To- lrsh_position (TotalDepth.RP66V1.core.LogicalFile.PositionEFLR attribute), 95
LrBase (class in <i>TotalDepth.LIS.core.LogiRec</i>), 41	To- lrsh_position (TotalDepth.RP66V1.Scan.LRSHSummary attribute), 114
LrBOT (class in <i>TotalDepth.LIS.core.LogiRec</i>), 42	To- lrsh_record_type (TotalDepth.RP66V1.Scan.LRSHSummary attribute), 114
LrBOTRead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 42	LRSHSummary (class in <i>TotalDepth.RP66V1.Scan</i>), 114
LrDataCount() (TotalDepth.util.plot.LogHeader.APIHeaderLIS method), 194	LrTable (class in <i>TotalDepth.LIS.core.LogiRec</i>), 46
LrDFSR (class in <i>TotalDepth.LIS.core.LogiRec</i>), 49	LrTableRead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 47
LrDFSRRead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 49	LrTableWrite (class in <i>TotalDepth.LIS.core.LogiRec</i>), 47
LrEOF (class in <i>TotalDepth.LIS.core.LogiRec</i>), 41	LrTapeHead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 43
LrEOFRead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 42	LrTapeHeadRead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 43
LrEOM (class in <i>TotalDepth.LIS.core.LogiRec</i>), 42	LrTapeHeadTail (class in <i>TotalDepth.LIS.core.LogiRec</i>), 43
LrEOMRead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 42	LrTapeTail (class in <i>TotalDepth.LIS.core.LogiRec</i>), 43
LrEOT (class in <i>TotalDepth.LIS.core.LogiRec</i>), 42	LrTapeTailRead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 44
LrEOTRead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 42	LrType (TotalDepth.LIS.core.FileIndexer.IndexObjBase property), 27
LrFactory (class in <i>TotalDepth.LIS.core.LogiRec</i>), 50	LrType (TotalDepth.LIS.TableHistogram.TableMatcher attribute), 83
LrFactoryRead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 50	LrTypeS (TotalDepth.LIS.core.FileIndexer.FileIndex property), 30
LrFileHead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 43	LrUnknown (class in <i>TotalDepth.LIS.core.LogiRec</i>), 44
LrFileHeadRead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 43	LrUnkownRead (class in <i>TotalDepth.LIS.core.LogiRec</i>), 45
LrFileHeadTail (class in <i>TotalDepth.LIS.core.LogiRec</i>), 42	LrWithDateField (class in <i>TotalDepth.LIS.core.LogiRec</i>), 43
LrFileTail (class in <i>TotalDepth.LIS.core.LogiRec</i>), 43	

talDepth.LIS.core.LogiRec), 42

M

m (*TotalDepth.LIS.core.Mnem.Mnem* property), 57

main() (in module *TotalDepth.BIT.ReadBIT*), 13

main() (in module *TotalDepth.BIT.ToLAS*), 13

main() (in module *TotalDepth.common.process*), 153

main() (in module *TotalDepth.DAT.DAT_parser*), 138

main() (in module *TotalDepth.LAS.ReadLASFiles*), 15

main() (in module *TotalDepth.RP66V1.LogRecIndex*), 114

main() (in module *TotalDepth.RP66V1.SearchFF01*), 117

main() (in module *TotalDepth.RP66V1.ToLAS*), 119

main() (in module *TotalDepth.util.CopyBinFiles*), 161

main() (in module *TotalDepth.util.FileStatus*), 170

main() (in module *TotalDepth.util.RemoveDupeFiles*), 173

mainPanePlotDepth (*TotalDepth.util.plot.Plot.PlotRoll* property), 199

Margin (class in *TotalDepth.util.plot.Coord*), 181

MarginQtrInch (in module *TotalDepth.util.plot.PlotConstants*), 201

markers() (*TotalDepth.LIS.core.TifMarker.TifMarkerBase* method), 72

mask_array() (*TotalDepth.common.LogPass.FrameArray* method), 143

mask_array() (*TotalDepth.common.LogPass.FrameChannel* method), 142

max (*TotalDepth.common.statistics.LengthDict* property), 156

MAX_BACKUP_TRACK_CROSSING_LINES (*TotalDepth.util.plot.Plot.Plot* attribute), 200

maxValue() (in module *TotalDepth.LIS.core.pRepCode*), 66

method (*TotalDepth.common.lookup_mnemonic.LoggingTool* attribute), 150

MICRO_MARGIN (*TotalDepth.util.plot.Plot.Plot* attribute), 200

min (*TotalDepth.common.statistics.LengthDict* property), 156

minMaxValue() (in module *TotalDepth.LIS.core.pRepCode*), 66

minValue() (in module *TotalDepth.LIS.core.pRepCode*), 66

mirrorPt() (in module *TotalDepth.util.plot.Coord*), 183

missingFields() (*TotalDepth.util.plot.LogHeader.APIHeaderBase* method), 194

missingFields() (*TotalDepth.util.plot.LogHeader.APIHeaderLAS* method), 194

missingFields() (*TotalDepth.util.plot.LogHeader.APIHeaderLIS*

method), 194

Mnem, 224

Mnem (class in *TotalDepth.LIS.core.Mnem*), 57

mnem (*TotalDepth.LAS.core.LASRead.SectLine* attribute), 17

mnem (*TotalDepth.util.plot.LogHeader.Static* attribute), 193

Mnemonic, 224

mnemonics() (*TotalDepth.LAS.core.LASRead.LASSection* method), 17

module

TotalDepth.BIT.ReadBIT, 7

TotalDepth.BIT.ToLAS, 13

TotalDepth.common.cmn_cmd_opts, 139

TotalDepth.common.colorama, 140

TotalDepth.common.data_table, 140

TotalDepth.common.LogPass, 141

TotalDepth.common.lookup_mnemonic, 144

TotalDepth.common.process, 151

TotalDepth.common.Rle, 153

TotalDepth.common.Slice, 154

TotalDepth.common.statistics, 156

TotalDepth.common.units, 157

TotalDepth.common.xml, 159

TotalDepth.common.xxd, 159

TotalDepth.DAT.DAT_parser, 137

TotalDepth.DeTif, 3

TotalDepth.LAS.core.LASRead, 15

TotalDepth.LAS.ReadLASFiles, 15

TotalDepth.LIS.core.cRepCode, 68

TotalDepth.LIS.core.EngVal, 21

TotalDepth.LIS.core.File, 24

TotalDepth.LIS.core.FileIndexer, 27

TotalDepth.LIS.core.FrameSet, 32

TotalDepth.LIS.core.LogiRec, 38

TotalDepth.LIS.core.LogPass, 50

TotalDepth.LIS.core.Mnem, 57

TotalDepth.LIS.core.PhysRec, 58

TotalDepth.LIS.core.pRepCode, 66

TotalDepth.LIS.core.RawStream, 62

TotalDepth.LIS.core.RepCode, 64

TotalDepth.LIS.core.Rle, 69

TotalDepth.LIS.core.TifMarker, 71

TotalDepth.LIS.core.Type01Plan, 73

TotalDepth.LIS.core.Units, 75

TotalDepth.LIS.DumpFrameSet, 78

TotalDepth.LIS.Index, 78

TotalDepth.LIS.lis_cmn_cmd_opts, 84

TotalDepth.LIS.LisToHtml, 79

TotalDepth.LIS.PlotLogPasses, 80

TotalDepth.LIS.ProcLISPath, 81

TotalDepth.LIS.RandomFrameSetRead, 82

TotalDepth.LIS.ScanLogiData, 82

TotalDepth.LIS.ScanLogiRec, 82

TotalDepth.LIS.ScanPhysRec, 82
 TotalDepth.LIS.TableHistogram, 83
 TotalDepth.LIS.ToLAS, 84
 TotalDepth.PlotLogs, 5
 TotalDepth.RP66V1.core.AbsentValue, 85
 TotalDepth.RP66V1.core.LogicalFile, 94
 TotalDepth.RP66V1.core.LogicalRecord.ComponentDetails, 102
 TotalDepth.RP66V1.core.LogicalRecord.EFLR, 104
 TotalDepth.RP66V1.core.LogicalRecord.EncryptionJob, 108
 TotalDepth.RP66V1.core.LogicalRecord.IFLR, 108
 TotalDepth.RP66V1.core.LogicalRecord.Semantics, 109
 TotalDepth.RP66V1.core.LogicalRecord.Types, 109
 TotalDepth.RP66V1.core.LogPass, 93
 TotalDepth.RP66V1.core.pFile, 85
 TotalDepth.RP66V1.core.pIndex, 91
 TotalDepth.RP66V1.core.RepCode, 97
 TotalDepth.RP66V1.core.stringify, 102
 TotalDepth.RP66V1.core.Units, 97
 TotalDepth.RP66V1.core.XAxis, 101
 TotalDepth.RP66V1.IndexPickle, 110
 TotalDepth.RP66V1.IndexXML, 111
 TotalDepth.RP66V1.LogRecIndex, 113
 TotalDepth.RP66V1.Scan, 114
 TotalDepth.RP66V1.ScanHTML, 116
 TotalDepth.RP66V1.SearchFF01, 117
 TotalDepth.RP66V1.ToLAS, 117
 TotalDepth.RP66V1.util.IndexXMLRead, 119
 TotalDepth.RP66V1.util.XMLReadUnits, 120
 TotalDepth.util.archive, 176
 TotalDepth.util.bin_file_type, 177
 TotalDepth.util.CopyBinFiles, 161
 TotalDepth.util.DictTree, 161
 TotalDepth.util.DirWalk, 165
 TotalDepth.util.EBCDIC, 166
 TotalDepth.util.ExecTimer, 168
 TotalDepth.util.FileBuffer, 169
 TotalDepth.util.FileStatus, 170
 TotalDepth.util.gnuplot, 178
 TotalDepth.util.Histogram, 171
 TotalDepth.util.HtmlUtils, 172
 TotalDepth.util.PatternSearch, 173
 TotalDepth.util.plot.AREACfg, 179
 TotalDepth.util.plot.Coord, 179
 TotalDepth.util.plot.FILMCfg, 185
 TotalDepth.util.plot.FILMCfgXML, 191
 TotalDepth.util.plot.LogHeader, 192
 TotalDepth.util.plot.Plot, 195
 TotalDepth.util.plot.PlotConstants, 201
 TotalDepth.util.plot.PRESCfg, 202
 TotalDepth.util.plot.PRESCfgXML, 210
 TotalDepth.util.plot.Stroke, 212
 TotalDepth.util.plot.SVGWriter, 212
 TotalDepth.util.plot.Track, 216
 TotalDepth.util.plot.XGrid, 220
 TotalDepth.util.plot.XMLCfg, 221
 TotalDepth.util.plot.XMLMatches, 222
 TotalDepth.util.RemoveDupeFiles, 173
 TotalDepth.util.XmlWrite, 174
name (*TotalDepth.util.ExecTimer.Timer* property), 168
multiprocessing_requested() (in module *TotalDepth.common.cmn_cmd_opts*), 140
N
name (*TotalDepth.BIT.ReadBIT.FileSizeTime* attribute), 12
name (*TotalDepth.common.units.Unit* attribute), 157
name (*TotalDepth.LIS.core.FileIndexer.IndexTable* property), 29
name (*TotalDepth.util.plot.FILMCfg.PhysFilmCfg* property), 189
NAME_UNITS_TYPE_MAP (in module *TotalDepth.DAT.DAT_parser*), 138
nameCol (*TotalDepth.LIS.TableHistogram.TableMatcher* attribute), 83
nameFromString() (in module *TotalDepth.util.XmlWrite*), 174
nameRow (*TotalDepth.LIS.TableHistogram.TableMatcher* attribute), 83
nameTable (*TotalDepth.LIS.TableHistogram.TableMatcher* attribute), 83
nbytes (*TotalDepth.LIS.core.FrameSet.FrameSet* property), 34
newEngValInOpticalUnits() (*TotalDepth.LIS.core.EngVal.EngVal* method), 23
newEngValInUnits() (*TotalDepth.LIS.core.EngVal.EngVal* method), 23
newPt() (in module *TotalDepth.util.plot.Coord*), 183
next (*TotalDepth.BIT.ReadBIT.TifMarker* attribute), 10
next (*TotalDepth.DeTif.TifMarker* attribute), 3
next_position (*TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHeader* property), 88
nextFileName (*TotalDepth.LIS.core.LogiRec.LrFileTail* property), 43
nextReelName (*TotalDepth.LIS.core.LogiRec.LrReelTail* property), 44
nextTapeName (*TotalDepth.LIS.core.LogiRec.LrTapeTail* property), 44
node (*TotalDepth.util.DictTree.DictTreeTableEvent* attribute), 162

normalise_integer() (*TotalDepth.LIS.core.PhysRec.PhysRecTail static method*), 61
normalise_units() (*TotalDepth.util.plot.Coord.Pt method*), 182
null_value (*TotalDepth.LAS.core.LASRead.LASBase property*), 18
nullValue (*TotalDepth.LIS.core.LogPass.LogPass property*), 51
num_frames() (*TotalDepth.RP66V1.core.LogicalFile.LogicalFile method*), 96
num_values() (*TotalDepth.common.Rle.RLE method*), 154
number_multiprocessing_jobs() (*in module TotalDepth.common.cmn_cmd_opts*), 140
number_of_data_points() (*TotalDepth.LAS.core.LASRead.LASRead method*), 19
number_of_frames() (*TotalDepth.LAS.core.LASRead.LASRead method*), 19
numBursts() (*TotalDepth.LIS.core.FrameSet.FrameSet method*), 35
numBytes (*TotalDepth.LIS.core.LogPass.LogPass property*), 51
numChannels (*TotalDepth.LIS.core.FrameSet.FrameSet property*), 34
numChannels (*TotalDepth.LIS.core.Type01Plan.FrameSetPlan property*), 74
numFrames (*TotalDepth.LIS.core.FrameSet.FrameSet property*), 34
numFrames (*TotalDepth.LIS.core.Rle.RLEItem type01 property*), 69
numFrames() (*TotalDepth.LIS.core.Type01Plan.FrameSetPlan method*), 74
numLogPasses() (*TotalDepth.LIS.core.FileIndexer.FileIndexer method*), 30
numLr (*TotalDepth.LIS.LisToHtml.FileInfo attribute*), 79
numPoints (*TotalDepth.LIS.PlotLogPasses.IndexTableValue attribute*), 80
numPoints (*TotalDepth.PlotLogs.IndexTableValue attribute*), 5
NUMPY_DATA_TYPE (*TotalDepth.LIS.core.FrameSet.FrameSet attribute*), 34
numpy_indexes() (*TotalDepth.common.LogPass.FrameChannel method*), 142
numSamples() (*TotalDepth.LIS.core.FrameSet.FrameSet method*), 35
numSubChannels (*TotalDepth.LIS.core.FrameSet.ChArTe property*), 33
numSubChannels() (*TotalDepth.LIS.core.FrameSet.FrameSet method*), 35
numValues (*TotalDepth.LIS.core.FrameSet.FrameSet property*), 34
numValues (*TotalDepth.LIS.core.FrameSet.SuChArTe property*), 33
O
Object (*class in TotalDepth.RP66V1.core.LogicalRecord.EFLR*), 106
offsetScale() (*TotalDepth.util.plot.PRESCfg.LineTransBase method*), 207
offset (*TotalDepth.common.units.Unit attribute*), 158
opacity (*TotalDepth.util.plot.Stroke.Stroke attribute*), 212
opticalLogScale (*TotalDepth.LIS.core.LogiRec.EntryBlockSet property*), 48
opticalUnits() (*in module TotalDepth.LIS.core.Units*), 77
ORDS_REPLACE (*in module TotalDepth.LIS.core.Mnem*), 57
ORDS_WS (*in module TotalDepth.LIS.core.Mnem*), 57
origin_logical_record (*TotalDepth.RP66V1.core.LogicalFile.LogicalFile property*), 96
osdd_data_file_path() (*in module TotalDepth.common.units*), 158
outPath (*TotalDepth.LIS.PlotLogPasses.IndexTableValue attribute*), 80
outPath (*TotalDepth.PlotLogs.IndexTableValue attribute*), 5
outpChIDs() (*TotalDepth.util.plot.PRESCfg.PresCfg method*), 209
outpCurveIDs() (*TotalDepth.util.plot.PRESCfg.PresCfg method*), 209
outpMnemS() (*TotalDepth.LIS.core.LogPass.LogPass method*), 52
P
packAndWrite() (*TotalDepth.LIS.core.RawStream.RawStream method*), 63
Pad (*class in TotalDepth.util.plot.Coord*), 181
pad_modulo (*TotalDepth.LIS.core.File.PhysicalRecordSettings attribute*), 25
pad_modulo (*TotalDepth.LIS.ScanPhysRec.PhysRecScanResultWithPad attribute*), 83
pad_non_null (*TotalDepth.LIS.core.File.PhysicalRecordSettings attribute*), 25
pad_non_null (*TotalDepth.LIS.ScanPhysRec.PhysRecScanResultWithPad attribute*), 83
Parameter (*class in TotalDepth.common.lookup_mnemonic*), 148
parameter (*TotalDepth.common.lookup_mnemonic.ParameterDescription attribute*), 149

ParameterDescription (class in <i>TotalDepth.common.lookup_mnemonic</i>), 149	plot() (<i>TotalDepth.util.plot.LogHeader.APIHeaderBase</i> method), 194
parse_file() (in module <i>TotalDepth.DAT.DAT_parser</i>), 138	plot_gnuplot() (in module <i>TotalDepth.RP66V1.LogRecIndex</i>), 114
parse_path() (in module <i>TotalDepth.DAT.DAT_parser</i>), 138	PlotLogInfo (class in <i>TotalDepth.LIS.PlotLogPasses</i>), 80
parse_timestamp() (in module <i>TotalDepth.common.process</i>), 152	PlotLogInfo (class in <i>TotalDepth.PlotLogs</i>), 5
path (<i>TotalDepth.LIS.ScanPhysRec.PhysRecScanResult</i> attribute), 82	PlotLogPasses (class in <i>TotalDepth.LIS.PlotLogPasses</i>), 81
path_in (<i>TotalDepth.RP66V1.IndexPickle.IndexResult</i> attribute), 110	PlotLogPasses (class in <i>TotalDepth.PlotLogs</i>), 6
path_in (<i>TotalDepth.RP66V1.LogRecIndex.IndexResult</i> attribute), 113	plotLogPassesMP() (in module <i>TotalDepth.LIS.PlotLogPasses</i>), 81
path_in (<i>TotalDepth.util.DirWalk.FileEvent</i> attribute), 165	plotLogPassesMP() (in module <i>TotalDepth.PlotLogs</i>), 6
path_in() (in module <i>TotalDepth.common.cmn_cmd_opts</i>), 139	plotLogPassLAS() (<i>TotalDepth.util.plot.Plot.Plot</i> method), 201
path_in_out() (in module <i>TotalDepth.common.cmn_cmd_opts</i>), 139	plotLogPassLIS() (<i>TotalDepth.util.plot.Plot.Plot</i> method), 200
path_in_out_required() (in module <i>TotalDepth.common.cmn_cmd_opts</i>), 139	PlotReadLIS (class in <i>TotalDepth.util.plot.Plot</i>), 201
path_input (<i>TotalDepth.RP66V1.IndexXML.IndexResult</i> attribute), 112	PlotReadXML (class in <i>TotalDepth.util.plot.Plot</i>), 201
path_out (<i>TotalDepth.util.DirWalk.FileEvent</i> attribute), 166	PlotRecordSet (class in <i>TotalDepth.LIS.core.FileIndexer</i>), 29
pathIn (<i>TotalDepth.LIS.LisToHtml.FileInfo</i> attribute), 79	PlotRoll (class in <i>TotalDepth.util.plot.Plot</i>), 197
pathOut (<i>TotalDepth.LIS.LisToHtml.FileInfo</i> attribute), 79	plotSVG() (<i>TotalDepth.util.plot.Track.Track</i> method), 220
pathSplit() (in module <i>TotalDepth.util.HtmlUtils</i>), 172	png_location() (in module <i>TotalDepth.util.plot.AREACfg</i>), 179
payload (<i>TotalDepth.BIT.ReadBIT.TifMarkedBytes</i> attribute), 11	polyLinePt() (<i>TotalDepth.util.plot.Plot.PlotRoll</i> method), 199
peek() (<i>TotalDepth.RP66V1.core.pFile.LogicalData</i> method), 89	populate_frame_array() (<i>TotalDepth.RP66V1.core.LogicalFile.LogicalFile</i> method), 96
PhysFilmCfg (class in <i>TotalDepth.util.plot.FILMCfg</i>), 189	position (<i>TotalDepth.RP66V1.core.pFile.LRPosDesc</i> attribute), 89
PhysFilmCfgLISRead (class in <i>TotalDepth.util.plot.FILMCfg</i>), 189	PositionEFLR (class in <i>TotalDepth.RP66V1.core.LogicalFile</i>), 94
PhysFilmCfgXMLRead (class in <i>TotalDepth.util.plot.FILMCfgXML</i>), 191	PR_ATTRIBUTE_BITS (in module <i>TotalDepth.LIS.core.PhysRec</i>), 58
Physical Record, 224	PR_ATTRIBUTE_UNUSED_MASK (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59
PhysicalRecordSettings (class in <i>TotalDepth.LIS.core.File</i>), 25	PR_ATTRIBUTE_UNUSED_ONLY_MASK (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59
PhysRecBase (class in <i>TotalDepth.LIS.core.PhysRec</i>), 60	PR_ATTRIBUTE_UNUSED_RESERVED_MASK (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59
PhysRecRead (class in <i>TotalDepth.LIS.core.PhysRec</i>), 60	PR_CHECKSUM_BIT (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59
PhysRecScanResult (class in <i>TotalDepth.LIS.ScanPhysRec</i>), 82	PR_CHECKSUM_UNDEFINED_BIT (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59
PhysRecScanResultWithPad (class in <i>TotalDepth.LIS.ScanPhysRec</i>), 83	pr_count (<i>TotalDepth.LIS.ScanPhysRec.PhysRecScanResult</i> attribute), 82
PhysRecTail (class in <i>TotalDepth.LIS.core.PhysRec</i>), 61	PR_FILE_NUMBER_BIT (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59
PhysRecWrite (class in <i>TotalDepth.LIS.core.PhysRec</i>), 61	PR_MAX_LENGTH (in module <i>TotalDepth.LIS.core.PhysRec</i>), 60
pI() (<i>TotalDepth.util.XmlWrite.XmlStream</i> method), 175	
Plot (class in <i>TotalDepth.util.plot.Plot</i>), 199	

PR_OLD_CHECK_ERROR_BIT (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59	<i>TotalDepth.BIT.ReadBIT</i>), 13
PR_OLD_PARITY_ERROR_BIT (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59	<i>print_process_file()</i> (in module <i>TotalDepth.BIT.ReadBIT</i>), 13
PR_PREDECESSOR_ATTRIBUTE_BIT (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59	<i>print_summarise_frame_array()</i> (in module <i>TotalDepth.BIT.ReadBIT</i>), 13
PR_PRH_ATTR_FORMAT (in module <i>TotalDepth.LIS.core.PhysRec</i>), 58	<i>processDir()</i> (<i>TotalDepth.util.FileStatus.FileInfoSet</i> method), 170
PR_PRH_LEN_FORMAT (in module <i>TotalDepth.LIS.core.PhysRec</i>), 58	<i>processFile()</i> (in module <i>TotalDepth.LIS.LisToHtml</i>), 80
PR_PRH_LENGTH (in module <i>TotalDepth.LIS.core.PhysRec</i>), 58	<i>ProcessLoggingThread</i> (class in <i>TotalDepth.common.process</i>), 152
PR_PRT_CHECKSUM_FORMAT (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59	<i>processPath()</i> (<i>TotalDepth.util.FileStatus.FileInfoSet</i> method), 170
PR_PRT_CHECKSUM_LEN (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59	<i>procLISPath()</i> (in module <i>TotalDepth.LIS.ProcLISPath</i>), 81
PR_PRT_FILE_NUM_FORMAT (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59	<i>ProcLISPathBase</i> (class in <i>TotalDepth.LIS.ProcLISPath</i>), 81
PR_PRT_FILE_NUM_LEN (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59	<i>procLISPathMP()</i> (in module <i>TotalDepth.LIS.ProcLISPath</i>), 82
PR_PRT_FILE_NUM_MAX (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59	PRODUCER_CODE_MAPPING_OF_UNIT_CODE (in module <i>TotalDepth.RP66V1.core.Units</i>), 100
PR_PRT_FILE_NUM_MIN (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59	<i>product</i> (<i>TotalDepth.common.lookup_mnemonic.ProductDescription</i> attribute), 147
PR_PRT_REC_NUM_FORMAT (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59	<i>ProductDescription</i> (class in <i>TotalDepth.common.lookup_mnemonic</i>), 147
PR_PRT_REC_NUM_LEN (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59	<i>property</i> (<i>TotalDepth.common.lookup_mnemonic.Channel</i> attribute), 148
PR_PRT_REC_NUM_MAX (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59	<i>property</i> (<i>TotalDepth.common.lookup_mnemonic.Parameter</i> attribute), 149
PR_PRT_REC_NUM_MIN (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59	<i>pStr()</i> (<i>TotalDepth.LIS.core.EngVal.EngVal</i> method), 22
PR_RECORD_NUMBER_BIT (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59	<i>pStr()</i> (<i>TotalDepth.LIS.core.Mnem.Mnem</i> method), 57
PR_SUCESSOR_ATTRIBUTE_BIT (in module <i>TotalDepth.LIS.core.PhysRec</i>), 58	PSUTIL_PROCESS_AS_DICT_KEYS (in module <i>TotalDepth.common.process</i>), 151
PR_TYPE_BIT (in module <i>TotalDepth.LIS.core.PhysRec</i>), 59	<i>Pt</i> (class in <i>TotalDepth.util.plot.Coord</i>), 182
PresCfg (class in <i>TotalDepth.util.plot.PRESCfg</i>), 209	PUBLIC_EFLR_TYPES (in module <i>TotalDepth.RP66V1.core.LogicalRecord.EFLR</i>), 107
PresCfgLISRead (class in <i>TotalDepth.util.plot.PRESCfg</i>), 210	<i>PublicEFLRType</i> (class in <i>TotalDepth.RP66V1.core.LogicalRecord.EFLR</i>), 106
PresCfgXMLRead (class in <i>TotalDepth.util.plot.PRESCfgXML</i>), 211	<i>pxBaseUnitsBox()</i> (in module <i>TotalDepth.util.plot.Coord</i>), 183
prev (<i>TotalDepth.BIT.ReadBIT.TifMarker</i> attribute), 10	<i>pxBaseUnitsDim()</i> (in module <i>TotalDepth.util.plot.Coord</i>), 183
prev (<i>TotalDepth.DeTif.TifMarker</i> attribute), 3	<i>pxBaseUnitsPad()</i> (in module <i>TotalDepth.util.plot.Coord</i>), 183
prevFileName (<i>TotalDepth.LIS.core.LogiRec.LrFileHead</i> property), 43	<i>pxBaseUnitsPt()</i> (in module <i>TotalDepth.util.plot.Coord</i>), 183
prevReelName (<i>TotalDepth.LIS.core.LogiRec.LrReelHead</i> property), 44	<i>pxUnitsDim()</i> (in module <i>TotalDepth.util.plot.Coord</i>), 183
prevTapeName (<i>TotalDepth.LIS.core.LogiRec.LrTapeHead</i> property), 43	
prhAttr (<i>TotalDepth.LIS.core.PhysRec.PhysRecTail</i> property), 61	
<i>print_process_directory()</i> (in module <i>TotalDepth.BIT.ReadBIT</i>), 13	

R

range() (*TotalDepth.common.Rle.RLEItem* method),

- 153
- ranges() (*TotalDepth.common.Rle.RLE method*), 154
- rAttr (*TotalDepth.util.plot.LogHeader.Static attribute*), 193
- RawStream (*class in TotalDepth.LIS.core.RawStream*), 62
- RE_CHANNEL_DEFINITION (*in module TotalDepth.DAT.DAT_parser*), 138
- RE_COMMENT (*in module TotalDepth.LAS.core.LASRead*), 16
- RE_COMPILED (*in module TotalDepth.util.bin_file_type*), 177
- RE_DATA_HEADER_DEFINITION (*in module TotalDepth.DAT.DAT_parser*), 138
- RE_DATE_STYLE_A (*in module TotalDepth.DAT.DAT_parser*), 138
- RE_DATE_STYLE_B (*in module TotalDepth.DAT.DAT_parser*), 138
- RE_LAS_VERSION_LINE (*in module TotalDepth.util.bin_file_type*), 177
- RE_LINE_FIELD_0 (*in module TotalDepth.LAS.core.LASRead*), 16
- RE_LINE_FIELD_1 (*in module TotalDepth.LAS.core.LASRead*), 16
- RE_LOG_LINE (*in module TotalDepth.common.process*), 151
- RE_SECT_HEAD (*in module TotalDepth.LAS.core.LASRead*), 16
- read() (*TotalDepth.LIS.core.LogiRec.LrFileHeadTail method*), 42
- read() (*TotalDepth.LIS.core.RawStream.RawStream method*), 63
- read() (*TotalDepth.LIS.core.TifMarker.TifMarkerRead method*), 73
- read() (*TotalDepth.RP66V1.core.LogicalRecord.EFLR.TemplateRead method*), 105
- read() (*TotalDepth.RP66V1.core.LogPass.RP66V1FrameArray method*), 93
- read() (*TotalDepth.RP66V1.core.LogPass.RP66V1FrameChannel method*), 93
- read() (*TotalDepth.RP66V1.core.pFile.LogicalData method*), 89
- read() (*TotalDepth.RP66V1.core.pFile.LogicalRecordSegmentHeader method*), 87
- read() (*TotalDepth.RP66V1.core.pFile.VisibleRecord method*), 87
- read49() (*in module TotalDepth.LIS.core.pRepCode*), 66
- read49() (*in module TotalDepth.LIS.core.RepCode*), 64
- read50() (*in module TotalDepth.LIS.core.pRepCode*), 66
- read50() (*in module TotalDepth.LIS.core.RepCode*), 64
- read56() (*in module TotalDepth.LIS.core.pRepCode*), 66
- read56() (*in module TotalDepth.LIS.core.RepCode*), 64
- read66() (*in module TotalDepth.LIS.core.pRepCode*), 67
- read66() (*in module TotalDepth.LIS.core.RepCode*), 64
- read68() (*in module TotalDepth.LIS.core.pRepCode*), 67
- read68() (*in module TotalDepth.LIS.core.RepCode*), 64
- read70() (*in module TotalDepth.LIS.core.pRepCode*), 67
- read70() (*in module TotalDepth.LIS.core.RepCode*), 64
- read73() (*in module TotalDepth.LIS.core.pRepCode*), 67
- read73() (*in module TotalDepth.LIS.core.RepCode*), 65
- read77() (*in module TotalDepth.LIS.core.pRepCode*), 67
- read77() (*in module TotalDepth.LIS.core.RepCode*), 65
- read79() (*in module TotalDepth.LIS.core.pRepCode*), 67
- read79() (*in module TotalDepth.LIS.core.RepCode*), 65
- read_a_single_file() (*in module TotalDepth.RP66V1.util.XMLReadUnits*), 136
- read_a_single_index() (*in module TotalDepth.RP66V1.util.IndexXMLRead*), 120
- read_bytes_from_offset() (*in module TotalDepth.BIT.ReadBIT*), 11
- read_float() (*in module TotalDepth.BIT.ReadBIT*), 10
- read_next() (*TotalDepth.RP66V1.core.pFile.VisibleRecord method*), 87
- read_osdd_static_data() (*in module TotalDepth.common.units*), 158
- read_partial() (*TotalDepth.RP66V1.core.LogPass.RP66V1FrameArray method*), 93
- read_x_axis() (*TotalDepth.RP66V1.core.LogPass.RP66V1FrameArray method*), 94
- readAndUnpack() (*TotalDepth.LIS.core.RawStream.RawStream method*), 63
- readBytes() (*in module TotalDepth.LIS.core.RepCode*), 65
- readBytes130() (*in module TotalDepth.LIS.core.RepCode*), 65
- readBytes234() (*in module TotalDepth.LIS.core.RepCode*), 65
- readBytes49() (*in module TotalDepth.LIS.core.RepCode*), 64
- readBytes50() (*in module TotalDepth.LIS.core.RepCode*), 64
- readBytes56() (*in module TotalDepth.LIS.core.RepCode*), 64
- readBytes66() (*in module TotalDepth.LIS.core.RepCode*), 64
- readBytes68() (*in module TotalDepth.LIS.core.RepCode*), 64
- readBytes70() (*in module TotalDepth.LIS.core.RepCode*), 64

<code>talDepth.LIS.core.RepCode)</code> , 65	<code>RepCode</code> , 224
<code>readBytes73()</code> (in module <code>talDepth.LIS.core.RepCode</code>), 65	<code>report0x80()</code> (in module <code>talDepth.util.PatternSearch</code>), 173
<code>readBytes77()</code> (in module <code>talDepth.LIS.core.RepCode</code>), 65	<code>reportAll()</code> (in module <code>TotalDepth.util.PatternSearch</code>), 173
<code>readBytes79()</code> (in module <code>talDepth.LIS.core.RepCode</code>), 65	<code>reportError()</code> (<code>TotalDepth.LIS.core.TifMarker.TifMarkerBase</code> method), 72
<code>readDir()</code> (<code>TotalDepth.util.plot.FILMCfgXML.FilmCfgXMLRead</code> method), 192	<code>reportIDENT()</code> (in module <code>talDepth.util.PatternSearch</code>), 173
<code>readFromFile()</code> (<code>TotalDepth.LIS.core.LogiRec.EntryBlock</code> method), 48	<code>Representation Code</code> , 224
<code>readLrBytes()</code> (<code>TotalDepth.LIS.core.File.FileRead</code> method), 24	<code>reset()</code> (<code>TotalDepth.LIS.core.TifMarker.TifMarkerBase</code> method), 72
<code>readLrBytes()</code> (<code>TotalDepth.LIS.core.PhysRec.PhysRecRead</code> method), 60	<code>reset()</code> (<code>TotalDepth.LIS.core.TifMarker.TifMarkerRead</code> method), 73
<code>readRepCode()</code> (in module <code>talDepth.LIS.core.RepCode</code>), 65	<code>reset()</code> (<code>TotalDepth.util.plot.Plot.CurvePlotScaleSlotMap</code> method), 197
<code>realUnitName()</code> (in module <code>TotalDepth.LIS.core.Units</code>), 77	<code>Restrictions</code> (class in <code>talDepth.RP66V1.core.LogicalRecord.Semantics</code>), 109
<code>RECT_ATTRS_FINE</code> (in module <code>talDepth.util.plot.LogHeader</code>), 193	<code>result</code> (<code>TotalDepth.LIS.ScanPhysRec.PhysRecScanResultWithPad</code> attribute), 83
<code>reduced_object_map()</code> (in module <code>talDepth.RP66V1.core.LogicalRecord.EFLR</code>), 108	<code>ret_int_dump_list()</code> (in module <code>talDepth.LIS.ScanLogiData</code>), 82
<code>reduced_power_2()</code> (<code>talDepth.common.statistics.LengthDict</code> method), 156	<code>ret_padding_options_with_max_records()</code> (in module <code>TotalDepth.LIS.core.File</code>), 26
<code>regexStdout()</code> (in module <code>talDepth.LIS.RandomFrameSetRead</code>), 82	<code>retAllFILMDestS()</code> (<code>talDepth.util.plot.FILMCfg.FilmCfg</code> method), 190
<code>related_channels</code> (<code>talDepth.common.lookup_mnemonic.LoggingTool</code> attribute), 150	<code>retAllFILMDestS()</code> (<code>talDepth.util.plot.FILMCfg.FilmCfgLISRead</code> method), 191
<code>related_parameters</code> (<code>talDepth.common.lookup_mnemonic.LoggingTool</code> attribute), 150	<code>retAllFILMDestS()</code> (<code>talDepth.util.plot.FILMCfgXML.FilmCfgXMLRead</code> method), 192
<code>related_products</code> (<code>talDepth.common.lookup_mnemonic.Channel</code> attribute), 148	<code>retExtChIndexList()</code> (<code>talDepth.LIS.core.LogPass.LogPass</code> method), 52
<code>related_products</code> (<code>talDepth.common.lookup_mnemonic.Parameter</code> attribute), 149	<code>retFILMDest()</code> (<code>TotalDepth.util.plot.FILMCfg.FilmCfg</code> method), 190
<code>related_tools</code> (<code>TotalDepth.common.lookup_mnemonic.Channel</code> attribute), 148	<code>retFILMDest()</code> (<code>TotalDepth.util.plot.FILMCfg.FilmCfgLISRead</code> method), 191
<code>remain</code> (<code>TotalDepth.RP66V1.core.pFile.LogicalData</code> property), 89	<code>retFILMDest()</code> (<code>TotalDepth.util.plot.FILMCfgXML.FilmCfgXMLRead</code> method), 192
<code>remove()</code> (<code>TotalDepth.util.DictTree.DictTree</code> method), 161	<code>retHeadPane()</code> (<code>TotalDepth.util.plot.Plot.PlotRoll</code> method), 199
<code>remove()</code> (<code>TotalDepth.util.DictTree.DictTreeHtmlTable</code> method), 164	<code>retHtmlFileLink()</code> (in module <code>talDepth.util.HtmlUtils</code>), 172
<code>remove_dupes()</code> (in module <code>talDepth.util.RemoveDupeFiles</code>), 173	<code>retHtmlFileName()</code> (in module <code>talDepth.util.HtmlUtils</code>), 172
<code>Rep Code</code> , 224	<code>retLegendPane()</code> (<code>TotalDepth.util.plot.Plot.PlotRoll</code> method), 199
<code>rep_code</code> (<code>TotalDepth.RP66V1.core.LogicalRecord.ComponentDescription</code> attribute), 102	<code>retLrFromFile()</code> (<code>talDepth.LIS.core.LISRepCodeChannel</code> method), 50

retMainPane() (*TotalDepth.util.plot.Plot.PlotRoll method*), 199
 retMainPaneStart() (*TotalDepth.util.plot.Plot.PlotRoll method*), 199
 retRelPath() (*TotalDepth.LIS.PlotLogPasses.PlotLogInfo method*), 80
 retRelPath() (*TotalDepth.PlotLogs.PlotLogInfo method*), 6
 retRowByMnem() (*TotalDepth.LIS.core.LogiRec.LrTable method*), 46
 retSVGAttrsFromStroke() (*in module TotalDepth.util.plot.Stroke*), 212
 retTailPane() (*TotalDepth.util.plot.Plot.PlotRoll method*), 199
 retUnitConvert() (*in module TotalDepth.LIS.core.Units*), 77
 retUnitConvertCategory() (*in module TotalDepth.LIS.core.Units*), 77
 rewind() (*TotalDepth.LIS.core.File.FileRead method*), 25
 rewind() (*TotalDepth.RP66V1.core.pFile.LogicalData method*), 89
 right (*TotalDepth.util.plot.Track.Track property*), 220
 rightL (*TotalDepth.util.plot.PRESCfg.LineTransBase property*), 207
 rightP (*TotalDepth.util.plot.PRESCfg.TrackWidthData attribute*), 208
 RLE (*class in TotalDepth.common.Rle*), 153
 rle (*TotalDepth.LIS.core.LogPass.LogPass property*), 51
 RLEItem (*class in TotalDepth.common.Rle*), 153
 RLEItemType01 (*class in TotalDepth.LIS.core.Rle*), 69
 RLEType01 (*class in TotalDepth.LIS.core.Rle*), 70
 role (*TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor property*), 103
 role (*TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor attribute*), 102
 RoleType (*class in TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor*), 102
 rootNode() (*TotalDepth.util.plot.FILMCfgXML.FilmCfgXML method*), 192
 ROW_CLOSE (*TotalDepth.util.DictTree.DictTreeHtmlTable attribute*), 164
 ROW_OPEN (*TotalDepth.util.DictTree.DictTreeHtmlTable attribute*), 164
 row_span (*TotalDepth.util.DictTree.DictTreeTableEvent attribute*), 162
 rowLabels() (*TotalDepth.LIS.core.LogiRec.LrTable method*), 47
 rowMnems() (*TotalDepth.LIS.core.LogiRec.LrTable method*), 47
 RP66, 224
 RP66V1, 224
 RP66V1.Absent Value, 224
 RP66V1.Attribute, 225
 RP66V1.Channel, 225
 RP66V1.Characteristic, 225
 RP66V1.Company Code, 225
 RP66V1.Component, 225
 RP66V1.Component Descriptor, 225
 RP66V1.Component Format, 225
 RP66V1.Component Role, 225
 RP66V1.Compound Representation Code, 225
 RP66V1.Consumer, 225
 RP66V1.Copy Number, 225
 RP66V1.Count, 225
 RP66V1.Data Descriptor Reference, 225
 RP66V1.Defining Origin, 225
 RP66V1.Descriptor, 225
 RP66V1.Dictionary, 225
 RP66V1.Dimension, 225
 RP66V1.EFLR, 226
 RP66V1.Element, 226
 RP66V1.Encryption Packet, 226
 RP66V1.Explicitly Formatted Logical Record, 226
 RP66V1.Format Version, 226
 RP66V1.Frame, 226
 RP66V1.Frame Data, 226
 RP66V1.Frame Number, 226
 RP66V1.Frame Type, 226
 RP66V1.Header (*Refer to Logical Record Segment Header*), 226
 RP66V1.Identifier, 226
 RP66V1.IFLR, 226
 RP66V1.Index Channel, 226
 RP66V1.Indirectly Formatted Data, 227
 RP66V1.Indirectly Formatted Logical Record, 227
 RP66V1.Invisible Envelope, 227
 RP66V1.Lexicon, 227
 RP66V1.Locus, 227
 RP66V1.Logical File, 227
 RP66V1.Logical Format, 227
 RP66V1.Logical Record, 227
 RP66V1.Logical Record Body, 227
 RP66V1.Logical Record Segment, 227
 RP66V1.Logical Record Segment Attributes, 227
 RP66V1.Logical Record Segment Body, 227
 RP66V1.Logical Record Segment Encryption Packet, 227
 RP66V1.Logical Record Segment Header, 227
 RP66V1.Logical Record Segment Length, 227
 RP66V1.Logical Record Segment Trailer, 228
 RP66V1.Logical Record Structure, 228
 RP66V1.Logical Record Type, 228
 RP66V1.Long Name, 228

RP66V1.Maximum Visible Record Length, [228](#)
 RP66V1.Minimum Visible Record Length, [228](#)
 RP66V1.Name, [228](#)
 RP66V1.Name Part Type, [228](#)
 RP66V1.Name Part Value, [228](#)
 RP66V1.Object, [228](#)
 RP66V1.Object Component, [228](#)
 RP66V1.Origin, [228](#)
 RP66V1.Pad Bytes, [228](#)
 RP66V1.Pad Count, [228](#)
 RP66V1.Padding, [228](#)
 RP66V1.Parent File, [228](#)
 RP66V1.Path, [228](#)
 RP66V1.Physical Format, [229](#)
 RP66V1.Predecessor, [229](#)
 RP66V1.Private, [229](#)
 RP66V1.Producer, [229](#)
 RP66V1.Public, [229](#)
 RP66V1.Radial Drift, [229](#)
 RP66V1.Record Structure, [229](#)
 RP66V1.Representation Code, [229](#)
 RP66V1.Sample (of a Channel), [229](#)
 RP66V1.Semantics, [229](#)
 RP66V1.Set, [229](#)
 RP66V1.Set Component, [229](#)
 RP66V1.Set Type, [229](#)
 RP66V1.Slot, [230](#)
 RP66V1.Splice, [230](#)
 RP66V1.Static Information, [230](#)
 RP66V1.Storage Set, [230](#)
 RP66V1.Storage Set Identifier, [230](#)
 RP66V1.Storage Unit, [230](#)
 RP66V1.Storage Unit Label, [230](#)
 RP66V1.Storage Unit Sequence Number, [230](#)
 RP66V1.Storage Unit Structure, [230](#)
 RP66V1.Subfield, [230](#)
 RP66V1.Successor, [230](#)
 RP66V1.Syntax, [230](#)
 RP66V1.Template, [230](#)
 RP66V1.Tool Zero Point, [230](#)
 RP66V1.Trailing Length, [230](#)
 RP66V1.Transient Information, [230](#)
 RP66V1.Unzoned, [230](#)
 RP66V1.Update, [231](#)
 RP66V1.Value (of an Attribute), [231](#)
 RP66V1.Vertical Depth, [231](#)
 RP66V1.Vertical Generatrix, [231](#)
 RP66V1.Visible Envelope, [231](#)
 RP66V1.Visible Record, [231](#)
 RP66V1.Visible Record Length, [231](#)
 RP66V1.Well Reference Point, [231](#)
 RP66V1.Zone, [231](#)
 RP66V1.Zoned, [231](#)

RP66V1FrameArray (class in *TotalDepth.RP66V1.core.LogPass*), [93](#)
 RP66V1FrameChannel (class in *TotalDepth.RP66V1.core.LogPass*), [93](#)
 RP66V2, [231](#)
 run() (*TotalDepth.common.process.ProcessLoggingThread* method), [152](#)

S

same_dimension() (in module *TotalDepth.common.units*), [158](#)
 Sample (class in *TotalDepth.common.Slice*), [155](#)
 samples() (*TotalDepth.LIS.core.LogiRec.DatumSpecBlock* method), [49](#)
 scale (*TotalDepth.common.units.Unit* attribute), [157](#)
 scale (*TotalDepth.LIS.PlotLogPasses.IndexTableValue* attribute), [80](#)
 scale (*TotalDepth.PlotLogs.IndexTableValue* attribute), [5](#)
 scale() (*TotalDepth.util.plot.Coord.Dim* method), [180](#)
 scale() (*TotalDepth.util.plot.Coord.Pt* method), [182](#)
 ScaleSliceCurve (class in *TotalDepth.util.plot.Plot*), [197](#)
 scan_a_single_file() (in module *TotalDepth.RP66V1.ScanHTML*), [116](#)
 scan_dir_multiprocessing() (in module *TotalDepth.RP66V1.ScanHTML*), [116](#)
 scan_dir_or_file() (in module *TotalDepth.RP66V1.ScanHTML*), [116](#)
 scan_file_no_output() (in module *TotalDepth.LIS.core.File*), [26](#)
 scan_file_with_different_padding() (in module *TotalDepth.LIS.core.File*), [26](#)
 scan_RP66V1_file_data_content() (in module *TotalDepth.RP66V1.Scan*), [115](#)
 scan_RP66V1_file_EFLR_IFLR() (in module *TotalDepth.RP66V1.Scan*), [115](#)
 scan_RP66V1_file_logical_data() (in module *TotalDepth.RP66V1.Scan*), [115](#)
 scan_RP66V1_file_visible_records() (in module *TotalDepth.RP66V1.Scan*), [114](#)
 scan_RP66V1_LRSH_consistency() (in module *TotalDepth.RP66V1.Scan*), [115](#)
 seal() (*TotalDepth.RP66V1.core.pFile.FileLogicalData* method), [90](#)
 SECT_DESCRIPTION_MAP (in module *TotalDepth.LAS.core.LASRead*), [16](#)
 SECT_TYPES (in module *TotalDepth.LAS.core.LASRead*), [16](#)
 SECT_TYPES_WITH_DATA_LINES (in module *TotalDepth.LAS.core.LASRead*), [16](#)
 SECT_TYPES_WITH_INDEX (in module *TotalDepth.LAS.core.LASRead*), [16](#)

section() (in module <i>TotalDepth.common.colorama</i>), 140	setLogicalRecord() (<i>TotalDepth.LIS.core.FileIndexer.IndexTable</i> method), 29
SECTION_MNEMONIC_ORDER_AND_VALUES (<i>TotalDepth.LAS.core.LASRead.LASSection</i> attribute), 17	setLogicalRecord() (<i>TotalDepth.LIS.core.FileIndexer.IndexUnknownInternalFormat</i> method), 28
SectLine (class in <i>TotalDepth.LAS.core.LASRead</i>), 16	setValue() (<i>TotalDepth.LIS.core.LogiRec.CbEngVal</i> method), 45
seek() (<i>TotalDepth.LIS.core.RawStream.RawStream</i> method), 62	shape() (<i>TotalDepth.RP66V1.core.pFile.LogicalData</i> property), 89
seek() (<i>TotalDepth.RP66V1.core.LogPass.RP66V1FrameChannel</i> method), 93	shape (<i>TotalDepth.common.LogPass.FrameArray</i> property), 143
seek() (<i>TotalDepth.RP66V1.core.pFile.LogicalData</i> method), 89	shape (<i>TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormatted</i> property), 108
seekCurrentLrStart() (<i>TotalDepth.LIS.core.File.FileRead</i> method), 24	single_bit_path_to_las_path() (in module <i>TotalDepth.BIT.ToLAS</i>), 13
seekCurrentLrStart() (<i>TotalDepth.LIS.core.PhysRec.PhysRecRead</i> method), 61	single_rp66v1_file_to_las() (in module <i>TotalDepth.RP66V1.ToLAS</i>), 119
seekLr() (<i>TotalDepth.LIS.core.File.FileRead</i> method), 24	size (<i>TotalDepth.BIT.ReadBIT.FileSizeTime</i> attribute), 12
seekLr() (<i>TotalDepth.LIS.core.PhysRec.PhysRecRead</i> method), 61	size (<i>TotalDepth.util.FileStatus.FileInfo</i> property), 170
SEMANTICS (in module <i>TotalDepth.RP66V1.core.LogicalRecord.Semantics</i>), 109	size (<i>TotalDepth.util.plot.LogHeader.Static</i> attribute), 193
Set (class in <i>TotalDepth.RP66V1.core.LogicalRecord.EFLR</i>), 104	size() (<i>TotalDepth.util.plot.LogHeader.APIHeaderBase</i> method), 194
set_log_level() (in module <i>TotalDepth.common.cmn_cmd_opts</i>), 140	size_index (<i>TotalDepth.RP66V1.IndexPickle.IndexResult</i> attribute), 110
set_row_column_span() (<i>TotalDepth.util.DictTree.DictTreeHtmlTable</i> method), 164	size_index (<i>TotalDepth.RP66V1.IndexXML.IndexResult</i> attribute), 112
setEntryBlock() (<i>TotalDepth.LIS.core.LogiRec.EntryBlockSet</i> method), 48	size_index (<i>TotalDepth.RP66V1.LogRecIndex.IndexResult</i> attribute), 113
setFrameBytes() (<i>TotalDepth.LIS.core.FrameSet.FrameSet</i> method), 35	size_index (<i>TotalDepth.RP66V1.util.IndexXMLRead.IndexResult</i> attribute), 119
setFrameSet() (<i>TotalDepth.LIS.core.LogPass.LogPass</i> method), 53	size_input (<i>TotalDepth.RP66V1.IndexPickle.IndexResult</i> attribute), 110
setFrameSetChX() (<i>TotalDepth.LIS.core.LogPass.LogPass</i> method), 53	size_input (<i>TotalDepth.RP66V1.IndexXML.IndexResult</i> attribute), 112
setIndirectX() (<i>TotalDepth.LIS.core.FrameSet.FrameSetsize_output</i> method), 35	size_input (<i>TotalDepth.RP66V1.LogRecIndex.IndexResult</i> attribute), 113
setLogicalRecord() (<i>TotalDepth.LIS.core.FileIndexer.IndexLrFull</i> method), 28	size_input (<i>TotalDepth.RP66V1.Scan.IndexResult</i> attribute), 115
setLogicalRecord() (<i>TotalDepth.LIS.core.FileIndexer.IndexNone</i> method), 27	size_input (<i>TotalDepth.RP66V1.util.IndexXMLRead.IndexResult</i> attribute), 120
setLogicalRecord() (<i>TotalDepth.LIS.core.FileIndexer.IndexObjBase</i> method), 27	size_output (<i>TotalDepth.RP66V1.Scan.IndexResult</i> attribute), 115
	sizeof_array (<i>TotalDepth.common.LogPass.FrameArray</i> property), 143
	sizeof_array (<i>TotalDepth.common.LogPass.FrameChannel</i> property), 142
	sizeof_frame (<i>TotalDepth.common.LogPass.FrameArray</i> property), 143
	sizeof_frame (<i>TotalDepth.common.LogPass.FrameChannel</i> property), 142
	skipLrBytes() (<i>TotalDepth.LIS.core.File.FileRead</i>

method), 24

skipLrBytes() (TotalDepth.LIS.core.PhysRec.PhysRecRead method), 60

skipToEndOfFrame() (TotalDepth.LIS.core.Type01Plan.FrameSetPlan method), 74

skipToNextLr() (TotalDepth.LIS.core.File.FileRead method), 24

skipToNextLr() (TotalDepth.LIS.core.PhysRec.PhysRecRead method), 60

slb_data_channel() (in module TotalDepth.common.lookup_mnemonic), 148

slb_load_units() (in module TotalDepth.common.units), 158

slb_logging_tool() (in module TotalDepth.common.lookup_mnemonic), 150

slb_parameter() (in module TotalDepth.common.lookup_mnemonic), 149

slb_standard_form_to_unit_code() (in module TotalDepth.common.units), 158

slb_units() (in module TotalDepth.common.units), 158

Slice (class in TotalDepth.common.Slice), 155

slice (TotalDepth.util.plot.Plot.ScaleSliceCurve attribute), 197

SliceABC (class in TotalDepth.common.Slice), 154

sliceDefaults() (in module TotalDepth.LIS.core.FrameSet), 32

sloc (TotalDepth.util.FileStatus.FileInfo property), 170

spacing (TotalDepth.BIT.ReadBIT.LogPassRange attribute), 11

span (TotalDepth.util.plot.Plot.ScaleSliceCurve attribute), 197

standard_form (TotalDepth.common.units.Unit attribute), 157

STANDARD_PAPER_DEPTH (in module TotalDepth.util.plot.PlotConstants), 202

STANDARD_PAPER_MARGIN (in module TotalDepth.util.plot.PlotConstants), 202

STANDARD_PAPER_WIDTH (in module TotalDepth.util.plot.PlotConstants), 202

start (TotalDepth.util.plot.Plot.ScaleSliceCurve attribute), 197

startNewRow() (TotalDepth.LIS.core.LogiRec.LrTable method), 47

Static (class in TotalDepth.util.plot.LogHeader), 192

status (TotalDepth.LIS.core.LogiRec.CbEngVal property), 45

step() (TotalDepth.common.Slice.Sample method), 155

step() (TotalDepth.common.Slice.Slice method), 155

step() (TotalDepth.common.Slice.SliceABC method), 154

step() (TotalDepth.util.FileBuffer.FileBuffer method), 169

stop() (TotalDepth.util.ExecTimer.Timer method), 168

stop() (TotalDepth.util.ExecTimer.TimerList method), 169

storage_unit_label (TotalDepth.RP66V1.core.LogicalFile.LogicalIndex property), 97

StorageUnitLabel (class in TotalDepth.RP66V1.core.pFile), 86

str() (TotalDepth.util.plot.XMLCfg.LgXMLBase method), 221

str_long() (TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyForm method), 108

stream (TotalDepth.LIS.core.RawStream.RawStream property), 62

strFormat() (TotalDepth.LIS.core.EngVal.EngVal method), 22

strHeader() (TotalDepth.LIS.core.PhysRec.PhysRecBase method), 60

strHeader() (TotalDepth.LIS.core.TifMarker.TifMarkerBase method), 72

string_to_value() (in module TotalDepth.LAS.core.LASRead), 16

stringify() (in module TotalDepth.LIS.ToLAS), 84

stringify_object_by_type() (in module TotalDepth.RP66V1.core.stringify), 102

stringify_value() (TotalDepth.RP66V1.core.LogicalRecord.EFLR.AttributeBase method), 105

strip_path() (in module TotalDepth.DeTif), 4

strip_tif() (in module TotalDepth.DeTif), 4

Stroke (class in TotalDepth.util.plot.Stroke), 212

StrokeBlackSolid (in module TotalDepth.util.plot.Stroke), 212

StrokeGreySolid (in module TotalDepth.util.plot.XGrid), 220

strRep() (TotalDepth.util.Histogram.Histogram method), 171

STRUCT_COMPONENT_BLOCK_PREAMBLE (in module TotalDepth.LIS.core.LogiRec), 41

STRUCT_DSB (in module TotalDepth.LIS.core.LogiRec), 41

STRUCT_ENTRY_BLOCK_PREAMBLE (in module TotalDepth.LIS.core.LogiRec), 41

STRUCT_LR_FILE_HEAD_TAIL (in module TotalDepth.LIS.core.LogiRec), 41

STRUCT_LR_HEAD (in module TotalDepth.LIS.core.LogiRec), 41

STRUCT_LR_REEL_TAPE_HEAD_TAIL (in module TotalDepth.LIS.core.LogiRec), 41

subChMnem() (TotalDepth.LIS.core.LogiRec.DatumSpecBlock method), 49

subChOffsRange() (TotalDepth.LIS.core.FrameSet.ChArTe method), 33

subChOffsSlice() (To-

talDepth.LIS.core.FrameSet.ChArTe method), 33
 SuChArTe (class in *TotalDepth.LIS.core.FrameSet*), 33
 sul (*TotalDepth.RP66V1.core.pIndex.LogicalRecordIndex* property), 92
 summary (*TotalDepth.RP66V1.core.XAxis.XAxis* property), 101
 summary_file_types_supported() (in module *TotalDepth.util.bin_file_type*), 178
 supportedFilmTracks() (*TotalDepth.util.plot.FILMCfg.PhysFilmCfgLISRead* Template method), 190
 SVGCircle (class in *TotalDepth.util.plot.SVGWriter*), 213
 SVGEllipse (class in *TotalDepth.util.plot.SVGWriter*), 213
 SVGGroup (class in *TotalDepth.util.plot.SVGWriter*), 213
 SVGLine (class in *TotalDepth.util.plot.SVGWriter*), 213
 SVGPointList (class in *TotalDepth.util.plot.SVGWriter*), 213
 SVGPolygon (class in *TotalDepth.util.plot.SVGWriter*), 214
 SVGPolyline (class in *TotalDepth.util.plot.SVGWriter*), 214
 SVGRect (class in *TotalDepth.util.plot.SVGWriter*), 213
 SVGText (class in *TotalDepth.util.plot.SVGWriter*), 214
 SVGWriter (class in *TotalDepth.util.plot.SVGWriter*), 212

T

table_as_strings() (*TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormattedLogRecord* method), 108
 TableMatcher (class in *TotalDepth.LIS.TableHistogram*), 83
 TableRow (class in *TotalDepth.LIS.core.LogiRec*), 46
 tAttr (*TotalDepth.util.plot.LogHeader.Static* attribute), 193
 technology (*TotalDepth.common.lookup_mnemonic.LoggingTool* attribute), 150
 tell (*TotalDepth.BIT.ReadBIT.TifMarkedBytes* attribute), 11
 tell (*TotalDepth.BIT.ReadBIT.TifMarker* attribute), 10
 tell (*TotalDepth.DeTif.TifMarker* attribute), 3
 tell (*TotalDepth.LIS.core.FileIndexer.IndexObjBase* property), 27
 tell() (*TotalDepth.LIS.core.File.FileRead* method), 24
 tell() (*TotalDepth.LIS.core.PhysRec.PhysRecRead* method), 61
 tell() (*TotalDepth.LIS.core.RawStream.RawStream* method), 62
 tell() (*TotalDepth.util.FileBuffer.FileBuffer* method), 169
 tellLr() (*TotalDepth.LIS.core.File.FileRead* method), 24
 tellLr() (*TotalDepth.LIS.core.PhysRec.PhysRecRead* method), 61
 tellLrForFrame() (*TotalDepth.LIS.core.Rle.RLEItemType01* method), 70
 tellLrForFrame() (*TotalDepth.LIS.core.Rle.RLEType01* method), 70
 Template (class in *TotalDepth.RP66V1.core.LogicalRecord.EFLR*), 105
 TemplateAttribute (class in *TotalDepth.RP66V1.core.LogicalRecord.EFLR*), 105
 text (*TotalDepth.util.plot.LogHeader.Static* attribute), 193
 TEXT_ATTRS_FINE (in module *TotalDepth.util.plot.LogHeader*), 193
 TEXT_ATTRS_LARGE (in module *TotalDepth.util.plot.LogHeader*), 193
 TEXT_ATTRS_LARGE_WOB (in module *TotalDepth.util.plot.LogHeader*), 193
 TIF_FILE_PREFIX (in module *TotalDepth.RP66V1.core.pFile*), 86
 TIF_FIRST_WORD_LIMIT (in module *TotalDepth.LIS.core.TifMarker*), 72
 TIF_NUM_WORDS (in module *TotalDepth.LIS.core.TifMarker*), 72
 tif_scan_file_object() (in module *TotalDepth.DeTif*), 4
 tif_scan_path() (in module *TotalDepth.DeTif*), 4
 TIF_TOTAL_BYTES (in module *TotalDepth.LIS.core.TifMarker*), 72
 tif_type (*TotalDepth.BIT.ReadBIT.TifMarkedBytes* attribute), 11
 TIF_WORD_ALL_FORMAT_WRONG_SEX (in module *TotalDepth.LIS.core.TifMarker*), 72
 TIF_WORD_BYTES (in module *TotalDepth.LIS.core.TifMarker*), 72
 TIF_WORD_FORMAT (in module *TotalDepth.LIS.core.TifMarker*), 72
 TIF_WORD_FORMAT_WRONG_SEX (in module *TotalDepth.LIS.core.TifMarker*), 72
 TifMarkedBytes (class in *TotalDepth.BIT.ReadBIT*), 11
 TifMarker (class in *TotalDepth.BIT.ReadBIT*), 10
 TifMarker (class in *TotalDepth.DeTif*), 3
 TifMarkerBase (class in *TotalDepth.LIS.core.TifMarker*), 72
 TifMarkerRead (class in *TotalDepth.LIS.core.TifMarker*), 72
 TifMarkerWrite (class in *TotalDepth.LIS.core.TifMarker*), 73

TifType (class in TotalDepth.BIT.ReadBIT), 11
 time (TotalDepth.BIT.ReadBIT.FileSizeTime attribute), 13
 time (TotalDepth.RP66V1.IndexXML.IndexResult attribute), 112
 time (TotalDepth.RP66V1.Scan.IndexResult attribute), 115
 time (TotalDepth.RP66V1.util.IndexXMLRead.IndexResult attribute), 120
 time_index (TotalDepth.RP66V1.IndexPickle.IndexResult attribute), 110
 time_read (TotalDepth.RP66V1.IndexPickle.IndexResult attribute), 110
 time_read_back (TotalDepth.RP66V1.LogRecIndex.IndexResult attribute), 113
 time_write (TotalDepth.RP66V1.IndexPickle.IndexResult attribute), 110
 time_write (TotalDepth.RP66V1.LogRecIndex.IndexResult attribute), 113
 Timer (class in TotalDepth.util.ExecTimer), 168
 timer (TotalDepth.util.ExecTimer.TimerList property), 169
 TimerList (class in TotalDepth.util.ExecTimer), 168
 TITLE_FONT_FAMILY (TotalDepth.util.plot.Plot.Plot attribute), 199
 TITLE_FONT_SIZE (TotalDepth.util.plot.Plot.Plot attribute), 199
 to49() (in module TotalDepth.LIS.core.pRepCode), 66
 to50() (in module TotalDepth.LIS.core.pRepCode), 66
 to56() (in module TotalDepth.LIS.core.pRepCode), 66
 to66() (in module TotalDepth.LIS.core.pRepCode), 67
 to68() (in module TotalDepth.LIS.core.cRepCode), 68
 to68() (in module TotalDepth.LIS.core.pRepCode), 67
 to70() (in module TotalDepth.LIS.core.pRepCode), 67
 to73() (in module TotalDepth.LIS.core.pRepCode), 67
 to77() (in module TotalDepth.LIS.core.pRepCode), 67
 to79() (in module TotalDepth.LIS.core.pRepCode), 67
 to_cartesian() (in module TotalDepth.util.plot.Coord), 182
 to_polar() (in module TotalDepth.util.plot.Coord), 182
 tocStr() (TotalDepth.LIS.core.FileIndexer.IndexFileHeadTotal method), 28
 tocStr() (TotalDepth.LIS.core.FileIndexer.IndexLogPassTotal method), 29
 tocStr() (TotalDepth.LIS.core.FileIndexer.IndexObjBaseTotal method), 27
 tocStr() (TotalDepth.LIS.core.FileIndexer.IndexReelHeadTotal method), 28
 tocStr() (TotalDepth.LIS.core.FileIndexer.IndexTableTotal method), 29
 tocStr() (TotalDepth.LIS.core.FileIndexer.IndexTapeHeadTotal method), 28
 tool (TotalDepth.common.lookup_mnemonic.ToolDescriptionTotal attribute), 148
 ToolDescription (class in TotalDepth.common.lookup_mnemonic), 148
 TotalDepth.BIT.ReadBIT module, 7
 TotalDepth.BIT.ToLAS module, 13
 TotalDepth.common.cmn_cmd_opts module, 139
 TotalDepth.common.colorama module, 140
 TotalDepth.common.data_table module, 140
 TotalDepth.common.LogPass module, 141
 TotalDepth.common.lookup_mnemonic module, 144
 TotalDepth.common.process module, 151
 TotalDepth.common.Rle module, 153
 TotalDepth.common.Slice module, 154
 TotalDepth.common.statistics module, 156
 TotalDepth.common.units module, 157
 TotalDepth.common.xml module, 159
 TotalDepth.common.xxd module, 159
 TotalDepth.DAT.DAT_parser module, 137
 TotalDepth.DeTif module, 3
 TotalDepth.LAS.core.LASRead module, 15
 TotalDepth.LAS.ReadLASFiles module, 15
 TotalDepth.LIS.core.cRepCode module, 68
 TotalDepth.LIS.core.EngVal module, 21
 TotalDepth.LIS.core.File module, 24
 TotalDepth.LIS.core.FileIndexer module, 27
 TotalDepth.LIS.core.FrameSet module, 32
 TotalDepth.LIS.core.LogiRec module, 38
 TotalDepth.LIS.core.LogPass module, 50
 TotalDepth.LIS.core.Mnem module, 57

TotalDepth.LIS.core.PhysRec module, 58	TotalDepth.RP66V1.core.LogicalRecord.Semantics module, 109
TotalDepth.LIS.core.pRepCode module, 66	TotalDepth.RP66V1.core.LogicalRecord.Types module, 109
TotalDepth.LIS.core.RawStream module, 62	TotalDepth.RP66V1.core.LogPass module, 93
TotalDepth.LIS.core.RepCode module, 64	TotalDepth.RP66V1.core.pFile module, 85
TotalDepth.LIS.core.Rle module, 69	TotalDepth.RP66V1.core.pIndex module, 91
TotalDepth.LIS.core.TifMarker module, 71	TotalDepth.RP66V1.core.RepCode module, 97
TotalDepth.LIS.core.Type01Plan module, 73	TotalDepth.RP66V1.core.stringify module, 102
TotalDepth.LIS.core.Units module, 75	TotalDepth.RP66V1.core.Units module, 97
TotalDepth.LIS.DumpFrameSet module, 78	TotalDepth.RP66V1.core.XAxis module, 101
TotalDepth.LIS.Index module, 78	TotalDepth.RP66V1.IndexPickle module, 110
TotalDepth.LIS.lis_cmn_cmd_opts module, 84	TotalDepth.RP66V1.IndexXML module, 111
TotalDepth.LIS.LisToHtml module, 79	TotalDepth.RP66V1.LogRecIndex module, 113
TotalDepth.LIS.PlotLogPasses module, 80	TotalDepth.RP66V1.Scan module, 114
TotalDepth.LIS.ProcLISPath module, 81	TotalDepth.RP66V1.ScanHTML module, 116
TotalDepth.LIS.RandomFrameSetRead module, 82	TotalDepth.RP66V1.SearchFF01 module, 117
TotalDepth.LIS.ScanLogiData module, 82	TotalDepth.RP66V1.ToLAS module, 117
TotalDepth.LIS.ScanLogiRec module, 82	TotalDepth.RP66V1.util.IndexXMLRead module, 119
TotalDepth.LIS.ScanPhysRec module, 82	TotalDepth.RP66V1.util.XMLReadUnits module, 120
TotalDepth.LIS.TableHistogram module, 83	TotalDepth.util.archive module, 176
TotalDepth.LIS.ToLAS module, 84	TotalDepth.util.bin_file_type module, 177
TotalDepth.PlotLogs module, 5	TotalDepth.util.CopyBinFiles module, 161
TotalDepth.RP66V1.core.AbsentValue module, 85	TotalDepth.util.DictTree module, 161
TotalDepth.RP66V1.core.LogicalFile module, 94	TotalDepth.util.DirWalk module, 165
TotalDepth.RP66V1.core.LogicalRecord.ComponentDescription module, 102	TotalDepth.util.EBCDIC module, 166
TotalDepth.RP66V1.core.LogicalRecord.EFLR module, 104	TotalDepth.util.ExecTimer module, 168
TotalDepth.RP66V1.core.LogicalRecord.Encryption module, 108	TotalDepth.util.FileBuffer module, 169
TotalDepth.RP66V1.core.LogicalRecord.IFLR module, 108	TotalDepth.util.FileStatus module, 170

- TotalDepth.util.gnuplot
 module, 178
 TotalDepth.util.Histogram
 module, 171
 TotalDepth.util.HtmlUtils
 module, 172
 TotalDepth.util.PatternSearch
 module, 173
 TotalDepth.util.plot.AREACfg
 module, 179
 TotalDepth.util.plot.Coord
 module, 179
 TotalDepth.util.plot.FILMCfg
 module, 185
 TotalDepth.util.plot.FILMCfgXML
 module, 191
 TotalDepth.util.plot.LogHeader
 module, 192
 TotalDepth.util.plot.Plot
 module, 195
 TotalDepth.util.plot.PlotConstants
 module, 201
 TotalDepth.util.plot.PRESCfg
 module, 202
 TotalDepth.util.plot.PRESCfgXML
 module, 210
 TotalDepth.util.plot.Stroke
 module, 212
 TotalDepth.util.plot.SVGWriter
 module, 212
 TotalDepth.util.plot.Track
 module, 216
 TotalDepth.util.plot.XGrid
 module, 220
 TotalDepth.util.plot.XMLCfg
 module, 221
 TotalDepth.util.plot.XMLMatches
 module, 222
 TotalDepth.util.RemoveDupeFiles
 module, 173
 TotalDepth.util.XmlWrite
 module, 174
 totalFrames (TotalDepth.LIS.core.LogPass.LogPass
 property), 52
 totalFrames() (TotalDepth.LIS.core.Rle.RLEItemType01
 method), 69
 totalFrames() (TotalDepth.LIS.core.Rle.RLEType01
 method), 70
 TRAC_STANDARD_SPAN (To-
 talDepth.util.plot.FILMCfgXML.PhysFilmCfgXMLRead
 attribute), 191
 TRAC_UNIQUEID_TO_NON_STANDARD_SPAN (To-
 talDepth.util.plot.FILMCfgXML.PhysFilmCfgXMLRead
 attribute), 191
 TRAC_UNIQUEID_TO_NON_STANDARD_START (To-
 talDepth.util.plot.FILMCfgXML.PhysFilmCfgXMLRead
 attribute), 191
 TRAC_XML_UNIQUEID_TO_PRES (To-
 talDepth.util.plot.PRESCfgXML.CurveCfgXMLRead
 attribute), 211
 Track (class in TotalDepth.util.plot.Track), 219
 trackTopLeft (TotalDepth.util.plot.Plot.PlotRoll prop-
 erty), 199
 TrackWidthData (class in To-
 talDepth.util.plot.PRESCfg), 208
 tracValueFunction() (To-
 talDepth.util.plot.PRESCfg.CurveCfg method),
 209
 tracWidthData() (To-
 talDepth.util.plot.PRESCfg.CurveCfg method),
 209
 type (TotalDepth.BIT.ReadBIT.TifMarker attribute), 10
 type (TotalDepth.DeTif.TifMarker attribute), 3
 type (TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.Comp
 property), 103
 type (TotalDepth.RP66V1.core.LogicalRecord.ComponentDescriptor.RoleT
 attribute), 102
 type (TotalDepth.RP66V1.core.LogicalRecord.EFLR.PublicEFLRType
 attribute), 106
 type (TotalDepth.RP66V1.core.LogicalRecord.Types.TypeDescriptionAllow
 attribute), 110
 type01Plan (TotalDepth.LIS.core.LogPass.LogPass
 property), 51
 TypeDescriptionAllowableSetTypes (class in To-
 talDepth.RP66V1.core.LogicalRecord.Types),
 109
 TypeDescriptionDataDescriptorObjectReferenceType
 (in module To-
 talDepth.RP66V1.core.LogicalRecord.Types),
 109
U
 uniqueIds() (TotalDepth.util.plot.FILMCfgXML.FilmCfgXMLRead
 method), 192
 Unit (class in TotalDepth.common.units), 157
 unit (TotalDepth.LAS.core.LASRead.SectLine attribute),
 17
 UNIT_MAP (in module TotalDepth.util.plot.Coord), 179
 UNIT_MAP_DEFAULT_FORMAT (in module To-
 talDepth.util.plot.Coord), 180
 UNIT_MAP_DEFAULT_FORMAT_WITH_UNITS (in module
 TotalDepth.util.plot.Coord), 180
 unit_quantity (TotalDepth.common.lookup_mnemonic.Channel
 attribute), 148
 unit_quantity (TotalDepth.common.lookup_mnemonic.Parameter
 attribute), 149
 UnitCategories() (in module To-
 talDepth.LIS.core.Units), 76

UnitConvert (class in TotalDepth.LIS.core.Units), 76
 UnitConvertCategory (class in TotalDepth.LIS.core.Units), 76
 unitConvertor() (TotalDepth.LIS.core.Units.UnitConvertCategory method), 76
 unitDescription() (in module TotalDepth.LIS.core.Units), 77
 units() (in module TotalDepth.LIS.core.Units), 77
 units() (in module TotalDepth.util.plot.Coord), 180
 units() (TotalDepth.LAS.core.LASRead.LASSection method), 17
 units() (TotalDepth.LIS.core.Units.UnitConvertCategory method), 76
 UNITS_LAS_TO_LIS (TotalDepth.LAS.core.LASRead.LASBase attribute), 18
 unknown_a (TotalDepth.BIT.ReadBIT.LogPassRange attribute), 11
 unknown_b (TotalDepth.BIT.ReadBIT.LogPassRange attribute), 11
 unpack() (TotalDepth.LIS.core.File.FileRead method), 25
 unpickle() (in module TotalDepth.RP66V1.IndexPickle), 111
 unpickle() (in module TotalDepth.RP66V1.LogRecIndex), 113
 UOM, 231
 usesInternalRecords (TotalDepth.PlotLogs.PlotLogPasses property), 6
 usesOutpChannel() (TotalDepth.util.plot.PRESCfg.PresCfg method), 209

V

validate() (TotalDepth.RP66V1.core.pIndex.LogicalRecord method), 92
 validate_positions() (TotalDepth.RP66V1.core.pFile.FileRead method), 91
 valu (TotalDepth.LAS.core.LASRead.SectLine attribute), 17
 value (TotalDepth.LIS.core.LogiRec.CbEngVal property), 45
 value (TotalDepth.LIS.core.LogiRec.LrTable property), 46
 value (TotalDepth.LIS.core.LogiRec.TableRow property), 46
 value (TotalDepth.util.EBCDIC.EbcdicAsciiTableEntry property), 167
 value() (TotalDepth.common.Rle.RLE method), 154
 value() (TotalDepth.common.Rle.RLEItem method), 153
 value() (TotalDepth.LIS.core.FrameSet.AccActivity method), 38
 value() (TotalDepth.LIS.core.FrameSet.AccBias method), 37
 value() (TotalDepth.LIS.core.FrameSet.AccCount method), 37
 value() (TotalDepth.LIS.core.FrameSet.AccDelta method), 37
 value() (TotalDepth.LIS.core.FrameSet.AccDrift method), 38
 value() (TotalDepth.LIS.core.FrameSet.AccMax method), 36
 value() (TotalDepth.LIS.core.FrameSet.AccMean method), 36
 value() (TotalDepth.LIS.core.FrameSet.AccMin method), 36
 value() (TotalDepth.LIS.core.FrameSet.AccStDev method), 37
 value() (TotalDepth.LIS.core.FrameSet.FrameSet method), 35
 value() (TotalDepth.LIS.core.Rle.RLEItemType01 method), 69
 value() (TotalDepth.util.DictTree.DictTree method), 161
 valueIdxInFrame() (TotalDepth.LIS.core.FrameSet.FrameSet method), 35
 valueIdxStartExtCh() (TotalDepth.LIS.core.FrameSet.FrameSet method), 35
 values() (TotalDepth.common.Rle.RLE method), 154
 values() (TotalDepth.common.Rle.RLEItem method), 153
 values() (TotalDepth.LIS.core.LogiRec.DatumSpecBlock method), 49
 values() (TotalDepth.LIS.core.Rle.RLEItemType01 method), 69
 values() (TotalDepth.util.DictTree.DictTree method), 161
 values_as_strings() (TotalDepth.RP66V1.core.LogicalRecord.EFLR.Object method), 106
 valuesPerFrame (TotalDepth.LIS.core.FrameSet.FrameSet property), 34
 version() (in module TotalDepth.util.gnuplot), 178
 VIEW_BOX_UNITS_PER_PLOT_UNITS (in module TotalDepth.util.plot.PlotConstants), 201
 view_remaining() (TotalDepth.RP66V1.core.pFile.LogicalData method), 89
 viewBox (TotalDepth.util.plot.Plot.PlotRoll property), 199
 viewPort() (TotalDepth.util.plot.LogHeader.APIHeaderBase method), 194
 visible_record_positions (To-

talDepth.RP66V1.core.LogicalFile.LogicalIndex property), 97
visible_record_positions (*TotalDepth.RP66V1.core.pIndex.LogicalRecordIndex* property), 92
VisibleRecord (class in *TotalDepth.RP66V1.core.pFile*), 87
vr_position (*TotalDepth.RP66V1.Scan.LRSHSummary* attribute), 114

W

w (*TotalDepth.util.plot.LogHeader.Static* attribute), 193
walk_row_col_span() (*TotalDepth.util.DictTree.DictTreeHtmlTable* method), 165
WELL_INFORMATION_FROM_ORIGIN (in module *TotalDepth.RP66V1.ToLAS*), 118
width (*TotalDepth.util.plot.Plot.PlotRoll* property), 199
width (*TotalDepth.util.plot.Stroke.Stroke* attribute), 212
widthDim (*TotalDepth.util.plot.Plot.PlotRoll* property), 199
wordLength() (in module *TotalDepth.LIS.core.pRepCode*), 66
wrapPos() (*TotalDepth.util.plot.PRESCfg.LineTransBase* method), 207
wrapPos() (*TotalDepth.util.plot.PRESCfg.LineTransLin* method), 208
wrapPos() (*TotalDepth.util.plot.PRESCfg.LineTransLog10* method), 208
write() (*TotalDepth.LIS.core.File.FileWrite* method), 25
write() (*TotalDepth.LIS.core.RawStream.RawStream* method), 63
write() (*TotalDepth.LIS.core.TifMarker.TifMarkerWrite* method), 73
write() (*TotalDepth.util.FileStatus.FileInfo* method), 170
write() (*TotalDepth.util.FileStatus.FileInfoSet* method), 170
write_las_file() (in module *TotalDepth.LIS.ToLAS*), 84
write_las_header() (in module *TotalDepth.RP66V1.ToLAS*), 117
write_logical_file_sequence_to_xml() (in module *TotalDepth.RP66V1.IndexXML*), 112
write_logical_index_to_las() (in module *TotalDepth.RP66V1.ToLAS*), 119
write_parameter_section_to_las() (in module *TotalDepth.RP66V1.ToLAS*), 118
write_test_file() (in module *TotalDepth.util.gnuplot*), 179
write_well_information_to_las() (in module *TotalDepth.RP66V1.ToLAS*), 118
writeBytes() (in module *TotalDepth.LIS.core.RepCode*), 65
writeBytes66() (in module *TotalDepth.LIS.core.RepCode*), 64
writeBytes68() (in module *TotalDepth.LIS.core.RepCode*), 64
writeBytes73() (in module *TotalDepth.LIS.core.RepCode*), 65
writeDictTreeAsTable() (in module *TotalDepth.util.HtmlUtils*), 172
writeECMAScript() (*TotalDepth.util.XmlWrite.XmlStream* method), 175
writeFileListAsTable() (in module *TotalDepth.util.HtmlUtils*), 172
writeFileListTrippleAsTable() (in module *TotalDepth.util.HtmlUtils*), 172
writeFilePathsAsTable() (in module *TotalDepth.util.HtmlUtils*), 172
writeHeader() (*TotalDepth.util.FileStatus.FileInfo* method), 170
writeHTML() (*TotalDepth.LIS.PlotLogPasses.PlotLogInfo* method), 80
writeHTML() (*TotalDepth.PlotLogs.PlotLogInfo* method), 6
writeHtmlFileAnchor() (in module *TotalDepth.util.HtmlUtils*), 172
writeHtmlFileLink() (in module *TotalDepth.util.HtmlUtils*), 172
writeLr() (*TotalDepth.LIS.core.PhysRec.PhysRecWrite* method), 62

X

x (*TotalDepth.util.plot.LogHeader.Static* attribute), 193
X Axis, 231
x_axis (*TotalDepth.RP66V1.core.XAxis.IFLRReference* attribute), 101
x_axis_len_input_bytes (*TotalDepth.RP66V1.core.LogPass.RP66V1FrameArray* property), 94
x_axis_start (*TotalDepth.LAS.core.LASRead.LASBase* property), 18
x_axis_step (*TotalDepth.LAS.core.LASRead.LASBase* property), 18
x_axis_stop (*TotalDepth.LAS.core.LASRead.LASBase* property), 18
x_axis_units (*TotalDepth.LAS.core.LASRead.LASBase* property), 18
Xaxis, 231
XAxis (class in *TotalDepth.RP66V1.core.XAxis*), 101
XAxisDecl (class in *TotalDepth.LIS.core.FrameSet*), 33
xAxisDecl (*TotalDepth.LIS.core.FrameSet.FrameSet* property), 35
xAxisFirst() (*TotalDepth.LIS.core.Rle.RLEItemType01* method), 70

[xAxisFirst\(\)](#) (*TotalDepth.LIS.core.Rle.RLEType01 method*), 70
[xAxisFirstEngVal](#) (*TotalDepth.LIS.core.LogPass.LogPass property*), 52
[xAxisFirstVal](#) (*TotalDepth.LIS.core.LogPass.LogPass property*), 52
[xAxisFirstValOptical](#) (*TotalDepth.LIS.core.LogPass.LogPass property*), 52
[xAxisIndex](#) (*TotalDepth.LIS.core.LogPass.LogPass property*), 51
[xAxisLast\(\)](#) (*TotalDepth.LIS.core.Rle.RLEItemType01 method*), 70
[xAxisLast\(\)](#) (*TotalDepth.LIS.core.Rle.RLEType01 method*), 70
[xAxisLastEngVal](#) (*TotalDepth.LIS.core.LogPass.LogPass property*), 52
[xAxisLastFrame\(\)](#) (*TotalDepth.LIS.core.Rle.RLEType01 method*), 70
[xAxisLastVal](#) (*TotalDepth.LIS.core.LogPass.LogPass property*), 52
[xAxisLastValOptical](#) (*TotalDepth.LIS.core.LogPass.LogPass property*), 52
[xAxisSpacing](#) (*TotalDepth.LIS.core.LogPass.LogPass property*), 52
[xAxisSpacingOptical](#) (*TotalDepth.LIS.core.LogPass.LogPass property*), 52
[xAxisStep\(\)](#) (*TotalDepth.LIS.core.FrameSet.FrameSet method*), 34
[xAxisUnits](#) (*TotalDepth.LIS.core.LogPass.LogPass property*), 52
[xAxisUnits](#) (*TotalDepth.LIS.core.Rle.RLEType01 property*), 70
[xAxisUnitsOptical](#) (*TotalDepth.LIS.core.LogPass.LogPass property*), 52
[xAxisValue\(\)](#) (*TotalDepth.LIS.core.FrameSet.FrameSet method*), 34
[xDepth\(\)](#) (*TotalDepth.util.plot.Plot.PlotRoll method*), 199
[XGrid](#) (class in *TotalDepth.util.plot.XGrid*), 220
[XhtmlStream](#) (class in *TotalDepth.util.XmlWrite*), 175
[xInc](#) (*TotalDepth.LIS.core.LogiRec.EntryBlockSet property*), 48
[XML_CODI_MAP](#) (in module *TotalDepth.util.plot.PRESCfgXML*), 210
[xml_write_value\(\)](#) (in module *TotalDepth.RP66V1.IndexXML*), 111
[xmlSpacePreserve\(\)](#) (*TotalDepth.util.XmlWrite.XmlStream method*), 175
[XmlStream](#) (class in *TotalDepth.util.XmlWrite*), 174
[xMnem](#) (*TotalDepth.util.plot.LogHeader.Static attribute*), 193
[xScale](#) (*TotalDepth.util.plot.FILMCfg.PhysFilmCfg property*), 189
[xScale\(\)](#) (*TotalDepth.util.plot.Plot.Plot method*), 200
[xxd\(\)](#) (in module *TotalDepth.common.xxd*), 159

Y

[y](#) (*TotalDepth.util.plot.LogHeader.Static attribute*), 193
[yield_tif_blocks\(\)](#) (in module *TotalDepth.BIT.ReadBIT*), 11
[ymd](#) (*TotalDepth.LIS.core.LogiRec.LrWithDateField property*), 42

Z

[zero_count](#) (*TotalDepth.common.statistics.LengthDict property*), 156
[zeroBaseUnitsBox\(\)](#) (in module *TotalDepth.util.plot.Coord*), 182
[zeroBaseUnitsDim\(\)](#) (in module *TotalDepth.util.plot.Coord*), 182
[zeroBaseUnitsPad\(\)](#) (in module *TotalDepth.util.plot.Coord*), 182
[zeroBaseUnitsPt\(\)](#) (in module *TotalDepth.util.plot.Coord*), 182